Review of `XYin.NumberField.FieldExamples`

Updated from the 2026-06-15 reports

1 Coverage overview

1.1 Source reports and verification

This document is regenerated from the seven per-field inventories in `.overview/` and the synthesis file `PROJECT_OVERVIEW.md`. I checked the inventories against the current Lean sources in `XYin/NumberField/FieldExamples`: `lake build XYin` completes successfully, and the current FieldExamples sorry ledger is exactly the six reported sorries:

- explicit fundamental unit certification:

  ```
  CubeRootThree/Units.fundamentalUnitSystem_isFundamental
  CyclotomicFive/Units.fundamentalUnitSystem_isFundamental
  CyclotomicEight/Units.fundamentalUnitSystem_isFundamental
  ```

- norm-Euclidean covering statements:

  ```
  CubeRootThree/Euclidean.normEuclidean
  CyclotomicFive/Euclidean.normEuclidean
  CyclotomicEight/Euclidean.normEuclidean
  ```

The report headings were also matched against the indicated Lean files and line ranges. The only non-declaration headings are intentional report blocks such as “finite-place examples” or “Different section”; the declaration statements used below match the current source layout.

1.2 Mathlib **`NumberField`** API coverage

The current examples exercise substantially more of the NumberField tree than the earlier version of this document did. In particular, CM fields, local–global/adelic objects, canonical embeddings, product formulas, finite and infinite completions, and Dedekind zeta smoke tests now appear in the inventories.

Basic field structure, degree, power bases. covered All seven folders fix a concrete model K, a distinguished generator, and the degree. Cyclotomic finrank is essentially API-driven; radical/adjoin-root presentations require explicit irreducibility and equivalence plumbing.

Ring of integers, integral bases, discriminants. covered Cyclotomic cases are especially natural through `IsCyclotomicExtension`; radical cases use Eisenstein or discriminant/index arguments. Matching a displayed basis to `NumberField.integralBasis` is still manual.

Prime splitting, inertia, ramification, different ideals. covered Concrete prime arithmetic is formalised throughout. The project now has a shared `Common.genResidueHom` Kummer–Dedekind-style residue map, although congruence splitting laws are still hand-assembled.

Class number, PID/UFD and negative results. covered Class number one, class number two, PID/UFD consequences, and the $\mathbb{Q}(\sqrt{-6})$ non-PID/non-UFD counterexample are all formalised with Mathlib's standard predicates.

Units and regulators. partial Abstract Dirichlet unit structure is proved in every folder. The regulator-value bridge itself is mature API: `regOfFamily_div_regulator` plus `regOfFamily_eq_det`. The remaining issue is not the regulator API but certification of the displayed fundamental unit in the cubic and totally complex cyclotomic cases.

CM fields and local–global layer. covered `NumberField.IsCMField`, finite/infinite places, completions, adeles, product formulas, canonical embeddings, convex-body objects, and Dedekind zeta examples are all exercised, often as anonymous `example` smoke tests.

Norm-Euclideanity. partial Small quadratic and $\mathbb{Q}(\zeta_3)$ Euclidean proofs are direct. The cubic and quartic cyclotomic Euclidean statements remain open because the required covering arguments are genuine geometry-of-numbers work; a dedicated predicate would improve the statement shape but not supply the proof.

1.3 Coverage matrix

Legend:
— means mathematically not applicable;
× means applicable but not explicitly stated in this folder;
△ means stated but the declaration itself contains, or depends essentially on a displayed concrete theorem with, a `sorry`;
✓ means stated without `sorry` and easy/formal API driven;
✓ means stated without `sorry` but proof is genuinely involved.

Statement/Object	$\mathbb{Q}(\sqrt{2})$	$\mathbb{Q}(\sqrt{5})$	$\mathbb{Q}(\sqrt{-6})$	$\mathbb{Q}(\sqrt[3]{3})$	$\mathbb{Q}(\zeta_3)$	$\mathbb{Q}(\zeta_5)$	$\mathbb{Q}(\zeta_8)$
Concrete field model and generator relation	✓	✓	✓	✓	✓	✓	✓
Degree, signature, and place counts	✓	✓	✓	✓	✓	✓	✓
Alternative presentations/equivalences	✓	✓	✓	✓	✓	✓	✓
Ring of integers as generated order	✓	✓	✓	✓	✓	✓	✓
Explicit integral basis	✓	✓	✓	✓	✓	✓	✓
Discriminant of field/integral basis	✓	✓	✓	✓	✓	✓	✓
Norm form and unit criterion	✓	✓	✓	✓	✓	✓	✓
Concrete prime splitting and ramification	✓	✓	✓	✓	✓	✓	✓
Congruence splitting law / degree-one criterion	✓	✓	✓	✓	✓	✓	✓
Class number, class group, PID/UFD consequence	✓	✓	✓	✓	✓	✓	✓
Torsion units and abstract unit group decomposition	✓	✓	✓	✓	✓	✓	✓
Displayed fundamental unit generates free part	✓	✓	—	△	—	△	△
Concrete regulator value	✓	✓	✓	△	✓	△	△
Norm-Euclidean or non-Euclidean statement	✓	✓	✓	△	✓	△	△
Galois group / automorphism structure	✓	✓	✓	✓	✓	✓	✓
Galois closure / named non-cyclic group	—	—	—	✓	—	—	✓
CM field structure	—	—	✓	—	✓	✓	✓
Quadratic subfields of cyclotomic fields	—	—	—	—	—	✓	✓
Local–global, adelic, product-formula, zeta smoke tests	✓	✓	✓	✓	✓	✓	✓

2 $\mathbb{Q}(\sqrt{2})$

Real quadratic field $\mathbb{Q}(\sqrt{2})$, built via `QuadraticAlgebra ℚ 2 0` (the radical model, generator $\sqrt{2}$ with $\sqrt{2}^2 = 2$). Twin of `SqrtFive` (which uses the golden-ratio model `QuadraticAlgebra ℚ 1 1`). The decisive difference: ring of integers is $\mathbb{Z}[\sqrt{2}]$ (monogenic at the *square root*, disc $8 = 2^3$), and the fundamental unit $1+\sqrt{2}$ is proved outright via mathlib's `Zsqrtd/Pell.Solution`$_1$ API – so `unitGroupStructure` ($\mu_2 \times \mathbb{Z}$) is fully formalised; `regulator_eq` is now also proved, making the folder sorry-free. Namespace `NumberField.SqrtTwo` throughout. Every file: `noncomputable section`, `attribute [-instance] DivisionRing.toRatAlgebra`, same open scopes.

All cross-references to `Common.lean` use namespace `NumberField.FieldExamples`. Common API actually used here: `eq_of_le_of_absNorm_eq`, `eq_prime_of_dvd_prime_sq_of_ne_one_of_ne_sq`, `not_isUnramifiedAt_of_map_span_natCast_le_sq_of_not_le_pow_succ`, `not_map_span_natCast_le_pow_succ_of_absNorm_span_eq_pow`, `abs_intNorm_lt_of_abs_norm_le`, `natAbs_lt_of_abs_ratCast_lt`, `torsionPowerHom`, `rankOnePowerHom`.

Each block below begins with the displayed Lean declaration or object definition. For theorem, lemma, instance-proof, and example blocks the proof body is omitted; for definitions and abbreviations the displayed RHS is retained, because it is part of the mathematical object being documented.

2.1 `Basic.lean`

```
instance : Fact (∀ r : ℚ, r ^ 2 ≠ (2 : ℚ) + (0 : ℚ) * r)
```

Mathematical meaning. the `QuadraticAlgebra ℚ 2 0` field-instance hypothesis (2 has no rational square root).

Implementation. from a root build `IsSquare (2:ℚ)` via `isSquare_iff_exists_sq` + `nlinarith`, then `norm_num`.

```
abbrev K : Type := QuadraticAlgebra ℚ (2 : ℚ) 0
```

Mathematical meaning. $\mathbb{Q}(\sqrt{2})$ as a quadratic algebra.

Implementation. This is a direct definition: the displayed RHS fixes the concrete model or object, and later proofs use it by unfolding or by its generated API.

```
instance : NumberField K :=
  NumberField.of_module_finite ℚ K
```

Mathematical meaning. `K` is a number field.

Implementation. This instance is supplied by `NumberField.of_module_finite` after the concrete field has been shown to satisfy its hypotheses.

```
abbrev sqrtTwoRoot : K := QuadraticAlgebra.omega
```

Mathematical meaning. the generator $\sqrt{2}$.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtTwoRoot_sq : sqrtTwoRoot ^ 2 = (2 : K)
```

Mathematical meaning. The displayed Lean declaration has type `sqrtTwoRoot ^ 2 = (2 : K)`.

Implementation. `pow_two` + `QuadraticAlgebra.omega_mul_omega_eq_add`, `ext`/`norm_num`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtTwoPolynomial : ℚ[X] := X ^ 2 - C (2 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `ℚ[X] := X^2 - C 2` – the minimal polynomial.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
instance : Fact (Irreducible sqrtTwoPolynomial)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `irreducible_of_degree_le_three_of_not_isRoot`; root would give `IsSquare 2`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev KAdjoinRoot : Type := AdjoinRoot sqrtTwoPolynomial

abbrev sqrtTwoAdjoinRoot : KAdjoinRoot := AdjoinRoot.root
    sqrtTwoPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `AdjoinRoot sqrtTwoPolynomial`; `AdjoinRoot.root and related terms`. Alternate presentation.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtTwoAdjoinRoot_sq : sqrtTwoAdjoinRoot ^ 2 = (2 : KAdjoinRoot)
```

Mathematical meaning. The displayed Lean declaration has type `sqrtTwoAdjoinRoot^2 = 2`.

Implementation. `AdjoinRoot.eval₂_root` + `calc`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def sqrtTwoAdjoinRootToQuadratic : KAdjoinRoot →ₐ[ℚ] K :=
  AdjoinRoot.liftAlgHom sqrtTwoPolynomial (Algebra.ofId ℚ K) sqrtTwoRoot
    (by
    calc
      sqrtTwoPolynomial.eval₂ (Algebra.ofId ℚ K) sqrtTwoRoot =
          sqrtTwoRoot ^ 2 - 2 := by
        simp [sqrtTwoPolynomial]
      _ = 0 := by rw [sqrtTwoRoot_sq]; ring)

@[simp]
theorem sqrtTwoAdjoinRootToQuadratic_root :
    sqrtTwoAdjoinRootToQuadratic sqrtTwoAdjoinRoot = sqrtTwoRoot
```

Mathematical meaning. The displayed Lean declaration has type `KAdjoinRoot →ₐ[ℚ] K`; root identity.

Implementation. `AdjoinRoot.liftAlgHom`; `AdjoinRoot.liftAlgHom_root`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def sqrtTwoQuadraticToAdjoinRoot : K →ₐ[ℚ] KAdjoinRoot :=
  (QuadraticAlgebra.lift (R := ℚ) (a := (2 : ℚ)) (b := 0))
    ⟨sqrtTwoAdjoinRoot, by
    rw [← pow_two, sqrtTwoAdjoinRoot_sq]
    simp⟩

@[simp]
theorem sqrtTwoQuadraticToAdjoinRoot_root :
    sqrtTwoQuadraticToAdjoinRoot sqrtTwoRoot = sqrtTwoAdjoinRoot
```

Mathematical meaning. The displayed Lean declaration has type `K →ₐ[ℚ] KAdjoinRoot`; root identity.

Implementation. The proof applies the packaged result `QuadraticAlgebra.lift`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
def sqrtTwoAdjoinRootEquivQuadratic : KAdjoinRoot ≃ₐ[ℚ] K where
  __ := sqrtTwoAdjoinRootToQuadratic
  invFun := sqrtTwoQuadraticToAdjoinRoot
  left_inv x := by
    have hcomp :
        sqrtTwoQuadraticToAdjoinRoot.comp sqrtTwoAdjoinRootToQuadratic =
          AlgHom.id ℚ KAdjoinRoot := by
      apply AdjoinRoot.algHom_ext
      change sqrtTwoQuadraticToAdjoinRoot
          (sqrtTwoAdjoinRootToQuadratic sqrtTwoAdjoinRoot) =
    sqrtTwoAdjoinRoot
```

```
      rw [sqrtTwoAdjoinRootToQuadratic_root,
    sqrtTwoQuadraticToAdjoinRoot_root]
    exact DFunLike.congr_fun hcomp x
  right_inv x := by
    have hcomp :
        sqrtTwoAdjoinRootToQuadratic.comp sqrtTwoQuadraticToAdjoinRoot =
          AlgHom.id ℚ K := by
      apply QuadraticAlgebra.algHom_ext
      change sqrtTwoAdjoinRootToQuadratic
          (sqrtTwoQuadraticToAdjoinRoot sqrtTwoRoot) = sqrtTwoRoot
      rw [sqrtTwoQuadraticToAdjoinRoot_root,
    sqrtTwoAdjoinRootToQuadratic_root]
    exact DFunLike.congr_fun hcomp x
```

Mathematical meaning. AdjoinRoot $\cong$ quadratic-algebra presentation.

Implementation. package the two AlgHoms; inverses via `AdjoinRoot.algHom_ext`/`QuadraticAlgebra.algHom_ext`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtTwoAdjoinRootPowerBasis : PowerBasis ℚ KAdjoinRoot :=
  AdjoinRoot.powerBasis (f := sqrtTwoPolynomial)
    ((Fact.out : Irreducible sqrtTwoPolynomial).ne_zero)

abbrev sqrtTwoPowerBasis : PowerBasis ℚ K :=
  sqrtTwoAdjoinRootPowerBasis.map sqrtTwoAdjoinRootEquivQuadratic
```

Mathematical meaning. The displayed Lean declaration has type `PowerBasis ℚ KAdjoinRoot` from `AdjoinRoot.powerBasis`; mapped to `PowerBasis ℚ K`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
@[simp]
theorem sqrtTwoPowerBasis_gen : sqrtTwoPowerBasis.gen = sqrtTwoRoot
```

Mathematical meaning. The displayed Lean declaration has type `sqrtTwoPowerBasis.gen = sqrtTwoRoot`.

Implementation. The proof applies the packaged result `sqrtTwoAdjoinRootToQuadratic_root`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem adjoin_sqrtTwoRoot_eq_top : Algebra.adjoin ℚ ({sqrtTwoRoot} : Set
    K) = ⊤
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.adjoin ℚ {sqrtTwoRoot} = ⊤`.

Implementation. The proof applies the packaged result `PowerBasis.adjoin_gen_eq_top`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem sqrtTwoPowerBasis_dim : sqrtTwoPowerBasis.dim = 2
```

Mathematical meaning. The displayed Lean declaration has type `sqrtTwoPowerBasis.dim = 2`.

Implementation. The proof applies the packaged result `natDegree_X_pow_sub_C`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem degree_eq_two : Module.finrank ℚ K = 2
```

Mathematical meaning. The displayed Lean declaration has type `Module.finrank ℚ K = 2`.

Implementation. `PowerBasis.finrank` + `sqrtTwoPowerBasis_dim`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : Algebra.IsQuadraticExtension ℚ K where
  finrank_eq_two' := degree_eq_two
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. This instance is supplied by `degree_eq_two` after the concrete field has been shown to satisfy its hypotheses.

```
abbrev sqrtTwoUnitPolynomial : ℚ[X] :=
  C (1 : ℚ) * X ^ 2 + C (-2 : ℚ) * X + C (-1 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `ℚ[X] := C 1 * X² + C(-2) * X + C(-1)` – the minimal polynomial X^2-2X-1 of $1+\sqrt{2}$.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
instance : Fact (Irreducible sqrtTwoUnitPolynomial)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `irreducible_of_degree_le_three_of_not_isRoot`; root would give `IsSquare 2` (witness `x-1`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev KUnitModel : Type := AdjoinRoot sqrtTwoUnitPolynomial

abbrev sqrtTwoUnitRoot : KUnitModel := AdjoinRoot.root
    sqrtTwoUnitPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `AdjoinRoot sqrtTwoUnitPolynomial`; `AdjoinRoot.root and related terms` (a root of X^2-2X-1, i.e. $1+\sqrt{2}$).

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtTwoUnitRoot_relation :
    sqrtTwoUnitRoot ^ 2 - 2 * sqrtTwoUnitRoot - 1 = 0
```

Mathematical meaning. The displayed Lean declaration has type $\texttt{sqrtTwoUnitRoot}^2 - 2\cdot\texttt{sqrtTwoUnitRoot} - 1 = 0$.

Implementation. `AdjoinRoot.eval`$_2$`_root`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def sqrtTwoToUnitModel : K →ₐ[ℚ] KUnitModel :=
  (QuadraticAlgebra.lift (R := ℚ) (a := (2 : ℚ)) (b := 0))
    ⟨sqrtTwoUnitRoot - 1, by
      calc
        _ = (2 : KUnitModel) := by
          calc
            _ = sqrtTwoUnitRoot ^ 2 - 2 * sqrtTwoUnitRoot + 1 := by ring
            _ = 2 := by
              have h := sqrtTwoUnitRoot_relation
              ring_nf at h |-
              linear_combination h
        _ = _ := by norm_num [Algebra.smul_def]⟩

@[simp]
theorem sqrtTwoToUnitModel_root :
    sqrtTwoToUnitModel sqrtTwoRoot = sqrtTwoUnitRoot - 1
```

Mathematical meaning. The displayed Lean declaration has type `K →ₐ[ℚ] KUnitModel` sending $\sqrt{2} \mapsto \beta-1$.

Implementation. `QuadraticAlgebra.lift`, `calc+linear_combination sqrtTwoUnitRoot_relation`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def unitModelToSqrtTwo : KUnitModel →ₐ[ℚ] K :=
  AdjoinRoot.liftAlgHom sqrtTwoUnitPolynomial (Algebra.ofId ℚ K)
    (sqrtTwoRoot + 1) (by
      calc
        sqrtTwoUnitPolynomial.eval₂ (Algebra.ofId ℚ K) (sqrtTwoRoot + 1)
            = (sqrtTwoRoot + 1) ^ 2 + -(2 * (sqrtTwoRoot + 1)) + -1 := by
              simp [sqrtTwoUnitPolynomial]
        _ = sqrtTwoRoot ^ 2 - 2 := by ring
        _ = 0 := by rw [sqrtTwoRoot_sq]; ring)

@[simp]
theorem unitModelToSqrtTwo_root :
    unitModelToSqrtTwo sqrtTwoUnitRoot = sqrtTwoRoot + 1
```

Mathematical meaning. The displayed Lean declaration has type `KUnitModel →ₐ[ℚ] K` sending $\beta \mapsto \sqrt{2}+1$.

Implementation. The proof applies the packaged result `AdjoinRoot.liftAlgHom`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
def sqrtTwoEquivUnitModel : K ≃ₐ[ℚ] KUnitModel where
  __ := sqrtTwoToUnitModel
  invFun := unitModelToSqrtTwo
  left_inv x := by
    have hcomp :
        unitModelToSqrtTwo.comp sqrtTwoToUnitModel = AlgHom.id ℚ K := by
      apply QuadraticAlgebra.algHom_ext
      change unitModelToSqrtTwo (sqrtTwoToUnitModel sqrtTwoRoot) =
    sqrtTwoRoot
      rw [sqrtTwoToUnitModel_root]
      simp only [map_sub, map_one, unitModelToSqrtTwo_root]
      ring
    change (unitModelToSqrtTwo.comp sqrtTwoToUnitModel) x = (AlgHom.id ℚ
    K) x
    rw [hcomp]
  right_inv x := by
    have hcomp :
        sqrtTwoToUnitModel.comp unitModelToSqrtTwo = AlgHom.id ℚ
    KUnitModel := by
      apply AdjoinRoot.algHom_ext
      change sqrtTwoToUnitModel (unitModelToSqrtTwo sqrtTwoUnitRoot) =
    sqrtTwoUnitRoot
      rw [unitModelToSqrtTwo_root]
```

```
      simp only [map_add, map_one, sqrtTwoToUnitModel_root]
      ring
    change (sqrtTwoToUnitModel.comp unitModelToSqrtTwo) x =
      (AlgHom.id ℚ KUnitModel) x
    rw [hcomp]

theorem sqrtTwoEquivUnitModel_root :
    sqrtTwoEquivUnitModel sqrtTwoRoot = sqrtTwoUnitRoot - 1
```

Mathematical meaning. the two AdjoinRoot presentations are equivalent.

Implementation. package AlgHoms; inverses via `QuadraticAlgebra.algHom_ext`/`AdjoinRoot.algHom_ext`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

2.2 `Galois.lean`

```
def sqrtTwoConjHom : K →ₐ[ℚ] K :=
  (QuadraticAlgebra.lift (R := ℚ) (a := (2 : ℚ)) (b := 0)) ⟨-sqrtTwoRoot,
    by
    rw [← pow_two, neg_sq, sqrtTwoRoot_sq]
    ext <;> norm_num⟩

@[simp]
theorem sqrtTwoConjHom_root : sqrtTwoConjHom sqrtTwoRoot = -sqrtTwoRoot
```

Mathematical meaning. The displayed Lean declaration has type `K →ₐ[ℚ] K` sending $\sqrt{2} \mapsto -\sqrt{2}$; `sqrtTwoConjHom sqrtTwoRoot = -sqrtTwoRoot`.

Implementation. `QuadraticAlgebra.lift` (verifying $(-\sqrt{2})^2=2$). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def sqrtTwoConj : K ≃ₐ[ℚ] K where
  __ := sqrtTwoConjHom
  invFun := sqrtTwoConjHom
  left_inv x := by
    have hcomp : sqrtTwoConjHom.comp sqrtTwoConjHom = AlgHom.id ℚ K := by
      apply QuadraticAlgebra.algHom_ext
      change sqrtTwoConjHom (sqrtTwoConjHom sqrtTwoRoot) = sqrtTwoRoot
      rw [sqrtTwoConjHom_root]
      simp only [map_neg, sqrtTwoConjHom_root, neg_neg]
    change (sqrtTwoConjHom.comp sqrtTwoConjHom) x = (AlgHom.id ℚ K) x
    rw [hcomp]
  right_inv x := by
```

```
  have hcomp : sqrtTwoConjHom.comp sqrtTwoConjHom = AlgHom.id ℚ K := by
    apply QuadraticAlgebra.algHom_ext
    change sqrtTwoConjHom (sqrtTwoConjHom sqrtTwoRoot) = sqrtTwoRoot
    rw [sqrtTwoConjHom_root]
    simp only [map_neg, sqrtTwoConjHom_root, neg_neg]
  change (sqrtTwoConjHom.comp sqrtTwoConjHom) x = (AlgHom.id ℚ K) x
  rw [hcomp]
```

Mathematical meaning. the nontrivial conjugation automorphism.

Implementation. package `sqrtTwoConjHom` as its own inverse (involution via `QuadraticAlgebra.algHom_ext`).

```
theorem sqrtTwoConj_root : sqrtTwoConj sqrtTwoRoot = -sqrtTwoRoot
```

Mathematical meaning. The displayed Lean declaration has type `sqrtTwoConj sqrtTwoRoot = -sqrtTwoRoot`.

Implementation. The proof applies the packaged result `sqrtTwoConjHom_root`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem isGalois : IsGalois ℚ K
```

Mathematical meaning. The displayed Lean declaration has type `IsGalois ℚ K`.

Implementation. `infer_instance` (quadratic extensions are Galois). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwo_gal_eq_id_or_conj (σ : Gal(K/ℚ)) :
    σ = 1 ∨ σ = sqrtTwoConj
```

Mathematical meaning. The displayed Lean declaration has type `(σ : Gal(K/ℚ)) → σ = 1 ∨ σ = sqrtTwoConj`.

Implementation. $\sigma\sqrt{2}$ satisfies $X^2=2$, factor $(\sigma\sqrt{2}-\sqrt{2})(\sigma\sqrt{2}+\sqrt{2})=0$, `mul_eq_zero`, extensionality via `QuadraticAlgebra.algHom_ext`.

```
private theorem sqrtTwoConj_ne_one : sqrtTwoConj ≠ (1 : Gal(K/ℚ))
```

Mathematical meaning. The displayed Lean declaration has type `sqrtTwoConj ≠ 1`.

Implementation. else $-\sqrt{2}=\sqrt{2} \Rightarrow 2\cdot\sqrt{2}=0 \Rightarrow \sqrt{2}=0 \Rightarrow 2=0$, contradiction.

```
theorem gal_natCard_eq_two : Nat.card (Gal(K/ℚ)) = 2
```

Mathematical meaning. The displayed Lean declaration has type `Nat.card (Gal(K/ℚ)) = 2`.

Implementation. `Nat.card_eq_two_iff` with witnesses `1`,`sqrtTwoConj + sqrtTwo_gal_eq_id_or_conj`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private abbrev CyclicTwo : Type := Multiplicative (ZMod 2)
```

Mathematical meaning. The displayed Lean declaration has type `Multiplicative (ZMod 2)`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem sqrtTwoGal_additive_generator :
    ∀ σ : Additive (Gal(K/ℚ)),
      σ ∈ AddSubgroup.zmultiples (Additive.ofMul sqrtTwoConj)
```

Mathematical meaning. The displayed Lean declaration has type every $\sigma \in$ `AddSubgroup.zmultiples (ofMul sqrtTwoConj)`.

Implementation. case on `sqrtTwo_gal_eq_id_or_conj`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private noncomputable abbrev cyclicTwoEquivSqrtTwoGal : CyclicTwo ≃*
    Gal(K/ℚ) :=
  AddEquiv.toMultiplicativeLeft <|
    zmodAddEquivOfGenerator sqrtTwoGal_additive_generator
    gal_natCard_eq_two

private noncomputable abbrev sqrtTwoGalEquivCyclicTwo : Gal(K/ℚ) ≃*
    CyclicTwo :=
  cyclicTwoEquivSqrtTwoGal.symm
```

Mathematical meaning. The displayed Lean declaration has type `CyclicTwo ≃* Gal(K/ℚ)` (via `zmodAddEquivOfGenerator`+`AddEquiv.toMultiplicativeLeft`); its `.symm`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem galoisGroupStructure : Nonempty (Gal(K/ℚ) ≃* Multiplicative (ZMod
    2))
```

Mathematical meaning. Galois group is cyclic of order 2.

Implementation. ⟨`sqrtTwoGalEquivCyclicTwo`⟩. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

2.3 Embeddings.lean

```
private theorem ringHom_ext_sqrtTwo {f g : K →+* ℂ} (h : f sqrtTwoRoot =
    g sqrtTwoRoot)
    :
    f = g
```

Mathematical meaning. ring homs K→ℂ agreeing on $\sqrt{2}$ are equal.

Implementation. `QuadraticAlgebra.mk_eq_add_smul_omega`, `eq_ratCast` on rational part. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isTotallyReal : NumberField.IsTotallyReal K
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.IsTotallyReal K`.

Implementation. each embedding is real – conjugate root satisfies $X^2=2$, factor, rule out `re=0` branch via `nlinarith [sq_nonneg z.im]`. Uses `ComplexEmbedding.conjugate_coe_eq`, `ringHom_ext_sqrtTwo`.

```
theorem nrComplexPlaces_eq_zero :
    NumberField.InfinitePlace.nrComplexPlaces K = 0
```

Mathematical meaning. The displayed Lean declaration has type `InfinitePlace.nrComplexPlaces K = 0`.

Implementation. The proof applies the packaged result `IsTotallyReal.nrComplexPlaces_eq_zero`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem nrRealPlaces_eq_two :
    NumberField.InfinitePlace.nrRealPlaces K = 2
```

Mathematical meaning. The displayed Lean declaration has type `InfinitePlace.nrRealPlaces K = 2`.

Implementation. `card_add_two_mul_card_eq_rank`, `nrComplexPlaces_eq_zero`, `finrank=2`, `omega`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem exists_realEmbedding_pos :
    ∃ σ : K →ₐ[ℚ] ℝ, σ sqrtTwoRoot = Real.sqrt 2

theorem exists_realEmbedding_neg :
    ∃ σ : K →ₐ[ℚ] ℝ, σ sqrtTwoRoot = -Real.sqrt 2
```

Mathematical meaning. The displayed Lean declaration has type `∃ σ : K →ₐ[ℚ] ℝ, σ sqrtTwoRoot = ±Real.sqrt 2`.

Implementation. `QuadraticAlgebra.lift` with `Real.mul_self_sqrt`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem realEmbedding_root_eq_pos_or_neg (σ : K →ₐ[ℚ] ℝ) :
    σ sqrtTwoRoot = Real.sqrt 2 ∨ σ sqrtTwoRoot = -Real.sqrt 2
```

Mathematical meaning. The displayed Lean declaration has type `(σ : K →ₐ[ℚ] ℝ) → σ sqrtTwoRoot = √2 ∨ = -√2`.

Implementation. The proof applies the packaged result `sq_eq_sq_iff_eq_or_eq_neg`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem infinitePlace_card_eq_two :
    Fintype.card (NumberField.InfinitePlace K) = 2
```

Mathematical meaning. The displayed Lean declaration has type `Fintype.card (InfinitePlace K) = 2`.

Implementation. `card_eq_nrRealPlaces_add_nrComplexPlaces` + counts. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_isReal (v : NumberField.InfinitePlace K) :
    NumberField.InfinitePlace.IsReal v
```

Mathematical meaning. The displayed Lean declaration has type `(v : InfinitePlace K) → IsReal v`.

Implementation. The proof applies the packaged result `IsTotallyReal.isReal`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem infinitePlace_apply_sqrtTwoRoot (v : NumberField.InfinitePlace K)
    :
    v sqrtTwoRoot = Real.sqrt 2
```

Mathematical meaning. The displayed Lean declaration has type `(v : InfinitePlace K) → v sqrtTwoRoot = Real.sqrt 2`.

Implementation. `norm_embedding_eq`, embedding-square is $2=\sqrt{2^2}$, `sq_eq_sq_iff_eq_or_eq_neg`, both branches give $\sqrt{2}$.

2.4 `RingOfIntegers.lean`

```
theorem sqrtTwoRoot_isIntegral : IsIntegral ℤ sqrtTwoRoot
```

Mathematical meaning. The displayed Lean declaration has type `IsIntegral ℤ sqrtTwoRoot`.

Implementation. witness `X^2-C 2`, `monic_X_pow_sub_C`, `sqrtTwoRoot_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtTwoInteger : 𝒪 K :=
  ⟨sqrtTwoRoot, sqrtTwoRoot_isIntegral⟩
```

Mathematical meaning. $\sqrt{2}$ in $\mathcal{O}$ K.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
local notation "𝒫₂" => Submodule.span ℤ ({(2 : ℤ)} : Set ℤ)

abbrev sqrtTwoIntegerPolynomial : ℤ[X] := X ^ 2 - C (2 : ℤ)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtTwo_minpoly_rat : minpoly ℚ sqrtTwoRoot = sqrtTwoPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `minpoly ℚ sqrtTwoRoot = sqrtTwoPolynomial`.

Implementation. transport along `sqrtTwoAdjoinRootEquivQuadratic` via `minpoly.algEquiv_eq` + `AdjoinRoot.minpoly_root`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwo_minpoly_int : minpoly ℤ sqrtTwoRoot =
    sqrtTwoIntegerPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `minpoly ℤ sqrtTwoRoot = sqrtTwoIntegerPolynomial`.

Implementation. `map_injective` + `minpoly.isIntegrallyClosed_eq_field_fractions'` + `sqrtTwo_minpoly_rat`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwo_minpoly_int_isEisenstein :
    (minpoly ℤ sqrtTwoRoot).IsEisensteinAt 𝒫₂
```

Mathematical meaning. X^2-2 is Eisenstein at 2.

Implementation. `monic.isEisensteinAt_of_mem_of_notMem`, membership via `Ideal.mem_span_singleton`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwoPowerBasis_discr :
    Algebra.discr ℚ sqrtTwoPowerBasis.basis = (8 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℚ sqrtTwoPowerBasis.basis = 8`.

Implementation. `discr_powerBasis_eq_norm`; compute `norm(1+1)=4`, `norm(√2)=−2` via `PowerBasis.norm_gen_eq_coeff_zero_minpoly`, `simp+norm_num`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwo_integral_mem_adjoin (z : K) (hz : IsIntegral ℤ z) :
    z ∈ Algebra.adjoin ℤ ({sqrtTwoRoot} : Set K)
```

Mathematical meaning. every algebraic integer lies in $\mathbb{Z}[\sqrt{2}]$.

Implementation. `Algebra.discr_mul_isIntegral_mem_adjoin` gives `8•z` $\in \mathbb{Z}[\sqrt{2}]$, then `mem_adjoin_of_smul_prime_pow_smul_of_minpoly_isEisensteinAt` (Eisenstein theorem at `p=2,n=3`) removes the 2^3.

```
theorem isIntegralClosure_adjoin_sqrtTwoRoot :
    IsIntegralClosure
      (Algebra.adjoin ℤ ({sqrtTwoRoot} : Set K)) ℤ K
```

Mathematical meaning. The displayed Lean declaration has type `IsIntegralClosure (Algebra.adjoin ℤ {sqrtTwoRoot}) ℤ K`.

Implementation. injectivity (`Subtype.ext`) + `sqrtTwo_integral_mem_adjoin`; reverse via `Algebra.IsIntegral.adjoin`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwoInteger_adjoin_eq_top :
    Algebra.adjoin ℤ ({sqrtTwoInteger} : Set (𝒪 K)) = ⊤
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.adjoin ℤ {sqrtTwoInteger} = ⊤`.

Implementation. `le_antisymm`; pull each `x∈𝒪 K` back via `sqrtTwo_integral_mem_adjoin` + `adjoin_singleton_eq_range_aeval`.

```
theorem sqrtTwoInteger_minpoly :
    minpoly ℤ sqrtTwoInteger = sqrtTwoIntegerPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `minpoly ℤ sqrtTwoInteger = sqrtTwoIntegerPolynomial`.

Implementation. `minpoly.isIntegrallyClosed_eq_field_fractions` (fully explicit `@`-application), `map_injective`, reduce to `sqrtTwo_minpoly_rat`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtTwoExplicitIntegralPowerBasis : PowerBasis ℤ (𝒪 K) :=
  PowerBasis.ofAdjoinEqTop'
    (Algebra.IsIntegral.isIntegral sqrtTwoInteger)
    sqrtTwoInteger_adjoin_eq_top
```

Mathematical meaning. The displayed Lean declaration has type `PowerBasis ℤ (𝒪 K) := PowerBasis.ofAdjoinEqTop' and related terms`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtTwoExplicitIntegralPowerBasis_dim :
    sqrtTwoExplicitIntegralPowerBasis.dim = 2
```

Mathematical meaning. The displayed Lean declaration has type `dim = 2`.

Implementation. `ofAdjoinEqTop'_dim` + `sqrtTwoInteger_minpoly` + `natDegree_X_pow_sub_C`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtTwoExplicitIntegralBasis : Basis (Fin 2) ℤ (𝒪 K) :=
  sqrtTwoExplicitIntegralPowerBasis.basis.reindex
    (finCongr sqrtTwoExplicitIntegralPowerBasis_dim)
```

Mathematical meaning. The displayed Lean declaration has type `Basis (Fin 2) ℤ (𝒪 K)` = $[1, \sqrt{2}]$ (reindexed power basis).

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtTwoExplicitIntegralBasis_zero :
    sqrtTwoExplicitIntegralBasis 0 = 1

theorem sqrtTwoExplicitIntegralBasis_one :
    sqrtTwoExplicitIntegralBasis 1 = sqrtTwoInteger
```

Mathematical meaning. The displayed Lean declaration has type basis 0 = 1; basis 1 = sqrtTwoInteger.

Implementation. `Basis.reindex_apply` + `basis_eq_pow`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwoInteger_sq : sqrtTwoInteger ^ 2 = (2 : 𝒪 K)
```

Mathematical meaning. The displayed Lean declaration has type `sqrtTwoInteger ^ 2 = (2 : 𝒪 K)`.

Implementation. `Subtype.ext` + `sqrtTwoRoot_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwoExplicitIntegralBasis_discr :
    Algebra.discr ℤ sqrtTwoExplicitIntegralBasis = (8 : ℤ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℤ sqrtTwoExplicitIntegralBasis = 8`.

Implementation. `Algebra.discr_localizationLocalization` + `Module.Basis.localizationLocalization` to reduce to `sqrtTwoPowerBasis_discr` via `discr_reindex`.

```
theorem sqrtTwo_numberField_discr :
    NumberField.discr K = 8
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.discr K = 8`.

Implementation. Transfer the explicit integral-basis discriminant by `discr_eq_discr`, then simplify with `sqrtTwoExplicitIntegralBasis_discr`.

```
attribute [local instance] DivisionRing.toRatAlgebra in
theorem integralBasis_discr :
    Algebra.discr ℚ (NumberField.integralBasis K) = (8 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℚ (integralBasis K) = 8`.

Implementation. `NumberField.coe_discr` + `sqrtTwo_numberField_discr`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
example
    (P : Matrix (Fin sqrtTwoPowerBasis.dim) (Fin sqrtTwoPowerBasis.dim)
    ℚ) :
    Algebra.discr ℚ (Matrix.mulVec (P.map (algebraMap ℚ K))
    sqrtTwoPowerBasis.basis) =
      P.det ^ 2 * Algebra.discr ℚ sqrtTwoPowerBasis.basis
```

```
example
    (P : Matrix (Fin sqrtTwoPowerBasis.dim) (Fin sqrtTwoPowerBasis.dim) ℚ)
    (hdet : P.det = 1 ∨ P.det = -1) :
    Algebra.discr ℚ (Matrix.mulVec (P.map (algebraMap ℚ K))
    sqrtTwoPowerBasis.basis) =
      (8 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type discr-of-`mulVec` = $\det^2 \cdot$`discr`; and =8 when `det`=$\pm$1 (change-of-basis invariance of disc).

Implementation. `Algebra.discr_of_matrix_mulVec` + `sqrtTwoPowerBasis_discr`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwoInteger_dvd_two : sqrtTwoInteger ∣ (2 : 𝒪 K)
```

Mathematical meaning. The displayed Lean declaration has type `sqrtTwoInteger ∣ (2 : 𝒪 K)`.

Implementation. witness `sqrtTwoInteger`, `sqrtTwoInteger_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem norm_eq_normForm (x : K) :
    Algebra.norm ℚ x = x.re ^ 2 - 2 * x.im ^ 2
```

Mathematical meaning. norm form = principal form of disc 8.

Implementation. `Algebra.norm_apply`, `QuadraticAlgebra.det_toLinearMap_eq_norm`, `QuadraticAlgebra.norm_def`, `ring`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem intNorm_eq_normForm (x y : ℤ) :
    Algebra.norm ℤ (x • (1 : 𝒪 K) + y • sqrtTwoInteger) = x ^ 2 - 2 * y ^
    2
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.norm ℤ (x•1 + y•sqrtTwoInteger) =` $x^2 - 2y^2$ (`x,y:`$\mathbb{Z}$).

Implementation. `Algebra.coe_norm_int`, cast to K, `norm_eq_normForm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isUnit_iff_normForm (x y : ℤ) :
    IsUnit (x • (1 : 𝒪 K) + y • sqrtTwoInteger) ↔
      x ^ 2 - 2 * y ^ 2 = 1 ∨ x ^ 2 - 2 * y ^ 2 = -1
```

Mathematical meaning. the Pell-equation unit criterion.

Implementation. `isUnit_iff_norm`, `intNorm_eq_normForm`, `abs_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

2.5 Discriminant.lean

2.6 PrimeSplitting.lean

```
theorem span_two_eq : Ideal.span {(2 : 𝒪 K)} = Ideal.span
    {sqrtTwoInteger} ^ 2
```

Mathematical meaning. 2 totally ramifies, $(2)=(\sqrt{2})^2$.

Implementation. `span_singleton_pow` + `sqrtTwoInteger_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_span_sqrtTwoInteger : Ideal.absNorm (Ideal.span
    {sqrtTwoInteger}) = 2
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (span {sqrtTwoInteger}) = 2`.

Implementation. `intNorm_eq_normForm 0 1` (= −2, abs 2), `absNorm_span_singleton`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isPrime_span_sqrtTwoInteger : (Ideal.span
    {sqrtTwoInteger}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `(span {sqrtTwoInteger}).IsPrime`.

Implementation. `isPrime_of_irreducible_absNorm` + `Nat.prime_two`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_span_sqrtTwoInteger :
    (Ideal.span {sqrtTwoInteger}).LiesOver (Ideal.span {(2 : ℤ)})

theorem inertiaDeg_span_sqrtTwoInteger :
    (Ideal.span {(2 : ℤ)}).inertiaDeg (Ideal.span {sqrtTwoInteger}) = 1

theorem ramificationIdx_span_sqrtTwoInteger :
    (Ideal.span {(2 : ℤ)}).ramificationIdx (Ideal.span {sqrtTwoInteger})
    = 2
```

Mathematical meaning. The displayed Lean declaration has type lies over `(2)`; `f=1`; `e=2`.

Implementation. `liesOver_span_iff`; `absNorm_eq_pow_inertiaDeg'`+`Nat.Prime.pow_eq_iff`; `ramificationIdx_spec` with norm upper bound $8 \nmid 4$ for the cube.

```
theorem span_seven_eq :
    Ideal.span {(7 : 𝒪 K)} =
      Ideal.span {3 + sqrtTwoInteger} * Ideal.span {3 - sqrtTwoInteger}
```

Mathematical meaning. 7 splits.

Implementation. `span_singleton_mul_span_singleton` + $(3+\sqrt{2})(3-\sqrt{2})=9-\sqrt{2}^2=7$. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_span_three_add_sqrtTwo :
    Ideal.absNorm (Ideal.span {3 + sqrtTwoInteger}) = 7

theorem isPrime_span_three_add_sqrtTwo : (Ideal.span {3 +
    sqrtTwoInteger}).IsPrime

theorem absNorm_span_three_sub_sqrtTwo :
    Ideal.absNorm (Ideal.span {3 - sqrtTwoInteger}) = 7

theorem isPrime_span_three_sub_sqrtTwo : (Ideal.span {3 -
    sqrtTwoInteger}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type each norm 7; each prime.

Implementation. `intNorm_eq_normForm 3 (±1)` (=7); `isPrime_of_irreducible_absNorm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_span_three_add_sqrtTwo :
    (Ideal.span {3 + sqrtTwoInteger}).LiesOver (Ideal.span {(7 : ℤ)})

theorem liesOver_span_three_sub_sqrtTwo :
    (Ideal.span {3 - sqrtTwoInteger}).LiesOver (Ideal.span {(7 : ℤ)})
```

Mathematical meaning. The displayed Lean declaration has type each lies over `(7)`.

Implementation. `liesOver_span_iff` + product =7. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem inertiaDeg_span_three_add_sqrtTwo :
    (Ideal.span {(7 : ℤ)}).inertiaDeg (Ideal.span {3 + sqrtTwoInteger}) =
    1

theorem inertiaDeg_span_three_sub_sqrtTwo :
    (Ideal.span {(7 : ℤ)}).inertiaDeg (Ideal.span {3 - sqrtTwoInteger}) =
    1
```

Mathematical meaning. The displayed Lean declaration has type each `f=1` over 7.

Implementation. `absNorm_eq_pow_inertiaDeg'`+`Nat.Prime.pow_eq_iff`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem absNorm_ne_three (P : Ideal (𝒪 K)) : Ideal.absNorm P ≠ 3
```

Mathematical meaning. The displayed Lean declaration has type no ideal of $\mathcal{O}$ K has norm 3.

Implementation. residue field $\simeq$ ZMod 3 (ZMod.ringEquivOfPrime), lift $\sqrt{2}$ to root of t^2=2 in ZMod 3, revert t; decide (no solution).

```
theorem isPrime_span_three : (Ideal.span {(3 : 𝒪 K)}).IsPrime
```

Mathematical meaning. 2 is a non-residue mod 3.

Implementation. norm 9, take maximal M$\geq$(3), rule out norms 1,3 (absNorm_ne_three), eq_of_le_of_absNorm_eq.

```
theorem absNorm_span_three : Ideal.absNorm (Ideal.span {(3 : 𝒪 K)}) = 9

theorem liesOver_span_three : (Ideal.span {(3 : 𝒪 K)}).LiesOver
    (Ideal.span {(3 : ℤ)})

theorem inertiaDeg_span_three :
    (Ideal.span {(3 : ℤ)}).inertiaDeg (Ideal.span {(3 : 𝒪 K)}) = 2
```

Mathematical meaning. The displayed Lean declaration has type norm 9; lies over (3); f=2.

Implementation. intNorm_eq_normForm 3 0; liesOver_span_iff; absNorm_eq_pow_inertiaDeg'+Nat.pow_right_injective. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem absNorm_span_natCast (p : ℕ) :
    Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K)}) = p ^ 2
```

Mathematical meaning. The displayed Lean declaration has type absNorm (span {(p:$\mathbb{N}$):$\mathcal{O}$ K}) = p^2.

Implementation. Ideal.absNorm_span_natCast, RingOfIntegers.rank, degree_eq_two. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem li_one_sqrtTwo : LinearIndependent ℤ ![(1 : 𝒪 K),
    sqrtTwoInteger]
```

Mathematical meaning. The displayed Lean declaration has type LinearIndependent $\mathbb{Z}$![1, sqrtTwoInteger].

Implementation. transport from sqrtTwoExplicitIntegralBasis.linearIndependent. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem exists_o2_coords (z : 𝒪 K) :
    ∃ A B : ℤ, z = A • (1 : 𝒪 K) + B • sqrtTwoInteger
```

Mathematical meaning. The displayed Lean declaration has type ∀ z:𝒪 K, ∃ A B:ℤ, z = A•1 + B•sqrtTwoInteger.

Implementation. repr/sum_repr of the explicit basis. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem unit_sqrtTwo_coeff_not_mem (p : ℕ) (hp : 2 ≤ p) (A B : ℤ)
    (hB : B = 1 ∨ B = -1) :
    ((A : ℤ) : 𝒪 K) + ((B : ℤ) : 𝒪 K) * sqrtTwoInteger ∉
      Ideal.span {((p : ℕ) : 𝒪 K)}
```

Mathematical meaning. The displayed Lean declaration has type A + B$\sqrt{2}$ with B=$\pm$1 not in span {p} for p$\geq$2.

Implementation. decompose membership, li_one_sqrtTwo, Int.le_of_dvd. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramified_iff_eq_two (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ^ 2) ↔ p =
    2
```

Mathematical meaning. p ramifies iff divides disc 8.

Implementation. forward via not_isUnramifiedAt_ and related terms + different-ideal divisibility (dvd_differentIdeal_iff, absNorm_differentIdeal, sqrtTwo_numberField_discr); backward via span_two_eq.

```
theorem ramificationIdx_span_three_add_sqrtTwo :
    (Ideal.span {(7 : ℤ)}).ramificationIdx (Ideal.span {3 +
    sqrtTwoInteger}) = 1

theorem ramificationIdx_span_three_sub_sqrtTwo :
    (Ideal.span {(7 : ℤ)}).ramificationIdx (Ideal.span {3 -
    sqrtTwoInteger}) = 1

theorem ramificationIdx_span_three :
    (Ideal.span {(3 : ℤ)}).ramificationIdx (Ideal.span {(3 : 𝒪 K)}) = 1
```

Mathematical meaning. The displayed Lean declaration has type each unramified (e=1).

Implementation. ramificationIdx_spec; upper bound by reducing to ramified_iff_eq_two (p$\neq$2). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem absNorm_span_prime_sqrtTwo_sub_of_mul (p : ℕ) (hp :
    p.Prime) (c k : ℤ)
    (hprod : (sqrtTwoInteger - (c : 𝒪 K)) * (sqrtTwoInteger + (c : 𝒪 K)) =
      ((p : ℕ) : 𝒪 K) * (k : 𝒪 K)) :
    Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K), sqrtTwoInteger - (c : 𝒪
    K)}) = p
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (span {p, √2−c}) = p` given `(√2−c)(√2+c)=p·k`.

Implementation. rule out norm 1 (`mem_span_pair+unit_sqrtTwo_coeff_not_mem`) and norm p^2 (`eq_of_le_of_absNorm_eq+unit_sqrtTwo_coeff_not_mem`), then Common `eq_prime_of_dvd_prime_sq_` and related terms.

```
private theorem pm_one_mod_eight_of_prime_norm (p : ℕ) (hp : p.Prime)
    (hp2 : p ≠ 2) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
    Ideal.absNorm P = p) →
      p % 8 = 1 ∨ p % 8 = 7
```

Mathematical meaning. The displayed Lean declaration has type `(∃ P prime, span{p}≤P, absNorm P = p) → p%8∈{1,7}` `(p≠2)`.

Implementation. residue field $\simeq$ `ZMod p`, lift $\sqrt{2}$ to `t` with $t^2=2$, `IsSquare (2:ZMod p)`, then `ZMod.exists_sq_eq_two_iff` (second supplement to QR).

```
private theorem exists_prime_norm_of_pm_one_mod_eight (p : ℕ) (hp :
    p.Prime) (hp2 : p ≠
    2) :
    p % 8 = 1 ∨ p % 8 = 7 →
      ∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
    Ideal.absNorm P = p
```

Mathematical meaning. The displayed Lean declaration has type `p%8∈{1,7} → ∃ P prime of norm p above p` `(p≠2)`.

Implementation. `ZMod.exists_sq_eq_two_iff` gives `c` with $c^2\equiv 2$, build `span {p, √2−c}`, factorization `(√2−c)(√2+c)=p·k`, `absNorm_span_prime_sqrtTwo_sub_of_mul` + `isPrime_of_irreducible_absNorm`.

```
theorem exists_prime_norm_iff_pm_one_mod_eight (p : ℕ) (hp : p.Prime)
    (hp2 : p ≠ 2) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
    Ideal.absNorm P = p) ↔
      p % 8 = 1 ∨ p % 8 = 7
```

Mathematical meaning. The displayed Lean declaration has type `(∃ P prime of norm p above p) ↔ p%8∈{1,7} (p≠2)`.

Implementation. combine the two private lemmas.

```
private theorem pm_one_mod_eight_of_splits (p : ℕ) (hp : p.Prime) (hp2 :
    p ≠ 2) :
    (∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) → p % 8 = 1 ∨ p % 8 = 7
```

Mathematical meaning. The displayed Lean declaration has type split ⇒ `p%8∈{1,7} (p≠2)`.

Implementation. norms multiply to p^2, one factor norm `p`, reduce to `exists_prime_norm_iff_` and `related terms`.

```
private theorem splits_of_pm_one_mod_eight (p : ℕ) (hp : p.Prime) (hp2 :
    p ≠ 2) :
    p % 8 = 1 ∨ p % 8 = 7 →
      ∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
        Ideal.span {(p : 𝒪 K)} = P * Q
```

Mathematical meaning. The displayed Lean declaration has type `p%8∈{1,7} ⇒ ∃ distinct primes P,Q, span{p}=P*Q (p≠2)`.

Implementation. get norm-`p` prime `P`, `dvd_iff_le` gives complement `Q`, norms force `absNorm Q = p`, `P≠Q` via `ramified_iff_eq_two`.

```
theorem splits_iff_pm_one_mod_eight (p : ℕ) (hp : p.Prime) (hp2 : p ≠ 2) :
    (∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) ↔ p % 8 = 1 ∨ p % 8 = 7
```

Mathematical meaning. the quadratic splitting law (`(2/p)=1 ⇔ p≡±1 mod 8`, second supplement).

Implementation. combine the two private lemmas.

```
abbrev sqrtTwoPolynomialMod (p : ℕ) : (ZMod p)[X] :=
  X ^ 2 - C (2 : ZMod p)
```

Mathematical meaning. The displayed Lean declaration has type `(ZMod p)[X] := X² - C 2`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtTwoInteger_exponent_eq_one :
    RingOfIntegers.exponent sqrtTwoInteger = 1
```

Mathematical meaning. $\sqrt{2}$ generates the full $\mathcal{O}$ K, so KD has no conductor primes.

Implementation. exponent_eq_one_iff + sqrtTwoInteger_adjoin_eq_top. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwoInteger_exponent_not_dvd (p : ℕ) [Fact p.Prime] :
    ¬ p ∣ RingOfIntegers.exponent sqrtTwoInteger
```

Mathematical meaning. The displayed Lean declaration has type ¬ p ∣ exponent sqrtTwoInteger.

Implementation. sqrtTwoInteger_exponent_eq_one + not_dvd_one. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwo_kummerDedekind_inertiaDeg
    {p : ℕ} [Fact p.Prime] {Q : (ZMod p)[X]}
    (hQ : Q ∈ RingOfIntegers.monicFactorsMod sqrtTwoInteger p) :
    Ideal.inertiaDeg (Ideal.span {(p : ℤ)})
        ((NumberField.Ideal.primesOverSpanEquivMonicFactorsMod
          (θ := sqrtTwoInteger)
          (sqrtTwoInteger_exponent_not_dvd p)).symm
            ⟨Q, hQ⟩ :
          Ideal (𝒪 K)) = Q.natDegree

theorem sqrtTwo_kummerDedekind_ramificationIdx
    {p : ℕ} [Fact p.Prime] {Q : (ZMod p)[X]}
    (hQ : Q ∈ RingOfIntegers.monicFactorsMod sqrtTwoInteger p) :
    Ideal.ramificationIdx (Ideal.span {(p : ℤ)})
        ((NumberField.Ideal.primesOverSpanEquivMonicFactorsMod
          (θ := sqrtTwoInteger)
          (sqrtTwoInteger_exponent_not_dvd p)).symm
            ⟨Q, hQ⟩ :
          Ideal (𝒪 K)) = multiplicity Q (sqrtTwoPolynomialMod p)
```

Mathematical meaning. The displayed Lean declaration has type KD identifies f with natDegree (resp. e with multiplicity) of a mod-p factor.

Implementation. Apply the Kummer–Dedekind equivalence for a generator with exponent prime to p; the ramification-index case then rewrites the generator's minpoly to sqrtTwoPolynomialMod p.

```
theorem sqrtTwoPolynomialMod_two :
    sqrtTwoPolynomialMod 2 = X ^ 2
```

Mathematical meaning. KD double root at 2 (ramification).

Implementation. (2:ZMod 2)=0, simp. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwoPolynomialMod_seven_factor :
    sqrtTwoPolynomialMod 7 =
      (X - C (3 : ZMod 7)) * (X + C (3 : ZMod 7))
```

Mathematical meaning. KD two roots ± 3 (splitting).

Implementation. `ring_nf` + $2=3^2$ in `ZMod 7`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwoPolynomialMod_three_irreducible :
    Irreducible (sqrtTwoPolynomialMod 3)
```

Mathematical meaning. KD irreducible at 3 (inert).

Implementation. `irreducible_of_degree_le_three_of_not_isRoot` + `fin_cases x` over `ZMod 3`, `decide`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem exists_root_mod_iff_exists_prime_norm
    (p : ℕ) (hp : p.Prime) (hp2 : p ≠ 2) :
    (∃ c : ZMod p, c ^ 2 = (2 : ZMod p)) ↔
      ∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
        Ideal.absNorm P = p
```

Mathematical meaning. KD root condition = the norm-p prime condition.

Implementation. `calc` through $p\%8\in\{1,7\}$ (`ZMod.exists_sq_eq_two_iff`) + `exists_prime_norm_iff_pm_one_mod_eight`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem differentIdeal_eq_span_two_mul_sqrtTwo :
    differentIdeal ℤ (𝒪 K) = Ideal.span {(2 : 𝒪 K) * sqrtTwoInteger}
```

Mathematical meaning. The displayed Lean declaration has type `differentIdeal ℤ (𝒪 K) = span` $\{2\cdot\sqrt{2}\}$.

Implementation. `conductor_eq_top_of_adjoin_eq_top`, `conductor_mul_differentIdeal`, `aeval` of derivative $2X$ of X^2-2. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem differentIdeal_eq_span_sqrtTwoInteger_pow_three :
    differentIdeal ℤ (𝒪 K) = Ideal.span {sqrtTwoInteger} ^ 3
```

Mathematical meaning. different is $(\sqrt{2})^3$ (matches disc 2^3).

Implementation. rewrite $2\cdot\sqrt{2} = \sqrt{2}^3$ via `sqrtTwoInteger_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_differentIdeal_eq_eight :
    Ideal.absNorm (differentIdeal ℤ (𝒪 K)) = 8
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (differentIdeal) = 8`.

Implementation. `differentIdeal_eq_span_sqrtTwoInteger_pow_three`, `map_pow`, `absNorm_span_sqrtTwoInteger`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem prime_dvd_differentIdeal_iff_eq_span_sqrtTwo
    (P : Ideal (𝒪 K)) (hP : P.IsPrime) (_hP0 : P ≠ ⊥) :
    P ∣ differentIdeal ℤ (𝒪 K) ↔ P = Ideal.span {sqrtTwoInteger}
```

Mathematical meaning. The displayed Lean declaration has type nonzero prime `P ∣ differentIdeal ↔ P = span {sqrtTwoInteger}`.

Implementation. `IsPrime.le_of_pow_le`, maximality (`isPrime_span_sqrtTwoInteger.isMaximal`), `differentIdeal_eq_` and related `terms_pow_three`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

2.7 `ClassGroup.lean`

```
abbrev sqrtTwoRootFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  FractionalIdeal.spanSingleton (𝒪 K)⁰ sqrtTwoRoot

abbrev ramifiedPrimeFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  (Ideal.span {sqrtTwoInteger} : FractionalIdeal (𝒪 K)⁰ K)
```

Mathematical meaning. The displayed Lean declaration has type `spanSingleton sqrtTwoRoot`; coe of `span {sqrtTwoInteger}`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtTwoRoot_mem_fractionalIdeal :
    sqrtTwoRoot ∈ sqrtTwoRootFractionalIdeal
```

Mathematical meaning. The displayed Lean declaration has type `sqrtTwoRoot ∈ sqrtTwoRootFractionalIdeal`.

Implementation. `mem_spanSingleton` (witness 1). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwoRootFractionalIdeal_absNorm :
    FractionalIdeal.absNorm sqrtTwoRootFractionalIdeal = |Algebra.norm ℚ
    sqrtTwoRoot|
```

Mathematical meaning. The displayed Lean declaration has type `absNorm = |Algebra.norm ℚ sqrtTwoRoot|`.

Implementation. The proof applies the packaged result `FractionalIdeal.absNorm_span_singleton`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem ramifiedPrimeFractionalIdeal_ne_zero :
    ramifiedPrimeFractionalIdeal ≠ 0
```

Mathematical meaning. The displayed Lean declaration has type `≠ 0`.

Implementation. `coeIdeal_ne_zero` + `absNorm_span_sqrtTwoInteger` (norm $2 \neq 0$). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝒪 K)
```

Mathematical meaning. ring-theoretic content of class number 1.

Implementation. `RingOfIntegers.isPrincipalIdealRing_of_abs_discr_lt`; bound `|disc|=8` (Minkowski bound $\frac{1}{2}\sqrt{8} \approx$ `1.41 < 2`), case-split `nrComplexPlaces ≤ 1`, `nlinarith [Real.pi_gt_three]`.

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝒪 K)
```

Mathematical meaning. The displayed Lean declaration has type `UniqueFactorizationMonoid (𝒪 K)`.

Implementation. PID ⇒ UFM (`infer_instance`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.classNumber K = 1`.

Implementation. The proof applies the packaged result `classNumber_eq_one_iff.mpr isPrincipalIdealRing`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem classGroupStructure : Nonempty (ClassGroup (𝒪 K) ≃*
    Multiplicative (ZMod 1))
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty (ClassGroup (𝒪 K) ≃* Multiplicative (ZMod 1))`.

Implementation. card 1 ⇒ `Subsingleton` ⇒ `MulEquiv.ofBijective` of the trivial hom.

```
abbrev unitIdealRepresentative : (Ideal (𝒪 K))⁰ := ⟨(⊤ : Ideal (𝒪 K)), by
    simp⟩
```

Mathematical meaning. The displayed Lean declaration has type `(Ideal (𝒪 K))⁰ := ⟨⊤, and related terms⟩`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem classGroupRepresentative (C : ClassGroup (𝒪 K)) :
    C = ClassGroup.mk0 unitIdealRepresentative
```

Mathematical meaning. The displayed Lean declaration has type `∀ C, C = ClassGroup.mk0 unitIdealRepresentative`.

Implementation. The proof applies the packaged result `Subsingleton.elim`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

2.8 `Units.lean`

```
abbrev ZSqrtTwo : Type := Zsqrtd (2 : ℤ)

abbrev zsqrtTwoToRingOfIntegers : ZSqrtTwo →+* 𝒪 K :=
  Zsqrtd.lift ⟨sqrtTwoInteger, by simpa [sq] using sqrtTwoInteger_sq⟩
```

Mathematical meaning. The displayed Lean declaration has type `Zsqrtd (2:ℤ)` (mathlib's concrete $\mathbb{Z}[\sqrt{2}]$); `Zsqrtd.lift ⟨sqrtTwoInteger, and related terms⟩ : ZSqrtTwo →+* 𝒪 K`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem zsqrtTwoToRingOfIntegers_apply (a b : ℤ) :
    zsqrtTwoToRingOfIntegers (⟨a, b⟩ : ZSqrtTwo) =
      (a : 𝒪 K) + b * sqrtTwoInteger
```

Mathematical meaning. The displayed Lean declaration has type `zsqrtTwoToRingOfIntegers ⟨a,b⟩ = a + b·√2`.

Implementation. The proof applies the packaged result `simp`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem two_not_int_square : ∀ n : ℤ, (2 : ℤ) ≠ n * n
```

Mathematical meaning. The displayed Lean declaration has type `∀ n:ℤ, 2 ≠ n*n`.

Implementation. `IsSquare 2` + `norm_num`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zsqrtTwoToRingOfIntegers_injective : Function.Injective
    zsqrtTwoToRingOfIntegers

theorem zsqrtTwoToRingOfIntegers_surjective : Function.Surjective
    zsqrtTwoToRingOfIntegers
```

Mathematical meaning. The displayed Lean declaration has type injective (`Zsqrtd.lift_injective` + `two_not_int_square`); surjective (decompose target on `sqrtTwoExplicitIntegralBasis`).

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
abbrev zsqrtTwoEquivRingOfIntegers : ZSqrtTwo ≃+* 𝒪 K :=
  RingEquiv.ofBijective zsqrtTwoToRingOfIntegers
    ⟨zsqrtTwoToRingOfIntegers_injective,
    zsqrtTwoToRingOfIntegers_surjective⟩
```

Mathematical meaning. The displayed Lean declaration has type `ZSqrtTwo ≃+* 𝒪 K := RingEquiv.ofBijective and related terms`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev sqrtTwoFundamentalUnit : (𝒪 K)ˣ where
  val := sqrtTwoInteger + 1
  inv := sqrtTwoInteger - 1
  val_inv := by
    apply Subtype.ext
    calc
      (sqrtTwoRoot + 1) * (sqrtTwoRoot - 1) = sqrtTwoRoot ^ 2 - 1 := by
    ring
      _ = (1 : K) := by rw [sqrtTwoRoot_sq]; ring
  inv_val := by
    apply Subtype.ext
    calc
      (sqrtTwoRoot - 1) * (sqrtTwoRoot + 1) = sqrtTwoRoot ^ 2 - 1 := by
    ring
      _ = (1 : K) := by rw [sqrtTwoRoot_sq]; ring
```

Mathematical meaning. the fundamental unit $1+\sqrt{2}$.

Implementation. `Subtype.ext` + $(\sqrt{2}+1)(\sqrt{2}-1)=\sqrt{2}^2-1=1$. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev zsqrtTwoFundamentalUnit : ZSqrtTwo× where
  val := ⟨1, 1⟩
  inv := ⟨-1, 1⟩
  val_inv := by
    ext <;> norm_num
  inv_val := by
    ext <;> norm_num
```

Mathematical meaning. $1+\sqrt{2}$ in the `Zsqrtd` model.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem zsqrtTwoFundamentalUnit_norm :
    Zsqrtd.norm (zsqrtTwoFundamentalUnit : ZSqrtTwo) = -1
```

Mathematical meaning. The displayed Lean declaration has type `Zsqrtd.norm ⟨1,1⟩ = -1`.

Implementation. The proof applies the packaged result `norm_num [Zsqrtd.norm]`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem zsqrtTwoEquivRingOfIntegers_fundamentalUnit :
    Units.mapEquiv zsqrtTwoEquivRingOfIntegers.toMulEquiv
    zsqrtTwoFundamentalUnit =
      sqrtTwoFundamentalUnit
```

Mathematical meaning. The displayed Lean declaration has type `Units.mapEquiv and related terms zsqrtTwoFundamentalUnit = sqrtTwoFundamentalUnit`.

Implementation. `Units.ext` + `zsqrtTwoToRingOfIntegers_apply`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zsqrtTwoEquivRingOfIntegers_unitsMap_neg (v : ZSqrtTwo×) :
    Units.mapEquiv zsqrtTwoEquivRingOfIntegers.toMulEquiv (-v) =
      -(Units.mapEquiv zsqrtTwoEquivRingOfIntegers.toMulEquiv v)
```

Mathematical meaning. The displayed Lean declaration has type `mapEquiv (-v) = -(mapEquiv v)`.

Implementation. `Units.ext` + `map_neg`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwoFundamentalUnit_sq :
    ((sqrtTwoFundamentalUnit ^ 2 : (𝒪 K)×) : 𝒪 K) =
      3 + 2 * sqrtTwoInteger
```

Mathematical meaning. The displayed Lean declaration has type `((1+√2)^2 : 𝒪 K) = 3 + 2√2`.

Implementation. `Subtype.ext`, expand, `sqrtTwoRoot_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev pellTwoFundamentalSolution : Pell.Solution₁ (2 : ℤ) :=
  Pell.Solution₁.mk 3 2 (by norm_num)
```

Mathematical meaning. $3+2\sqrt{2}$, the fundamental norm-one Pell solution of $x^2-2y^2=1$.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem pellTwoFundamentalSolution_isFundamental :
    Pell.IsFundamental pellTwoFundamentalSolution
```

Mathematical meaning. The displayed Lean declaration has type `Pell.IsFundamental pellTwoFundamentalSolution`.

Implementation. `Pell.IsFundamental` triple – positivity + minimality of `x`-coord (rule out `x=2` via $2y^2=3$, `omega`).

```
theorem pellTwoFundamentalSolution_generates_normOnePell
    (a : Pell.Solution₁ (2 : ℤ)) :
    ∃ n : ℤ, a = pellTwoFundamentalSolution ^ n ∨
      a = -pellTwoFundamentalSolution ^ n
```

Mathematical meaning. The displayed Lean declaration has type `∀ a:Solution₁ 2, ∃ n, a = pell^n ∨ a = -pell^n`.

Implementation. The proof applies the packaged result `IsFundamental.eq_zpow_or_neg_zpow`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem zsqrtTwoFundamentalUnit_sq_toPell :
    (Unitary.toUnits pellTwoFundamentalSolution : ZSqrtTwo^×) =
      zsqrtTwoFundamentalUnit ^ 2
```

Mathematical meaning. The displayed Lean declaration has type `Unitary.toUnits pellTwoFundamentalSolution = zsqrtTwoFundamentalUnit`^{2}.

Implementation. `ext` + `norm_num`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zsqrtTwo_normOneUnit_generated_by_square
    (u : ZSqrtTwo^×) (hnorm : Zsqrtd.norm (u : ZSqrtTwo) = 1) :
    ∃ n : ℤ, u = zsqrtTwoFundamentalUnit ^ (2 * n) ∨
      u = -(zsqrtTwoFundamentalUnit ^ (2 * n))
```

Mathematical meaning. The displayed Lean declaration has type norm-one ZSqrtTwo$^\times$ units = $\pm$zsqrtTwoFundamentalUnit^(2n).

Implementation. pass to Solution$_1$ (Zsqrtd.norm_eq_one_iff_mem_unitary), pellTwoFundamentalSolution_generates_normOnePell, map_zpow/zpow_mul. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zsqrtTwo_units_generated_by_fundamental (u : ZSqrtTwo×) :
    ∃ n : ℤ, u = zsqrtTwoFundamentalUnit ^ n ∨
      u = -(zsqrtTwoFundamentalUnit ^ n)
```

Mathematical meaning. The displayed Lean declaration has type every ZSqrtTwo$^\times$ unit = $\pm$zsqrtTwoFundamentalUnit^n.

Implementation. norm is ± 1 (Zsqrtd.isUnit_iff_norm_isUnit, Int.isUnit_eq_one_or); norm -1 case multiplies by the fundamental unit to reach norm 1, then shifts the exponent.

```
theorem sqrtTwoFundamentalUnit_generates_unitGroup (u : (𝒪 K)×) :
    ∃ n : ℤ, u = sqrtTwoFundamentalUnit ^ n ∨
      u = -(sqrtTwoFundamentalUnit ^ n)
```

Mathematical meaning. every unit is a signed power of the fundamental unit – fully proved (the analogue is a SORRY in every other folder).

Implementation. transport zsqrtTwo_units_generated_by_fundamental along Units.mapEquiv zsqrtTwoEquivRingOfIntegers. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtTwoFundamentalUnit_isFundamental :
    ((sqrtTwoFundamentalUnit ^ 2 : (𝒪 K)×) : 𝒪 K) =
        3 + 2 * sqrtTwoInteger ∧
      ∀ a : Pell.Solution₁ (2 : ℤ), ∃ n : ℤ,
        a = pellTwoFundamentalSolution ^ n ∨
          a = -pellTwoFundamentalSolution ^ n
```

Mathematical meaning. The displayed Lean declaration has type $((1+\sqrt{2})^2=3+2\sqrt{2}) \wedge (\forall$ Pell solution = $\pm$pell^n).

Implementation. ⟨sqrtTwoFundamentalUnit_sq, pellTwoFundamentalSolution_generates_normOnePell⟩. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem sqrtTwo_torsion_eq_one_or_neg_one
    (x : Units.torsion K) : (x : (𝒪 K)×) = 1 ∨ (x : (𝒪 K)×) = -1
```

Mathematical meaning. The displayed Lean declaration has type `(x : torsion K) → (x:(𝒪 K)ˣ) = 1 ∨ = -1`.

Implementation. if `orderOf > 2`, a primitive root of order >2 would force `nrRealPlaces = 0` (`IsPrimitiveRoot.nrRealPlaces_eq_zero_of_two_lt`), contradicting `nrRealPlaces_eq_two`; else `interval_cases` on order ∈{0,1,2}.

```
theorem torsionOrder_eq_two : Units.torsionOrder K = 2
```

Mathematical meaning. The displayed Lean declaration has type `Units.torsionOrder K = 2`.

Implementation. `Finset.card_eq_two` with {1,−1} + `sqrtTwo_torsion_eq_one_or_neg_one`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem unitRank_eq_one : Units.rank K = 1
```

Mathematical meaning. The displayed Lean declaration has type `Units.rank K = 1`.

Implementation. `rank = card places − 1`, `nrRealPlaces_eq_two`/`nrComplexPlaces_eq_zero`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem exists_eq_torsion_mul_fundSystem_zpow
    (i₀ : Fin (NumberField.Units.rank K)) (u : (𝒪 K)ˣ) :
    ∃ ζ ∈ NumberField.Units.torsion K, ∃ n : ℤ,
      u = ζ * NumberField.Units.fundSystem K i₀ ^ n
```

Mathematical meaning. The displayed Lean declaration has type any unit `u = ζ * fundSystem K i₀ ^ n`, ζ torsion.

Implementation. `closure_fundSystem_sup_torsion_eq_top`, `Subgroup.mem_sup`, subsingleton of `Fin (rank)`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem fundSystem_zpow_mem_torsion
    (i₀ : Fin (NumberField.Units.rank K)) (n : ℤ)
    (hn : NumberField.Units.fundSystem K i₀ ^ n ∈
      NumberField.Units.torsion K) : n = 0
```

Mathematical meaning. The displayed Lean declaration has type `fundSystem K i₀ ^ n` torsion ⇒ `n=0`.

Implementation. quotient by torsion is free, `basisModTorsion.repr`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem eq_one_or_neg_one_of_mem_torsion {ζ : (𝒪 K)ˣ}
    (hζ : ζ ∈ NumberField.Units.torsion K) : ζ = 1 ∨ ζ = -1
```

Mathematical meaning. The displayed Lean declaration has type $\zeta \in$ `torsion` $\Rightarrow \zeta =$ `±1`.

Implementation. The proof applies the packaged result `sqrtTwo_torsion_eq_one_or_neg_one`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem mem_torsion_iff_eq_one_or_neg_one (u : (𝒪 K)ˣ) :
    u ∈ NumberField.Units.torsion K ↔ u = 1 ∨ u = -1
```

Mathematical meaning. The displayed Lean declaration has type `u ∈ torsion ↔ u = 1 ∨ u = -1`.

Implementation. `eq_one_or_neg_one_of_mem_torsion` + `neg_one_mem_torsion`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem regulator_eq : Units.regulator K = Real.log (1 + Real.sqrt 2)
```

Mathematical meaning. the regulator is `log(1+√2) ≈ 0.8814`.

Implementation. pick the real place `wR`; show the fundamental-unit family generates the units mod torsion (from the Pell-certified `sqrtTwoFundamentalUnit_generates_unitGroup`); then `regOfFamily_div_regulator` (index = 1) ⇒ `regulator = regOfFamily`, `regOfFamily_eq_det`, and `Matrix.det_eq_elem_of_subsingleton` on the rank-1 `1×1` matrix evaluate it to `|log (wR (1+√2))| = log(1+√2)`.

```
theorem regulator_eq : Units.regulator K = Real.log (1 + Real.sqrt 2)
```

Mathematical meaning. The displayed Lean declaration has type `(torsionPowerHom 2 (-1) _).coprod (rankOnePowerHom η)` is bijective.

Implementation. injectivity – `fundSystem_zpow_mem_torsion` kills the free part, then `ZMod 2`-parity kills the torsion part (using `(-1:𝒪 K)=1 ⇒ 2=0`); surjectivity – `exists_eq_torsion_mul_fundSystem_zpow` + `eq_one_or_neg_one_of_mem_torsion`.

```
private theorem unitGroupStructure_bijective
    (i₀ : Fin (NumberField.Units.rank K)) (η : (𝒪 K)ˣ)
    (hη : η = NumberField.Units.fundSystem K i₀)
    (hneg_tor : (-1 : (𝒪 K)ˣ) ∈ NumberField.Units.torsion K) :
    Function.Bijective
      ((torsionPowerHom 2 (-1 : (𝒪 K)ˣ) (by rw [neg_one_sq])).coprod
    (rankOnePowerHom
    η))
```

Mathematical meaning. the unit group is $\mu_2 \times \mathbb{Z}$ as an abstract abelian group – fully proved, no sorry.

Implementation. package ψ_1`.coprod` ψ_2 (`torsionPowerHom`/`rankOnePowerHom`), `unitGroupStructure_bijective`, `MulEquiv.ofBijective`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

2.9 `Euclidean.lean`

```
private theorem abs_norm_int_lt_of_remainder_eq {b r : 𝒪 K} {bK e : K} {u
    v : ℚ}
    (hbK : bK = algebraMap (𝒪 K) K b)
    (hrK : algebraMap (𝒪 K) K r = bK * e)
    (hb_pos : 0 < |Algebra.norm ℚ bK|)
    (hnorm_e :
      |Algebra.norm ℚ e| =
        |(u - ((round u : ℤ) : ℚ)) ^ 2 - 2 * (v - ((round v : ℤ) : ℚ)) ^
    2|)
    (hfrac :
      |(u - ((round u : ℤ) : ℚ)) ^ 2 - 2 * (v - ((round v : ℤ) : ℚ)) ^ 2|
    ≤ 1 / 2) :
    |((Algebra.norm ℤ r : ℤ) : ℚ)| < |((Algebra.norm ℤ b : ℤ) : ℚ)|
```

Mathematical meaning. The displayed Lean declaration has type packaging lemma: from `bK = algebraMap b`, `algebraMap r = bK·e`, `|norm bK|>0`, `|norm e| = |(u−round u)² − 2(v−round v)²|`, and that fraction $\leq$ `1/2`, conclude `|norm ℤ r| < |norm ℤ b|`.

Implementation. applies Common `abs_intNorm_lt_of_abs_norm_le` with the threshold `1/2 < 1`.

```
theorem normEuclidean (a b : 𝒪 K) (hb : b ≠ 0) :
    ∃ q r : 𝒪 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

Mathematical meaning. $\mathbb{Q}(\sqrt{2})$ is norm-Euclidean (the real quadratic field of smallest discriminant).

Implementation. take `q` = nearest-integer rounding of the coordinates of `a/b`; the remainder `e` has `|norm e| = |(u−round u)² − 2(v−round v)²| ≤ 1/2 < 1` (`nlinarith` with `abs_sub_round` bounds and `sq_nonneg`), then the packaging lemma + Common `natAbs_lt_of_abs_ratCast_lt`.

2.10 `LocalGlobal.lean`

```
def vTwo : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {sqrtTwoInteger}
  isPrime := isPrime_span_sqrtTwoInteger
  ne_bot := by
    intro h
    have hnorm := absNorm_span_sqrtTwoInteger
    rw [h] at hnorm
    norm_num at hnorm

def vThree : HeightOneSpectrum (𝒪 K) where
```

```
  asIdeal := Ideal.span {(3 : 𝒪 K)}
  isPrime := isPrime_span_three
  ne_bot := by
    intro h
    have hnorm := absNorm_span_three
    rw [h] at hnorm
    norm_num at hnorm

def vSevenPlus : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {3 + sqrtTwoInteger}
  isPrime := isPrime_span_three_add_sqrtTwo
  ne_bot := by
    intro h
    have hnorm := absNorm_span_three_add_sqrtTwo
    rw [h] at hnorm
    norm_num at hnorm

def vSevenMinus : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {3 - sqrtTwoInteger}
  isPrime := isPrime_span_three_sub_sqrtTwo
  ne_bot := by
    intro h
    have hnorm := absNorm_span_three_sub_sqrtTwo
    rw [h] at hnorm
    norm_num at hnorm
```

Mathematical meaning. The displayed Lean declaration has type `HeightOneSpectrum (𝒪 K)` for the primes $(\sqrt{2})$, (3), $(3+\sqrt{2})$, $(3-\sqrt{2})$.

Implementation. package `isPrime_ and related terms` + `ne_bot` (via nonzero `absNorm`).

```
abbrev finitePlaceTwo : NumberField.FinitePlace K :=
  NumberField.FinitePlace.mk vTwo

abbrev finitePlaceThree : NumberField.FinitePlace K :=
  NumberField.FinitePlace.mk vThree

abbrev finitePlaceSevenPlus : NumberField.FinitePlace K :=
  NumberField.FinitePlace.mk vSevenPlus

abbrev finitePlaceSevenMinus : NumberField.FinitePlace K :=
  NumberField.FinitePlace.mk vSevenMinus
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.FinitePlace K :=`

`FinitePlace.mk v and related terms`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example :
    (Ideal.span {(2 : ℤ)}).ramificationIdx vTwo.asIdeal = 2

example :
    (Ideal.span {(2 : ℤ)}).inertiaDeg vTwo.asIdeal = 1

example :
    (Ideal.span {(3 : ℤ)}).ramificationIdx vThree.asIdeal = 1

example :
    (Ideal.span {(3 : ℤ)}).inertiaDeg vThree.asIdeal = 2

example :
    (Ideal.span {(7 : ℤ)}).ramificationIdx vSevenPlus.asIdeal = 1

example :
    (Ideal.span {(7 : ℤ)}).ramificationIdx vSevenMinus.asIdeal = 1

example :
    (Ideal.span {(7 : ℤ)}).inertiaDeg vSevenPlus.asIdeal = 1

example :
    (Ideal.span {(7 : ℤ)}).inertiaDeg vSevenMinus.asIdeal = 1
```

Mathematical meaning. The displayed Lean declaration has type `ramificationIdx`/`inertiaDeg` of each `v and related terms.asIdeal` equal the computed `e`,`f` (`2,1` / `1,2` / `1,1`,`1,1`).

Implementation. cite the PrimeSplitting lemmas.

```
abbrev CompletionAtTwo : Type :=
  vTwo.adicCompletion K

abbrev CompletionAtThree : Type :=
  vThree.adicCompletion K

abbrev CompletionAtSevenPlus : Type :=
  vSevenPlus.adicCompletion K

abbrev CompletionAtSevenMinus : Type :=
  vSevenMinus.adicCompletion K
```

Mathematical meaning. The displayed Lean declaration has type `v and related terms. adicCompletion K`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example :
    (Ideal.span {(2 : ℤ)}).ramificationIdx vTwo.asIdeal = 2

example :
    (Ideal.span {(3 : ℤ)}).inertiaDeg vThree.asIdeal = 2

example :
    (Ideal.span {(7 : ℤ)}).inertiaDeg vSevenPlus.asIdeal = 1 ∧
      (Ideal.span {(7 : ℤ)}).inertiaDeg vSevenMinus.asIdeal = 1
```

Mathematical meaning. The displayed Lean declaration has type 2-adic remembers `e=2`; 3-adic inert `f=2`; both 7-completions `f=1`.

Implementation. The proof applies the packaged result `cite PrimeSplitting lemmas`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
abbrev InfiniteCompletion (v : NumberField.InfinitePlace K) : Type :=
  v.Completion
```

Mathematical meaning. The displayed Lean declaration has type `(v : InfinitePlace K) → Type := v.Completion`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example (v : NumberField.InfinitePlace K) :
    Nonempty (InfiniteCompletion v ≃+* ℝ)
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty (InfiniteCompletion v ≃+* ℝ)`.

Implementation. The proof applies the packaged result `Completion.ringEquivRealOfIsReal (infinitePlace_isReal v)`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem infiniteCompletion_sqrtTwoRoot_norm (v :
    NumberField.InfinitePlace K) :
    ‖(algebraMap K (InfiniteCompletion v) sqrtTwoRoot)‖ = Real.sqrt 2
```

Mathematical meaning. The displayed Lean declaration has type ‖algebraMap K (InfiniteCompletion v) sqrtTwoRoot‖ = Real.sqrt 2.

Implementation. Completion.norm_coe, WithAbs.equiv, infinitePlace_apply_sqrtTwoRoot. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_sum_inertiaDeg_eq_two (v :
    NumberField.InfinitePlace ℚ) :
    ∑ w ∈ v.placesOver K, v.inertiaDeg w = 2
```

Mathematical meaning. The displayed Lean declaration has type (v : InfinitePlace ℚ) → ∑ w ∈ v.placesOver K, v.inertiaDeg w = 2.

Implementation. sum_inertiaDeg_eq_finrank + degree_eq_two. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
example :
    (∏ w : NumberField.InfinitePlace K, w sqrtTwoRoot ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K, v sqrtTwoRoot = 1

example :
    (∏ w : NumberField.InfinitePlace K,
          w (algebraMap (𝒪 K) K (sqrtTwoFundamentalUnit : 𝒪 K)) ^ w.mult)
    *
        ∏ᶠ v : NumberField.FinitePlace K,
          v (algebraMap (𝒪 K) K (sqrtTwoFundamentalUnit : 𝒪 K)) = 1
```

Mathematical meaning. The displayed Lean declaration has type product formula (∏ infinite)(∏ finite) = 1 for $\sqrt{2}$ and for the fundamental unit $1+\sqrt{2}$.

Implementation. NumberField.prod_abs_eq_one (nonzero witness). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem finite_support_sqrtTwoRoot_only_vTwo
    (v : NumberField.FinitePlace K) (hv : v ≠ finitePlaceTwo) :
    v sqrtTwoRoot = 1
```

Mathematical meaning. the finite-place support of $\sqrt{2}$ is exactly vTwo.

Implementation. norm_embedding_eq, norm_eq_one_iff_notMem; if $\sqrt{2} \in$ maximal ideal then span{$\sqrt{2}$} ≤ that ideal, and maximality of ($\sqrt{2}$) forces the ideal = vTwo.

```
abbrev SqrtTwoInfiniteAdeleRing : Type :=
  NumberField.InfiniteAdeleRing K
```

```
abbrev infinitePrincipalAdele_sqrtTwoRoot : SqrtTwoInfiniteAdeleRing :=
  algebraMap K SqrtTwoInfiniteAdeleRing sqrtTwoRoot

abbrev infinitePrincipalAdele_fundamentalUnit : SqrtTwoInfiniteAdeleRing
    :=
  algebraMap K SqrtTwoInfiniteAdeleRing
    (algebraMap (𝒪 K) K (sqrtTwoFundamentalUnit : 𝒪 K))
```

Mathematical meaning. The displayed Lean declaration has type `InfiniteAdeleRing K`; principal infinite adeles of $\sqrt{2}$ and $1+\sqrt{2}$.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example :
    ‖infinitePrincipalAdele_sqrtTwoRoot‖ = |Algebra.norm ℚ sqrtTwoRoot|

example :
    ‖infinitePrincipalAdele_fundamentalUnit‖ =
      |Algebra.norm ℚ (algebraMap (𝒪 K) K (sqrtTwoFundamentalUnit : 𝒪 K))|

example :
    DenseRange (algebraMap K SqrtTwoInfiniteAdeleRing)
```

Mathematical meaning. The displayed Lean declaration has type `‖adele‖ = |norm ℚ ·|` for $\sqrt{2}$ and the fundamental unit; `DenseRange (algebraMap K and related terms)`.

Implementation. `InfiniteAdeleRing.coe_norm_eq_abs_norm`, `denseRange_algebraMap`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem complexEmbedding_norm_sqrtTwoRoot (φ : K →+* ℂ) :
    ‖φ sqrtTwoRoot‖ = Real.sqrt 2
```

Mathematical meaning. The displayed Lean declaration has type `(φ : K →+* ℂ) → ‖φ sqrtTwoRoot‖ = Real.sqrt 2`.

Implementation. `infinitePlace_apply_sqrtTwoRoot` on `InfinitePlace.mk φ`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem canonicalEmbedding_norm_sqrtTwoRoot :
    ‖NumberField.canonicalEmbedding K sqrtTwoRoot‖ = Real.sqrt 2
```

Mathematical meaning. the house of $\sqrt{2}$.

Implementation. `le_antisymm`; `canonicalEmbedding.norm_le_iff` + `norm_le_pi_norm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem canonicalEmbedding_sqrtTwoInteger_mem_integerLattice :
    NumberField.canonicalEmbedding K (algebraMap (𝒪 K) K sqrtTwoInteger) ∈
      NumberField.canonicalEmbedding.integerLattice K
```

Mathematical meaning. The displayed Lean declaration has type `canonicalEmbedding K (algebraMap sqrtTwoInteger) ∈ integerLattice K`.

Implementation. exhibit the preimage in `RingHom.range`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem canonicalEmbedding_fundamentalUnit_coordinates :
    ∃ σ τ : K →+* ℂ,
      σ (algebraMap (𝒪 K) K (sqrtTwoFundamentalUnit : 𝒪 K)) =
          (1 + Real.sqrt 2 : ℝ) ∧
        τ (algebraMap (𝒪 K) K (sqrtTwoFundamentalUnit : 𝒪 K)) =
          (1 - Real.sqrt 2 : ℝ)
```

Mathematical meaning. The displayed Lean declaration has type $\exists\ \sigma\ \tau$ `: K →+* ℂ,` $\sigma(1+\sqrt{2}) = 1+\sqrt{2}$ and $\tau(1+\sqrt{2}) = 1-\sqrt{2}$.

Implementation. `exists_realEmbedding_pos/_neg`, compose with `Complex.ofRealHom`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev mixedEmbedding_sqrtTwoRoot : NumberField.mixedEmbedding.mixedSpace
    K :=
  NumberField.mixedEmbedding K sqrtTwoRoot
```

Mathematical meaning. The displayed Lean declaration has type `mixedEmbedding.mixedSpace K := mixedEmbedding K sqrtTwoRoot`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example
    (f : NumberField.InfinitePlace K → NNReal)
    (hf : ∀ w : NumberField.InfinitePlace K, w sqrtTwoRoot < f w) :
    mixedEmbedding_sqrtTwoRoot ∈ NumberField.mixedEmbedding.convexBodyLT
    K f

example :
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone K)

example :
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone.normLeOne K)
```

Mathematical meaning. The displayed Lean declaration has type `mixedEmbedding_sqrtTwoRoot ∈ convexBodyLT K f` under `w √2 < f w; MeasurableSet (fundamentalCone K); MeasurableSet (fundamentalCone.normLeOne K)`.

Implementation. `convexBodyLT_mem`, `measurableSet_fundamentalCone`, `fundamentalCone.measurableSet_normLeOne`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem polarSpace_nonempty :
    Nonempty (NumberField.mixedEmbedding.polarSpace K)
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty (mixedEmbedding.polarSpace K)`.

Implementation. The proof applies the packaged result `infer_instance`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem unit_smul_mem_fundamentalCone_iff_eq_one_or_neg_one
    {x : NumberField.mixedEmbedding.mixedSpace K}
    (hx : x ∈ NumberField.mixedEmbedding.fundamentalCone K)
    (u : (𝒪 K)ˣ) :
    u • x ∈ NumberField.mixedEmbedding.fundamentalCone K ↔ u = 1 ∨ u = -1
```

Mathematical meaning. The displayed Lean declaration has type for `x` in the fundamental cone, `u • x ∈ cone ↔ u = 1 ∨ u = -1`.

Implementation. `fundamentalCone.unit_smul_mem_iff_mem_torsion` + `mem_torsion_iff_eq_one_or_neg_one`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem house_sqrtTwoRoot :
    NumberField.house sqrtTwoRoot = Real.sqrt 2
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.house sqrtTwoRoot = Real.sqrt 2`.

Implementation. unfold `house` + `canonicalEmbedding_norm_sqrtTwoRoot`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem house_sqrtTwoInteger :
    NumberField.house (algebraMap (𝒪 K) K sqrtTwoInteger) = Real.sqrt 2
```

Mathematical meaning. The displayed Lean declaration has type `house (algebraMap sqrtTwoInteger) = Real.sqrt 2`.

Implementation. `simpa` from `house_sqrtTwoRoot`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem house_sqrtTwoFundamentalUnit :
    NumberField.house (algebraMap (𝒪 K) K (sqrtTwoFundamentalUnit : 𝒪 K))
    =
      1 + Real.sqrt 2
```

Mathematical meaning. The displayed Lean declaration has type `house (algebraMap (1+√2)) = 1 + Real.sqrt 2`.

Implementation. `le_antisymm`; upper bound `norm_add_le` per embedding (each $\|\varphi\sqrt{2}\|=\sqrt{2}$); lower bound from the real embedding $\sigma(1+\sqrt{2})=1+\sqrt{2}$ of `canonicalEmbedding_fundamentalUnit_coordinates`.

```
abbrev SqrtTwoAdeleRing : Type :=
  NumberField.AdeleRing (𝒪 K) K

abbrev principalAdele_sqrtTwoRoot : SqrtTwoAdeleRing :=
  algebraMap K SqrtTwoAdeleRing sqrtTwoRoot

abbrev principalAdele_fundamentalUnit : SqrtTwoAdeleRing :=
  algebraMap K SqrtTwoAdeleRing (algebraMap (𝒪 K) K
    (sqrtTwoFundamentalUnit : 𝒪 K))
```

Mathematical meaning. The displayed Lean declaration has type `AdeleRing (𝒪 K) K`; principal adeles of $\sqrt{2}$ and $1+\sqrt{2}$.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem principalAdele_sqrtTwoRoot_ne_zero :
    principalAdele_sqrtTwoRoot ≠ 0
```

Mathematical meaning. The displayed Lean declaration has type `principalAdele_sqrtTwoRoot ≠ 0`.

Implementation. `AdeleRing.algebraMap_injective`; $\sqrt{2}=0 \Rightarrow 2=0$. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem principalAdele_fundamentalUnit_is_unit_everywhere :
    ∀ v : NumberField.FinitePlace K,
      v (algebraMap (𝒪 K) K (sqrtTwoFundamentalUnit : 𝒪 K)) = 1
```

Mathematical meaning. the unit is a local unit at every finite place.

Implementation. `norm_embedding_eq`, `norm_eq_one_iff_notMem`; if $(1+\sqrt{2}) \in$ maximal ideal then so is $(1+\sqrt{2})(\sqrt{2}-1)=1$, contradiction.

```
abbrev sqrtTwoDedekindZeta (s : ℂ) : ℂ :=
  NumberField.dedekindZeta K s

abbrev sqrtTwoDedekindZetaResidue : ℝ :=
  NumberField.dedekindZeta_residue K
```

Mathematical meaning. The displayed Lean declaration has type `dedekindZeta K s`; `dedekindZeta_residue K`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example :
    0 < sqrtTwoDedekindZetaResidue

example :
    sqrtTwoDedekindZetaResidue =
      (2 ^ NumberField.InfinitePlace.nrRealPlaces K *
          (2 * Real.pi) ^ NumberField.InfinitePlace.nrComplexPlaces K *
          NumberField.Units.regulator K * NumberField.classNumber K) /
        (NumberField.Units.torsionOrder K * Real.sqrt |NumberField.discr
    K|)
```

Mathematical meaning. The displayed Lean declaration has type residue `> 0`; the class-number-formula expansion `residue = (2^r₁ (2π)^r₂ · reg · h)/(w · √|disc|)`.

Implementation. `dedekindZeta_residue_pos`, `dedekindZeta_residue_def`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

2.11 Summary for $\mathbb{Q}(\sqrt{2})$

Real quadratic field $\mathbb{Q}(\sqrt{2})$, built via `QuadraticAlgebra ℚ 2 0` (the radical model, generator $\sqrt{2}$ with $\sqrt{2}^2 = 2$). Twin of `SqrtFive` (which uses the golden-ratio model `QuadraticAlgebra ℚ 1 1`). The decisive difference: ring of integers is $\mathbb{Z}[\sqrt{2}]$ (monogenic at the *square root*, disc $8 = 2^3$), and the fundamental unit $1+\sqrt{2}$ is proved outright via mathlib's `Zsqrtd/Pell.Solution₁` API – so `unitGroupStructure` ($\mu_2 \times \mathbb{Z}$) is fully formalised; `regulator_eq` is now also proved, making the folder sorry-free. Namespace `NumberField.SqrtTwo` throughout. Every file: `noncomputable section`, `attribute [-instance] DivisionRing.toRatAlgebra`, same open scopes.

All cross-references to `Common.lean` use namespace `NumberField.FieldExamples`. Common API actually used here: `eq_of_le_of_absNorm_eq`, `eq_prime_of_dvd_prime_sq_of_ne_one_of_ne_sq`, `not_isUnramifiedAt_of_map_span_natCast_le_sq_of_not_le_pow_succ`, `not_map_span_natCast_le_pow_succ_of_absNorm_span_eq_pow`, `abs_intNorm_lt_of_abs_norm_le`, `natAbs_lt_of_abs_ratCast_lt`, `torsionPowerHom`, `rankOnePowerHom`.

3 $\mathbb{Q}(\sqrt{5})$

Folder: `XYin/NumberField/FieldExamples/SqrtFive/` Field: real quadratic $\mathbb{Q}(\sqrt{5})$, built via `QuadraticAlgebra ℚ 1 1` (golden-ratio model, generator φ with φ^2 `= φ + 1`). Ring of integers $\mathbb{Z}[\varphi]$ = $\mathbb{Z}$`[(1+`$\sqrt{5}$`)/2]`, disc `5` (smallest of any real quadratic field). Twin of SqrtTwo (`QuadraticAlgebra ℚ 2 0`); SqrtFive additionally carries a `RadicalModel = QuadraticAlgebra ℚ 5 0` (generator $\sqrt{5}$) and an `AdjoinRoot (X²−5)` presentation, plus the golden `AdjoinRoot (X²−X−1)` presentation, all proved equivalent. Namespace throughout: `NumberField.SqrtFive`. Files open `QuadraticAlgebra`, `NumberField`, `Module`, `Polynomial` (and `NumberField.FieldExamples` = Common where relevant). All core files are `noncomputable section` and remove `DivisionRing.toRatAlgebra`. Cross-reference – `Common.lean` API used: `eq_prime_of_dvd_prime_sq_of_ne_one_of_ne_sq`, `left_eq_prime_of_mul_eq_prime_sq_of_ne_one`, `exists_int_inverse_mod_prime`, `eq_of_le_of_absNorm_eq`, `natAbs_lt_of_abs_ratCast_lt`, `torsionPowerHom`, `rankOnePowerHom`, `abs_intNorm_lt_of_abs_norm_le`, `not_isUnramifiedAt_of_map_span_natCast_le_sq_of_not_le_pow_succ`, `not_map_span_natCast_le_pow_succ_of_absNorm_span_eq_pow`. Sorry-free (as of the 2026-06-15 update). The two former Units.lean sorries are now closed. `fundamentalUnit_generates` is proved by a self-contained real-embedding window-reduction argument: every unit maps under `reEmb : ℚ(`$\sqrt{5}$`) → ℝ` (sending $\varphi \mapsto \Phi$ `= (1+`$\sqrt{5}$`)/2`); a power of φ reduces it into the window `[1, Φ)`; and the norm form `x²+xy−y² = ±1` (`isUnit_iff_normForm`) forces the only unit there to be `1`. This is the standard textbook proof, formalised faithfully – it does not use the `Zsqrtd/Pell` `x²−5y²=1` machinery (which only covers the index-2 suborder $\mathbb{Z}$`[`$\sqrt{5}$`]`), and needs no missing API. `regulator_eq` then follows from mature mathlib API (`regOfFamily_div_regulator` → `regOfFamily_eq_det` → `Matrix.det_eq_elem_of_subsingleton`). SqrtFive is the proof that, for a real quadratic field, ”this explicit unit is fundamental” is a matter of *intrinsic mathematical work*, not API immaturity.

Each block below begins with the displayed Lean declaration or object definition. For theorem, lemma, instance-proof, and example blocks the proof body is omitted; for definitions and abbreviations the displayed RHS is retained, because it is part of the mathematical object being documented.

3.1 `Basic.lean`

```
instance : Fact (∀ r : ℚ, r ^ 2 ≠ (1 : ℚ) + (1 : ℚ) * r)
```

Mathematical meaning. `X²−X−1` has no rational root; data instance feeding `QuadraticAlgebra` that the field is genuinely degree 2.

Implementation. from `r²=r+1` build `IsSquare (5:ℚ)` via `2r−1`, contradiction by `norm_num`.

```
abbrev K : Type := QuadraticAlgebra ℚ (1 : ℚ) (1 : ℚ)
```

Mathematical meaning. the field $\mathbb{Q}$`(`$\sqrt{5}$`)`. –

Implementation. The displayed RHS is the packaged construction `definitional. --` specialized to this field.

```
instance : FiniteDimensional ℚ K :=
  (QuadraticAlgebra.basis (1 : ℚ) (1 : ℚ)).finiteDimensional_of_finite
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `QuadraticAlgebra.basis (1) (1) |>.finiteDimensional_of_finite`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : NumberField K :=
  NumberField.of_module_finite ℚ K
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `NumberField.of_module_finite ℚ K`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem degree_eq_two : Module.finrank ℚ K = 2
```

Mathematical meaning. The displayed Lean declaration has type `Module.finrank ℚ K = 2` –.

Implementation. `finrank_eq_card_basis` + `Fintype.card_fin`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : Algebra.IsQuadraticExtension ℚ K where
  finrank_eq_two'
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `finrank_eq_two' := degree_eq_two`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev goldenRoot : K := QuadraticAlgebra.omega
```

Mathematical meaning. distinguished φ. –

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem goldenRoot_sq : goldenRoot ^ 2 = goldenRoot + 1
```

Mathematical meaning. The displayed Lean declaration has type `goldenRoot ^ 2 = goldenRoot + 1` –.

Implementation. `change` to $\langle$`0,1`$\rangle$, `ext/simp [pow_two]`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev goldenPolynomial : ℚ[X] :=
  C (1 : ℚ) * X ^ 2 + C (-1 : ℚ) * X + C (-1 : ℚ)
```

Mathematical meaning. defining poly X^2-X-1. –

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
instance : Fact (Irreducible goldenPolynomial)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `Polynomial.X_pow_sub_X_sub_one_irreducible_rat (n:=2)` + `simpa`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev KAdjoinRoot : Type := AdjoinRoot goldenPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `:= AdjoinRoot goldenPolynomial` – AdjoinRoot presentation. –.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev goldenAdjoinRoot : KAdjoinRoot := AdjoinRoot.root goldenPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `KAdjoinRoot := AdjoinRoot.root goldenPolynomial` –.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem goldenAdjoinRoot_sq :
    goldenAdjoinRoot ^ 2 = goldenAdjoinRoot + 1
```

Mathematical meaning. The displayed Lean declaration has type `goldenAdjoinRoot ^ 2 = goldenAdjoinRoot + 1` –.

Implementation. `AdjoinRoot.eval₂_root` + `linear_combination`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def goldenAdjoinRootEquivGolden :
    KAdjoinRoot ≃ₐ[ℚ] K := by
  let f : KAdjoinRoot →ₐ[ℚ] K :=
    AdjoinRoot.liftAlgHom goldenPolynomial (Algebra.ofId ℚ K) goldenRoot
    (by
```

```
      simp [goldenPolynomial, goldenRoot_sq])
  let g : K →ₐ[ℚ] KAdjoinRoot :=
    (QuadraticAlgebra.lift (R := ℚ) (a := (1 : ℚ)) (b := 1))
    ⟨goldenAdjoinRoot, by
      rw [← pow_two, goldenAdjoinRoot_sq]
      simp
      ring_nf⟩
  refine { __ := f, invFun := g, left_inv := ?_, right_inv := ?_ }
  · intro x
    have hcomp : g.comp f = AlgHom.id ℚ KAdjoinRoot := by
      apply AdjoinRoot.algHom_ext
      change g (f goldenAdjoinRoot) = goldenAdjoinRoot
      simp [f, g, goldenAdjoinRoot]
    exact DFunLike.congr_fun hcomp x
  · intro x
    have hcomp : f.comp g = AlgHom.id ℚ K := by
      apply QuadraticAlgebra.algHom_ext
      change f (g goldenRoot) = goldenRoot
      simp [f, g, goldenRoot, goldenAdjoinRoot]
    exact DFunLike.congr_fun hcomp x
```

Mathematical meaning. AdjoinRoot ≅ QuadraticAlgebra model.

Implementation. fwd `AdjoinRoot.liftAlgHom`, back `QuadraticAlgebra.lift`; `algHom_ext+DFunLike.congr_fun`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem goldenAdjoinRootEquivGolden_root :
    goldenAdjoinRootEquivGolden goldenAdjoinRoot = goldenRoot
```

Mathematical meaning. The displayed Lean declaration has type `and related variants goldenAdjoinRoot = goldenRoot` –.

Implementation. The proof applies the packaged result `simp. ––`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
abbrev sqrtFiveRoot : K := 2 * goldenRoot - 1
```

Mathematical meaning. $\sqrt{5} = 2\varphi - 1$. –

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtFiveRoot_sq : sqrtFiveRoot ^ 2 = (5 : K)
```

Mathematical meaning. The displayed Lean declaration has type `sqrtFiveRoot ^ 2 = (5 : K)` –.

Implementation. `calc` expand + `goldenRoot_sq` + `ring`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem goldenRoot_eq_half_sqrtFiveRoot_add_one :
    goldenRoot = (sqrtFiveRoot + 1) / 2
```

Mathematical meaning. The displayed Lean declaration has type `goldenRoot = (sqrtFiveRoot + 1) / 2` –.

Implementation. `field_simp`/`ring`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem goldenRoot_isIntegral : IsIntegral ℤ goldenRoot
```

Mathematical meaning. The displayed Lean declaration has type `IsIntegral ℤ goldenRoot` –.

Implementation. witness X^2-X-1, `monicity`, `simp [goldenRoot_sq]`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : Fact (∀ r : ℚ, r ^ 2 ≠ (5 : ℚ) + (0 : ℚ) * r)
```

Mathematical meaning. 5 not a rational square (for `RadicalModel`).

Implementation. `IsSquare (5:ℚ)`, `norm_num`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev RadicalModel : Type := QuadraticAlgebra ℚ (5 : ℚ) 0
```

Mathematical meaning. $\sqrt{5}$-generator presentation. –

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev sqrtFiveRadicalRoot : RadicalModel := QuadraticAlgebra.omega
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `change`/`ext`/`simp`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtFive : RadicalModel := sqrtFiveRadicalRoot
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev goldenRadicalRoot : RadicalModel := (1 + sqrtFive) / 2
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `field_simp+linear_combination sqrtFive_sq`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtFivePolynomial : ℚ[X] :=
  X ^ 2 - C (5 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `ℚ[X] := X² - C 5` – defining poly of $\sqrt{5}$. –.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev RadicalAdjoinRoot : Type := AdjoinRoot sqrtFivePolynomial
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private def sqrtFiveAdjoinRootToRadical : RadicalAdjoinRoot →ₐ[ℚ]
    RadicalModel :=
  AdjoinRoot.liftAlgHom sqrtFivePolynomial (Algebra.ofId ℚ RadicalModel)
    sqrtFive (by
    calc
      sqrtFivePolynomial.eval₂ (Algebra.ofId ℚ RadicalModel) sqrtFive =
    sqrtFive ^ 2 -
    5 := by
        simp [sqrtFivePolynomial]
      _ = 0 := by rw [sqrtFive_sq]; ring)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private def sqrtFiveRadicalToAdjoinRoot : RadicalModel →ₐ[ℚ]
    RadicalAdjoinRoot :=
  (QuadraticAlgebra.lift (R := ℚ) (a := (5 : ℚ)) (b := 0))
    ⟨sqrtFiveAdjoinRoot, by
    rw [← pow_two, sqrtFiveAdjoinRoot_sq]
    simp [Algebra.smul_def, map_ofNat]⟩
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
noncomputable def sqrtFiveRadicalEquivAdjoinRoot : RadicalModel ≃ₐ[ℚ]
    RadicalAdjoinRoot
    := by
  refine
    { __ := sqrtFiveRadicalToAdjoinRoot
      invFun := sqrtFiveAdjoinRootToRadical
      left_inv := ?_
      right_inv := ?_ }
  · intro x
    have hcomp :
        sqrtFiveAdjoinRootToRadical.comp sqrtFiveRadicalToAdjoinRoot =
          AlgHom.id ℚ RadicalModel := by
      apply QuadraticAlgebra.algHom_ext
      change sqrtFiveAdjoinRootToRadical
          (sqrtFiveRadicalToAdjoinRoot sqrtFive) = sqrtFive
      rw [sqrtFiveRadicalToAdjoinRoot_root,
    sqrtFiveAdjoinRootToRadical_root]
    exact DFunLike.congr_fun hcomp x
  · intro x
    have hcomp :
        sqrtFiveRadicalToAdjoinRoot.comp sqrtFiveAdjoinRootToRadical =
          AlgHom.id ℚ RadicalAdjoinRoot := by
      apply AdjoinRoot.algHom_ext
      change sqrtFiveRadicalToAdjoinRoot
          (sqrtFiveAdjoinRootToRadical sqrtFiveAdjoinRoot) =
    sqrtFiveAdjoinRoot
      rw [sqrtFiveAdjoinRootToRadical_root,
    sqrtFiveRadicalToAdjoinRoot_root]
    exact DFunLike.congr_fun hcomp x
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean dec-

laration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
def sqrtFiveRadicalToGolden : RadicalModel →ₐ[ℚ] K :=
  (QuadraticAlgebra.lift (R := ℚ) (a := (5 : ℚ)) (b := 0)) ⟨sqrtFiveRoot,
    by
    rw [← pow_two, sqrtFiveRoot_sq]
    ext <;> norm_num⟩
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
def goldenToSqrtFiveRadical : K →ₐ[ℚ] RadicalModel :=
  (QuadraticAlgebra.lift (R := ℚ) (a := (1 : ℚ)) (b := 1))
    ⟨(sqrtFiveRadicalRoot + 1) / 2, by
      calc
        ((sqrtFiveRadicalRoot + 1) / 2) * ((sqrtFiveRadicalRoot + 1) / 2)
            = (sqrtFiveRadicalRoot ^ 2 + 2 * sqrtFiveRadicalRoot + 1) / 4
    := by ring
        _ = 1 • (1 : RadicalModel) + 1 • ((sqrtFiveRadicalRoot + 1) / 2)
    := by
          rw [sqrtFiveRadicalRoot_sq]
          ring⟩
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
def sqrtFiveRadicalEquivGolden : RadicalModel ≃ₐ[ℚ] K where
  __ := sqrtFiveRadicalToGolden
  invFun := goldenToSqrtFiveRadical
  left_inv x := by
    have hcomp :
        goldenToSqrtFiveRadical.comp sqrtFiveRadicalToGolden =
          AlgHom.id ℚ RadicalModel := by
      apply QuadraticAlgebra.algHom_ext
      change goldenToSqrtFiveRadical
```

```
          (sqrtFiveRadicalToGolden sqrtFiveRadicalRoot) =
    sqrtFiveRadicalRoot
      rw [sqrtFiveRadicalToGolden_root, sqrtFiveRoot, map_sub, map_mul,
    map_ofNat,
        goldenToSqrtFiveRadical_root, map_one]
      ring
    exact DFunLike.congr_fun hcomp x
  right_inv x := by
    have hcomp :
        sqrtFiveRadicalToGolden.comp goldenToSqrtFiveRadical =
          AlgHom.id ℚ K := by
      apply QuadraticAlgebra.algHom_ext
      change sqrtFiveRadicalToGolden
          (goldenToSqrtFiveRadical goldenRoot) = goldenRoot
      rw [goldenToSqrtFiveRadical_root, map_div₀, map_add, map_one,
        sqrtFiveRadicalToGolden_root]
      have hmap_two : sqrtFiveRadicalToGolden (2 : RadicalModel) = (2 :
    K) := by
        simpa using sqrtFiveRadicalToGolden.commutes (2 : ℚ)
      rw [hmap_two]
      exact goldenRoot_eq_half_sqrtFiveRoot_add_one.symm
    exact DFunLike.congr_fun hcomp x
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. fwd/back homs, `algHom_ext`+`DFunLike`, `goldenRoot_eq_half_sqrtFiveRoot_add_one`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def sqrtFiveAdjoinRootEquivGolden : RadicalAdjoinRoot ≃ₐ[ℚ]
    K :=
  sqrtFiveRadicalEquivAdjoinRoot.symm.trans sqrtFiveRadicalEquivGolden
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

3.2 `Galois.lean`

```
private noncomputable def conjHom : K →ₐ[ℚ] K :=
  (QuadraticAlgebra.lift (R := ℚ) (a := (1 : ℚ)) (b := 1)) ⟨1 -
    goldenRoot, by
```

```
    rw [← pow_two]
    rw [show (1 - goldenRoot) ^ 2 = 1 - 2 * goldenRoot + goldenRoot ^ 2
    by ring]
    rw [goldenRoot_sq]
    simp
    ring⟩
```

Mathematical meaning. The displayed Lean declaration has type `K →ₐ[ℚ] K (φ ↦ 1 - φ)` –.

Implementation. `QuadraticAlgebra.lift` with `(1−φ)²=1−2φ+φ²`, `goldenRoot_sq`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem conjHom_root : conjHom goldenRoot = 1 - goldenRoot
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
noncomputable def conjugation : K ≃ₐ[ℚ] K where
  __ := conjHom
  invFun := conjHom
  left_inv x := DFunLike.congr_fun conjHom_comp x
  right_inv x := DFunLike.congr_fun conjHom_comp x
```

Mathematical meaning. The displayed Lean declaration has type `K ≃ₐ[ℚ] K` – involutive conjugation.

Implementation. `invFun := conjHom`, `left/right_inv` from `conjHom_comp`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem conjugation_root : conjugation goldenRoot = 1 - goldenRoot
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
noncomputable def idAut : K ≃ₐ[ℚ] K := AlgEquiv.refl
```

Mathematical meaning. The displayed Lean declaration has type `K ≃ₐ[ℚ] K := AlgEquiv.refl`. –.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem conjugation_apply_goldenRoot : conjugation goldenRoot = 1 -
    goldenRoot
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem conjugation_apply_sqrtFiveRoot : conjugation sqrtFiveRoot =
    -sqrtFiveRoot
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `map_sub`/`map_mul` + `conjugation_apply_goldenRoot` + `ring`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem exists_conj :
    ∃ σ : K ≃ₐ[ℚ] K, σ goldenRoot = 1 - goldenRoot
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. ⟨`conjugation, conjugation_root`⟩. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem aut_apply_goldenRoot (σ : K ≃ₐ[ℚ] K) :
    σ goldenRoot = goldenRoot ∨ σ goldenRoot = 1 - goldenRoot
```

Mathematical meaning. classifies action on generator.

Implementation. $\sigma\varphi$ satisfies $x^2=x+1$; factor $(\sigma\varphi-\varphi)(\sigma\varphi-(1-\varphi))=0$ by `linear_combination`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem aut_eq_or (σ : K ≃ₐ[ℚ] K) :
    σ = idAut ∨ σ = conjugation
```

Mathematical meaning. The displayed Lean declaration has type $(\sigma) \to \sigma$ `= idAut` $\vee \sigma =$ `conjugation` –.

Implementation. `QuadraticAlgebra.algHom_ext` ⇒ determined by image of φ; case-split `aut_apply_goldenRoot`. –

```
theorem isGalois : IsGalois ℚ K
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `infer_instance` (quadratic char-0 $\Rightarrow$ Galois). – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem galoisGroupStructure :
    Nonempty ((K ≃ₐ[ℚ] K) ≃* Multiplicative (ZMod 2))
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty ((K ≃ₐ[ℚ] K) ≃* Multiplicative (ZMod 2))` –.

Implementation. `conjugation ≠ idAut` (else $2\varphi=1$, check `im`); `Nat.card_eq_two_iff` from `aut_eq_or`; `mulEquivOfPrimeCardEq` + `Nat.card_zmod`. –

3.3 `Embeddings.lean`

```
private theorem ringHom_ext_golden {f g : K →+* ℂ} (h : f goldenRoot = g
    goldenRoot) :
    f = g
```

Mathematical meaning. ring homs to $\mathbb{C}$ determined by image of φ.

Implementation. `QuadraticAlgebra.mk_eq_add_smul_omega`, `eq_ratCast` on $\mathbb{Q}$-part. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isTotallyReal : NumberField.IsTotallyReal K
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.IsTotallyReal K` –.

Implementation. each embedding fixes φ under conjugation (factor `(z̄−z)(z̄+z−1)=0`); exclude `re=1/2` branch by `nlinarith [sq_nonneg z.im]`. –

```
theorem nrRealPlaces_eq_two : InfinitePlace.nrRealPlaces K = 2
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `IsTotallyReal.nrComplexPlaces_eq_zero` + `card_add_two_mul_card_eq_rank` + `degree_eq_two`, `omega`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem nrComplexPlaces_eq_zero : InfinitePlace.nrComplexPlaces K = 0
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `IsTotallyReal.nrComplexPlaces_eq_zero`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_card_eq_two :
    Fintype.card (NumberField.InfinitePlace K) = 2
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `card_eq_nrRealPlaces_add_nrComplexPlaces` + the two counts. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_isReal (v : NumberField.InfinitePlace K) :
    NumberField.InfinitePlace.IsReal v
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `isTotallyReal.isReal`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem exists_realEmbedding_goldenRatio :
    ∃ σ : K →ₐ[ℚ] ℝ, σ goldenRoot = (1 + Real.sqrt 5) / 2
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `QuadraticAlgebra.lift` with the relation checked via `Real.mul_self_sqrt`, `nlinarith`. –

3.4 `RingOfIntegers.lean`

```
abbrev GoldenOrder : Type := QuadraticAlgebra ℤ (1 : ℤ) (1 : ℤ)
```

Mathematical meaning. concrete $\mathbb{Z}[\varphi]$ model (`a+bφ`, `a b : ℤ`). –

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev goldenInteger : GoldenOrder := QuadraticAlgebra.omega
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem goldenInteger_isIntegral : IsIntegral ℤ goldenInteger
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. witness X^2-X-1, `monicity`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : Algebra.IsIntegral ℤ GoldenOrder
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. each $\langle a,b\rangle$ = `algebraMap a + b•`φ, both integral. –

```
def goldenOrderToField : GoldenOrder →+* K where
  toFun z := ⟨z.re, z.im⟩
  map_zero' := by ext <;> simp
  map_one' := by
    change (⟨((1 : GoldenOrder).re : ℚ), ((1 : GoldenOrder).im : ℚ)⟩ : K)
    = ⟨1, 0⟩
    ext <;> simp only [QuadraticAlgebra.re_one, QuadraticAlgebra.im_one,
    Int.cast_one,
      Int.cast_zero]
  map_add' x y := by ext <;> simp
  map_mul' x y := by ext <;> simp
```

Mathematical meaning. The displayed Lean declaration has type `GoldenOrder →+* K` ($\langle a,b\rangle \mapsto \langle a,b\rangle$ over $\mathbb{Q}$) –.

Implementation. explicit `RingHom` w/ `ext`/`simp`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : Algebra GoldenOrder K :=
  goldenOrderToField.toAlgebra

instance : IsScalarTower ℤ GoldenOrder K :=
  IsScalarTower.of_algebraMap_eq fun _ => by ext <;> simp
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem goldenOrderToField_injective : Function.Injective
    goldenOrderToField
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. compare `re`/`im`, `exact_mod_cast`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev goldenPowerBasis : PowerBasis ℚ K where
  gen := goldenRoot
  dim := 2
  basis := QuadraticAlgebra.basis (1 : ℚ) (1 : ℚ)
  basis_eq_pow i := by
    fin_cases i <;> ext <;>
      simp [QuadraticAlgebra.basis, QuadraticAlgebra.linearEquivTuple,
    goldenRoot]
```

Mathematical meaning. The displayed Lean declaration has type `PowerBasis ℚ K` (gen φ, dim 2, basis = `QuadraticAlgebra.basis`). –.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem adjoin_goldenRoot_eq_top : Algebra.adjoin ℚ ({goldenRoot} : Set
    K) = ⊤
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `PowerBasis.adjoin_gen_eq_top`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem goldenPowerBasis_basis_zero : goldenPowerBasis.basis 0 = (1 : K)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `change`+`ext`/`simp`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem golden_trace_eq (x : K) :
    Algebra.trace ℚ K x = 2 * x.re + x.im
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.trace ℚ K x = 2·x.re + x.im` –.

Implementation. `trace_eq_matrix_trace`, `leftMulMatrix_eq_repr_mul`, `Matrix.trace_fin_two`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem golden_norm_eq (x : K) :
    Algebra.norm ℚ x = x.re ^ 2 + x.re * x.im - x.im ^ 2
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.norm ℚ x = x.re`^{2} `+ x.re·x.im − x.im`^{2} –.

Implementation. `norm_eq_matrix_det`,`Matrix.det_fin_two`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem goldenPowerBasis_discr :
    Algebra.discr ℚ goldenPowerBasis.basis = (5 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℚ goldenPowerBasis.basis = 5` –.

Implementation. traceMatrix `!![2,1;1,3]` (entrywise via `golden_trace_eq`), `det_fin_two_of`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem five_dvd_coords_of_trace_norm {m n T N : ℤ}
    (hT : 2 * m + n = 5 * T)
    (hN : m * m + m * n - n * n = 25 * N) :
    (5 : ℤ) ∣ m ∧ (5 : ℤ) ∣ n
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. substitute `n=5T−2m`, `m`^{2}`=5·` `and related terms` $\Rightarrow$ `5`$\mid$`m`^{2}, `Prime 5`; then `5`$\mid$`n`. –

```
theorem int_eq_of_rat_div_five_eq {a b : ℤ} (h : ((a : ℚ) / 5) = (b : ℚ))
    :
    a = 5 * b
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `norm_num`/`linear_combination`/`exact_mod_cast`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem five_dvd_coords_of_integral {x : K} (hx : IsIntegral ℤ x)
    {m n : ℤ} (hre : (m : ℚ) = 5 * x.re) (him : (n : ℚ) = 5 * x.im) :
    (5 : ℤ) ∣ m ∧ (5 : ℤ) ∣ n
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `trace`/`norm` of `x` are integers, push to `five_dvd_coords_of_trace_norm`. –

```
theorem integral_mem_goldenOrderToField_range {x : K} (hx : IsIntegral ℤ
    x) :
    ∃ y : GoldenOrder, algebraMap GoldenOrder K y = x
```

Mathematical meaning. The displayed Lean declaration has type integral `x` ⇒ ∃ `y : GoldenOrder, algebraMap y = x` –.

Implementation. `discr·x ∈ adjoin` (`Algebra.discr_mul_isIntegral_mem_adjoin`, disc 5), divisibility of coords by 5 cancels the 5•. –

```
instance : IsIntegralClosure GoldenOrder ℤ K where
  algebraMap_injective
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `algebraMap_injective` + `isIntegral_iff` both directions. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def integralBasisVectors : Fin 2 → K := ![1, goldenRoot]
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `le_antisymm`; for ≥ decompose via `integral_mem_goldenOrderToField_range`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev concreteRingOfIntegers : Subalgebra ℤ K :=
  Algebra.adjoin ℤ ({goldenRoot} : Set K)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem ringOfIntegers_eq : integralClosure ℤ K = concreteRingOfIntegers
```

Mathematical meaning. The displayed Lean declaration has type `integralClosure ℤ K = concreteRingOfIntegers –`.

Implementation. `le_antisymm`; integral ⇒ in adjoin via `mk_eq_add_smul_omega`, `intCast_mem/zsmul_mem`. –

```
theorem span_integralBasisVectors_eq_integralBasis :
    Submodule.span ℤ (Set.range integralBasisVectors) =
      Submodule.span ℤ (Set.range (NumberField.integralBasis K))
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `mem_span_integralBasis`, `RingHom.mem_range`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem adjoin_sqrtFive_ne_integralClosure :
    Algebra.adjoin ℤ ({sqrtFiveRoot} : Set K) ≠ integralClosure ℤ K
```

Mathematical meaning. $\mathbb{Z}[\sqrt{5}] \subsetneq \mathbb{Z}[\varphi]$ (index 2).

Implementation. build subalgebra `S = {re∈ℤ, im∈2ℤ}` (with explicit `mul_mem` using $\varphi^2=\varphi+1$), $\sqrt{5}=-1+2\varphi \in S$, but $\varphi \notin S$ (im=1 not even). –

```
abbrev sqrtFiveExplicitIntegralBasis : Basis (Fin 2) ℤ GoldenOrder :=
  QuadraticAlgebra.basis (1 : ℤ) (1 : ℤ)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem goldenOrderToField_basis_zero :
    goldenOrderToField (sqrtFiveExplicitIntegralBasis 0) = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
noncomputable def goldenOrderEquivRingOfIntegers : GoldenOrder ≃ₐ[ℤ] 𝒪 K
    :=
  IsIntegralClosure.equiv ℤ GoldenOrder K (𝒪 K)
```

Mathematical meaning. The displayed Lean declaration has type `GoldenOrder` $\simeq_a$ `[ℤ] 𝒪 K := IsIntegralClosure.equiv and related terms`. –.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
noncomputable def concreteIntegralBasis : Basis (Fin 2) ℤ (𝒪 K) :=
  sqrtFiveExplicitIntegralBasis.map
    goldenOrderEquivRingOfIntegers.toLinearEquiv
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `IsIntegralClosure.algebraMap_equiv`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem concreteIntegralBasis_discr :
    Algebra.discr ℤ concreteIntegralBasis = (5 : ℤ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℤ concreteIntegralBasis = 5` –.

Implementation. `Algebra.discr_localizationLocalization` ($\mathcal{O}$ K $\to \mathbb{Q}\otimes$) + `Module.Basis.localizationLocalization`, transfer to `goldenPowerBasis_discr`. –

```
theorem numberField_discr_eq : NumberField.discr K = 5
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.discr K = 5` –.

Implementation. `discr_eq_discr K concreteIntegralBasis + concreteIntegralBasis_discr`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem integralBasis_discr :
    Algebra.discr ℚ (NumberField.integralBasis K) = (5 : ℚ)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `NumberField.coe_discr + numberField_discr_eq`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def goldenInt0 : 𝒪 K := ⟨goldenRoot, goldenRoot_isIntegral⟩
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `Subtype.ext/push_cast`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem norm_eq_normForm (x y : ℚ) :
    Algebra.norm ℚ (x • (1 : K) + y • goldenRoot) = x ^ 2 + x * y - y ^ 2
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.norm ℚ (x•1 + y•φ) = x² + xy - y²` –.

Implementation. `golden_norm_eq` + `re/im_add`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem intNorm_eq_normForm (x y : ℤ) :
    Algebra.norm ℤ (x • (1 : 𝒪 K) + y • goldenInt0) = x ^ 2 + x * y - y ^
    2
```

Mathematical meaning. The displayed Lean declaration has type integral version `Algebra.norm ℤ (x•1 + y•φ) = x² + xy - y² (x y : ℤ)` –.

Implementation. push to ℚ via `Algebra.coe_norm_int`, `norm_eq_normForm`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isUnit_iff_normForm (x y : ℤ) :
    IsUnit (x • (1 : 𝒪 K) + y • goldenInt0) ↔
      x ^ 2 + x * y - y ^ 2 = 1 ∨ x ^ 2 + x * y - y ^ 2 = -1
```

Mathematical meaning. Pell-type unit equation.

Implementation. `isUnit_iff_norm`, `intNorm_eq_normForm`, `abs_eq`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

3.5 PrimeSplitting.lean

```
noncomputable def sqrtFiveInt0 : 𝒪 K := 2 * goldenInt0 - 1
```

Mathematical meaning. The displayed Lean declaration has type `𝒪 K := 2 * goldenInt0 - 1` – $\sqrt{5}$ in `𝒪 K`. –.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem span_five_eq :
    Ideal.span {(5 : 𝒪 K)} = Ideal.span {sqrtFiveInt0} ^ 2
```

Mathematical meaning. 5 totally ramifies.

Implementation. `span_singleton_pow`, $(2\varphi-1)^2=5$ via `sqrtFiveRoot_sq`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_span_sqrtFiveInt0 : Ideal.absNorm (Ideal.span
    {sqrtFiveInt0}) = 5
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. $\sqrt{5}$ `= −1+2`φ, `intNorm_eq_normForm (−1) 2`, `absNorm_span_singleton`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isPrime_span_sqrtFiveInt0 : (Ideal.span {sqrtFiveInt0}).IsPrime
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `isPrime_of_irreducible_absNorm` + `Nat.Prime 5`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_span_sqrtFiveInt0 :
    (Ideal.span {sqrtFiveInt0}).LiesOver (Ideal.span {(5 : ℤ)})
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `liesOver_span_iff`; `absNorm_eq_pow_inertiaDeg'`; `ramificationIdx_spec` with norm 5/125/25 divisibility. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem absNorm_ne_two (P : Ideal (𝒪 K)) : Ideal.absNorm P ≠ 2
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. residue field $\cong$ `ZMod 2/3`, φ would solve `t`^{2}`=t+1`, `decide`. –

```
theorem isPrime_span_two : (Ideal.span {(2 : 𝒪 K)}).IsPrime
```

Mathematical meaning. 2 inert.

Implementation. `absNorm = 4`, any maximal `M` $\supseteq$ `(2)` has norm $|4$, $\neq 1$,$\neq 2 \Rightarrow = 4 \Rightarrow$ `M=(2)`.

```
theorem absNorm_span_two : Ideal.absNorm (Ideal.span {(2 : 𝒪 K)}) = 4
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `intNorm_eq_normForm 2 0`; `liesOver_span_iff`; `absNorm_eq_pow_inertiaDeg'` + `pow_right_injective`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isPrime_span_three : (Ideal.span {(3 : 𝒪 K)}).IsPrime
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. analogous to `(2)` (5 not a square mod 3).

```
theorem eleven_eq_mul : (11 : 𝒪 K) = (3 + goldenInt0) * (4 - goldenInt0)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `Subtype.ext`, expand, `goldenRoot_sq`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_eleven_eq :
    Ideal.span {(11 : 𝒪 K)} =
      Ideal.span {3 + goldenInt0} * Ideal.span {4 - goldenInt0}

theorem absNorm_span_three_add_golden :
    Ideal.absNorm (Ideal.span {3 + goldenInt0}) = 11

theorem isPrime_span_three_add_golden : (Ideal.span {3 +
    goldenInt0}).IsPrime

theorem absNorm_span_four_sub_golden :
    Ideal.absNorm (Ideal.span {4 - goldenInt0}) = 11

theorem isPrime_span_four_sub_golden : (Ideal.span {4 -
    goldenInt0}).IsPrime

theorem liesOver_span_three_add_golden :
    (Ideal.span {3 + goldenInt0}).LiesOver (Ideal.span {(11 : ℤ)})

theorem liesOver_span_four_sub_golden :
    (Ideal.span {4 - goldenInt0}).LiesOver (Ideal.span {(11 : ℤ)})
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `span_singleton_mul`, `intNorm_eq_normForm 3 1 / 4 (-1)`, `isPrime_of_irreducible_absNorm`. –

```
theorem primesAboveEleven_ne :
    Ideal.span ({3 + goldenInt0} : Set (𝒪 K)) ≠ Ideal.span {4 -
    goldenInt0}
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. else both contain their difference $\sqrt{5}$, then 5, then 11, then $1 \Rightarrow$ contradiction with norm 11. –

```
private theorem absNorm_span_natCast (p : ℕ) :
    Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K)}) = p ^ 2
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `absNorm_span_natCast` (mathlib) + `RingOfIntegers.rank` + `degree_eq_two`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem concreteIntegralBasis_zero_eq_one :
    concreteIntegralBasis 0 = (1 : 𝒪 K)

private theorem concreteIntegralBasis_one_eq_goldenInt0 :
    concreteIntegralBasis 1 = goldenInt0

private theorem li_one_golden : LinearIndependent ℤ ![(1 : 𝒪 K),
    goldenInt0]

private theorem exists_o5_coords (z : 𝒪 K) :
    ∃ A B : ℤ, z = A • (1 : 𝒪 K) + B • goldenInt0
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `Subtype.ext`, `Fintype.linearIndependent_iff`, `concreteIntegralBasis.repr`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem unit_golden_coeff_not_mem (p : ℕ) (hp : 2 ≤ p) (A B : ℤ)
    (hB : B = 1 ∨ B = -1) :
    ((A : ℤ) : 𝒪 K) + ((B : ℤ) : 𝒪 K) * goldenInt0 ∉
      Ideal.span {((p : ℕ) : 𝒪 K)}
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `mem_span_singleton`, coords via `li_one_golden`, $p|1 \Rightarrow$contra. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramified_iff_eq_five (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧
      Ideal.span {(p : 𝒪 K)} ≤ P ^ 2) ↔ p = 5
```

Mathematical meaning. ramification $\Leftrightarrow$ `p | disc = 5`.

Implementation. ($\Rightarrow$) `not_isUnramifiedAt_ and related terms ⇒ P | differentIdeal, absNorm P | 5`, so `p=5`; ($\Leftarrow$) use `span_five_eq`. – Uses (incl. Common): [absNorm_span_natCast, not_isUnramifiedAt_of_map_span_natCast_le_sq_of_not_le_pow_succ, not_map_span_natCast_le_pow_succ_of_absNorm_span_eq_pow, numberField_discr_eq, absNorm_differentIdeal, sqrtFiveIntO, span_five_eq, isPrime_span_sqrtFiveIntO] – public –

```
theorem ramificationIdx_span_two :
    (Ideal.span {(2 : ℤ)}).ramificationIdx (Ideal.span {(2 : 𝒪 K)}) = 1

theorem ramificationIdx_span_three :
    (Ideal.span {(3 : ℤ)}).ramificationIdx (Ideal.span {(3 : 𝒪 K)}) = 1

theorem ramificationIdx_span_three_add_golden :
    (Ideal.span {(11 : ℤ)}).ramificationIdx (Ideal.span {3 + goldenIntO})
    = 1

theorem ramificationIdx_span_four_sub_golden :
    (Ideal.span {(11 : ℤ)}).ramificationIdx (Ideal.span {4 - goldenIntO})
    = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `ramificationIdx_spec`; the `>` direction reduces to `ramified_iff_eq_five` giving `p=5` contradiction. –

```
private theorem golden_sub_mem_span_natCast_of_one_mem_span_pair (p : ℕ)
    (c c' k₁ k₂ :
    ℤ)
    (hprod : (goldenIntO - (c : 𝒪 K)) *
        (goldenIntO - (c' : 𝒪 K)) =
      ((p : ℕ) : 𝒪 K) *
        ((k₁ : 𝒪 K) * goldenIntO + (k₂ : 𝒪 K)))
    (h1 : (1 : 𝒪 K) ∈
      Ideal.span {((p : ℕ) : 𝒪 K), goldenIntO - (c : 𝒪 K)}) :
    goldenIntO - (c' : 𝒪 K) ∈ Ideal.span {((p : ℕ) : 𝒪 K)}
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `mem_span_pair`, `mem_span_singleton`, `linear_combination`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem absNorm_span_prime_golden_sub_of_mul (p : ℕ) (hp :
    p.Prime)
    (c c' k₁ k₂ : ℤ)
    (hprod : (goldenIntO - (c : 𝒪 K)) *
        (goldenIntO - (c' : 𝒪 K)) =
      ((p : ℕ) : 𝒪 K) *
        ((k₁ : 𝒪 K) * goldenIntO + (k₂ : 𝒪 K))) :
    Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K),
      goldenIntO - (c : 𝒪 K)}) = p
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. norm | p^2, $\neq 1$ (`unit_golden_coeff_not_mem`), $\neq p^2$ (`eq_of_le_of_absNorm_eq`), `eq_prime_of_dvd_prime_sq_` and related terms. – Uses (incl. Common): [absNorm_span_natCast, unit_golden_coeff_not_mem, eq_of_le_of_absNorm_eq, eq_prime_of_dvd_prime_sq_of_ne_one_of_ne_sq, golden_sub_mem_span_natCast_of_one_mem_span_pair, goldenIntO] – private –

```
private theorem exists_golden_root_of_absNorm_eq_prime (p : ℕ) (hp :
    p.Prime)
    (P : Ideal (𝒪 K)) (hNP : Ideal.absNorm P = p) :
    ∃ t : ZMod p, t ^ 2 = t + 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. residue field $\cong$ `ZMod p`, image of φ. –

```
private theorem isSquare_five_of_absNorm_eq_prime (p : ℕ) (hp : p.Prime)
    (P : Ideal (𝒪 K)) (hNP : Ideal.absNorm P = p) :
    IsSquare ((5 : ℕ) : ZMod p)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private theorem pm_one_mod_five_of_splits (p : ℕ) (hp : p.Prime) (hp5 : p
    ≠ 5) :
    (∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) → p % 5 = 1 ∨ p % 5 = 4
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. norm-`p` factor + `isSquare_five`, quadratic reciprocity `ZMod.exists_sq_eq_prime_iff_of_mod_four_eq_one`, exclude `2,3` mod 5 by `decide`. – Uses (incl. Common): [left_eq_prime_of_mul_eq_prime_sq_of_ne_one, absNorm_span_natCast, exists_golden_root_of_absNorm_eq_prime, isSquare_five_of_absNorm_eq_prime] – private –

```
private theorem exists_golden_root_of_pm_one_mod_five (p : ℕ) (hp :
    p.Prime) :
    p % 5 = 1 ∨ p % 5 = 4 → ∃ t : ZMod p, t ^ 2 = t + 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. reciprocity gives 5 square mod `p`, set `t=(1+s)/2`, clear 4. –

```
private theorem golden_root_ne_one_sub_of_prime_ne_five (p : ℕ) (hp :
    p.Prime) (hp5 : p
    ≠ 5)
    {t : ZMod p} (ht2 : t ^ 2 = t + 1) : t ≠ 1 - t
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. else `2t=1`, derive `5≡0 mod p` ⇒ contradiction. –

```
private theorem span_prime_golden_sub_ne_of_root (p : ℕ) (hp : p.Prime)
    (hp5 : p ≠ 5)
    (t : ZMod p) (ht2 : t ^ 2 = t + 1) (c c' : ℤ)
    (hct : ((c : ℤ) : ZMod p) = t) (hc't : ((c' : ℤ) : ZMod p) = 1 - t)
    (hNP : Ideal.absNorm
      (Ideal.span {((p : ℕ) : 𝒪 K), goldenInt0 - (c : 𝒪 K)}) = p) :
    Ideal.span {((p : ℕ) : 𝒪 K), goldenInt0 - (c : 𝒪 K)} ≠
      Ideal.span {((p : ℕ) : 𝒪 K), goldenInt0 - (c' : 𝒪 K)}
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. inverse of `c-c'` mod p (`exists_int_inverse_mod_prime`), else `1 ∈ P`. – Uses (incl. Common): [exists_int_inverse_mod_prime, golden_root_ne_one_sub_of_prime_ne_five, goldenInt0] – private –

```
private theorem span_natCast_eq_mul_span_golden_sub_of_mul (p : ℕ) (hp :
    p.Prime)
    (c c' k₁ k₂ : ℤ)
    (hprod : (goldenInt0 - (c : 𝒪 K)) *
```

```
      (goldenInt0 - (c' : 𝒪 K)) =
    ((p : ℕ) : 𝒪 K) *
      ((k₁ : 𝒪 K) * goldenInt0 + (k₂ : 𝒪 K)))
  (hNP : Ideal.absNorm
    (Ideal.span {((p : ℕ) : 𝒪 K), goldenInt0 - (c : 𝒪 K)}) = p)
  (hNQ : Ideal.absNorm
    (Ideal.span {((p : ℕ) : 𝒪 K), goldenInt0 - (c' : 𝒪 K)}) = p) :
  Ideal.span {((p : ℕ) : 𝒪 K)} =
    Ideal.span {((p : ℕ) : 𝒪 K), goldenInt0 - (c : 𝒪 K)} *
      Ideal.span {((p : ℕ) : 𝒪 K), goldenInt0 - (c' : 𝒪 K)}
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `mul_le` + `mem_span_pair`/`linear_combination`, `eq_of_le_of_absNorm_eq` with norm p^2. – Uses (incl. Common): [eq_of_le_of_absNorm_eq, absNorm_span_natCast] – private –

```
private theorem exists_golden_split_data (p : ℕ) [Fact p.Prime] (t : ZMod
    p)
    (ht2 : t ^ 2 = t + 1) :
    ∃ c c' k₁ k₂ : ℤ,
      ((c : ℤ) : ZMod p) = t ∧ ((c' : ℤ) : ZMod p) = 1 - t ∧
        (goldenInt0 - (c : 𝒪 K)) * (goldenInt0 - (c' : 𝒪 K)) =
          ((p : ℕ) : 𝒪 K) *
            ((k₁ : 𝒪 K) * goldenInt0 + (k₂ : 𝒪 K))
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `ZMod.natCast_rightInverse`, divisibilities `p|1-c-c'`,`p|1+cc'`, `linear_combination`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem splits_of_pm_one_mod_five (p : ℕ) (hp : p.Prime) (hp5 : p
    ≠ 5) :
    p % 5 = 1 ∨ p % 5 = 4 →
      ∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
        Ideal.span {(p : 𝒪 K)} = P * Q
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. assemble all the above. –

```
theorem splits_iff_pm_one_mod_five (p : ℕ) (hp : p.Prime) (hp5 : p ≠ 5) :
    (∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) ↔ p % 5 = 1 ∨ p % 5 = 4
```

Mathematical meaning. the quadratic splitting law (reciprocity).

Implementation. combines the two private directions. –

```
abbrev goldenPolynomialMod (p : ℕ) : (ZMod p)[X] :=
  X ^ 2 - X - C (1 : ZMod p)

private theorem goldenPolynomialMod_isRoot_iff (p : ℕ) (t : ZMod p) :
    (goldenPolynomialMod p).IsRoot t ↔ t ^ 2 = t + 1

private theorem goldenPolynomialMod_natDegree (p : ℕ) [Fact p.Prime] :
    (goldenPolynomialMod p).natDegree = 2

private theorem goldenPolynomialMod_roots_eq_zero_iff_no_root (p : ℕ)
    [Fact p.Prime] :
    (goldenPolynomialMod p).roots = 0 ↔ ¬ ∃ t : ZMod p, t ^ 2 = t + 1

private theorem goldenPolynomialMod_no_root_iff_two_or_three_mod_five
    (p : ℕ) (hp : p.Prime) (hp5 : p ≠ 5) :
    (¬ ∃ t : ZMod p, t ^ 2 = t + 1) ↔ p % 5 = 2 ∨ p % 5 = 3
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private theorem goldenInt0_adjoin_eq_top :
    Algebra.adjoin ℤ ({goldenInt0} : Set (𝒪 K)) = ⊤

private theorem goldenInt0_minpoly :
    minpoly ℤ goldenInt0 = X ^ 2 - X - 1

theorem differentIdeal_eq_span_sqrtFive :
    differentIdeal ℤ (𝒪 K) = Ideal.span {sqrtFiveInt0}

theorem absNorm_differentIdeal_eq_five :
    Ideal.absNorm (differentIdeal ℤ (𝒪 K)) = 5

theorem prime_dvd_differentIdeal_iff_eq_span_sqrtFive
```

```
    (P : Ideal (𝒪 K)) (_hP : P.IsPrime) (_hP0 : P ≠ ⊥) :
    P ∣ differentIdeal ℤ (𝒪 K) ↔ P = Ideal.span {sqrtFiveInt0}
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

3.6 `ClassGroup.lean`

```
abbrev goldenRootFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  FractionalIdeal.spanSingleton (𝒪 K)⁰ goldenRoot
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem goldenRoot_mem_fractionalIdeal :
    goldenRoot ∈ goldenRootFractionalIdeal
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `mem_spanSingleton_self`; `simp`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramifiedPrimeFractionalIdeal_ne_zero :
    ramifiedPrimeFractionalIdeal ≠ 0
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `coeIdeal_ne_zero`; $(\sqrt{5})=0 \Rightarrow 5=0$. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

Mathematical meaning. trivial class group.

Implementation. `classNumber_eq_one_iff + isPrincipalIdealRing_of_ and related terms_mem_Icc`; Minkowski bound $\sqrt{5}/2 < 2$ so $\lfloor \text{bound} \rfloor < 2$, no primes in `Icc 2 1`.

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝒪 K)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. ← `classNumber_eq_one_iff`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝒪 K)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. PID ⇒ UFM, `infer_instance`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classGroupStructure :
    Nonempty (ClassGroup (𝒪 K) ≃* Multiplicative (ZMod 1))
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty (ClassGroup (𝒪 K) ≃* Multiplicative (ZMod 1))` –.

Implementation. `Nat.card = 1` ⇒ `Subsingleton/Unique` ⇒ `MulEquiv.ofUnique`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev unitIdealRepresentative : (Ideal (𝒪 K))⁰ :=
  ⟨(⊤ : Ideal (𝒪 K)), by simp⟩
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `Subsingleton.elim`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

3.7 **Units.lean**

```
noncomputable def fundamentalUnit : (𝒪 K)ˣ where
  val := goldenInt0
  inv := goldenInt0 - 1
  val_inv := by
    rw [show goldenInt0 * (goldenInt0 - 1) = goldenInt0 ^ 2 - goldenInt0
    by ring]
    rw [goldenInt0_sq]
    ring
  inv_val := by
    rw [show (goldenInt0 - 1) * goldenInt0 = goldenInt0 ^ 2 - goldenInt0
    by ring]
    rw [goldenInt0_sq]
    ring
```

Mathematical meaning. the fundamental unit φ.

Implementation. `val_inv`/`inv_val` from `goldenInt0_sq` ($\varphi(\varphi-1)=\varphi^2-\varphi=1$). – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev torsionGenerator : (𝓞 K)ˣ := -1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private noncomputable def realEmb : K →ₐ[ℚ] ℝ :=
  QuadraticAlgebra.lift ⟨(1 + Real.sqrt 5) / 2, by
    have hs : Real.sqrt 5 * Real.sqrt 5 = 5 := Real.mul_self_sqrt (by
    norm_num)
    rw [show ((1 + Real.sqrt 5) / 2) * ((1 + Real.sqrt 5) / 2) =
        (3 + Real.sqrt 5) / 2 by nlinarith]
    simp
    ring⟩
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `QuadraticAlgebra.lift`, `Real.mul_self_sqrt`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem torsionUnits_eq : {u : (𝓞 K)ˣ | IsOfFinOrder u} = torsionUnits
```

Mathematical meaning. torsion = $\{\pm 1\}$.

Implementation. real embedding e; finite-order $\Rightarrow$ $|e\ x|=1 \Rightarrow e\ x=\pm 1 \Rightarrow x=\pm 1$ (injectivity). –

```
theorem torsionOrder_eq_two : Units.torsionOrder K = 2
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `Nat.card (torsion) = 2` via `card_eq_two_iff` (± 1, distinct since $2\neq 0$ in K), `torsionUnits_eq` for surjectivity. –

```
theorem unitRank_eq_one : Units.rank K = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `rank = #places − 1`, `nrRealPlaces=2`, `nrComplexPlaces=0`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def fundamentalUnitSystem : Fin (Units.rank K) → (𝒪 K)ˣ :=
  fun _ => fundamentalUnit
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem intNorm_goldenInt : Algebra.norm ℤ goldenInt0 = -1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `intNorm_eq_normForm 0 1` ($0^2+0-1^2=-1$). – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem normFundamentalUnit_map :
    Units.map (Algebra.norm ℤ : (𝒪 K) →* ℤ) fundamentalUnit = (-1 : ℤˣ)

private theorem normNegOneUnit_map :
    Units.map (Algebra.norm ℤ : (𝒪 K) →* ℤ) (-1 : (𝒪 K)ˣ) = 1

private theorem even_of_negOneUnit_zpow_eq_one {n : ℤ} (h : ((-1 : ℤˣ) ^
    n) = 1) :
    ∃ k : ℤ, n = 2 * k

private theorem normFundamentalUnit_zpow_map (n : ℤ) :
    Units.map (Algebra.norm ℤ : (𝒪 K) →* ℤ) (fundamentalUnit ^ n) =
      (-1 : ℤˣ) ^ n

private theorem normNegFundamentalUnit_zpow_map (n : ℤ) :
    Units.map (Algebra.norm ℤ : (𝒪 K) →* ℤ) (-(fundamentalUnit ^ n)) =
      (-1 : ℤˣ) ^ n
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `Units.ext`, `map_zpow`, `Odd.neg_one_zpow`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem exists_eq_torsion_mul_fundSystem_zpow (i₀ : Fin (Units.rank K))
    (u : (𝒪 K)ˣ) :
    ∃ ζ ∈ NumberField.Units.torsion K, ∃ n : ℤ,
      u = ζ * NumberField.Units.fundSystem K i₀ ^ n
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `closure_fundSystem_sup_torsion_eq_top`, `mem_sup`, `range fundSystem = {fundSystem i₀}` (subsingleton via rank 1), `mem_closure_singleton`. –

```
theorem fundSystem_zpow_mem_torsion (i₀ : Fin (Units.rank K)) (n : ℤ)
    (hn : NumberField.Units.fundSystem K i₀ ^ n ∈
      NumberField.Units.torsion K) : n = 0
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. image in `(𝒪 K)ˣ/torsion` via `basisModTorsion`, `repr_self`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem eq_one_or_neg_one_of_mem_torsion {ζ : (𝒪 K)ˣ}
    (hζ : ζ ∈ NumberField.Units.torsion K) : ζ = 1 ∨ ζ = -1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `torsionUnits_eq` + `fin_cases`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem fundamentalUnit_generates (u : (𝒪 K)ˣ) :
    ∃ n : ℤ, u = fundamentalUnit ^ n ∨ u = -(fundamentalUnit ^ n)
```

Mathematical meaning. every unit is $\pm\varphi^n$.

Implementation. a real-embedding window-reduction (≈35 lines of supporting `private` lemmas: `reEmb`, `fhom`, `exists_coords`, `minimality`, `reduction`, `pos_case`). Send units to $\mathbb{R}$ via `reEmb : φ ↦ Φ`; `exists_mem_Ico_zpow` reduces any positive-image unit into `[1, Φ)` by a power of φ; `minimality` uses the norm form $x^2+xy-y^2 = \pm 1$ (`isUnit_iff_normForm`) + bounds on the conjugate to force the reduced unit to 1; sign handled by `fhom_neg`. –

```
theorem fundamentalUnitSystem_isFundamental :
    Subgroup.closure (Set.range fundamentalUnitSystem) ⊔
    NumberField.Units.torsion K =
      ⊤
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `fundamentalUnit_generates` + `mem_sup`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem unit_eq_torsion_mul_fundamentalUnit_zpow (u : (𝒪 K)ˣ) :
    ∃ ζ : (𝒪 K)ˣ, ζ ∈ NumberField.Units.torsion K ∧
      ∃ n : ℤ, u = ζ * fundamentalUnit ^ n
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `fundamentalUnit_generates`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem normOneUnit_generated_by_fundamentalUnit_sq (u : (𝒪 K)ˣ)
    (hu : Algebra.norm ℤ (u : 𝒪 K) = 1) :
    ∃ n : ℤ, u = fundamentalUnit ^ (2 * n) ∨
      u = -(fundamentalUnit ^ (2 * n))
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `fundamentalUnit_generates` + `even_of_negOneUnit_zpow_eq_one` via the norm map. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem regulator_eq :
    Units.regulator K = Real.log ((1 + Real.sqrt 5) / 2)
```

Mathematical meaning. regulator = `log` $\varphi \approx$ `0.4812` (smallest of any real quadratic field).

Implementation. choose the real place `wR` (`exists_realEmbedding_goldenRatio`); `fundamentalUnitSystem_isFundamental` gives index 1, so `regOfFamily_div_regulator` $\Rightarrow$ `regulator = regOfFamily`; `regOfFamily_eq_det` + `Matrix.det_eq_elem_of_subsingleton` evaluate the rank-1 1×1 matrix to `|log(wR` φ`)| = log` Φ. –

```
private theorem sqrt5_sq : Real.sqrt 5 ^ 2 = 5

private theorem sqrt5_lo : (2 : ℝ) < Real.sqrt 5

private theorem sqrt5_hi : Real.sqrt 5 < 3

private theorem phi_gt_one : (1 : ℝ) < (1 + Real.sqrt 5) / 2

private theorem minimality (a b : ℤ) (sq5 : ℝ) (hsq : sq5 ^ 2 = 5) (hlo :
    2 < sq5)
    (hhi : sq5 < 3)
    (hprod : ((a : ℝ) + b * ((1 + sq5) / 2)) * ((a : ℝ) + b * ((1 - sq5)
    / 2)) = 1 ∨
```

```
    ((a : ℝ) + b * ((1 + sq5) / 2)) * ((a : ℝ) + b * ((1 - sq5) / 2)) =
  -1)
  (h1 : 1 ≤ (a : ℝ) + b * ((1 + sq5) / 2))
  (h2 : (a : ℝ) + b * ((1 + sq5) / 2) < (1 + sq5) / 2) :
  a = 1 ∧ b = 0
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. injective via `fundSystem_zpow_mem_torsion` + `−1` order 2; surjective via `exists_eq_torsion_mul_fundSystem_zpow` + `eq_one_or_neg_one_of_mem_torsion`. – Uses (incl. Common): [torsionPowerHom, rankOnePowerHom, fundSystem_zpow_mem_torsion, exists_eq_torsion_mul_fundSystem_zpow, eq_one_or_neg_one_of_mem_torsion] – private –

```
private theorem reduction (u : (𝒪 K)ˣ) (h1 : 1 ≤ fhom u)
  (h2 : fhom u < (1 + Real.sqrt 5) / 2) : u = 1
```

Mathematical meaning. $(\mathcal{O}\ K)^\times \cong \mu_2 \times \mathbb{Z}$.

Implementation. assemble `torsionPowerHom`/`rankOnePowerHom` coprod, `MulEquiv.ofBijective`. – Uses (incl. Common): [torsionPowerHom, rankOnePowerHom, unitGroupStructure_bijective, unitRank_eq_one] – public –

3.8 `Euclidean.lean`

```
private theorem abs_norm_int_lt_of_remainder_eq {b r : 𝒪 K} {bK : K}
  {u v : ℚ}
  (hbK : bK = algebraMap (𝒪 K) K b)
  (hrK :
    algebraMap (𝒪 K) K r =
      bK * ((u - ((round u : ℤ) : ℚ)) • (1 : K) +
        (v - ((round v : ℤ) : ℚ)) • goldenRoot))
  (hb_pos : 0 < |Algebra.norm ℚ bK|)
  (hfrac :
    |(u - ((round u : ℤ) : ℚ)) ^ 2 +
      (u - ((round u : ℤ) : ℚ)) * (v - ((round v : ℤ) : ℚ)) -
      (v - ((round v : ℤ) : ℚ)) ^ 2| ≤ 3 / 4) :
  |((Algebra.norm ℤ r : ℤ) : ℚ)| < |((Algebra.norm ℤ b : ℤ) : ℚ)|
```

Mathematical meaning. The displayed Lean declaration has type division-with-remainder norm bound: if `r = b·(frac)` and the golden norm form of the fractional part $\leq$ `3/4`, then `|N(r)| < |N(b)|`.

Implementation. `Common.abs_intNorm_lt_of_abs_norm_le` with `3/4 < 1`, rewriting the norm via `norm_eq_normForm`. – Uses (incl. Common): [abs_intNorm_lt_of_abs_norm_le, norm_eq_normForm, goldenRoot] – private –

```
theorem normEuclidean (a b : 𝒪 K) (hb : b ≠ 0) :
    ∃ q r : 𝒪 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

Mathematical meaning. $\mathbb{Z}[\varphi]$ is norm-Euclidean.

Implementation. take `q` by rounding the `(re, im)` coordinates of `a/b` in the $1, \varphi$ basis; the fractional parts `(s,t)` satisfy $|s^2+st-t^2| \leq 3/4$ (the golden norm form on $[-1/2,1/2]^2$, shown by `nlinarith` from `abs_sub_round`), then apply the bound lemma. – Uses (incl. Common): [goldenRoot, goldenIntO, norm_eq_normForm, abs_norm_int_lt_of_remainder_eq, natAbs_lt_of_abs_ratCast_lt] – public –

3.9 `LocalGlobal.lean`

```
def heightOneFive : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {sqrtFiveIntO}
  isPrime := by
    exact isPrime_span_sqrtFiveIntO
  ne_bot := by
    intro hbot
    have hzero : sqrtFiveIntO = 0 := by
      simpa [Ideal.span_singleton_eq_bot] using hbot
    have hsqrt : sqrtFiveIntO ^ 2 = (5 : 𝒪 K) := by
      apply Subtype.ext
      change (2 * goldenRoot - 1) ^ 2 = (5 : K)
      exact sqrtFiveRoot_sq
    rw [hzero] at hsqrt
    norm_num at hsqrt
```

Mathematical meaning. The displayed Lean declaration has type `HeightOneSpectrum (𝒪 K)` (asIdeal $(\sqrt{5})$) – the finite place over 5.

Implementation. `isPrime_span_sqrtFiveIntO`, `ne_bot` from $(\sqrt{5})=0 \Rightarrow 5=0$. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev vFive : NumberField.FinitePlace K :=
  NumberField.FinitePlace.mk heightOneFive
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example :
    (Ideal.span {(5 : ℤ)}).ramificationIdx heightOneFive.asIdeal = 2
```

```
example :
    (Ideal.span {(5 : ℤ)}).inertiaDeg heightOneFive.asIdeal = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. the PrimeSplitting lemmas. –

```
abbrev CompletionAtFive : Type :=
  heightOneFive.adicCompletion K
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev InfiniteCompletion (v : NumberField.InfinitePlace K) : Type :=
  v.Completion
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example :
    ((∏ w : NumberField.InfinitePlace K, w goldenRoot ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K, v goldenRoot) = 1

example :
    ((∏ w : NumberField.InfinitePlace K,
          w (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪 K)) ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K,
          v (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪 K))) = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `NumberField.prod_abs_eq_one` (nonzero from φ^2=φ+1 / `Units.ne_zero`). – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev SqrtFiveInfiniteAdeleRing : Type :=
  NumberField.InfiniteAdeleRing K
```

```
abbrev infinitePrincipalAdele_goldenRoot : SqrtFiveInfiniteAdeleRing :=
  algebraMap K SqrtFiveInfiniteAdeleRing goldenRoot

abbrev infinitePrincipalAdele_fundamentalUnit : SqrtFiveInfiniteAdeleRing
    :=
  algebraMap K SqrtFiveInfiniteAdeleRing (algebraMap (𝒪 K) K
    (fundamentalUnit : 𝒪 K))

example :
    ‖infinitePrincipalAdele_goldenRoot‖ = |Algebra.norm ℚ goldenRoot|

example :
    DenseRange (algebraMap K SqrtFiveInfiniteAdeleRing)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `coe_norm_eq_abs_norm`, `denseRange_algebraMap`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem canonicalEmbedding_goldenInt0_mem_integerLattice :
    NumberField.canonicalEmbedding K (algebraMap (𝒪 K) K goldenInt0) ∈
      NumberField.canonicalEmbedding.integerLattice K
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `simp [integerLattice]`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev mixedEmbedding_goldenRoot : NumberField.mixedEmbedding.mixedSpace
    K :=
  NumberField.mixedEmbedding K goldenRoot
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `Set.mem_prod`/`mem_pi`, totally-real reduces complex coordinates (`infinitePlace_isReal`). – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
example :
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone K)

example :
```

```
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone.normLeOne K)

theorem polarSpace_nonempty :
    Nonempty (NumberField.mixedEmbedding.polarSpace K)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. mathlib lemmas / `infer_instance`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
example :
    0 < NumberField.house goldenRoot
```

Mathematical meaning. archimedean size of the fundamental unit.

Implementation. every embedding sends φ to `(1±√5)/2` (`sq_eq_sq_iff_eq_or_eq_neg`); the conjugate `(1−√5)/2` has smaller norm; `le_antisymm` via `canonicalEmbedding.norm_le_iff` and `exists_realEmbedding_goldenRatio`. –

```
abbrev SqrtFiveAdeleRing : Type :=
  NumberField.AdeleRing (𝒪 K) K

abbrev principalAdele_goldenRoot : SqrtFiveAdeleRing :=
  algebraMap K SqrtFiveAdeleRing goldenRoot

abbrev principalAdele_fundamentalUnit : SqrtFiveAdeleRing :=
  algebraMap K SqrtFiveAdeleRing (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪
    K))

theorem principalAdele_goldenRoot_ne_zero : principalAdele_goldenRoot ≠ 0

theorem principalAdele_fundamentalUnit_is_unit_everywhere :
    ∀ v : NumberField.FinitePlace K,
      v (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪 K)) = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `AdeleRing.algebraMap_injective`; `norm_eq_one_iff_notMem` (if $\varphi \in \mathfrak{m}$ then $1 \in \mathfrak{m}$ via the inverse). –

```
abbrev sqrtFiveDedekindZeta (s : ℂ) : ℂ :=
  NumberField.dedekindZeta K s
```

```
abbrev sqrtFiveDedekindZetaResidue : ℝ :=
  NumberField.dedekindZeta_residue K

example :
    0 < sqrtFiveDedekindZetaResidue

example :
    sqrtFiveDedekindZetaResidue =
      (2 ^ NumberField.InfinitePlace.nrRealPlaces K *
          (2 * Real.pi) ^ NumberField.InfinitePlace.nrComplexPlaces K *
          NumberField.Units.regulator K * NumberField.classNumber K) /
        (NumberField.Units.torsionOrder K * Real.sqrt |NumberField.discr
    K|)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `dedekindZeta_residue_pos`, `dedekindZeta_residue_def`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

3.10 Summary for $\mathbb{Q}(\sqrt{5})$

Folder: `XYin/NumberField/FieldExamples/SqrtFive/` Field: real quadratic $\mathbb{Q}(\sqrt{5})$, built via `QuadraticAlgebra ℚ 1 1` (golden-ratio model, generator φ with φ^2 `= φ + 1`). Ring of integers $\mathbb{Z}[\varphi]$ `= ℤ[(1+√5)/2]`, disc `5` (smallest of any real quadratic field). Twin of SqrtTwo (`QuadraticAlgebra ℚ 2 0`); SqrtFive additionally carries a `RadicalModel = QuadraticAlgebra ℚ 5 0` (generator $\sqrt{5}$) and an `AdjoinRoot (X²−5)` presentation, plus the golden `AdjoinRoot (X²−X−1)` presentation, all proved equivalent. Namespace throughout: `NumberField.SqrtFive`. Files open `QuadraticAlgebra`, `NumberField`, `Module`, `Polynomial` (and `NumberField.FieldExamples` = Common where relevant). All core files are `noncomputable section` and remove `DivisionRing.toRatAlgebra`. Cross-reference – `Common.lean` API used: `eq_prime_of_dvd_prime_sq_of_ne_one_of_ne_sq`, `left_eq_prime_of_mul_eq_prime_sq_of_ne_one`, `exists_int_inverse_mod_prime`, `eq_of_le_of_absNorm_eq`, `natAbs_lt_of_abs_ratCast_lt`, `torsionPowerHom`, `rankOnePowerHom`, `abs_intNorm_lt_of_abs_norm_le`, `not_isUnramifiedAt_of_map_span_natCast_le_sq_of_not_le_pow_succ`, `not_map_span_natCast_le_pow_succ_of_absNorm_span_eq_pow`. Sorry-free (as of the 2026-06-15 update). The two former Units.lean sorries are now closed. `fundamentalUnit_generates` is proved by a self-contained real-embedding window-reduction argument: every unit maps under `reEmb : ℚ(√5) →+* ℝ` (sending $\varphi \mapsto \Phi$ `= (1+√5)/2`); a power of φ reduces it into the window `[1, Φ)`; and the norm form `x²+xy−y² = ±1` (`isUnit_iff_normForm`) forces the only unit there to be `1`. This is the standard textbook proof, formalised faithfully – it does not use the `Zsqrtd/Pell` $x^2-5y^2=1$ machinery (which only covers the index-2 suborder $\mathbb{Z}[\sqrt{5}]$), and needs no missing API. `regulator_eq` then follows from mature mathlib API (`regOfFamily_div_regulator` → `regOfFamily_eq_det` → `Matrix.det_eq_elem_of_subsingleton`). SqrtFive is the proof that, for a real quadratic field, ”this explicit unit is fundamental” is a matter of *intrinsic mathematical work*, not API

immaturity.

4 $\mathbb{Q}(\sqrt{-6})$

Imaginary quadratic field $\mathbb{Q}(\sqrt{-6})$, built via `QuadraticAlgebra ℚ (-6) 0` (radical model, generator θ with θ^2 `= -6`). Signature `(0,1)`; ring of integers $\mathbb{Z}[\sqrt{-6}]$ (maximal order, since `-6 ≡ 2 (mod 4)`); discriminant `-24`. The only non-PID case study: class number `2`, class group $\cong \mathbb{Z}/2$, *not* a UFD, *not* norm-Euclidean – the mathematically interesting negative results. Namespace `NumberField.SqrtNegSix`. Cross-refs to `Common.lean` via `NumberField.FieldExamples`. Import chain: `Basic` → `Galois` → `Embeddings` → `RingOfIntegers` → `PrimeSplitting` → `ClassGroup` → `Units` → `Euclidean` → `LocalGlobal`, plus special-topic files `CMField` and `UniqueFactorization`. Sorry-free (confirmed: zero `sorry`/`admit`).

Each block below begins with the displayed Lean declaration or object definition. For theorem, lemma, instance-proof, and example blocks the proof body is omitted; for definitions and abbreviations the displayed RHS is retained, because it is part of the mathematical object being documented.

4.1 `Basic.lean`

```
instance : Fact (∀ r : ℚ, r ^ 2 ≠ (-6 : ℚ) + (0 : ℚ) * r)
```

Mathematical meaning. Nonsquare hypothesis making `QuadraticAlgebra ℚ (-6) 0` a field (`-6` is not a rational square).

Implementation. `sq_nonneg r` + `nlinarith` (no real square root of a negative).

```
abbrev K : Type := QuadraticAlgebra ℚ (-6 : ℚ) 0
```

Mathematical meaning. the field $\mathbb{Q}(\sqrt{-6})$ (radical model).

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev θ : K := QuadraticAlgebra.omega
```

Mathematical meaning. distinguished $\sqrt{-6}$.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
instance : Field K
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `infer_instance` (uses the `Fact` above). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : FiniteDimensional ℚ K :=
  (QuadraticAlgebra.basis (-6 : ℚ) (0 : ℚ)).finiteDimensional_of_finite
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `QuadraticAlgebra.basis (-6) 0 |>.finiteDimensional_of_finite`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : NumberField K :=
  NumberField.of_module_finite ℚ K
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `NumberField.of_module_finite ℚ K`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem degree_eq_two : Module.finrank ℚ K = 2
```

Mathematical meaning. The displayed Lean declaration has type `Module.finrank ℚ K = 2`.

Implementation. `Module.finrank_eq_card_basis` on `QuadraticAlgebra.basis` + `Fintype.card_fin`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : Algebra.IsQuadraticExtension ℚ K where
  finrank_eq_two' := degree_eq_two
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. This instance is supplied by `degree_eq_two` after the concrete field has been shown to satisfy its hypotheses.

```
theorem theta_sq : θ ^ 2 = -6
```

Mathematical meaning. The displayed Lean declaration has type `θ ^ 2 = -6`.

Implementation. `sq` + `QuadraticAlgebra.omega_mul_omega_eq_add` + `simp`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtNegSixPolynomial : Polynomial ℚ :=
  Polynomial.X ^ 2 + Polynomial.C (6 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `ℚ[X] := X^2 + C 6` – defining polynomial X^2+6.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
instance : Fact (Irreducible sqrtNegSixPolynomial)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `irreducible_of_degree_le_three_of_not_isRoot` (degree 2, no rational root since $x^2+6=0$ impossible via `sq_nonneg+nlinarith`).

```
abbrev KAdjoinRoot : Type := AdjoinRoot sqrtNegSixPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `AdjoinRoot sqrtNegSixPolynomial`. Alternate presentation.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev thetaAdjoinRoot : KAdjoinRoot := AdjoinRoot.root
    sqrtNegSixPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `AdjoinRoot.root sqrtNegSixPolynomial`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem thetaAdjoinRoot_sq :
    thetaAdjoinRoot ^ 2 = (-6 : KAdjoinRoot)
```

Mathematical meaning. The displayed Lean declaration has type `thetaAdjoinRoot ^ 2 = (-6 : KAdjoinRoot)`.

Implementation. `AdjoinRoot.eval₂_root` + `linear_combination`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem sqrtNegSixPolynomial_aeval_theta :
    Polynomial.eval₂ (Algebra.ofId ℚ K) θ sqrtNegSixPolynomial = 0
```

Mathematical meaning. The displayed Lean declaration has type `eval₂ (Algebra.ofId ℚ K) θ sqrtNegSixPolynomial = 0`.

Implementation. `calc` to θ^2+6 then `theta_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem thetaAdjoinRoot_mul_self :
    thetaAdjoinRoot * thetaAdjoinRoot =
      (-6 : ℚ) • (1 : KAdjoinRoot) + (0 : ℚ) • thetaAdjoinRoot
```

Mathematical meaning. The displayed Lean declaration has type `thetaAdjoinRoot * thetaAdjoinRoot = (-6)•1 + 0•thetaAdjoinRoot`.

Implementation. `thetaAdjoinRoot_sq` + `simp`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def adjoinRootEquivQuadratic : KAdjoinRoot ≃ₐ[ℚ] K := by
  let f : KAdjoinRoot →ₐ[ℚ] K :=
    AdjoinRoot.liftAlgHom sqrtNegSixPolynomial (Algebra.ofId ℚ K) θ
      sqrtNegSixPolynomial_aeval_theta
  let g : K →ₐ[ℚ] KAdjoinRoot :=
    QuadraticAlgebra.lift ⟨thetaAdjoinRoot, thetaAdjoinRoot_mul_self⟩
  have hfroot : f thetaAdjoinRoot = θ := by
    change (AdjoinRoot.liftAlgHom sqrtNegSixPolynomial (Algebra.ofId ℚ K)
    θ
        sqrtNegSixPolynomial_aeval_theta) (AdjoinRoot.root
    sqrtNegSixPolynomial) = θ
    exact AdjoinRoot.liftAlgHom_root sqrtNegSixPolynomial (Algebra.ofId ℚ
    K) θ
      sqrtNegSixPolynomial_aeval_theta
  have hgroot : g θ = thetaAdjoinRoot := by
    change (QuadraticAlgebra.omega : K).re • (1 : KAdjoinRoot) +
      (QuadraticAlgebra.omega : K).im • thetaAdjoinRoot = thetaAdjoinRoot
    simp
  refine AlgEquiv.ofAlgHom f g ?_ ?_
  · apply QuadraticAlgebra.algHom_ext
    change f (g θ) = θ
    rw [hgroot]
    exact hfroot
  · apply AdjoinRoot.algHom_ext
    change g (f thetaAdjoinRoot) = thetaAdjoinRoot
    rw [hfroot]
    exact hgroot
```

Mathematical meaning. equivalence of AdjoinRoot and quadratic-algebra presentations.

Implementation. `AdjoinRoot.liftAlgHom` and `QuadraticAlgebra.lift`; inverses via `algHom_ext` on both sides.

```
theorem adjoinRootEquivQuadratic_root :
    adjoinRootEquivQuadratic thetaAdjoinRoot = θ
```

Mathematical meaning. The displayed Lean declaration has type `adjoinRootEquivQuadratic thetaAdjoinRoot = θ`.

Implementation. The proof applies the packaged result `AdjoinRoot.liftAlgHom_root`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

4.2 `Galois.lean`

```
noncomputable def idAut : K ≃ₐ[ℚ] K := AlgEquiv.refl
```

Mathematical meaning. The displayed Lean declaration has type `K ≃ₐ[ℚ] K := AlgEquiv.refl`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private noncomputable def conjugationHom : K →ₐ[ℚ] K :=
  QuadraticAlgebra.lift ⟨-QuadraticAlgebra.omega, by
    have h := QuadraticAlgebra.omega_mul_omega_eq_add
      (R := ℚ) (a := (-6 : ℚ)) (b := (0 : ℚ))
    simpa [neg_mul_neg] using h⟩
```

Mathematical meaning. $\theta \mapsto -\theta$ via `QuadraticAlgebra.lift ⟨-omega, and related terms⟩`.

Implementation. relation from `omega_mul_omega_eq_add` + `neg_mul_neg`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem conjugationHom_omega : conjugationHom θ = -θ
```

Mathematical meaning. The displayed Lean declaration has type `conjugationHom θ = -θ`.

Implementation. `change` to component form + `simp`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem conjugationHom_conjugationHom :
    conjugationHom.comp conjugationHom = AlgHom.id ℚ K
```

Mathematical meaning. The displayed Lean declaration has type `conjugationHom.comp conjugationHom = AlgHom.id`.

Implementation. `QuadraticAlgebra.algHom_ext` + `conjugationHom_omega`, `neg_neg`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def conjugation : K ≃ₐ[ℚ] K :=
  AlgEquiv.ofAlgHom conjugationHom conjugationHom
    conjugationHom_conjugationHom conjugationHom_conjugationHom
```

Mathematical meaning. nontrivial conjugation $\theta \mapsto -\theta$ (complex conj. under any embedding).

Implementation. The displayed RHS is the packaged construction `AlgEquiv.ofAlgHom` specialized to this field.

`theorem conjugation_apply_theta : conjugation` θ `= -`θ

Mathematical meaning. The displayed Lean declaration has type `conjugation` θ `= -`θ.

Implementation. The proof applies the packaged result `conjugationHom_omega`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

`theorem isGalois : IsGalois` $\mathbb{Q}$ `K`

Mathematical meaning. The displayed Lean declaration has type `IsGalois` $\mathbb{Q}$ `K`.

Implementation. `infer_instance` (separable + normal for quadratic). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

`theorem aut_eq_or (`σ `: K` $\simeq_a$`[`$\mathbb{Q}$`] K) :` σ `= idAut` $\vee$ σ `= conjugation`

Mathematical meaning. only two automorphisms.

Implementation. $(\sigma\theta)^2$`=-6` $\Rightarrow$ $(\sigma\theta-\theta)(\sigma\theta+\theta)$`=0`; `QuadraticAlgebra.algHom_ext` pins down σ by its value on θ.

`private theorem conjugation_ne_id : conjugation` $\neq$ `idAut`

Mathematical meaning. The displayed Lean declaration has type `conjugation` $\neq$ `idAut`.

Implementation. would give 2θ`=0`; rule out via `theta_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

`theorem prod_auts (z : K) :` $\prod$ σ `: K` $\simeq_a$`[`$\mathbb{Q}$`] K,` σ `z = z *  conjugation z`

Mathematical meaning. The displayed Lean declaration has type $\prod$ σ`, `σ `z = z *  conjugation z`.

Implementation. `Finset.univ = {idAut, conjugation}` (from `aut_eq_or`), `prod_insert`/`prod_singleton`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

`theorem galoisGroupStructure : Nonempty ((K` $\simeq_a$`[`$\mathbb{Q}$`] K)` $\simeq*$ `Multiplicative (ZMod 2))`

Mathematical meaning. The displayed Lean declaration has type `Nonempty ((K` $\simeq_a$`[`$\mathbb{Q}$`] K)` $\simeq*$ `Multiplicative (ZMod 2))`.

Implementation. `Nat.card = 2` (from `aut_eq_or`), `mulEquivOfPrimeCardEq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

4.3 Embeddings.lean

```
theorem isTotallyComplex : NumberField.IsTotallyComplex K
```

Mathematical meaning. every infinite place is complex.

Implementation. under a real embedding `z=`$\sigma\theta$ satisfies `z`^{2}`=-6` with `z.im=0`, contradicting `z.re`^{2}`<0` via `nlinarith`.

```
theorem nrRealPlaces_eq_zero : NumberField.InfinitePlace.nrRealPlaces K =
    0
```

Mathematical meaning. The displayed Lean declaration has type `nrRealPlaces K = 0`.

Implementation. The proof applies the packaged result `IsTotallyComplex.nrRealPlaces_eq_zero`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem nrComplexPlaces_eq_one :
    NumberField.InfinitePlace.nrComplexPlaces K = 1
```

Mathematical meaning. The displayed Lean declaration has type `nrComplexPlaces K = 1`.

Implementation. `card_add_two_mul_card_eq_rank` + `nrRealPlaces_eq_zero` + `degree_eq_two` + `omega`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_card_eq_one :
    Fintype.card (NumberField.InfinitePlace K) = 1
```

Mathematical meaning. The displayed Lean declaration has type `Fintype.card (InfinitePlace K) = 1`.

Implementation. The proof applies the packaged result `card_eq_nrRealPlaces_add_nrComplexPlaces`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem infinitePlace_isComplex (v : NumberField.InfinitePlace K) :
    NumberField.InfinitePlace.IsComplex v
```

Mathematical meaning. The displayed Lean declaration has type `∀ v, IsComplex v`.

Implementation. The proof applies the packaged result `isTotallyComplex.isComplex`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem exists_complexEmbedding_theta_pos :
    ∃ σ : K →+* ℂ, σ θ = Complex.I * (Real.sqrt 6 : ℂ)
```

Mathematical meaning. The displayed Lean declaration has type `∃ σ : K →+* ℂ, σ θ = I·√6`.

Implementation. `QuadraticAlgebra.lift ⟨I·√6, and related terms⟩`; `(I·√6)²=−6` via `I_mul_I+sq_sqrt`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem exists_complexEmbedding_theta_neg :
    ∃ σ : K →+* ℂ, σ θ = -Complex.I * (Real.sqrt 6 : ℂ)
```

Mathematical meaning. The displayed Lean declaration has type `∃ σ : K →+* ℂ, σ θ = −I·√6`.

Implementation. twin of pos, witness `−I·√6`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

4.4 `RingOfIntegers.lean`

```
theorem theta_integral : θ ∈ integralClosure ℤ K
```

Mathematical meaning. The displayed Lean declaration has type `θ ∈ integralClosure ℤ K`.

Implementation. root of monic `X²+6` (`monicity! + theta_sq`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def thetaInt : 𝒪 K := ⟨θ, theta_integral⟩
```

Mathematical meaning. $\sqrt{-6}$ in the ring of integers.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem thetaInt_sq : thetaInt ^ 2 = -6
```

Mathematical meaning. The displayed Lean declaration has type `thetaInt ^ 2 = −6`.

Implementation. `Subtype.ext + theta_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def integralBasisVectors : Fin 2 → K := ![1, θ]
```

Mathematical meaning. The displayed Lean declaration has type `Fin 2 → K := ![1, θ]`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
local notation "𝒫₂" => Submodule.span ℤ ({(2 : ℤ)} : Set ℤ)

local notation "𝒫₃" => Submodule.span ℤ ({(3 : ℤ)} : Set ℤ)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private theorem quadraticBasis_zero : (QuadraticAlgebra.basis (-6 : ℚ) (0
    : ℚ)) 0 = (1
    : K)

private theorem quadraticBasis_one : (QuadraticAlgebra.basis (-6 : ℚ) (0
    : ℚ)) 1 = θ
```

Mathematical meaning. The displayed Lean declaration has type `QuadraticAlgebra.basis (-6) 0` sends $0 \mapsto 1$, $1 \mapsto \theta$.

Implementation. `change+simp [basis, ofEquivFun, linearEquivTuple]`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def sqrtNegSixPowerBasis : PowerBasis ℚ K where
  gen := θ
  dim := 2
  basis := QuadraticAlgebra.basis (-6 : ℚ) (0 : ℚ)
  basis_eq_pow := by
    intro i
    fin_cases i
    · exact quadraticBasis_zero
    · simpa [pow_one] using quadraticBasis_one
```

Mathematical meaning. The displayed Lean declaration has type `PowerBasis ℚ K` with `gen`=θ, `dim`=2.

Implementation. `basis_eq_pow` via the two `quadraticBasis_*`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtNegSixIntegerPolynomial : ℤ[X] := Polynomial.X ^ 2 +
    Polynomial.C (6 : ℤ)
```

Mathematical meaning. The displayed Lean declaration has type `ℤ[X] := X^2 + C 6`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtNegSix_minpoly_rat : minpoly ℚ θ = sqrtNegSixPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `minpoly ℚ θ = sqrtNegSixPolynomial`.

Implementation. `minpoly.eq_of_irreducible_of_monic` (Fact instance + `theta_sq` + `monicity!`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtNegSix_minpoly_int : minpoly ℤ θ = sqrtNegSixIntegerPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `minpoly ℤ θ = sqrtNegSixIntegerPolynomial`.

Implementation. `map_injective` + `minpoly.isIntegrallyClosed_eq_field_fractions'` + `sqrtNegSix_minpoly_rat`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtNegSix_minpoly_int_isEisensteinAt_two :
    (minpoly ℤ θ).IsEisensteinAt 𝒫₂
```

Mathematical meaning. The displayed Lean declaration has type `(minpoly ℤ θ).IsEisensteinAt` $\mathcal{P}_2$.

Implementation. `monic_X_pow_add_C.isEisensteinAt_of_mem_of_notMem` ($6\in$`(2)`, $6\notin$`(2)`^{2}`=(4)`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtNegSix_minpoly_int_isEisensteinAt_three :
    (minpoly ℤ θ).IsEisensteinAt 𝒫₃
```

Mathematical meaning. The displayed Lean declaration has type `(minpoly ℤ θ).IsEisensteinAt` $\mathcal{P}_3$.

Implementation. twin at 3 ($6\in$`(3)`, $6\notin$`(9)`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem algebraNorm_eq_quadraticNorm (z : K) :
    Algebra.norm ℚ z = QuadraticAlgebra.norm z
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.norm ℚ z = QuadraticAlgebra.norm z`.

Implementation. `norm_apply` + `det_toLinearMap_eq_norm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtNegSixPowerBasis_discr :
    Algebra.discr ℚ sqrtNegSixPowerBasis.basis = (-24 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℚ sqrtNegSixPowerBasis.basis = −24`.

Implementation. `discr_powerBasis_eq_norm`; `minpoly' = 2X`, `aeval = 2`θ, `norm(2`θ`)=24` via the quadratic norm form.

```
private theorem integralClosure_eq_adjoin :
    integralClosure ℤ K = Algebra.adjoin ℤ ({θ} : Set K)
```

Mathematical meaning. maximality of $\mathbb{Z}[\theta]$.

Implementation. `discr_mul_isIntegral_mem_adjoin` gives `24z ∈ adjoin`; strip 2^3 and 3 via `mem_adjoin_of_smul_prime_pow_smul_of_minpoly_isEisensteinAt` at both Eisenstein primes.

```
private theorem thetaInt_adjoin_eq_top : Algebra.adjoin ℤ ({thetaInt} :
    Set (𝒪 K)) = ⊤
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.adjoin ℤ {thetaInt} = ⊤`.

Implementation. pull back `integralClosure_eq_adjoin` through `adjoin_singleton_eq_range_aeval`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem thetaInt_minpoly : minpoly ℤ thetaInt =
    sqrtNegSixIntegerPolynomial
```

Mathematical meaning. The displayed Lean declaration has type `minpoly ℤ thetaInt = sqrtNegSixIntegerPolynomial`.

Implementation. `minpoly.isIntegrallyClosed_eq_field_fractions + sqrtNegSix_minpoly_rat`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private noncomputable def sqrtNegSixExplicitIntegralPowerBasis :
    PowerBasis ℤ (𝒪 K) :=
  PowerBasis.ofAdjoinEqTop' (Algebra.IsIntegral.isIntegral thetaInt)
    thetaInt_adjoin_eq_top
```

Mathematical meaning. The displayed Lean declaration has type `PowerBasis ℤ (𝒪 K)`.

Implementation. The displayed RHS is the packaged construction `PowerBasis.ofAdjoinEqTop'` specialized to this field.

```
private theorem sqrtNegSixExplicitIntegralPowerBasis_dim :
    sqrtNegSixExplicitIntegralPowerBasis.dim = 2
```

Mathematical meaning. The displayed Lean declaration has type `dim = 2`.

Implementation. `ofAdjoinEqTop'_dim` + `thetaInt_minpoly` + `natDegree_X_pow_add_C`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private noncomputable def sqrtNegSixExplicitIntegralBasis : Basis (Fin 2)
    ℤ (𝒪 K) :=
  sqrtNegSixExplicitIntegralPowerBasis.basis.reindex
    (finCongr sqrtNegSixExplicitIntegralPowerBasis_dim)
```

Mathematical meaning. The displayed Lean declaration has type `Basis (Fin 2) ℤ (𝒪 K)`.

Implementation. reindex the power basis along `dim=2`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem sqrtNegSixExplicitIntegralBasis_zero :
    sqrtNegSixExplicitIntegralBasis 0 = 1

private theorem sqrtNegSixExplicitIntegralBasis_one :
    sqrtNegSixExplicitIntegralBasis 1 = thetaInt
```

Mathematical meaning. The displayed Lean declaration has type basis sends `0`↦`1`, `1`↦`thetaInt`.

Implementation. `basis_eq_pow` + `ofAdjoinEqTop'_gen`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private noncomputable def integralBasisVectorsInt : Fin 2 → 𝒪 K := ![1,
    thetaInt]
```

Mathematical meaning. The displayed Lean declaration has type `Fin 2 → 𝒪 K := ![1, thetaInt]`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem algebraMap_integralBasisVectorsInt :
    (fun i => (algebraMap (𝒪 K) K) (integralBasisVectorsInt i)) =
    integralBasisVectors
```

Mathematical meaning. The displayed Lean declaration has type maps to `integralBasisVectors` in `K`.

Implementation. `fin_cases+simp`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private noncomputable def ringOfIntegersLinearMap : 𝒪 K →ₗ[ℤ] K :=
  (Algebra.linearMap (𝒪 K) K).restrictScalars ℤ
```

Mathematical meaning. The displayed Lean declaration has type `𝒪 K →ₗ[ℤ] K`.

Implementation. The displayed RHS is the packaged construction `Algebra.linearMap.restrictScalars` specialized to this field.

```
private theorem span_integralBasisVectorsInt_eq_top :
    Submodule.span ℤ (Set.range integralBasisVectorsInt) = ⊤
```

Mathematical meaning. The displayed Lean declaration has type `span ℤ (range integralBasisVectorsInt) = ⊤`.

Implementation. `Basis.span_eq` of the explicit basis. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem sqrtNegSixExplicitIntegralBasis_discr :
    Algebra.discr ℤ sqrtNegSixExplicitIntegralBasis = (-24 : ℤ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℤ and related terms = -24`.

Implementation. `discr_localizationLocalization` from $\mathbb{Z}\to\mathbb{Q}$, transfer to `sqrtNegSixPowerBasis_discr` via `Basis.localizationLocalization`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_integralBasisVectors_eq :
    Submodule.span ℤ (Set.range integralBasisVectors) =
      Subalgebra.toSubmodule (integralClosure ℤ K)
```

Mathematical meaning. $1, \theta$ is an integral basis.

Implementation. push `span_integralBasisVectorsInt_eq_top` through `ringOfIntegersLinearMap`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def concreteIntegralBasis : Basis (Fin 2) ℤ (𝒪 K) := by
  exact sqrtNegSixExplicitIntegralBasis
```

Mathematical meaning. The displayed Lean declaration has type `Basis (Fin 2) ℤ (𝒪 K) := sqrtNegSixExplicitIntegralBasis`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem concreteIntegralBasis_apply (i : Fin 2) :
    (algebraMap (𝒪 K) K) (concreteIntegralBasis i) = integralBasisVectors
    i
```

Mathematical meaning. The displayed Lean declaration has type `algebraMap (concreteIntegralBasis i) = integralBasisVectors i`.

Implementation. `fin_cases+simp`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem exists_eq_smul_one_add_smul_theta (z : 𝒪 K) :
    ∃ x y : ℤ, z = x • (1 : 𝒪 K) + y • thetaInt
```

Mathematical meaning. The displayed Lean declaration has type `∀ z:𝒪 K, ∃ x y:ℤ, z = x•1 + y•thetaInt`.

Implementation. `concreteIntegralBasis.repr` + `sum_repr`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem linearIndependent_one_theta : LinearIndependent ℤ ![(1 : 𝒪 K),
    thetaInt]
```

Mathematical meaning. The displayed Lean declaration has type `LinearIndependent ℤ ![1, thetaInt]`.

Implementation. transport `concreteIntegralBasis.linearIndependent`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_integralBasisVectors_eq_integralBasis :
    Submodule.span ℤ (Set.range integralBasisVectors) =
      Submodule.span ℤ (Set.range (NumberField.integralBasis K))
```

Mathematical meaning. The displayed Lean declaration has type span of `1,`θ = span of mathlib's `NumberField.integralBasis K`.

Implementation. `span_integralBasisVectors_eq` + `mem_span_integralBasis`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev concreteRingOfIntegers : Subalgebra ℤ K := Algebra.adjoin ℤ ({θ} :
    Set K)
```

Mathematical meaning. the order $\mathbb{Z}[\theta]$.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem ringOfIntegers_eq : integralClosure ℤ K = concreteRingOfIntegers
```

Mathematical meaning. The displayed Lean declaration has type `integralClosure ℤ K = concreteRingOfIntegers`.

Implementation. The proof applies the packaged result `integralClosure_eq_adjoin`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem numberField_discr_eq : NumberField.discr K = (-24 : ℤ)
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.discr K = -24`.

Implementation. `discr_eq_discr K concreteIntegralBasis` + `sqrtNegSixExplicitIntegralBasis_discr`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem concreteIntegralBasis_discr :
    Algebra.discr ℤ concreteIntegralBasis = (-24 : ℤ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℤ concreteIntegralBasis = -24`.

Implementation. The proof applies the packaged result `sqrtNegSixExplicitIntegralBasis_discr`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem integralBasis_discr :
    Algebra.discr ℚ (NumberField.integralBasis K) = (-24 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℚ (NumberField.integralBasis K) = -24`.

Implementation. `coe_discr` + `numberField_discr_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem norm_eq_normForm (x y : ℚ) :
    Algebra.norm ℚ (x • (1 : K) + y • θ) = x ^ 2 + 6 * y ^ 2
```

Mathematical meaning. the (positive-definite) norm form.

Implementation. `det_toLinearMap_eq_norm` + `QuadraticAlgebra.norm_def`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem intNorm_eq_normForm (x y : ℤ) :
    Algebra.norm ℤ (x • (1 : 𝒪 K) + y • thetaInt) = x ^ 2 + 6 * y ^ 2
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.norm ℤ (x•1 + y•thetaInt) = x^2 + 6y^2 (x,y:ℤ)`.

Implementation. `coe_norm_int` cast from `norm_eq_normForm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem normForm_ne_two (x y : ℤ) : x ^ 2 + 6 * y ^ 2 ≠ 2
```

Mathematical meaning. The displayed Lean declaration has type $x^2 + 6y^2 \neq 2$.

Implementation. forces `y=0`, x^2=2, `interval_cases`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem normForm_ne_three (x y : ℤ) : x ^ 2 + 6 * y ^ 2 ≠ 3
```

Mathematical meaning. The displayed Lean declaration has type $x^2 + 6y^2 \neq 3$.

Implementation. forces $y=0$, $x^2=3$, `interval_cases`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem normForm_ne_five (x y : ℤ) : x ^ 2 + 6 * y ^ 2 ≠ 5
```

Mathematical meaning. The displayed Lean declaration has type $x^2 + 6y^2 \neq 5$.

Implementation. forces $y=0$, $x^2=5$, `interval_cases`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isUnit_iff_normForm (x y : ℤ) :
    IsUnit (x • (1 : 𝒪 K) + y • thetaInt) ↔ x ^ 2 + 6 * y ^ 2 = 1
```

Mathematical meaning. The displayed Lean declaration has type `IsUnit (x•1 + y•thetaInt)` $\leftrightarrow$ $x^2+6y^2 = 1$.

Implementation. `isUnit_iff_norm` + `intNorm_eq_normForm`; positive-definite $\Rightarrow$ no -1 case. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem intNorm_nonneg (z : 𝒪 K) : 0 ≤ Algebra.norm ℤ z
```

Mathematical meaning. The displayed Lean declaration has type $0 \leq$ `Algebra.norm` $\mathbb{Z}$ `z`.

Implementation. reduce to $x^2+6y^2 \geq 0$. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isUnit_iff_intNorm_eq_one (z : 𝒪 K) :
    IsUnit z ↔ Algebra.norm ℤ z = 1
```

Mathematical meaning. The displayed Lean declaration has type `IsUnit z` $\leftrightarrow$ `Algebra.norm` $\mathbb{Z}$ `z = 1`.

Implementation. reduce to `isUnit_iff_normForm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem intNorm_ne_two (z : 𝒪 K) : Algebra.norm ℤ z ≠ 2

theorem intNorm_ne_three (z : 𝒪 K) : Algebra.norm ℤ z ≠ 3

theorem intNorm_ne_five (z : 𝒪 K) : Algebra.norm ℤ z ≠ 5
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.norm` $\mathbb{Z}$ `z` $\neq$ `2 / 3 / 5`.

Implementation. reduce via coordinates to `normForm_ne_*`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

4.5 PrimeSplitting.lean

```
private theorem absNorm_span_natCast (p : ℕ) :
    Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K)}) = p ^ 2
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (span {(p:ℕ):𝒪 K}) = p`^{2}.

Implementation. `absNorm_span_natCast` + `RingOfIntegers.rank` + `degree_eq_two`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem unit_theta_coeff_not_mem (p : ℕ) (hp : 2 ≤ p) (A B : ℤ)
    (hB : B = 1 ∨ B = -1) :
    ((A : ℤ) : 𝒪 K) + ((B : ℤ) : 𝒪 K) * thetaInt ∉
      Ideal.span {((p : ℕ) : 𝒪 K)}
```

Mathematical meaning. The displayed Lean declaration has type `A + Bθ` with `B=±1` is never in `(p)` for `p≥2`.

Implementation. coordinates via `exists_eq_smul_one_add_smul_theta`; `linearIndependent_one_theta` gives `p∣1` contradiction. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem isSquare_neg_six_iff_mod_twentyfour
    (p : ℕ) (hp : p.Prime) (hp2 : p ≠ 2) (hp3 : p ≠ 3) :
    IsSquare ((-6 : ℤ) : ZMod p) ↔
      p % 24 = 1 ∨ p % 24 = 5 ∨ p % 24 = 7 ∨ p % 24 = 11
```

Mathematical meaning. The displayed Lean declaration has type `IsSquare ((-6:ℤ):ZMod p) ↔ p%24 ∈ {1,5,7,11}`.

Implementation. `jacobiSym.mod_right`, case-split `p%24`, `nonsquare_iff_jacobiSym_eq_neg_one` / `isSquare_of_jacobiSym_eq_one`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def primeAboveTwo : Ideal (𝒪 K) := Ideal.span {(2 : 𝒪 K), thetaInt}
```

Mathematical meaning. The displayed Lean declaration has type `Ideal (𝒪 K) := span {2, thetaInt}` = `(2,θ)`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
def primeAboveThree : Ideal (𝒪 K) := Ideal.span {(3 : 𝒪 K), thetaInt}
```

Mathematical meaning. The displayed Lean declaration has type `span {3, thetaInt}` = `(3,`θ`)`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem span_two_eq : Ideal.span {(2 : 𝒪 K)} = primeAboveTwo ^ 2
```

Mathematical meaning. 2 ramifies.

Implementation. `span_pair_mul_span_pair`, `le_antisymm`, `thetaInt_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isPrime_primeAboveTwo : primeAboveTwo.IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `primeAboveTwo.IsPrime`.

Implementation. `isPrime_of_irreducible_absNorm`, `absNorm=2` prime. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_primeAboveTwo : Ideal.absNorm primeAboveTwo = 2
```

Mathematical meaning. The displayed Lean declaration has type `absNorm primeAboveTwo = 2`.

Implementation. `absNorm(2)=4`, `span_two_eq` $\Rightarrow$ `absNorm`^{2}`=4`, `nlinarith`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_primeAboveTwo : primeAboveTwo.LiesOver (Ideal.span {(2 :
    ℤ)})
```

Mathematical meaning. The displayed Lean declaration has type `primeAboveTwo.LiesOver (span {(2:`$\mathbb{Z}$`)})`.

Implementation. `liesOver_span_iff` + `2`$\in$`(2,`θ`)`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem inertiaDeg_primeAboveTwo :
    (Ideal.span {(2 : ℤ)}).inertiaDeg primeAboveTwo = 1
```

Mathematical meaning. The displayed Lean declaration has type `inertiaDeg = 1`.

Implementation. `absNorm_eq_pow_inertiaDeg'` + `Prime.pow_eq_iff`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramificationIdx_primeAboveTwo :
    (Ideal.span {(2 : ℤ)}).ramificationIdx primeAboveTwo = 2
```

Mathematical meaning. The displayed Lean declaration has type `ramificationIdx = 2`.

Implementation. `ramificationIdx_spec` ($(2)\leq P^2$ from `span_two_eq`; $\neg(2)\leq P^3$ via norm $8\nmid 4$). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem not_isPrincipal_primeAboveTwo : ¬ Submodule.IsPrincipal
    primeAboveTwo
```

Mathematical meaning. THE central negative result – exhibits the nontrivial ideal class.

Implementation. a principal generator would have $|\text{norm}|=\text{absNorm}=2$, but $x^2+6y^2=2$ is unsolvable (`normForm_ne_two`).

```
theorem span_three_eq : Ideal.span {(3 : 𝒪 K)} = primeAboveThree ^ 2
```

Mathematical meaning. The displayed Lean declaration has type $(3) = \text{primeAboveThree}^2$.

Implementation. twin of `span_two_eq` ($-2\cdot\theta^2$ term). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isPrime_primeAboveThree : primeAboveThree.IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `primeAboveThree.IsPrime`.

Implementation. $\text{absNorm}=3$ prime. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_primeAboveThree : Ideal.absNorm primeAboveThree = 3
```

Mathematical meaning. The displayed Lean declaration has type `absNorm primeAboveThree = 3`.

Implementation. $\text{absNorm}(3)=9$, `nlinarith`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_primeAboveThree : primeAboveThree.LiesOver (Ideal.span
    {(3 : ℤ)})

theorem inertiaDeg_primeAboveThree :
    (Ideal.span {(3 : ℤ)}).inertiaDeg primeAboveThree = 1

theorem ramificationIdx_primeAboveThree :
    (Ideal.span {(3 : ℤ)}).ramificationIdx primeAboveThree = 2
```

Mathematical meaning. The displayed Lean declaration has type lies over (3); $\text{inertiaDeg}=1$; $\text{ramificationIdx}=2$.

Implementation. twins of the `Two` versions. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_theta_eq : Ideal.span {thetaInt} = primeAboveTwo *
    primeAboveThree
```

Mathematical meaning. the principal ideal $(\sqrt{-6})$ factors as the product of the two ramified primes.

Implementation. `span_pair_mul_span_pair` + `thetaInt_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ideal_eq_primeAboveTwo_of_absNorm_eq_two
    (I : Ideal (𝒪 K)) (hI : Ideal.absNorm I = 2) : I = primeAboveTwo
```

Mathematical meaning. The displayed Lean declaration has type `absNorm I = 2 ⇒ I = primeAboveTwo`.

Implementation. $2\in$`I`, $\theta\in$`I` (from $\theta^2=-6\in$`I` in $\mathcal{O}$`K/I`), then `eq_of_le_of_absNorm_eq` (Common).

```
theorem ideal_eq_primeAboveThree_of_absNorm_eq_three
    (I : Ideal (𝒪 K)) (hI : Ideal.absNorm I = 3) : I = primeAboveThree
```

Mathematical meaning. The displayed Lean declaration has type `absNorm I = 3 ⇒ I = primeAboveThree`.

Implementation. The proof applies the packaged result `twin`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem seven_eq_mul : (7 : 𝒪 K) = (1 + thetaInt) * (1 - thetaInt)
```

Mathematical meaning. The displayed Lean declaration has type $7 = (1+\theta)(1-\theta)$.

Implementation. `ring_nf`+`thetaInt_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_seven_eq :
    Ideal.span {(7 : 𝒪 K)} = Ideal.span {1 + thetaInt} * Ideal.span {1 -
    thetaInt}
```

Mathematical meaning. The displayed Lean declaration has type $(7) = (1+\theta)(1-\theta)$ (principal factors).

Implementation. `span_singleton_mul_span_singleton` + `seven_eq_mul`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_span_one_add_theta : Ideal.absNorm (Ideal.span {1 +
    thetaInt}) = 7

theorem isPrime_span_one_add_theta : (Ideal.span {1 + thetaInt}).IsPrime

theorem absNorm_span_one_sub_theta : Ideal.absNorm (Ideal.span {1 -
    thetaInt}) = 7
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (1±θ) = 7`.

Implementation. `absNorm_span_singleton` + `intNorm_eq_normForm 1 (±1)`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isPrime_span_one_add_theta : (Ideal.span {1 + thetaInt}).IsPrime

theorem absNorm_span_one_sub_theta : Ideal.absNorm (Ideal.span {1 -
    thetaInt}) = 7

theorem isPrime_span_one_sub_theta : (Ideal.span {1 - thetaInt}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `(1±θ).IsPrime`.

Implementation. `isPrime_of_irreducible_absNorm` (norm 7). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_span_one_add_theta :
    (Ideal.span {1 + thetaInt}).LiesOver (Ideal.span {(7 : ℤ)})

theorem liesOver_span_one_sub_theta :
    (Ideal.span {1 - thetaInt}).LiesOver (Ideal.span {(7 : ℤ)})
```

Mathematical meaning. The displayed Lean declaration has type lie over `(7)`.

Implementation. `7∈(1±θ)` via `seven_eq_mul`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem inertiaDeg_span_one_add_theta :
    (Ideal.span {(7 : ℤ)}).inertiaDeg (Ideal.span {1 + thetaInt}) = 1

theorem inertiaDeg_span_one_sub_theta :
    (Ideal.span {(7 : ℤ)}).inertiaDeg (Ideal.span {1 - thetaInt}) = 1
```

Mathematical meaning. The displayed Lean declaration has type `inertiaDeg = 1`.

Implementation. The proof applies the packaged result `absNorm_eq_pow_inertiaDeg'`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
def primeAboveFiveSubTwo : Ideal (𝒪 K) := Ideal.span {(5 : 𝒪 K), thetaInt
    - 2}

def primeAboveFiveAddTwo : Ideal (𝒪 K) := Ideal.span {(5 : 𝒪 K), thetaInt
    + 2}
```

Mathematical meaning. the *non-principal* primes above the split prime 5.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem span_five_eq :
    Ideal.span {(5 : 𝒪 K)} =
      Ideal.span {(5 : 𝒪 K), thetaInt - 2} * Ideal.span {(5 : 𝒪 K),
    thetaInt + 2}
```

Mathematical meaning. The displayed Lean declaration has type $(5) = (5,\theta-2)(5,\theta+2)$.

Implementation. `span_pair_mul_span_pair` + `thetaInt_sq` ($(\theta-2)(\theta+2)=\theta^2-4=-10$). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem span_five_theta_sub_two_ne_top :
    Ideal.span {(5 : 𝒪 K), thetaInt - 2} ≠ ⊤

private theorem span_five_theta_add_two_ne_top :
    Ideal.span {(5 : 𝒪 K), thetaInt + 2} ≠ ⊤
```

Mathematical meaning. The displayed Lean declaration has type $(5,\theta\mp2) \neq \top$.

Implementation. $1\in I \Rightarrow$ coordinates $\Rightarrow 5|1$, contradiction (`linearIndependent_one_theta`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_primeAboveFiveSubTwo : Ideal.absNorm primeAboveFiveSubTwo
    = 5

theorem absNorm_primeAboveFiveAddTwo : Ideal.absNorm primeAboveFiveAddTwo
    = 5
```

Mathematical meaning. The displayed Lean declaration has type `absNorm = 5`.

Implementation. divides 25, $\neq 1$ (proper), $\neq 25$ (else $\theta\mp2\in(5)$, ruled out by `unit_theta_coeff_not_mem`); `dvd_prime_pow`+`interval_cases`.

```
theorem isPrime_primeAboveFiveSubTwo : primeAboveFiveSubTwo.IsPrime

theorem isPrime_primeAboveFiveAddTwo : primeAboveFiveAddTwo.IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `.IsPrime`.

Implementation. `absNorm=5` prime. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_primeAboveFiveSubTwo :
    primeAboveFiveSubTwo.LiesOver (Ideal.span {(5 : ℤ)})

theorem liesOver_primeAboveFiveAddTwo :
    primeAboveFiveAddTwo.LiesOver (Ideal.span {(5 : ℤ)})

theorem inertiaDeg_primeAboveFiveSubTwo :
    (Ideal.span {(5 : ℤ)}).inertiaDeg primeAboveFiveSubTwo = 1

theorem inertiaDeg_primeAboveFiveAddTwo :
    (Ideal.span {(5 : ℤ)}).inertiaDeg primeAboveFiveAddTwo = 1
```

Mathematical meaning. The displayed Lean declaration has type lie over `(5)`; `inertiaDeg=1`.

Implementation. The proof applies the packaged result `standard`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
private theorem intNorm_five : Algebra.norm ℤ (5 : 𝒪 K) = 25

private theorem intNorm_theta_sub_two : Algebra.norm ℤ (thetaInt - 2) = 10
```

Mathematical meaning. The displayed Lean declaration has type `norm(5)=25`, `norm(`θ`−2)=10`.

Implementation. The proof applies the packaged result `intNorm_eq_normForm`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem not_isPrincipal_primeAboveFive :
    ¬ Submodule.IsPrincipal (Ideal.span {(5 : 𝒪 K), thetaInt - 2})
```

Mathematical meaning. second non-principality witness (the split prime over `5`).

Implementation. a generator `a` would satisfy `A|25` and `A|10`, with `A≠1` (proper), `A≠5` (`intNorm_ne_five`); `interval_cases A+omega` exhausts.

```
private theorem absNorm_span_thirteen_aux :
    Ideal.absNorm (Ideal.span {(13 : 𝒪 K)}) = 169
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (13) = 169`.

Implementation. The proof applies the packaged result `intNorm_eq_normForm 13 0`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem isPrime_span_thirteen : (Ideal.span {(13 : 𝒪 K)}).IsPrime
```

Mathematical meaning. `13` is inert.

Implementation. any maximal `M ⊇ (13)` has `absNorm|169`; `≠1`, `≠13` (else `ZMod 13` would contain a root of `t`^{2}`+6` – ruled out by `decide`), so `=169`, forcing `M=(13)`.

```
theorem absNorm_span_thirteen : Ideal.absNorm (Ideal.span {(13 : 𝒪 K)}) =
    169
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (13) = 169`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem liesOver_span_thirteen :
    (Ideal.span {(13 : 𝒪 K)}).LiesOver (Ideal.span {(13 : ℤ)})

theorem inertiaDeg_span_thirteen :
    (Ideal.span {(13 : ℤ)}).inertiaDeg (Ideal.span {(13 : 𝒪 K)}) = 2
```

Mathematical meaning. The displayed Lean declaration has type lies over `(13)`; `inertiaDeg = 2`.

Implementation. `pow_right_injective` on `13^2=13^f`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem mod_twentyfour_of_splits (p : ℕ) (hp : p.Prime) (hp2 : p
    ≠ 2)
    (hp3 : p ≠ 3)
    (h : ∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) :
    p % 24 = 1 ∨ p % 24 = 5 ∨ p % 24 = 7 ∨ p % 24 = 11
```

Mathematical meaning. The displayed Lean declaration has type split ($p\%24 \notin \{2,3\}$) $\Rightarrow$ $p\%24 \in \{1,5,7,11\}$.

Implementation. `absNorm P = p` $\Rightarrow$ `ZMod p` $\cong$ `𝒪K/P` has a root of t^2+6 $\Rightarrow$ `IsSquare (-6)` $\Rightarrow$ `isSquare_neg_six_iff_mod_twentyfour`.

```
private theorem exists_root_data (p : ℕ) [NeZero p] {s : ZMod p}
    (hs : ((-6 : ℤ) : ZMod p) = s * s) :
    ∃ c d k₁ k₂ : ℤ,
      ((c : ℤ) : ZMod p) = s ∧ ((d : ℤ) : ZMod p) = -s ∧
        (thetaInt - (c : 𝒪 K)) * (thetaInt - (d : 𝒪 K)) =
          ((p : ℕ) : 𝒪 K) * (-(k₁ : 𝒪 K) * thetaInt + (k₂ : 𝒪 K))
```

Mathematical meaning. The displayed Lean declaration has type from $-6=s^2 \bmod p$, produces $c,d,k_1,k_2:\mathbb{Z}$ with $(\theta-c)(\theta-d)=p\cdot$(and related terms).

Implementation. `c=s.val`, `d=(-s).val`, $p \mid c+d$, $p \mid cd-6$; `linear_combination`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem absNorm_span_pair_not_one (p : ℕ) (hp2le : 2 ≤ p)
    (c d k₁ k₂ : ℤ)
    (hprod : (thetaInt - (c : 𝒪 K)) * (thetaInt - (d : 𝒪 K)) =
      ((p : ℕ) : 𝒪 K) * (-(k₁ : 𝒪 K) * thetaInt + (k₂ : 𝒪 K))) :
    Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K), thetaInt - (c : 𝒪 K)}) ≠ 1
```

Mathematical meaning. The displayed Lean declaration has type $(p,\theta-c) \neq \top$ (given the product relation).

Implementation. $1\in I \Rightarrow \theta-d\in(p)$, ruled out by `unit_theta_coeff_not_mem`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem absNorm_span_pair_ne_prime_sq (p : ℕ) (hp : p.Prime) (c :
    ℤ) :
    Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K), thetaInt - (c : 𝒪 K)}) ≠
    p ^ 2
```

Mathematical meaning. The displayed Lean declaration has type $\mathrm{absNorm}\ (p,\theta-c) \neq p^2$.

Implementation. would give $(p,\theta-c)=(p) \Rightarrow \theta-c\in(p)$, contradiction. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem absNorm_span_pair_eq_prime (p : ℕ) (hp : p.Prime)
    (c d k₁ k₂ : ℤ)
    (hprod : (thetaInt - (c : 𝒪 K)) * (thetaInt - (d : 𝒪 K)) =
      ((p : ℕ) : 𝒪 K) * (-(k₁ : 𝒪 K) * thetaInt + (k₂ : 𝒪 K))) :
    Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K), thetaInt - (c : 𝒪 K)}) = p
```

Mathematical meaning. The displayed Lean declaration has type $\mathrm{absNorm}\ (p,\theta-c) = p$.

Implementation. `Common.eq_prime_of_dvd_prime_sq_of_ne_one_of_ne_sq` (divides p^2, $\neq 1$, $\neq p^2$). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem root_ne_neg_of_square_neg_six (p : ℕ) (hp : p.Prime) (hp2
    : p ≠ 2)
    (hp3 : p ≠ 3) {s : ZMod p} (hs : ((-6 : ℤ) : ZMod p) = s * s) :
    s ≠ -s
```

Mathematical meaning. The displayed Lean declaration has type for $p\notin\{2,3\}$, $s \neq -s$.

Implementation. $s=-s \Rightarrow 2s=0 \Rightarrow s=0 \Rightarrow p|6$, contradiction. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem span_pair_ne_of_roots_ne (p : ℕ) (hp : p.Prime) {c d : ℤ}
    (hcd : ((c - d : ℤ) : ZMod p) ≠ 0)
    (hNP : Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K), thetaInt - (c : 𝒪
```

```
    K)}) = p) :
    Ideal.span {((p : ℕ) : 𝒪 K), thetaInt - (c : 𝒪 K)} ≠
      Ideal.span {((p : ℕ) : 𝒪 K), thetaInt - (d : 𝒪 K)}
```

Mathematical meaning. The displayed Lean declaration has type `c≠d mod p ⇒ (p,θ−c) ≠ (p,θ−d)`.

Implementation. `exists_int_inverse_mod_prime` (Common) builds `1∈I`, contradicting norm `p`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem span_pair_mul_eq_span_natCast (p : ℕ) (hp : p.Prime) (c d
    k₁ k₂ : ℤ)
    (hprod : (thetaInt - (c : 𝒪 K)) * (thetaInt - (d : 𝒪 K)) =
      ((p : ℕ) : 𝒪 K) * (-(k₁ : 𝒪 K) * thetaInt + (k₂ : 𝒪 K)))
    (hNP : Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K), thetaInt - (c : 𝒪
    K)}) = p)
    (hNQ : Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K), thetaInt - (d : 𝒪
    K)}) = p) :
    Ideal.span {((p : ℕ) : 𝒪 K)} =
      Ideal.span {((p : ℕ) : 𝒪 K), thetaInt - (c : 𝒪 K)} *
        Ideal.span {((p : ℕ) : 𝒪 K), thetaInt - (d : 𝒪 K)}
```

Mathematical meaning. The displayed Lean declaration has type `(p) = (p,θ−c)(p,θ−d)`.

Implementation. $\leq$ directly; norms force equality via `eq_of_le_of_absNorm_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem splits_of_square_neg_six (p : ℕ) (hp : p.Prime) (hp2 : p
    ≠ 2)
    (hp3 : p ≠ 3) {s : ZMod p} (hs : ((-6 : ℤ) : ZMod p) = s * s) :
    ∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q
```

Mathematical meaning. The displayed Lean declaration has type $-6=s^2 \bmod p$ (`p∉{2,3}`) $\Rightarrow$ `(p)=P·Q`, `P,Q` distinct primes.

Implementation. assembles `exists_root_data`, the pair-norm lemmas, distinctness, product. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem splits_iff_mod_twentyfour (p : ℕ) (hp : p.Prime) (hp2 : p ≠ 2)
    (hp3 : p ≠ 3) :
    (∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) ↔
        p % 24 = 1 ∨ p % 24 = 5 ∨ p % 24 = 7 ∨ p % 24 = 11
```

Mathematical meaning. The displayed Lean declaration has type for p∉{2,3}: (p) splits into two distinct primes ↔ p%24∈{1,5,7,11}.

Implementation. mod_twentyfour_of_splits (⇒) and splits_of_square_neg_six (⇐). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem eq_two_or_eq_three_of_prime_dvd_twentyfour {p : ℕ} (hp :
    p.Prime)
    (hpdvd : p ∣ 24) : p = 2 ∨ p = 3
```

Mathematical meaning. The displayed Lean declaration has type p prime | 24 ⇒ p=2 ∨ p=3.

Implementation. 24=$2^3\cdot$3, Prime.dvd_mul. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramified_iff (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ^ 2) ↔ p =
    2 ∨ p = 3
```

Mathematical meaning. The displayed Lean declaration has type (∃ P prime, (p)≤P^2) ↔ p∈{2,3} (p ∣ disc −24).

Implementation. (⇒) P∣differentIdeal via not_isUnramifiedAt_of_ and related terms (Common), absNorm(different)=24, so p∣24. (⇐) exhibit primeAboveTwo/Three with span_two_eq/span_three_eq.

```
theorem ramificationIdx_span_one_add_theta :
    (Ideal.span {(7 : ℤ)}).ramificationIdx (Ideal.span {1 + thetaInt}) = 1

theorem ramificationIdx_span_one_sub_theta :
    (Ideal.span {(7 : ℤ)}).ramificationIdx (Ideal.span {1 - thetaInt}) = 1

theorem ramificationIdx_primeAboveFiveSubTwo :
    (Ideal.span {(5 : ℤ)}).ramificationIdx primeAboveFiveSubTwo = 1

theorem ramificationIdx_primeAboveFiveAddTwo :
    (Ideal.span {(5 : ℤ)}).ramificationIdx primeAboveFiveAddTwo = 1

theorem ramificationIdx_span_thirteen :
    (Ideal.span {(13 : ℤ)}).ramificationIdx (Ideal.span {(13 : 𝒪 K)}) = 1
```

Mathematical meaning. The displayed Lean declaration has type ramification index 1 (the split/inert primes are unramified).

Implementation. ramificationIdx_spec; over-2 case ruled out by ramified_iff (7,5,13∉{2,3}). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtNegSixPolynomialMod (p : ℕ) : (ZMod p)[X] :=
  X ^ 2 + C (6 : ZMod p)
```

Mathematical meaning. reduction of X^2+6 mod p.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem sqrtNegSixPolynomialMod_isRoot_iff (p : ℕ) (t : ZMod p) :
    (sqrtNegSixPolynomialMod p).IsRoot t ↔ t ^ 2 = ((-6 : ℤ) : ZMod p)
```

Mathematical meaning. The displayed Lean declaration has type `IsRoot t ↔ t² = ((-6:ℤ):ZMod p)`.

Implementation. `eval+linear_combination`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem sqrtNegSixPolynomialMod_natDegree (p : ℕ) [Fact p.Prime] :
    (sqrtNegSixPolynomialMod p).natDegree = 2
```

Mathematical meaning. The displayed Lean declaration has type `natDegree = 2`.

Implementation. The proof applies the packaged result `simp`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
private theorem sqrtNegSixPolynomialMod_roots_eq_zero_iff_no_root (p : ℕ)
    [Fact
    p.Prime] :
    (sqrtNegSixPolynomialMod p).roots = 0 ↔
      ¬ ∃ t : ZMod p, t ^ 2 = ((-6 : ℤ) : ZMod p)
```

Mathematical meaning. The displayed Lean declaration has type `roots = 0 ↔ ¬∃ t, t²=-6`.

Implementation. `mem_roots`/`eq_zero_iff_forall_notMem`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem sqrtNegSixPolynomialMod_no_root_iff_inert_mod_twentyfour
    (p : ℕ) (hp : p.Prime) (hp2 : p ≠ 2) (hp3 : p ≠ 3) :
    (¬ ∃ t : ZMod p, t ^ 2 = ((-6 : ℤ) : ZMod p)) ↔
      p % 24 = 13 ∨ p % 24 = 17 ∨ p % 24 = 19 ∨ p % 24 = 23
```

Mathematical meaning. The displayed Lean declaration has type `(¬∃ t, t²=-6) ↔ p%24∈{13,17,19,23}`.

Implementation. case-split `p%24`, dispatch via `isSquare_neg_six_iff_mod_twentyfour`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_differentIdeal_eq_twentyFour :
    Ideal.absNorm (differentIdeal ℤ (𝒪 K)) = 24
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (differentIdeal ℤ (𝒪 K)) = 24`.

Implementation. `NumberField.absNorm_differentIdeal` + `numberField_discr_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem prime_dvd_differentIdeal_iff_eq_ramifiedPrime
    (P : Ideal (𝒪 K)) (_hP : P.IsPrime) (_hP0 : P ≠ ⊥) :
    P ∣ differentIdeal ℤ (𝒪 K) ↔ P = primeAboveTwo ∨ P = primeAboveThree
```

Mathematical meaning. The displayed Lean declaration has type P∣`different` ↔ P∈{`primeAboveTwo, primeAboveThree`}.

Implementation. (⇒) `absNorm(under P)`∣24 ⇒ `under=2 or 3` ⇒ `P=primeAboveTwo/Three` (maximality via `span_*_eq`). (⇐) `dvd_differentIdeal_iff` via `ramificationIdx=2>1`.

4.6 `ClassGroup.lean`

```
abbrev thetaFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  FractionalIdeal.spanSingleton (𝒪 K)⁰ θ
```

Mathematical meaning. The displayed Lean declaration has type `FractionalIdeal (𝒪 K)⁰ K := spanSingleton θ`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev primeAboveTwoFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  (primeAboveTwo : FractionalIdeal (𝒪 K)⁰ K)

abbrev primeAboveThreeFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  (primeAboveThree : FractionalIdeal (𝒪 K)⁰ K)
```

Mathematical meaning. The displayed Lean declaration has type coercions of `primeAboveTwo/Three` to fractional ideals.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem theta_mem_fractionalIdeal :
    θ ∈ thetaFractionalIdeal
```

Mathematical meaning. The displayed Lean declaration has type $\theta \in$ `thetaFractionalIdeal`.

Implementation. The proof applies the packaged result `mem_spanSingleton_self`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem thetaFractionalIdeal_absNorm :
    FractionalIdeal.absNorm thetaFractionalIdeal = |Algebra.norm ℚ θ|
```

Mathematical meaning. The displayed Lean declaration has type `absNorm = |Algebra.norm ℚ θ|`.

Implementation. The proof applies the packaged result `simp`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem ramifiedFractionalIdeals_ne_zero :
    primeAboveTwoFractionalIdeal ≠ 0 ∧ primeAboveThreeFractionalIdeal ≠ 0
```

Mathematical meaning. The displayed Lean declaration has type both ramified fractional ideals $\neq$ `0`.

Implementation. `coeIdeal_ne_zero` + `absNorm` nonzero. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def ramifiedIdealOverTwo : (Ideal (𝒪 K))⁰ :=
  ⟨primeAboveTwo, by
    rw [← Ideal.absNorm_ne_zero_iff_mem_nonZeroDivisors]
    rw [absNorm_primeAboveTwo]
    norm_num⟩
```

Mathematical meaning. `(2,θ)` as a nonzero ideal – concrete representative of the nontrivial class.

Implementation. `absNorm=2≠0`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def ramifiedIdealOverThree : (Ideal (𝒪 K))⁰ :=
  ⟨primeAboveThree, by
    rw [← Ideal.absNorm_ne_zero_iff_mem_nonZeroDivisors]
    rw [absNorm_primeAboveThree]
    norm_num⟩
```

Mathematical meaning. `(3,θ)` nonzero ideal (same class as `OverTwo`).

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev unitIdealRepresentative : (Ideal (𝒪 K))⁰ := ⟨(⊤ : Ideal (𝒪 K)), by
    simp⟩
```

Mathematical meaning. representative of the trivial class.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem thetaInt_ne_zero : thetaInt ≠ 0
```

Mathematical meaning. The displayed Lean declaration has type `thetaInt ≠ 0`.

Implementation. else θ=0 contradicts `theta_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem minkowskiBound_lt_four :
    (4 / Real.pi) ^ NumberField.InfinitePlace.nrComplexPlaces K *
        (((Module.finrank ℚ K).factorial : ℝ) /
          ((Module.finrank ℚ K : ℝ)) ^ (Module.finrank ℚ K) *
            Real.sqrt (|NumberField.discr K| : ℝ)) < 4
```

Mathematical meaning. The displayed Lean declaration has type Minkowski bound `< 4`.

Implementation. `(2/`π`)`$\sqrt{24}$ `< 4` via $\sqrt{24}$<5 and π>3 (`nlinarith`); plugs `nrComplexPlaces=1`, `finrank=2`, disc −24.

```
theorem classGroup_mk0_ramified_ne_one :
    ClassGroup.mk0 ramifiedIdealOverTwo ≠ 1
```

Mathematical meaning. The displayed Lean declaration has type `ClassGroup.mk0 ramifiedIdealOverTwo ≠ 1`.

Implementation. `=1` would force `primeAboveTwo` principal (`mk0_eq_one_iff`), contradicting `not_isPrincipal_primeAboveTwo`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classGroup_mk0_ramified_sq_eq_one :
    ClassGroup.mk0 ramifiedIdealOverTwo ^ 2 = 1
```

Mathematical meaning. the nontrivial class has order dividing 2.

Implementation. `mk0_eq_mk0_inv_iff` with witness 2 (since `(2,`θ`)`^{2}`=(2)` principal $\Rightarrow$ class is its own inverse).

```
theorem orderOf_classGroup_mk0_ramified :
    orderOf (ClassGroup.mk0 ramifiedIdealOverTwo) = 2
```

Mathematical meaning. The displayed Lean declaration has type `orderOf = 2`.

Implementation. `orderOf_eq_prime` (sq=1, $\neq$1). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem class_mk0_primeAboveThree_eq_ramified :
    ClassGroup.mk0 ramifiedIdealOverThree = ClassGroup.mk0
    ramifiedIdealOverTwo
```

Mathematical meaning. The displayed Lean declaration has type class of $(3,\theta)$ = class of $(2,\theta)$.

Implementation. $(2,\theta)$ self-inverse and $(2,\theta)(3,\theta)=(\theta)$ principal (span_theta_eq) $\Rightarrow$ $(3,\theta)$ is the inverse of the inverse.

```
private theorem classGroupRepresentatives_aux (C : ClassGroup (𝒪 K)) :
    C = 1 ∨ C = ClassGroup.mk0 ramifiedIdealOverTwo
```

Mathematical meaning. The displayed Lean declaration has type $\forall$ C, C=1 $\vee$ C=mk0 ramifiedIdealOverTwo.

Implementation. exists_ideal_in_class_of_norm_le, absNorm $\leq$ 3 (minkowskiBound_lt_four); interval_cases on norm 1,2,3 using Ideal.absNorm_eq_one_iff, ideal_eq_primeAboveTwo/Three_of_absNorm_eq_$*$, class_mk0_primeAboveThree_eq_ramified.

```
theorem classNumber_eq_two : NumberField.classNumber K = 2
```

Mathematical meaning. THE headline result.

Implementation. Nat.card (ClassGroup) = 2 via card_eq_two_iff with {1, mk0 ramifiedIdealOverTwo}, completeness from classGroupRepresentatives_aux, nontriviality from classGroup_mk0_ramified_ne_one.

```
theorem classGroupStructure : Nonempty (ClassGroup (𝒪 K) ≃*
    Multiplicative (ZMod 2))
```

Mathematical meaning. The displayed Lean declaration has type Nonempty (ClassGroup (𝒪 K) $\simeq*$ Multiplicative (ZMod 2)).

Implementation. Nat.card=2 + mulEquivOfPrimeCardEq. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classGroup_mk0_ramified_ne :
    ClassGroup.mk0 ramifiedIdealOverTwo ≠ ClassGroup.mk0
    unitIdealRepresentative
```

Mathematical meaning. The displayed Lean declaration has type mk0 ramifiedIdealOverTwo $\neq$ mk0 unitIdealRepresentative.

Implementation. else primeAboveTwo principal, contradiction. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classGroupRepresentatives (C : ClassGroup (𝒪 K)) :
    C = ClassGroup.mk0 unitIdealRepresentative ∨ C = ClassGroup.mk0
    ramifiedIdealOverTwo
```

Mathematical meaning. The displayed Lean declaration has type `∀ C, C = mk0 unitIdealRepresentative ∨ C = mk0 ramifiedIdealOverTwo`.

Implementation. `Nat.card=2` + `card_eq_two_iff'`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

4.7 `Units.lean`

```
abbrev torsionGenerator : (𝒪 K)ˣ := -1
```

Mathematical meaning. torsion generator (-1).

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
def torsionUnits : Set ((𝒪 K)ˣ) := {u | ∃ k : Fin 2, u = torsionGenerator
    ^ k.val}
```

Mathematical meaning. $\{1,-1\}$.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem torsionUnits_eq : {u : (𝒪 K)ˣ | IsOfFinOrder u} = torsionUnits
```

Mathematical meaning. torsion subgroup is exactly $\{\pm 1\}$.

Implementation. unit $\Rightarrow$ $x^2+6y^2=1$ (`isUnit_iff_normForm`) $\Rightarrow$ $y=0$, $x=\pm 1$ (`interval_cases`); conversely $(\pm 1)^2=1$.

```
theorem torsionOrder_eq_two : NumberField.Units.torsionOrder K = 2
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.Units.torsionOrder K = 2`.

Implementation. `Nat.card (torsion) = 2` via $\{1,-1\}$ distinct ($2\neq 0$ in K); completeness from `torsionUnits_eq`.

```
theorem unitRank_eq_zero : NumberField.Units.rank K = 0
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.Units.rank K = 0`.

Implementation. `rank = #places - 1`; `nrRealPlaces=0`, `nrComplexPlaces=1`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem unit_eq_one_or_neg_one (u : (𝒪 K)ˣ) : u = 1 ∨ u = -1
```

Mathematical meaning. The displayed Lean declaration has type `∀ u:(𝒪 K)ˣ, u=1 ∨ u=-1`.

Implementation. `exist_unique_eq_mul_prod`; rank `0` ⇒ empty fundamental product ⇒ `u` is torsion ⇒ `torsionUnits_eq`.

```
theorem unitGroupStructure : Nonempty ((𝒪 K)ˣ ≃* Multiplicative (ZMod 2))
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty ((𝒪 K)ˣ ≃* Multiplicative (ZMod 2))`.

Implementation. `Nat.card=2` (from `unit_eq_one_or_neg_one`) + `mulEquivOfPrimeCardEq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem regulator_eq_one : NumberField.Units.regulator K = 1
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.Units.regulator K = 1`.

Implementation. `regulator_eq_det'`; rank `0` ⇒ empty `0×0` determinant `=1` (`det_eq_one_of_card_eq_zero`).

4.8 **Euclidean.lean**

```
theorem not_isPrincipalIdealRing : ¬ IsPrincipalIdealRing (𝒪 K)
```

Mathematical meaning. $\mathbb{Z}[\sqrt{-6}]$ is not a PID.

Implementation. a PID would make every ideal principal, contradicting `not_isPrincipal_primeAboveTwo` applied to `(2,θ)`.

```
theorem not_normEuclidean :
    ¬ ∀ a b : 𝒪 K, b ≠ 0 →
      ∃ q r : 𝒪 K, a = b * q + r ∧
        (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

Mathematical meaning. $\mathbb{Z}[\sqrt{-6}]$ is not norm-Euclidean.

Implementation. take `a=thetaInt`, `b=2`: any remainder `r=θ−2q` has coordinates `(−2x, 1−2y)` (odd second coordinate); `norm r = (−2x)²+6(1−2y)² ≥ 6 > 4 = |norm 2|` (`nlinarith`, since `(1−2y)²≥1`), contradicting the Euclidean inequality.

4.9 `UniqueFactorization.lean`

`theorem six_eq_neg_theta_sq : (6 : 𝒪 K) = -(thetaInt * thetaInt)`

Mathematical meaning. `6 = -√-6·√-6` – the second factorization of `6`.

Implementation. `thetaInt_sq` (θ^2=−6). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

`private theorem intNorm_two : Algebra.norm ℤ (2 : 𝒪 K) = 4`

`private theorem intNorm_three : Algebra.norm ℤ (3 : 𝒪 K) = 9`

`private theorem intNorm_thetaInt : Algebra.norm ℤ thetaInt = 6`

Mathematical meaning. The displayed Lean declaration has type `norm(2)=4`, `norm(3)=9`, `norm(`θ`)=6`.

Implementation. The proof applies the packaged result `intNorm_eq_normForm`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

`theorem irreducible_two : Irreducible (2 : 𝒪 K)`

Mathematical meaning. The displayed Lean declaration has type `Irreducible (2 : 𝒪 K)`.

Implementation. not a unit (`norm 4`≠1); any factor norm divides 4, is ≠1 (non-unit), ≠2 (`intNorm_ne_two`), so one factor has norm 1 ⇒ unit; `interval_cases+nlinarith`.

`theorem irreducible_three : Irreducible (3 : 𝒪 K)`

Mathematical meaning. The displayed Lean declaration has type `Irreducible (3 : 𝒪 K)`.

Implementation. twin (`norm 9`, ≠3 via `intNorm_ne_three`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

`theorem irreducible_theta : Irreducible thetaInt`

Mathematical meaning. The displayed Lean declaration has type `Irreducible thetaInt`.

Implementation. `norm 6 = 2·3`; factor norms must avoid 2 and 3 (`intNorm_ne_two/three`), so one is 1 ⇒ unit.

`theorem two_not_prime : ¬ Prime (2 : 𝒪 K)`

Mathematical meaning. 2 irreducible but not prime.

Implementation. `2` | $\theta\cdot\theta$ `= -6` but `2` ∤ θ (else $\theta\in$`(2)`, but `absNorm(2,`θ`)=2` ≠ `4=absNorm(2)`, so `(2,`θ`)`≠`(2)`); `dvd_or_dvd` fails.

```
theorem not_uniqueFactorizationMonoid : ¬ UniqueFactorizationMonoid (𝒪 K)
```

Mathematical meaning. $\mathbb{Z}[\sqrt{-6}]$ is not a UFD.

Implementation. a UFM Dedekind domain would be a PID (`IsPrincipalIdealRing.of_isDedekindDomain_of_uniqueFactorizationMonoid`), contradicting `not_isPrincipalIdealRing`.

4.10 `CMField.lean`

```
theorem isCMField : NumberField.IsCMField K
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.IsCMField K`.

Implementation. totally complex (`isTotallyComplex`) quadratic over $\mathbb{Q}$ $\Rightarrow$ `IsCMField.ofCMExtension ℚ K`.

```
theorem cmField_isTotallyComplex : NumberField.IsTotallyComplex K
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.IsTotallyComplex K`.

Implementation. The proof applies the packaged result `isTotallyComplex`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem cmField_quadratic_over_maximalRealSubfield :
    Algebra.IsQuadraticExtension (NumberField.maximalRealSubfield K) K
```

Mathematical meaning. The displayed Lean declaration has type `IsQuadraticExtension (maximalRealSubfield K) K`.

Implementation. `IsCMField.isQuadraticExtension` (max real subfield is $\mathbb{Q}$). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cmField_unitsRank_eq_maximalRealSubfield :
    NumberField.Units.rank (NumberField.maximalRealSubfield K) =
      NumberField.Units.rank K
```

Mathematical meaning. The displayed Lean declaration has type `Units.rank (maximalRealSubfield K) = Units.rank K`.

Implementation. The proof applies the packaged result `IsCMField.units_rank_eq_units_rank`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

4.11 `LocalGlobal.lean`

```
def heightOneTwo : HeightOneSpectrum (𝒪 K) where
  asIdeal := primeAboveTwo
  isPrime := by
    exact isPrime_primeAboveTwo
  ne_bot := by
    intro h
    have hnorm := absNorm_primeAboveTwo
    rw [h] at hnorm
    norm_num at hnorm

def heightOneThree : HeightOneSpectrum (𝒪 K) where
  asIdeal := primeAboveThree
  isPrime := by
    exact isPrime_primeAboveThree
  ne_bot := by
    intro h
    have hnorm := absNorm_primeAboveThree
    rw [h] at hnorm
    norm_num at hnorm
```

Mathematical meaning. the finite places over 2/3, *represented by the non-principal ramified primes*.

Implementation. `isPrime_*`, `ne_bot` from nonzero `absNorm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev vTwo : NumberField.FinitePlace K :=
  NumberField.FinitePlace.mk heightOneTwo

abbrev vThree : NumberField.FinitePlace K :=
  NumberField.FinitePlace.mk heightOneThree
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.FinitePlace K := FinitePlace.mk heightOne*`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example :
    (Ideal.span {(2 : ℤ)}).ramificationIdx heightOneTwo.asIdeal = 2

example :
    (Ideal.span {(3 : ℤ)}).ramificationIdx heightOneThree.asIdeal = 2
```

```
example :
    (Ideal.span {(2 : ℤ)}).inertiaDeg heightOneTwo.asIdeal = 1

example :
    (Ideal.span {(3 : ℤ)}).inertiaDeg heightOneThree.asIdeal = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. the `PrimeSplitting` ramification/inertia lemmas. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev CompletionAtTwo : Type :=
  heightOneTwo.adicCompletion K

abbrev CompletionAtThree : Type :=
  heightOneThree.adicCompletion K
```

Mathematical meaning. The displayed Lean declaration has type `heightOne*.adicCompletion K`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev InfiniteCompletion (v : NumberField.InfinitePlace K) : Type :=
  v.Completion
```

Mathematical meaning. The displayed Lean declaration has type `(v : InfinitePlace K) → v.Completion`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example (v : NumberField.InfinitePlace K) :
    Nonempty (InfiniteCompletion v ≃+* ℂ)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `Completion.ringEquivComplexOfIsComplex` + `infinitePlace_isComplex`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
example :
    ((∏ w : NumberField.InfinitePlace K, w θ ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K, v θ) = 1
```

```
example :
    ((∏ w : NumberField.InfinitePlace K,
          w (algebraMap (𝓞 K) K (torsionGenerator : 𝓞 K)) ^ w.mult) *
        ∏'[f] v : NumberField.FinitePlace K,
          v (algebraMap (𝓞 K) K (torsionGenerator : 𝓞 K))) = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof applies the packaged result `NumberField.prod_abs_eq_one`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
abbrev SqrtNegSixInfiniteAdeleRing : Type :=
  NumberField.InfiniteAdeleRing K

abbrev infinitePrincipalAdele_theta : SqrtNegSixInfiniteAdeleRing :=
  algebraMap K SqrtNegSixInfiniteAdeleRing θ

abbrev infinitePrincipalAdele_torsionGenerator :
    SqrtNegSixInfiniteAdeleRing :=
  algebraMap K SqrtNegSixInfiniteAdeleRing (algebraMap (𝓞 K) K
    (torsionGenerator : 𝓞 K))
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example :
    ‖infinitePrincipalAdele_theta‖ = |Algebra.norm ℚ θ|

example :
    DenseRange (algebraMap K SqrtNegSixInfiniteAdeleRing)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `coe_norm_eq_abs_norm`, `denseRange_algebraMap`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem canonicalEmbedding_thetaInt_mem_integerLattice :
    NumberField.canonicalEmbedding K (algebraMap (𝓞 K) K thetaInt) ∈
      NumberField.canonicalEmbedding.integerLattice K
```

Mathematical meaning. The displayed Lean declaration has type `canonicalEmbedding K (algebraMap thetaInt) ∈ integerLattice K`.

Implementation. The proof applies the packaged result `simp`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
abbrev mixedEmbedding_theta : NumberField.mixedEmbedding.mixedSpace K :=
  NumberField.mixedEmbedding K θ
```

Mathematical meaning. The displayed Lean declaration has type `mixedEmbedding.mixedSpace K := mixedEmbedding K θ`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem mixedEmbedding_theta_mem_convexBodyLT
    (f : NumberField.InfinitePlace K → NNReal)
    (hf : ∀ w : NumberField.InfinitePlace K, w θ < f w) :
    mixedEmbedding_theta ∈ NumberField.mixedEmbedding.convexBodyLT K f
```

Mathematical meaning. The displayed Lean declaration has type if `∀ w, w θ < f w` then `mixedEmbedding_theta ∈ convexBodyLT K f`.

Implementation. unfold convex-body membership; only complex places (`infinitePlace_isComplex`), `norm_embedding_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
example :
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone K)

example :
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone.normLeOne K)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `measurableSet_fundamentalCone`, `measurableSet_normLeOne`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem polarSpace_nonempty :
    Nonempty (NumberField.mixedEmbedding.polarSpace K)
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty (polarSpace K)`.

Implementation. The proof applies the packaged result `infer_instance`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
example :
    0 < NumberField.house θ
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof applies the packaged result `one_le_house_of_isIntegral theta_integral`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem house_torsionGenerator :
    NumberField.house (algebraMap (𝒪 K) K (torsionGenerator : 𝒪 K)) = 1
```

Mathematical meaning. The displayed Lean declaration has type `house (algebraMap torsionGenerator) = 1`.

Implementation. The proof applies the packaged result `house_intCast (-1)`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
abbrev SqrtNegSixAdeleRing : Type :=
  NumberField.AdeleRing (𝒪 K) K

abbrev principalAdele_theta : SqrtNegSixAdeleRing :=
  algebraMap K SqrtNegSixAdeleRing θ

abbrev principalAdele_torsionGenerator : SqrtNegSixAdeleRing :=
  algebraMap K SqrtNegSixAdeleRing (algebraMap (𝒪 K) K (torsionGenerator
    : 𝒪 K))
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem principalAdele_theta_ne_zero : principalAdele_theta ≠ 0
```

Mathematical meaning. The displayed Lean declaration has type `principalAdele_theta ≠ 0`.

Implementation. `AdeleRing.algebraMap_injective` + `theta_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem principalAdele_torsionGenerator_is_unit_everywhere :
    ∀ v : NumberField.FinitePlace K,
      v (algebraMap (𝒪 K) K (torsionGenerator : 𝒪 K)) = 1
```

Mathematical meaning. The displayed Lean declaration has type ∀ `v finite place, v (algebraMap torsionGenerator) = 1`.

Implementation. `simp [torsionGenerator]` (−1 is a unit everywhere). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtNegSixDedekindZeta (s : ℂ) : ℂ :=
  NumberField.dedekindZeta K s

abbrev sqrtNegSixDedekindZetaResidue : ℝ :=
  NumberField.dedekindZeta_residue K
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example :
    0 < sqrtNegSixDedekindZetaResidue

example :
    sqrtNegSixDedekindZetaResidue =
      (2 ^ NumberField.InfinitePlace.nrRealPlaces K *
          (2 * Real.pi) ^ NumberField.InfinitePlace.nrComplexPlaces K *
          NumberField.Units.regulator K * NumberField.classNumber K) /
        (NumberField.Units.torsionOrder K * Real.sqrt |NumberField.discr
    K|)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `dedekindZeta_residue_pos`, `dedekindZeta_residue_def`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

4.12 Summary for $\mathbb{Q}(\sqrt{-6})$

Imaginary quadratic field $\mathbb{Q}(\sqrt{-6})$, built via `QuadraticAlgebra ℚ (-6) 0` (radical model, generator θ with $\theta^2 = -6$). Signature `(0,1)`; ring of integers $\mathbb{Z}[\sqrt{-6}]$ (maximal order, since $-6 \equiv 2 \pmod 4$); discriminant -24. The only non-PID case study: class number 2, class group $\cong \mathbb{Z}/2$, *not* a UFD, *not* norm-Euclidean – the mathematically interesting negative results. Namespace `NumberField.SqrtNegSix`. Cross-refs to `Common.lean` via `NumberField.FieldExamples`. Import chain: `Basic` → `Galois` → `Embeddings` → `RingOfIntegers` → `PrimeSplitting` → `ClassGroup` → `Units` → `Euclidean` → `LocalGlobal`, plus special-topic files `CMField` and `UniqueFactorization`. Sorry-free (confirmed: zero `sorry`/`admit`).

5 $\mathbb{Q}(\sqrt[3]{3})$

Field: `K = ℚ(∛3) = AdjoinRoot (X³ − 3)`, generator θ `= ∛3`. Pure cubic field (the only degree>2 and only non-Galois example in `FieldExamples`): signature `(r₁,r₂) = (1,1)`, ring of integers $\mathbb{Z}[\theta]$ (monogenic, maximal order), discriminant `−243 = −3⁵`, class number `1`, unit group $\mu_2 \times \mathbb{Z}$ (Dirichlet rank `1`, fundamental unit θ^2 `− 2 = ∛9 − 2`). `K/ℚ` is not Galois (`Aut = 1`); its Galois closure is the S_3-splitting field $\mathbb{Q}(\sqrt[3]{3}, \zeta_3)$ of degree `6`. Two genuine sorries remain: `fundamentalUnitSystem_isFundamental` (Artin-type unit-index bound) and `normEuclidean` (geometry-of-numbers covering argument).

All declarations live in `namespace NumberField.CubeRootThree` unless noted. $\mathcal{O}$ `K` denotes `NumberField.RingOfIntegers K`; θ `= cubeRootThree`, `thetaInt` its integral lift in $\mathcal{O}$ `K`. Common helpers referenced live in `namespace NumberField.FieldExamples` (file `FieldExamples/Common.lean`). Import chain: `Basic` → `Galois` → `Embeddings` → `RingOfIntegers` → `Discriminant` → `PrimeSplitting` → `ClassGroup` → `Units` → `Euclidean` → `LocalGlobal`.

Each block below begins with the displayed Lean declaration or object definition. For theorem, lemma, instance-proof, and example blocks the proof body is omitted; for definitions and abbreviations the displayed RHS is retained, because it is part of the mathematical object being documented.

5.1 `Basic.lean`

```
abbrev cubePolynomial : ℚ[X] := X ^ 3 - C (3 : ℚ)
```

Mathematical meaning. Defining polynomial `X³ − 3` of $\mathbb{Q}(\sqrt[3]{3})$.

Implementation. The displayed RHS is the packaged construction `definitional abbrev` specialized to this field.

```
instance : Fact (Irreducible cubePolynomial)
```

Mathematical meaning. `X³ − 3` irreducible over $\mathbb{Q}$.

Implementation. `X_pow_sub_C_irreducible_of_prime` (3 prime); reduce "3 has no rational cube root" to integral case via `IsIntegrallyClosed.isIntegral_iff`, then `interval_cases n <;> omega` over divisors of 3.

```
abbrev K : Type := AdjoinRoot cubePolynomial
```

Mathematical meaning. The number field $\mathbb{Q}(\sqrt[3]{3})$.

Implementation. The displayed RHS is the packaged construction `definitional` specialized to this field.

```
abbrev cubeRootThree : K := AdjoinRoot.root cubePolynomial
```

Mathematical meaning. The distinguished cube root θ `= ∛3`.

Implementation. The displayed RHS is the packaged construction `definitional` specialized to this field.

```
instance : Field K
```

```
instance : NumberField K
```

Mathematical meaning. Field / number-field structure on `K`.

Implementation. `infer_instance` (both from `AdjoinRoot` + irreducibility `Fact`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cubePolynomial_ne_zero : cubePolynomial ≠ 0
```

Mathematical meaning. Defining polynomial is nonzero.

Implementation. `congrArg natDegree` (degree $3 \neq 0$). Recomputed inline (as `hf`) in several later files.

```
abbrev cubeRootThreePowerBasis : PowerBasis ℚ K :=
  AdjoinRoot.powerBasis cubePolynomial_ne_zero
```

Mathematical meaning. Power basis `1, θ, θ^2`.

Implementation. The displayed RHS is the packaged construction `AdjoinRoot.powerBasis cubePolynomial_ne_zero` specialized to this field.

```
@[simp]
theorem cubeRootThreePowerBasis_gen : cubeRootThreePowerBasis.gen =
    cubeRootThree
```

Mathematical meaning. Power-basis generator is θ.

Implementation. The proof applies the packaged result `AdjoinRoot.powerBasis_gen`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem cubeRootThreePowerBasis_dim : cubeRootThreePowerBasis.dim = 3
```

Mathematical meaning. Power basis has dimension 3.

Implementation. `AdjoinRoot.powerBasis_dim` + `natDegree_X_pow_sub_C`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem degree_eq_three : Module.finrank ℚ K = 3
```

Mathematical meaning. `[ℚ(∛3) : ℚ] = 3`. Workhorse used in almost every later file.

Implementation. `PowerBasis.finrank` + `cubeRootThreePowerBasis_dim`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cubeRootThree_cube : cubeRootThree ^ 3 = 3
```

Mathematical meaning. $\theta^3 = 3$. Fundamental algebraic relation, used pervasively.

Implementation. `AdjoinRoot.mk_self` of $X^3 - 3$, then `linear_combination`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cubeRootThree_minpoly_rat : minpoly ℚ cubeRootThree =
    cubePolynomial
```

Mathematical meaning. Rational minimal polynomial of θ.

Implementation. `AdjoinRoot.minpoly_root` (monic so $C\ \text{leadingCoeff}^{-1} = 1$). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem adjoin_cubeRootThree_eq_top : Algebra.adjoin ℚ ({cubeRootThree} :
    Set K) = ⊤
```

Mathematical meaning. K generated by θ over $\mathbb{Q}$.

Implementation. `PowerBasis.adjoin_gen_eq_top` rewritten through the gen lemma. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

5.2 **Galois.lean**

```
noncomputable def idAut : K ≃ₐ[ℚ] K := AlgEquiv.refl
```

Mathematical meaning. Identity automorphism.

Implementation. The displayed RHS is the packaged construction `definitional` specialized to this field.

```
theorem no_primitive_cube_root (z : K) : z ^ 2 + z + 1 ≠ 0
```

Mathematical meaning. K contains no primitive cube root of unity ($\zeta_3 \notin K$); this is what makes $K/\mathbb{Q}$ non-normal.

Implementation. such z would have minpoly $\text{cyclotomic } 3 = X^2+X+1$ (irreducible over $\mathbb{Q}$), degree $2 \nmid 3 = [K:\mathbb{Q}]$ – contradiction via `minpoly.degree_dvd`.

```
theorem cube_root_one (w : K) (hw : w ^ 3 = 1) : w = 1
```

Mathematical meaning. Only cube root of 1 in K is 1.

Implementation. factor $(w-1)(w^2+w+1)=0$; second factor nonzero by `no_primitive_cube_root`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem aut_eq_id (σ : K ≃ₐ[ℚ] K) : σ = idAut
```

Mathematical meaning. Every $\mathbb{Q}$-automorphism is the identity (so `Aut(K/ℚ)` is trivial – the cubic field is rigid).

Implementation. $\sigma\theta/\theta$ is a cube root of 1, hence `=1` by `cube_root_one`, so $\sigma\theta$=θ; `AdjoinRoot.algHom_ext`.

```
theorem not_isGalois : ¬ IsGalois ℚ K
```

Mathematical meaning. $\mathbb{Q}(\sqrt[3]{3})/\mathbb{Q}$ is not Galois.

Implementation. Galois $\Longrightarrow$ `card Aut = finrank = 3`, but `Aut` is `Subsingleton` (via `aut_eq_id`), so `card = 1`; `omega`.

```
theorem galoisGroupStructure : Nonempty ((K ≃ₐ[ℚ] K) ≃* Multiplicative
    (ZMod 1))
```

Mathematical meaning. Automorphism group of `K` is trivial (≅ `ZMod 1`).

Implementation. `Unique (K ≃ₐ[ℚ] K)` from `aut_eq_id`, then `MulEquiv.ofUnique`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev galoisClosure : Type := cubePolynomial.SplittingField
```

Mathematical meaning. The Galois closure of `K`, mathematically $\mathbb{Q}(\sqrt[3]{3},\ \zeta_3)$ – the splitting field of X^3-3. This is how the S_3 closure is formalised: as mathlib's `Polynomial.SplittingField`, not a bespoke field.

Implementation. The displayed RHS is the packaged construction `definitional` specialized to this field.

```
theorem isGalois_galoisClosure : IsGalois ℚ galoisClosure
```

Mathematical meaning. The closure is Galois over $\mathbb{Q}$.

Implementation. `IsGalois.of_separable_splitting_field` (X^3-3 separable in char 0). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem nonempty_algHom_galoisClosure : Nonempty (K →ₐ[ℚ] galoisClosure)
```

Mathematical meaning. `K` embeds in its closure (as one of three conjugate cubic subfields).

Implementation. closure splits X^3-3, pick a root via `splits.exists_eval_eq_zero`, lift `AdjoinRoot.liftAlgHom`.

```
theorem galoisClosureGroupStructure :
    Nonempty ((galoisClosure ≃ₐ[ℚ] galoisClosure) ≃* Equiv.Perm (Fin 3))
```

Mathematical meaning. Galois group of the closure is the full symmetric group S_3, acting by permuting the three cube roots of 3.

Implementation. `Polynomial.Gal.galActionHom_bijective_of_prime_degree` – X^3-3 irreducible of prime degree 3 with exactly 2 non-real roots (computes `card rootSet ℂ = 3`, `card rootSet ℝ = 1` via `3^(1/3)` real-root uniqueness from `Odd.strictMono_pow`); then `Fintype.equivFinOfCardEq.permCongrHom`.

```
theorem finrank_galoisClosure : Module.finrank ℚ galoisClosure = 6
```

Mathematical meaning. $[\mathbb{Q}(\sqrt[3]{3},\zeta_3) : \mathbb{Q}] = 6 = |S_3|$.

Implementation. closure Galois $\Longrightarrow$ `finrank = card Aut = card (Perm (Fin 3)) = 3! = 6`.

5.3 **Embeddings.lean**

```
theorem not_totallyReal : ¬ NumberField.IsTotallyReal K
```

Mathematical meaning. `K` is not totally real (it has a complex place).

Implementation. build `c` $\in \mathbb{C}$ with `c^3 = 3`, `c.im` $\neq$ `0` (using a primitive cube root ζ_3 of unity scaled into a non-real cube root of 3); the corresponding embedding `AdjoinRoot.lift` is not real (its conjugate disagrees on θ).

```
theorem nrComplexPlaces_eq_one :
    NumberField.InfinitePlace.nrComplexPlaces K = 1
```

Mathematical meaning. Exactly one complex place (r_2 = 1).

Implementation. signature relation r_1 + 2 r_2 = finrank = 3 (`card_add_two_mul_card_eq_rank`) with $r_2 \neq$ `0` (`not_totallyReal` via `nrComplexPlaces_eq_zero_iff`); `omega`.

```
theorem nrRealPlaces_eq_one : NumberField.InfinitePlace.nrRealPlaces K = 1
```

Mathematical meaning. Exactly one real place (r_1 = 1).

Implementation. same signature relation with r_2 = 1; `omega`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_card_eq_two :
    Fintype.card (NumberField.InfinitePlace K) = 2
```

Mathematical meaning. `K` has 2 infinite places (r_1 + r_2 = 2). Used by regulator computation.

Implementation. `card_eq_nrRealPlaces_add_nrComplexPlaces` + the two counts. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_signature :
    NumberField.InfinitePlace.nrRealPlaces K = 1 ∧
      NumberField.InfinitePlace.nrComplexPlaces K = 1
```

Mathematical meaning. Packaged signature `(1,1)`.

Implementation. conjunction of the two count lemmas.

```
theorem exists_realEmbedding_cubeRootThree :
    ∃ σ : K →ₐ[ℚ] ℝ, σ cubeRootThree = (3 : ℝ) ^ ((1 : ℝ) / 3)
```

Mathematical meaning. The real embedding sends $\theta \mapsto \sqrt[3]{3}$. Used by regulator computation.

Implementation. `AdjoinRoot.liftAlgHom` to $\mathbb{R}$ at `3^(1/3)` (`rpow` identity `(3^(1/3))^3 = 3`).

```
theorem exists_complexEmbedding_cubeRootThree :
    ∃ σ : K →+* ℂ, σ cubeRootThree ^ 3 = 3
```

Mathematical meaning. A complex embedding with $\sigma\theta$ a cube root of 3.

Implementation. compose the real embedding with `Complex.ofRealHom`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

5.4 `RingOfIntegers.lean`

```
theorem cubeRootThree_integral : cubeRootThree ∈ integralClosure ℤ K
```

Mathematical meaning. θ is integral over $\mathbb{Z}$ (root of monic X^3-3).

Implementation. exhibit `X^3-C 3` monic with θ a root via `cubeRootThree_cube`.

```
noncomputable def thetaInt : 𝒪 K := ⟨cubeRootThree,
    cubeRootThree_integral⟩
```

Mathematical meaning. θ as an element of $\mathcal{O}$ `K`. Central object throughout PrimeSplitting/ClassGroup/Units.

Implementation. The displayed RHS is the packaged construction `subtype constructor` specialized to this field.

```
theorem thetaInt_cube : thetaInt ^ 3 = 3
```

Mathematical meaning. $\theta^3 = 3$ inside $\mathcal{O}$ `K`. Used in nearly every ideal computation.

Implementation. push through injective `algebraMap (𝒪 K) K` to `cubeRootThree_cube`.

```
def integralBasisVectors : Fin 3 → K := ![1, cubeRootThree, cubeRootThree
    ^ 2]
```

Mathematical meaning. Displayed integral basis vectors `1, θ, θ^2` in `K`.

Implementation. The displayed RHS is the packaged construction `definitional` specialized to this field.

```
theorem mem_adjoin_iff (x : K) :
    x ∈ Algebra.adjoin ℤ ({cubeRootThree} : Set K) ↔
      ∃ a b c : ℤ, x = a • (1 : K) + b • cubeRootThree + c •
    cubeRootThree ^ 2
```

Mathematical meaning. $\mathbb{Z}[\theta]$ is exactly $\mathbb{Z}$-combinations of `1, θ, θ^2`.

Implementation. `Algebra.adjoin_induction`; mul-case uses θ^3=3, θ^4=3θ reductions + `ring`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_adjoin_eq :
    Subalgebra.toSubmodule (Algebra.adjoin ℤ ({cubeRootThree} : Set K)) =
      Submodule.span ℤ (Set.range integralBasisVectors)
```

Mathematical meaning. submodule form of `mem_adjoin_iff`.

Implementation. `mem_adjoin_iff` + `Submodule.span_induction`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem norm_eq_normForm (x y z : ℚ) :
    Algebra.norm ℚ (x • (1 : K) + y • cubeRootThree + z • cubeRootThree ^
    2) =
      x ^ 3 + 3 * y ^ 3 + 9 * z ^ 3 - 9 * x * y * z
```

Mathematical meaning. The cubic norm form of $\mathbb{Q}(\sqrt[3]{3})$. Backbone of all norm/ideal computations.

Implementation. `leftMulMatrix` of θ is the companion matrix `!![0,0,3;1,0,0;0,1,0]`; `Algebra.norm_eq_matrix_det` + `Matrix.det_fin_three` + `ring`.

```
theorem discr_powerBasis_eq :
    Algebra.discr ℚ (AdjoinRoot.powerBasis
      (show cubePolynomial ≠ 0 by intro h; simpa using congrArg natDegree
    h)).basis
      = (-243 : ℚ)
```

Mathematical meaning. Discriminant of the power basis `1, θ, θ^2` is `-243 = -3^5`.

Implementation. `Algebra.discr_powerBasis_eq_norm` → `N(f'(θ)) = N(3θ^2) = 243` via `norm_eq_normForm 0 0 3`, sign from `(-1)^(3·2/2)`.

```
theorem three_dvd_of_normForm_eq (a b c m : ℤ)
    (h : a ^ 3 + 3 * b ^ 3 + 9 * c ^ 3 - 9 * a * b * c = 27 * m) :
    3 ∣ a ∧ 3 ∣ b ∧ 3 ∣ c
```

Mathematical meaning. divisibility input to the peeling step.

Implementation. `Int.prime_three`; successively extract 3|a, then 3|b, then 3|c by `dvd_of_dvd_pow` + cancellation.

```
theorem exists_intCast_norm (z : K) (hz : IsIntegral ℤ z) :
    ∃ m : ℤ, Algebra.norm ℚ z = (m : ℚ)
```

Mathematical meaning. norm of an integral element is a rational integer.

Implementation. `Algebra.isIntegral_norm` + `IsIntegrallyClosed.isIntegral_iff`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem peel (z : K) (hz : IsIntegral ℤ z)
    (h3z : (3 : K) * z ∈ Algebra.adjoin ℤ ({cubeRootThree} : Set K)) :
    z ∈ Algebra.adjoin ℤ ({cubeRootThree} : Set K)
```

Mathematical meaning. peeling lemma – heart of the index argument ([$\mathcal{O}$ K : $\mathbb{Z}[\theta]$] not divisible by 3).

Implementation. write `3z = a•1+b•θ+c•θ²`; `N(3z)=27 N(z)` so norm form $\equiv$`0 (mod 27)`; `three_dvd_of_normForm_eq` gives 3|a,b,c; divide.

```
theorem integralClosure_eq_adjoin :
    integralClosure ℤ K = Algebra.adjoin ℤ ({cubeRootThree} : Set K)
```

Mathematical meaning. $\mathbb{Z}[\theta]$ is the maximal order – the field is monogenic ($3 \not\equiv \pm 1 \bmod 9$).

Implementation. $\supseteq$ trivial; $\subseteq$: `Algebra.discr_mul_isIntegral_mem_adjoin` puts `discr·z = −243 z` $\in \mathbb{Z}[\theta]$, then five rounds of `peel` strip 3^5.

```
theorem span_integralBasisVectors_eq :
    Submodule.span ℤ (Set.range integralBasisVectors) =
      Subalgebra.toSubmodule (integralClosure ℤ K)
```

Mathematical meaning. 1, θ, θ^2 span $\mathcal{O}$ K as $\mathbb{Z}$-lattice in K.

Implementation. `integralClosure_eq_adjoin` + `span_adjoin_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def integralBasisVectorsInt : Fin 3 → 𝒪 K := ![1, thetaInt,
    thetaInt ^ 2]
```

Mathematical meaning. integral basis vectors as elements of $\mathcal{O}$ K.

Implementation. The displayed RHS is the packaged construction `definitional` specialized to this field.

```
noncomputable def ringOfIntegersLinearMap : 𝒪 K →ₗ[ℤ] K :=
  (Algebra.linearMap (𝒪 K) K).restrictScalars ℤ
```

Mathematical meaning. $\mathbb{Z}$-linear embedding $\mathcal{O}$ K $\hookrightarrow$ K.

Implementation. `(Algebra.linearMap (𝒪 K) K).restrictScalars ℤ`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ringOfIntegersLinearMap_injective :
    Function.Injective ringOfIntegersLinearMap
```

Mathematical meaning. that embedding is injective.

Implementation. The proof applies the packaged result `FaithfulSMul.algebraMap_injective`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem algebraMap_integralBasisVectorsInt :
    (fun i => (algebraMap (𝒪 K) K) (integralBasisVectorsInt i)) =
    integralBasisVectors
```

Mathematical meaning. embedding sends $\mathcal{O}$ K-generators to K-vectors.

Implementation. `fin_cases` + `RingOfIntegers.map_mk`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem linearIndependent_integralBasisVectorsInt :
    LinearIndependent ℤ integralBasisVectorsInt
```

Mathematical meaning. `1, θ, θ^2` $\mathbb{Z}$-independent in $\mathcal{O}$ K.

Implementation. $\mathbb{Q}$-independence of power basis $\to$ `restrict_scalars'` $\to$ `of_comp` along injective embedding.

```
theorem span_integralBasisVectorsInt_eq_top :
    Submodule.span ℤ (Set.range integralBasisVectorsInt) = ⊤
```

Mathematical meaning. `1, θ, θ^2` span $\mathcal{O}$ K.

Implementation. push span through injective embedding to `span_integralBasisVectors_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def concreteIntegralBasis : Basis (Fin 3) ℤ (𝒪 K) :=
  Basis.mk linearIndependent_integralBasisVectorsInt
    (span_integralBasisVectorsInt_eq_top).ge
```

Mathematical meaning. genuine $\mathbb{Z}$-basis `1, θ, θ^2` of $\mathcal{O}$ `K`. Used to compute `finrank ℤ (𝓞 K) = 3`.

Implementation. `Basis.mk` from independence + spanning. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem concreteIntegralBasis_apply (i : Fin 3) :
    (algebraMap (𝓞 K) K) (concreteIntegralBasis i) = integralBasisVectors
    i
```

Mathematical meaning. basis vectors are the displayed `1, θ, θ^2`.

Implementation. `Basis.mk_apply` + `algebraMap_integralBasisVectorsInt`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_integralBasisVectors_eq_integralBasis :
    Submodule.span ℤ (Set.range integralBasisVectors) =
      Submodule.span ℤ (Set.range (NumberField.integralBasis K))
```

Mathematical meaning. displayed basis spans the same lattice as mathlib's `NumberField.integralBasis K` (unimodular change of basis, not a reindexing).

Implementation. `span_integralBasisVectors_eq` + `NumberField.mem_span_integralBasis` both directions. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev concreteRingOfIntegers : Subalgebra ℤ K := Algebra.adjoin ℤ
    ({cubeRootThree} :
    Set K)
```

Mathematical meaning. the order $\mathbb{Z}[\theta]$ inside `K`.

Implementation. The displayed RHS is the packaged construction `definitional` specialized to this field.

```
theorem ringOfIntegers_eq : integralClosure ℤ K = concreteRingOfIntegers
```

Mathematical meaning. $\mathcal{O}$ `K` = $\mathbb{Z}[\theta]$ (subalgebra restatement; monogenic).

Implementation. The proof applies the packaged result `integralClosure_eq_adjoin`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem intNorm_eq_normForm (x y z : ℤ) :
    Algebra.norm ℤ (x • (1 : 𝓞 K) + y • thetaInt + z • thetaInt ^ 2) =
      x ^ 3 + 3 * y ^ 3 + 9 * z ^ 3 - 9 * x * y * z
```

Mathematical meaning. integral norm form on $\mathcal{O}$ `K`. The single most-reused lemma in PrimeSplitting.

Implementation. cast to $\mathbb{Q}$ via `Algebra.coe_norm_int`, reduce to `norm_eq_normForm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isUnit_iff_normForm (x y z : ℤ) :
    IsUnit (x • (1 : 𝒪 K) + y • thetaInt + z • thetaInt ^ 2) ↔
      x ^ 3 + 3 * y ^ 3 + 9 * z ^ 3 - 9 * x * y * z = 1 ∨
        x ^ 3 + 3 * y ^ 3 + 9 * z ^ 3 - 9 * x * y * z = -1
```

Mathematical meaning. units of $\mathbb{Z}[\theta] \leftrightarrow x^3+3y^3+9z^3-9xyz = \pm 1$.

Implementation. `NumberField.isUnit_iff_norm` + `intNorm_eq_normForm` + `abs_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

5.5 `Discriminant.lean`

```
theorem numberField_discr_eq : NumberField.discr K = (-243 : ℤ)
```

Mathematical meaning. field discriminant $\mathrm{disc}(\mathbb{Q}(\sqrt[3]{3})) = -243 = -3^5$ (so 3 is the only ramified prime).

Implementation. power basis $1, \theta, \theta^2$ and `NumberField.integralBasis K` span the same lattice (unimodular change-of-basis), so `Algebra.discr_eq_discr_of_toMatrix_coeff_isIntegral` transfers `discr_powerBasis_eq = -243`; cast $\mathbb{Q} \to \mathbb{Z}$.

```
theorem integralBasis_discr :
    Algebra.discr ℚ (NumberField.integralBasis K) = (-243 : ℚ)
```

Mathematical meaning. discriminant of mathlib's canonical integral basis is -243.

Implementation. `NumberField.coe_discr` + `numberField_discr_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

5.6 `PrimeSplitting.lean`

```
theorem span_three_eq : Ideal.span {(3 : 𝒪 K)} = Ideal.span {thetaInt} ^ 3
```

Mathematical meaning. 3 totally ramified: $(3) = (\theta)^3$.

Implementation. `Ideal.span_singleton_pow` + `thetaInt_cube`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_span_theta : Ideal.absNorm (Ideal.span {thetaInt}) = 3
```

Mathematical meaning. (θ) has norm 3 (`f = 1`).

Implementation. `intNorm_eq_normForm 0 1 0` $(= 3)$ + `absNorm_span_singleton`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isPrime_span_theta : (Ideal.span {thetaInt}).IsPrime
```

Mathematical meaning. (θ) is prime.

Implementation. `Ideal.isPrime_of_irreducible_absNorm` (norm 3 prime). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_span_theta :
    (Ideal.span {thetaInt}).LiesOver (Ideal.span {(3 : ℤ)})
```

Mathematical meaning. (θ) lies over (3).

Implementation. `Ideal.liesOver_span_iff` + `span_three_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem inertiaDeg_span_theta :
    (Ideal.span {(3 : ℤ)}).inertiaDeg (Ideal.span {thetaInt}) = 1
```

Mathematical meaning. residue degree $f = 1$ over 3.

Implementation. `Ideal.absNorm_eq_pow_inertiaDeg'` ($3 = 3^f \implies f = 1$). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramificationIdx_span_theta :
    (Ideal.span {(3 : ℤ)}).ramificationIdx (Ideal.span {thetaInt}) = 3
```

Mathematical meaning. ramification index $e = 3$ (totally ramified).

Implementation. `Ideal.ramificationIdx_spec`: $(3) \subseteq (\theta)^3$ from `span_three_eq`, and $\not\subseteq (\theta)^4$ via norm bound ($81 \nmid 27$).

```
theorem eq_span_theta_of_three_mem (P : Ideal (𝓞 K)) (hP : P.IsPrime) (h3
    : (3 : 𝓞 K) ∈
    P) :
    P = Ideal.span {thetaInt}
```

Mathematical meaning. uniqueness of the prime above 3.

Implementation. $3 = \theta^3 \in P \implies \theta \in P$ (`mem_of_pow_mem`), then maximality `eq_of_le`.

```
theorem two_eq_mul : (2 : 𝓞 K) = (thetaInt - 1) * (thetaInt ^ 2 +
    thetaInt + 1)
```

Mathematical meaning. factorization of 2 in $\mathbb{Z}[\theta]$.

Implementation. The proof applies the packaged result `linear_combination -thetaInt_cube`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem span_two_eq :
    Ideal.span {(2 : 𝒪 K)} =
      Ideal.span {thetaInt - 1} * Ideal.span {thetaInt ^ 2 + thetaInt + 1}
```

Mathematical meaning. `2` splits as degree-1 $\times$ degree-2 prime (matching `X^3-3 ≡ (X+1)(X^2+X+1) mod 2`).

Implementation. `span_singleton_mul_span_singleton` + `two_eq_mul`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_span_theta_sub_one : Ideal.absNorm (Ideal.span {thetaInt
    - 1}) = 2
```

Mathematical meaning. `(θ-1)` has norm 2 (`f = 1`).

Implementation. `intNorm_eq_normForm (-1) 1 0`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isPrime_span_theta_sub_one : (Ideal.span {thetaInt - 1}).IsPrime
```

Mathematical meaning. `(θ-1)` prime.

Implementation. `isPrime_of_irreducible_absNorm` (norm 2). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_span_theta_sq_add_theta_add_one :
    Ideal.absNorm (Ideal.span {thetaInt ^ 2 + thetaInt + 1}) = 4
```

Mathematical meaning. degree-2 prime above 2 has norm 4 (`f = 2`).

Implementation. `intNorm_eq_normForm 1 1 1`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_ne_two_of_quadFactor_mem (P : Ideal (𝒪 K)) (hP :
    P.IsMaximal)
    (hmem : thetaInt ^ 2 + thetaInt + 1 ∈ P) : Ideal.absNorm P ≠ 2
```

Mathematical meaning. no norm-2 prime contains $\theta^2+\theta+1$ (formal "`X^2+X+1` irreducible mod 2").

Implementation. residue field $\mathbb{F}_2$ has only `0,1`; `t^2+t+1 = 0` has no root (case split).

```
theorem eq_span_quadFactor_of_mem (P : Ideal (𝒪 K)) (hP : P.IsPrime)
    (hmem : thetaInt ^ 2 + thetaInt + 1 ∈ P) :
    P = Ideal.span {thetaInt ^ 2 + thetaInt + 1}
```

Mathematical meaning. uniqueness of the degree-2 prime above 2.

Implementation. norm P | 4, $\neq$1, $\neq$2 $\Longrightarrow$ =4; cofactor norm 1 hence $\top$.

```
theorem isPrime_span_theta_sq_add_theta_add_one :
    (Ideal.span {thetaInt ^ 2 + thetaInt + 1}).IsPrime
```

Mathematical meaning. (θ^2+θ+1) prime.

Implementation. enclose in maximal P, eq_span_quadFactor_of_mem forces equality. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_span_theta_sub_one :
    (Ideal.span {thetaInt - 1}).LiesOver (Ideal.span {(2 : ℤ)})

theorem liesOver_span_theta_sq_add_theta_add_one :
    (Ideal.span {thetaInt ^ 2 + thetaInt + 1}).LiesOver (Ideal.span {(2 :
    ℤ)})
```

Mathematical meaning. both primes lie over (2).

Implementation. liesOver_span_iff + two_eq_mul membership. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem inertiaDeg_span_theta_sub_one :
    (Ideal.span {(2 : ℤ)}).inertiaDeg (Ideal.span {thetaInt - 1}) = 1

theorem inertiaDeg_span_theta_sq_add_theta_add_one :
    (Ideal.span {(2 : ℤ)}).inertiaDeg (Ideal.span {thetaInt ^ 2 +
    thetaInt + 1}) = 2
```

Mathematical meaning. residue degrees of the two primes above 2.

Implementation. absNorm_eq_pow_inertiaDeg' (2 = 2^f, 4 = 2^f); Nat.pow_right_injective. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem eq_of_two_mem (P : Ideal (𝒪 K)) (hP : P.IsPrime) (h2 : (2 : 𝒪 K)
    ∈ P) :
    P = Ideal.span {thetaInt - 1} ∨ P = Ideal.span {thetaInt ^ 2 +
    thetaInt + 1}
```

Mathematical meaning. only primes above 2 are these two principal ones (drives class-number argument at p=2).

Implementation. two_eq_mul $\in$ P $\Longrightarrow$ one factor in P; then the respective uniqueness lemma.

```
theorem absNorm_span_seven : Ideal.absNorm (Ideal.span {(7 : 𝒪 K)}) = 343
```

Mathematical meaning. `(7)` has norm 7^3 (inert).

Implementation. `intNorm_eq_normForm 7 0 0` = 343. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_ne_seven (P : Ideal (𝒪 K)) (_hP : P.IsMaximal) :
    Ideal.absNorm P ≠ 7
```

Mathematical meaning. no prime with residue field $\mathbb{F}_7$: 3 is not a cube mod 7.

Implementation. `ZMod.ringEquivOfPrime` gives `(𝒪 K)/P ≅ ZMod 7`; image `s` of θ satisfies `s^3 = 3`; `revert s; decide` (no solution).

```
theorem isPrime_span_seven : (Ideal.span {(7 : 𝒪 K)}).IsPrime
```

Mathematical meaning. 7 inert (`(7)` prime, `f = 3`).

Implementation. maximal `P ⊇ (7)` has norm | 7^3, ≠`1,7,49` (no norm-7 prime; norm-49 would force one), hence `343 ⟹ P = (7)`.

```
theorem liesOver_span_seven :
    (Ideal.span {(7 : 𝒪 K)}).LiesOver (Ideal.span {(7 : ℤ)})

theorem inertiaDeg_span_seven :
    (Ideal.span {(7 : ℤ)}).inertiaDeg (Ideal.span {(7 : 𝒪 K)}) = 3
```

Mathematical meaning. inert prime data.

Implementation. `liesOver_span_iff`; `absNorm_eq_pow_inertiaDeg'` (7^3 = `7^f`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def primeAbove61 (r : ℤ) : Ideal (𝒪 K) := Ideal.span {(61 : 𝒪 K),
    thetaInt - (r : 𝒪 K)}
```

Mathematical meaning. the prime `(61, θ − r)` above 61, for `r` a cube root of 3 mod 61 (`r ∈ {4,5,52}`).

Implementation. The displayed RHS is the packaged construction `definitional` specialized to this field.

```
theorem absNorm_span_theta_sub_four : Ideal.absNorm (Ideal.span {thetaInt
    - 4}) = 61

theorem absNorm_span_gen_five :
    Ideal.absNorm (Ideal.span {1 + 2 * thetaInt + 2 * thetaInt ^ 2}) = 61

theorem absNorm_span_gen_fiftytwo :
    Ideal.absNorm (Ideal.span {2 - 2 * thetaInt - thetaInt ^ 2}) = 61
```

Mathematical meaning. the three principal generators above 61 each have norm 61.

Implementation. `intNorm_eq_normForm` at `(-4,1,0)`, `(1,2,2)`, `(2,-2,-1)`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem primeAbove61_four_eq : primeAbove61 4 = Ideal.span {thetaInt - 4}

theorem primeAbove61_five_eq :
    primeAbove61 5 = Ideal.span {1 + 2 * thetaInt + 2 * thetaInt ^ 2}

theorem primeAbove61_fiftytwo_eq :
    primeAbove61 52 = Ideal.span {2 - 2 * thetaInt - thetaInt ^ 2}
```

Mathematical meaning. each `(61, θ-r)` is principal (concrete split-prime witnesses).

Implementation. `le_antisymm`: `61 ∈ (gen)` via `absNorm_mem`, and explicit `linear_combination`/`mem_span_pair` cofactor identities (e.g. `θ-5 = (1+2θ+2θ²)(1-θ)`).

```
theorem span_sixtyOne_eq :
    Ideal.span {(61 : 𝒪 K)} = primeAbove61 4 * primeAbove61 5 *
    primeAbove61 52
```

Mathematical meaning. 61 splits completely: `61 ≡ 1 (mod 3)` and 3 is a cubic residue mod 61.

Implementation. rewrite each prime as a principal span; product `linear_combination` from θ^3=3.

```
theorem isPrime_primeAbove61 (r : ℤ) (hr : r = 4 ∨ r = 5 ∨ r = 52) :
    (primeAbove61 r).IsPrime

theorem absNorm_primeAbove61 (r : ℤ) (hr : r = 4 ∨ r = 5 ∨ r = 52) :
    Ideal.absNorm (primeAbove61 r) = 61

theorem liesOver_primeAbove61 (r : ℤ) (hr : r = 4 ∨ r = 5 ∨ r = 52) :
    (primeAbove61 r).LiesOver (Ideal.span {(61 : ℤ)})

theorem inertiaDeg_primeAbove61 (r : ℤ) (hr : r = 4 ∨ r = 5 ∨ r = 52) :
    (Ideal.span {(61 : ℤ)}).inertiaDeg (primeAbove61 r) = 1
```

Mathematical meaning. split-prime data above 61.

Implementation. reduce to principal forms; `isPrime_of_irreducible_absNorm`; `absNorm_eq_pow_inertiaDeg'`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem primeAbove61_pairwise_ne :
    primeAbove61 4 ≠ primeAbove61 5 ∧ primeAbove61 4 ≠ primeAbove61 52 ∧
      primeAbove61 5 ≠ primeAbove61 52
```

Mathematical meaning. confirms `61` splits completely (3 distinct primes, unramified).

Implementation. differences θ−`r` − θ−`s` give a unit in any shared prime ($\mathbb{Z}$-combination yielding 1) – contradiction with `ne_top`.

```
private theorem adjoin_thetaInt_eq_top : Algebra.adjoin ℤ ({thetaInt} :
    Set (𝒪 K)) = ⊤
```

Mathematical meaning. $\mathcal{O}$ `K` = $\mathbb{Z}$`[thetaInt]` at the $\mathcal{O}$ `K`-level (for the power basis below).

Implementation. `span_integralBasisVectorsInt_eq_top` + `Submodule.span_induction`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private noncomputable abbrev thetaIntPowerBasis : PowerBasis ℤ (𝒪 K) :=
  PowerBasis.ofAdjoinEqTop' (NumberField.RingOfIntegers.isIntegral
    thetaInt)
    adjoin_thetaInt_eq_top

private theorem thetaIntPowerBasis_gen : thetaIntPowerBasis.gen = thetaInt
```

Mathematical meaning. the $\mathbb{Z}$-power-basis of $\mathcal{O}$ `K` generated by θ. Feeds Common's `ResidueHom` machinery.

Implementation. `PowerBasis.ofAdjoinEqTop'` from `adjoin_thetaInt_eq_top`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem thetaInt_minpoly_eq : minpoly ℤ thetaInt = X ^ 3 - C (3 :
    ℤ)
```

Mathematical meaning. integral minimal polynomial of θ.

Implementation. `minpoly.isIntegrallyClosed_eq_field_fractions` lifts the $\mathbb{Q}$-minpoly; `map_injective` $\mathbb{Z}\to\mathbb{Q}$. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem isMaximal_span_pair (p : ℕ) [Fact p.Prime] (r : ℤ)
    (hr : ((r : ZMod p)) ^ 3 = 3) :
    (Ideal.span {(p : 𝒪 K), thetaInt - (r : 𝒪 K)}).IsMaximal

private theorem absNorm_span_pair (p : ℕ) (r : ℤ) (hr : ((r : ZMod p)) ^
    3 = 3) :
    Ideal.absNorm (Ideal.span {(p : 𝒪 K), thetaInt - (r : 𝒪 K)}) = p
```

Mathematical meaning. generic Kummer-Dedekind degree-1 prime constructor.

Implementation. delegate to Common `isMaximal_span_gen_sub` / `absNorm_span_gen_sub` on `thetaIntPowerBasis` (root condition from `thetaInt_minpoly_eq`).

```
theorem exists_degree_one_prime_iff_cube_root_mod_p (p : ℕ) (hp :
    p.Prime) (hp3 : p ≠
    3) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
      Ideal.absNorm P = p) ↔ ∃ r : ZMod p, r ^ 3 = (3 : ZMod p)
```

Mathematical meaning. the cubic residue splitting law away from the ramified prime.

Implementation. $\to$ residue field `ZMod p` (`ringEquivOfPrime`) gives $s^3 = 3$; $\leftarrow$ build `(p, θ−r)` via `isMaximal_span_pair`/`absNorm_span_pair`.

```
theorem ramified_iff_eq_three (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ^ 2) ↔ p =
    3
```

Mathematical meaning. the ramification law: p ramifies iff $p = 3$ (iff $p \mid \mathrm{disc} = -3^5$).

Implementation. $\to$ ramified $\Longrightarrow \neg$ `IsUnramifiedAt` (Common helpers `not_map_span_natCast_le_pow_succ_of_absNorm_span_eq_pow`, `not_isUnramifiedAt_of_` and related variants) $\Longrightarrow$ $P \mid$ `differentIdeal`; `absNorm P` $\mid 243$ (via `absNorm_differentIdeal`) forces $p \mid 3$. $\leftarrow$ `span_three_eq`.

```
theorem ramificationIdx_span_theta_sub_one :
    (Ideal.span {(2 : ℤ)}).ramificationIdx (Ideal.span {thetaInt - 1}) = 1

theorem ramificationIdx_span_theta_sq_add_theta_add_one :
    (Ideal.span {(2 : ℤ)}).ramificationIdx
      (Ideal.span {thetaInt ^ 2 + thetaInt + 1}) = 1

theorem ramificationIdx_span_seven :
    (Ideal.span {(7 : ℤ)}).ramificationIdx (Ideal.span {(7 : 𝒪 K)}) = 1

theorem ramificationIdx_primeAbove61 (r : ℤ) (hr : r = 4 ∨ r = 5 ∨ r =
    52) :
    (Ideal.span {(61 : ℤ)}).ramificationIdx (primeAbove61 r) = 1
```

Mathematical meaning. confirm 2, 7, 61 unramified.

Implementation. `ramificationIdx_spec`: $(p) \subseteq P^1$, and $\not\subset P^2$ since `ramified_iff_eq_three` would force $p = 3$.

```
abbrev cubeRootThreePolynomialMod (p : ℕ) : (ZMod p)[X] :=
  X ^ 3 - C (3 : ZMod p)
```

Mathematical meaning. reduction of X^3-3 mod p.

Implementation. The displayed RHS is the packaged construction `definitional` specialized to this field.

```
theorem absNorm_differentIdeal_eq_twoHundredFortyThree :
    Ideal.absNorm (differentIdeal ℤ (𝒪 K)) = 243
```

Mathematical meaning. the different has norm $243 = |\mathrm{disc}|$.

Implementation. `NumberField.absNorm_differentIdeal` + `numberField_discr_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem prime_dvd_differentIdeal_iff_eq_span_theta
    (P : Ideal (𝒪 K)) (hP : P.IsPrime) (hP0 : P ≠ ⊥) :
    P ∣ differentIdeal ℤ (𝒪 K) ↔ P = Ideal.span {thetaInt}
```

Mathematical meaning. (θ) is the only prime dividing the different (only ramified prime).

Implementation. $\to$ `absNorm P ∣ 243 = 3`$^5 \Longrightarrow$ `3 ∈ P` $\Longrightarrow$ `P = (`θ`)`; $\leftarrow$ `dvd_differentIdeal_iff` + `ramificationIdx_span_theta = 3 ≠ 1`.

5.7 `ClassGroup.lean`

```
abbrev cubeRootThreeFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  FractionalIdeal.spanSingleton (𝒪 K)⁰ cubeRootThree

abbrev ramifiedPrimeFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  (Ideal.span {thetaInt} : FractionalIdeal (𝒪 K)⁰ K)
```

Mathematical meaning. principal fractional ideal $(\sqrt[3]{3})$; and (θ) as a fractional ideal.

Implementation. `FractionalIdeal.spanSingleton`; coercion of `Ideal.span {thetaInt}`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cubeRootThree_mem_fractionalIdeal :
    cubeRootThree ∈ cubeRootThreeFractionalIdeal

theorem cubeRootThreeFractionalIdeal_absNorm :
    FractionalIdeal.absNorm cubeRootThreeFractionalIdeal = |Algebra.norm ℚ
    cubeRootThree|
```

Mathematical meaning. basic facts on the principal fractional ideal $(\sqrt[3]{3})$.

Implementation. `mem_spanSingleton_self`; `FractionalIdeal.absNorm_span_singleton`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramifiedPrimeFractionalIdeal_ne_zero :
    ramifiedPrimeFractionalIdeal ≠ 0
```

Mathematical meaning. (θ) nonzero as fractional ideal.

Implementation. `coeIdeal_ne_zero` + `absNorm_span_theta = 3 ≠ 0`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrt_243_bounds : (15.58 : ℝ) < Real.sqrt 243 ∧ Real.sqrt 243 <
    15.59
```

Mathematical meaning. numerical bound for the Minkowski computation ($\sqrt{243} = 9\sqrt{3} \approx 15.588$).

Implementation. `Real.sq_sqrt` + `nlinarith`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

Mathematical meaning. class number 1.

Implementation. `classNumber_eq_one_iff` + `isPrincipalIdealRing_of_isPrincipal_of_pow_le_of_mem_primesOver_of_mem_Icc`; Minkowski bound $(4/\pi)\cdot(3!/3^3)\cdot\sqrt{243} \approx 4.41 < 5$ (`Nat.floor_lt`, `pi_gt_d2`, `sqrt_243_bounds`), so only `p = 2, 3` matter; their primes are principal (`eq_of_two_mem`, `eq_span_theta_of_three_mem`, explicit generators $\theta-1$, $\theta^2+\theta+1$, θ).

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝒪 K)
```

Mathematical meaning. $\mathbb{Z}[\theta]$ is a PID.

Implementation. `classNumber_eq_one_iff` ← `classNumber_eq_one`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝒪 K)
```

Mathematical meaning. $\mathbb{Z}[\theta]$ is a UFD.

Implementation. `infer_instance` from PID. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classGroupStructure : Nonempty (ClassGroup (𝒪 K) ≃*
    Multiplicative (ZMod 1))
```

Mathematical meaning. class group trivial.

Implementation. `card = 1` $\Longrightarrow$ `Unique` $\Longrightarrow$ `MulEquiv.ofUnique`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev unitIdealRepresentative : (Ideal (𝒪 K))⁰ := ⟨(⊤ : Ideal (𝒪 K)), by
    simp⟩

theorem classGroupRepresentative (C : ClassGroup (𝒪 K)) :
    C = ClassGroup.mk0 unitIdealRepresentative
```

Mathematical meaning. unit ideal represents the unique class.

Implementation. The proof applies the packaged result `Subsingleton.elim`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

5.8 `Units.lean`

```
abbrev torsionGenerator : (𝒪 K)ˣ := -1

def torsionUnits : Set ((𝒪 K)ˣ) := {u | ∃ k : Fin 2, u = torsionGenerator
    ^ k.val}
```

Mathematical meaning. torsion generator -1 and the concrete torsion set $\{1,-1\}$.

Implementation. The displayed RHS is the packaged construction `definitional` specialized to this field.

```
theorem torsionUnits_eq : {u : (𝒪 K)ˣ | IsOfFinOrder u} = torsionUnits
```

Mathematical meaning. torsion of $\mathcal{O}\ K^\times$ is exactly $\{\pm 1\}$ (odd-degree field has a real place $\Longrightarrow$ only roots of unity ± 1).

Implementation. `torsion_eq_one_or_neg_one_of_odd_finrank` (degree 3 odd). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem torsionOrder_eq_two : NumberField.Units.torsionOrder K = 2
```

Mathematical meaning. $|\mu(K)| = 2$.

Implementation. The proof applies the packaged result `torsionOrder_eq_two_of_odd_finrank`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem unitRank_eq_one : NumberField.Units.rank K = 1
```

Mathematical meaning. Dirichlet rank 1 ($r_1 + r_2 - 1 = 1$).

Implementation. `Units.rank` def + `card InfinitePlace = 2` (from signature `(1,1)`).

```
noncomputable def fundamentalUnit : (𝒪 K)ˣ where
  val := thetaInt ^ 2 - 2
  inv := 2 * thetaInt ^ 2 + 3 * thetaInt + 4
```

```
  val_inv := by have h := thetaInt_cube; linear_combination (2 * thetaInt
    + 3) * h
  inv_val := by have h := thetaInt_cube; linear_combination (2 * thetaInt
    + 3) * h
```

Mathematical meaning. the fundamental unit $\theta^2 - 2 = \sqrt[3]{9} - 2$ (norm 1), with explicit inverse.

Implementation. `val_inv`/`inv_val` by `linear_combination (2θ+3)·thetaInt_cube`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def fundamentalUnitSystem : Fin (NumberField.Units.rank K)
    → (𝒪 K)ˣ :=
  fun _ => fundamentalUnit
```

Mathematical meaning. the rank-1 system of fundamental units (single unit θ^2-2).

Implementation. The displayed RHS is the packaged construction `constant function` specialized to this field.

```
theorem fundamentalUnitSystem_isFundamental :
    Subgroup.closure (Set.range fundamentalUnitSystem) ⊔
    NumberField.Units.torsion K = ⊤
```

Mathematical meaning. θ^2-2 (with torsion) generates all units – i.e. θ^2-2 is a genuine fundamental unit (not a proper power times a root of unity).

Implementation. `sorry`. Needs: an Artin-type unit-index bound – that no unit η with `1 < η_real < |∛9−2|` exists below the candidate, equivalently `[𝒪 Kˣ : ⟨−1, θ²−2⟩] = 1`. Mathlib has `fundSystem K` (an abstract fundamental system) but no lemma identifying a given explicit unit as fundamental via a regulator/embedding lower bound. Discharging it needs either a covolume/regulator-minimality comparison or an effective lower bound on unit logarithms – neither in mathlib.

```
theorem unit_eq_torsion_mul_fundamentalUnit_zpow (u : (𝒪 K)ˣ) :
    ∃ ζ : (𝒪 K)ˣ, ζ ∈ NumberField.Units.torsion K ∧ ∃ n : ℤ, u = ζ *
    fundamentalUnit ^ n
```

Mathematical meaning. every unit is $\pm(\theta^2-2)^n$ (all solutions of the cubic Pell eqn).

Implementation. `Subgroup.mem_sup` + `mem_closure_singleton` on `fundamentalUnitSystem_isFundamental` (inherits the sorry). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem exists_eq_torsion_mul_fundSystem_zpow (i₀ : Fin
    (NumberField.Units.rank K))
    (u : (𝒪 K)ˣ) :
    ∃ ζ ∈ NumberField.Units.torsion K, ∃ n : ℤ,
      u = ζ * NumberField.Units.fundSystem K i₀ ^ n
```

Mathematical meaning. rank-1 specialization using mathlib's abstract `fundSystem` (sorry-free).

Implementation. `closure_fundSystem_sup_torsion_eq_top` + `Subsingleton (Fin (rank K))` + `mem_closure_singleton`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem fundSystem_zpow_mem_torsion (i₀ : Fin (NumberField.Units.rank K))
    (n : ℤ)
    (hn : NumberField.Units.fundSystem K i₀ ^ n ∈
    NumberField.Units.torsion K) : n = 0
```

Mathematical meaning. the abstract fundamental unit has infinite order mod torsion (image is a basis vector).

Implementation. project to free quotient `(𝒪 K)ˣ/torsion` via `basisModTorsion`; `repr_self`.

```
theorem eq_one_or_neg_one_of_mem_torsion {ζ : (𝒪 K)ˣ}
    (hζ : ζ ∈ NumberField.Units.torsion K) : ζ = 1 ∨ ζ = -1
```

Mathematical meaning. pointwise torsion classification.

Implementation. `torsionUnits_eq` + `fin_cases`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem unitGroupStructure_bijective
    (i₀ : Fin (NumberField.Units.rank K)) (η : (𝒪 K)ˣ)
    (hη : η = NumberField.Units.fundSystem K i₀)
    (hneg_tor : (-1 : (𝒪 K)ˣ) ∈ NumberField.Units.torsion K) :
    Function.Bijective
      ((torsionPowerHom 2 (-1 : (𝒪 K)ˣ) (by rw [neg_one_sq])).coprod
        (rankOnePowerHom η))
```

Mathematical meaning. the structure map $\mu_2 \times \mathbb{Z} \to \mathcal{O}\ \mathrm{K}^\times$ is bijective.

Implementation. injectivity from `fundSystem_zpow_mem_torsion` + `(-1)` not in trivial-exponent image; surjectivity from `exists_eq_torsion_mul_fundSystem_zpow` + `eq_one_or_neg_one_of_mem_torsion`. Uses Common `torsionPowerHom`, `rankOnePowerHom`.

```
theorem unitGroupStructure :
    Nonempty ((𝒪 K)ˣ ≃* (Multiplicative (ZMod 2) × Multiplicative (Fin 1
    → ℤ)))
```

Mathematical meaning. unit group $\cong \mu_2 \times \mathbb{Z}$ (rank-1 structure). Sorry-free (uses abstract `fundSystem`).

Implementation. `MulEquiv.ofBijective` of the coprod map, via `unitGroupStructure_bijective`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem regulator_eq :
    NumberField.Units.regulator K = |Real.log ((3 : ℝ) ^ ((2 : ℝ) / 3) -
    2)|
```

Mathematical meaning. regulator = `|log(∛9 − 2)|` $\approx$ `2.5246` (covolume of the unit lattice). Analytic restatement that θ^2-2 is fundamental.

Implementation. real embedding σ sends the fundamental unit to `3^(2/3) − 2 > 0`; `regOfFamily_div_regulator` with index 1 (depends on `fundamentalUnitSystem_isFundamental` SORRY), then `regOfFamily_eq_det` collapses the 1×1 log matrix (`Matrix.det_eq_elem_of_subsingleton`).

5.9 `Euclidean.lean`

```
theorem normEuclidean (a b : 𝒪 K) (hb : b ≠ 0) :
    ∃ q r : 𝒪 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

Mathematical meaning. $\mathbb{Q}(\sqrt[3]{3})$ is norm-Euclidean – division-with-remainder in $\mathbb{Z}[\theta]$ for `|field norm|`. Classical for the smallest-discriminant pure cubic; strengthens `isPrincipalIdealRing`.

Implementation. `sorry`. Needs: a geometry-of-numbers / covering argument – for every `x ∈ K` there is `m ∈ ℤ[θ]` with `|N(x − m)| < 1` (inhomogeneous minimum of the cubic norm form `< 1`), by partitioning the fundamental domain and bounding $|x^3+3y^3+9z^3-9xyz|$ over the unit cube. Mathlib has no norm-Euclidean API and exposes no effective Minkowski/covering bound for an explicit norm form (only `EuclideanDomain` structures and the abstract `convexBodyLT`/`fundamentalCone` measure theory). Would be built from scratch as an interval/case analysis over the cube.

5.10 `LocalGlobal.lean`

```
def heightOneThree : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {thetaInt}
  isPrime := by
    exact isPrime_span_theta
  ne_bot := by
    intro h
    have hnorm := absNorm_span_theta
    rw [h, Ideal.absNorm_bot] at hnorm
    norm_num at hnorm

def heightOneTwoLinear : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {thetaInt - 1}
  isPrime := by
    exact isPrime_span_theta_sub_one
  ne_bot := by
```

```
    intro h
    have hnorm := absNorm_span_theta_sub_one
    rw [h, Ideal.absNorm_bot] at hnorm
    norm_num at hnorm

def heightOneTwoQuadratic : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {thetaInt ^ 2 + thetaInt + 1}
  isPrime := by
    exact isPrime_span_theta_sq_add_theta_add_one
  ne_bot := by
    intro h
    have hnorm := absNorm_span_theta_sq_add_theta_add_one
    rw [h, Ideal.absNorm_bot] at hnorm
    norm_num at hnorm

def heightOneSeven : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {(7 : 𝒪 K)}
  isPrime := by
    exact isPrime_span_seven
  ne_bot := by
    intro h
    have hnorm := absNorm_span_seven
    rw [h, Ideal.absNorm_bot] at hnorm
    norm_num at hnorm

def heightOneSixtyOneFour : HeightOneSpectrum (𝒪 K) where
  asIdeal := primeAbove61 4
  isPrime := by
    exact isPrime_primeAbove61 4 (Or.inl rfl)
  ne_bot := by
    intro h
    have hnorm := absNorm_primeAbove61 4 (Or.inl rfl)
    rw [h, Ideal.absNorm_bot] at hnorm
    norm_num at hnorm

def heightOneSixtyOneFive : HeightOneSpectrum (𝒪 K) where
  asIdeal := primeAbove61 5
  isPrime := by
    exact isPrime_primeAbove61 5 (Or.inr (Or.inl rfl))
  ne_bot := by
    intro h
    have hnorm := absNorm_primeAbove61 5 (Or.inr (Or.inl rfl))
```

```
    rw [h, Ideal.absNorm_bot] at hnorm
    norm_num at hnorm

def heightOneSixtyOneFiftyTwo : HeightOneSpectrum (𝒪 K) where
  asIdeal := primeAbove61 52
  isPrime := by
    exact isPrime_primeAbove61 52 (Or.inr (Or.inr rfl))
  ne_bot := by
    intro h
    have hnorm := absNorm_primeAbove61 52 (Or.inr (Or.inr rfl))
    rw [h, Ideal.absNorm_bot] at hnorm
    norm_num at hnorm
```

Mathematical meaning. the worked finite places – (θ) over 3, $(\theta-1)/(\theta^2+\theta+1)$ over 2, (7) inert, the three primes over 61.

Implementation. package each prime ideal with `isPrime_*` and `ne_bot` from its nonzero norm.

```
abbrev vThree : NumberField.FinitePlace K :=
  NumberField.FinitePlace.mk heightOneThree

example :
    (Ideal.span {(3 : ℤ)}).ramificationIdx heightOneThree.asIdeal = 3

example :
    (Ideal.span {(3 : ℤ)}).inertiaDeg heightOneThree.asIdeal = 1

example :
    (Ideal.span {(2 : ℤ)}).inertiaDeg heightOneTwoLinear.asIdeal = 1

example :
    (Ideal.span {(2 : ℤ)}).inertiaDeg heightOneTwoQuadratic.asIdeal = 2

example :
    (Ideal.span {(7 : ℤ)}).inertiaDeg heightOneSeven.asIdeal = 3

example :
    (Ideal.span {(61 : ℤ)}).inertiaDeg heightOneSixtyOneFour.asIdeal = 1 ∧
      (Ideal.span {(61 : ℤ)}).inertiaDeg heightOneSixtyOneFive.asIdeal =
    1 ∧
        (Ideal.span {(61 : ℤ)}).inertiaDeg
    heightOneSixtyOneFiftyTwo.asIdeal = 1
```

Mathematical meaning. confirm the splitting data at the finite-place level.

Implementation. discharge by the corresponding PrimeSplitting `ramificationIdx_*`/`inertiaDeg_*` lemmas. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev CompletionAtThree : Type :=
  heightOneThree.adicCompletion K
```

Mathematical meaning. completion of K at the totally-ramified place over 3.

Implementation. The displayed RHS is the packaged construction `definitional` specialized to this field.

```
abbrev InfiniteCompletion (v : NumberField.InfinitePlace K) : Type :=
  v.Completion

example
    (v : NumberField.InfinitePlace K) (hv :
    NumberField.InfinitePlace.IsReal v) :
    Nonempty (InfiniteCompletion v ≃+* ℝ)

example
    (v : NumberField.InfinitePlace K) (hv :
    NumberField.InfinitePlace.IsComplex v) :
    Nonempty (InfiniteCompletion v ≃+* ℂ)
```

Mathematical meaning. archimedean completions are $\mathbb{R}$ (the one real place) and $\mathbb{C}$ (the one complex place).

Implementation. `ringEquivRealOfIsReal` / `ringEquivComplexOfIsComplex`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem product_formula_cubeRootThree :
    ((∏ w : NumberField.InfinitePlace K, w cubeRootThree ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K, v cubeRootThree) = 1

example :
    ((∏ w : NumberField.InfinitePlace K,
          w (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪 K)) ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K,
          v (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪 K))) = 1
```

Mathematical meaning. the product formula for $\sqrt[3]{3}$ and for θ^2-2.

Implementation. `NumberField.prod_abs_eq_one` ($\sqrt[3]{3} \neq 0$, fundamental unit $\neq 0$). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev CubeRootThreeInfiniteAdeleRing : Type :=
  NumberField.InfiniteAdeleRing K

abbrev infinitePrincipalAdele_cubeRootThree :
    CubeRootThreeInfiniteAdeleRing :=
  algebraMap K CubeRootThreeInfiniteAdeleRing cubeRootThree

abbrev infinitePrincipalAdele_fundamentalUnit :
    CubeRootThreeInfiniteAdeleRing :=
  algebraMap K CubeRootThreeInfiniteAdeleRing (algebraMap (𝒪 K) K
    (fundamentalUnit : 𝒪
    K))

example :
    ‖infinitePrincipalAdele_cubeRootThree‖ = |Algebra.norm ℚ
    cubeRootThree|

example :
    DenseRange (algebraMap K CubeRootThreeInfiniteAdeleRing)
```

Mathematical meaning. infinite-adele packaging.

Implementation. `coe_norm_eq_abs_norm`; `denseRange_algebraMap`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem canonicalEmbedding_thetaInt_mem_integerLattice :
    NumberField.canonicalEmbedding K (algebraMap (𝒪 K) K thetaInt) ∈
      NumberField.canonicalEmbedding.integerLattice K
```

Mathematical meaning. θ lands in the integer lattice.

Implementation. The proof applies the packaged result `simp [integerLattice]`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
abbrev mixedEmbedding_cubeRootThree :
    NumberField.mixedEmbedding.mixedSpace K :=
  NumberField.mixedEmbedding K cubeRootThree

example
    (f : NumberField.InfinitePlace K → NNReal)
    (hf : ∀ w : NumberField.InfinitePlace K, w cubeRootThree < f w) :
    mixedEmbedding_cubeRootThree ∈
    NumberField.mixedEmbedding.convexBodyLT K f
```

```
example :
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone K)

example :
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone.normLeOne K)

theorem polarSpace_nonempty :
    Nonempty (NumberField.mixedEmbedding.polarSpace K)
```

Mathematical meaning. Minkowski-geometry packaging in the mixed space.

Implementation. `convexBodyLT_mem`; `measurableSet_*`; `infer_instance`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem house_cubeRootThree_pos :
    0 < NumberField.house cubeRootThree

theorem house_thetaInt_pos :
    0 < NumberField.house (algebraMap (𝒪 K) K thetaInt)
```

Mathematical meaning. the house of $\sqrt[3]{3}$ is positive.

Implementation. `one_le_house_of_isIntegral` ($\sqrt[3]{3}$ integral, nonzero). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev CubeRootThreeAdeleRing : Type :=
  NumberField.AdeleRing (𝒪 K) K

abbrev principalAdele_cubeRootThree : CubeRootThreeAdeleRing :=
  algebraMap K CubeRootThreeAdeleRing cubeRootThree

abbrev principalAdele_fundamentalUnit : CubeRootThreeAdeleRing :=
  algebraMap K CubeRootThreeAdeleRing (algebraMap (𝒪 K) K
    (fundamentalUnit : 𝒪 K))

theorem principalAdele_cubeRootThree_ne_zero :
    principalAdele_cubeRootThree ≠ 0

theorem principalAdele_fundamentalUnit_is_unit_everywhere :
    ∀ v : NumberField.FinitePlace K,
      v (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪 K)) = 1
```

Mathematical meaning. full-adele packaging; the fundamental unit is a unit at every finite place.

Implementation. `AdeleRing.algebraMap_injective`; `norm_eq_one_iff_notMem` (a unit lies in no maximal ideal).

```
abbrev cubeRootThreeDedekindZeta (s : ℂ) : ℂ :=
  NumberField.dedekindZeta K s

abbrev cubeRootThreeDedekindZetaResidue : ℝ :=
  NumberField.dedekindZeta_residue K

example :
    0 < cubeRootThreeDedekindZetaResidue

example :
    cubeRootThreeDedekindZetaResidue =
      (2 ^ NumberField.InfinitePlace.nrRealPlaces K *
          (2 * Real.pi) ^ NumberField.InfinitePlace.nrComplexPlaces K *
          NumberField.Units.regulator K * NumberField.classNumber K) /
        (NumberField.Units.torsionOrder K * Real.sqrt |NumberField.discr
    K|)
```

Mathematical meaning. Dedekind zeta and its residue via the analytic class number formula, all invariants instantiated (r_1=r_2=1, Reg, h=1, w=2, |disc|=243).

Implementation. `dedekindZeta_residue_pos`; the formula by `rfl`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

5.11 Summary for $\mathbb{Q}(\sqrt[3]{3})$

Field: `K` = $\mathbb{Q}(\sqrt[3]{3})$ = `AdjoinRoot (X^3 - 3)`, generator θ = $\sqrt[3]{3}$. Pure cubic field (the only degree>2 and only non-Galois example in `FieldExamples`): signature `(r₁,r₂) = (1,1)`, ring of integers $\mathbb{Z}[\theta]$ (monogenic, maximal order), discriminant `-243 = -3^5`, class number 1, unit group $\mu_2 \times \mathbb{Z}$ (Dirichlet rank 1, fundamental unit θ^2 `- 2 =` $\sqrt[3]{9}$ `- 2`). `K`/$\mathbb{Q}$ is not Galois (`Aut = 1`); its Galois closure is the S_3-splitting field $\mathbb{Q}(\sqrt[3]{3}, \zeta_3)$ of degree 6. Two genuine sorries remain: `fundamentalUnitSystem_isFundamental` (Artin-type unit-index bound) and `normEuclidean` (geometry-of-numbers covering argument).

All declarations live in `namespace NumberField.CubeRootThree` unless noted. $\mathcal{O}$ `K` denotes `NumberField.RingOfIntegers K`; θ = `cubeRootThree`, `thetaInt` its integral lift in $\mathcal{O}$ `K`. Common helpers referenced live in `namespace NumberField.FieldExamples` (file `FieldExamples/Common.lean`). Import chain: `Basic` → `Galois` → `Embeddings` → `RingOfIntegers` → `Discriminant` → `PrimeSplitting` → `ClassGroup` → `Units` → `Euclidean` → `LocalGlobal`.

6 $\mathbb{Q}(\zeta_3)$

Folder: `XYin/NumberField/FieldExamples/CyclotomicThree/` Field: `K = CyclotomicField 3 ℚ`, the third cyclotomic field = $\mathbb{Q}(\sqrt{-3})$, built via `IsCyclotomicExtension {3} ℚ K`; generator ζ a primitive cube root with Φ_3 = `X^2+X+1` (ζ^2+ζ+1 = 0). Degree 2, signature `(0,1)`, ring

of integers $\mathbb{Z}[\zeta]$ (Eisenstein integers), disc -3, class number 1, unit group μ_6 (rank 0, regulator 1), norm-Euclidean, CM field. Twin of `CubeRootThree` ($\mathbb{Q}(\sqrt[3]{3})$): there the cubic relation `r^3=3` drives everything; here the quadratic cyclotomic relation `r^2+r+1=0` plays the analogous structural role. All declarations in `namespace NumberField.CyclotomicThree` unless noted. Shared helpers from `XYin/NumberField/FieldExamples/Common.lean` (`namespace NumberField.FieldExamples`). Sorry-free: confirmed (grep clean). Rank-0 units, so no Artin-bound blockers (unlike ζ_5/ζ_8). Reduction-mod-p / Kummer-Dedekind machinery uses Common's ResidueHom (`isMaximal_span_gen_sub`, `absNorm_span_gen_sub`) rather than bespoke residue maps; only thin local wrappers remain.

Each block below begins with the displayed Lean declaration or object definition. For theorem, lemma, instance-proof, and example blocks the proof body is omitted; for definitions and abbreviations the displayed RHS is retained, because it is part of the mathematical object being documented.

6.1 `Basic.lean`

```
abbrev K : Type := CyclotomicField 3 ℚ
```

Mathematical meaning. The third cyclotomic field = $\mathbb{Q}(\sqrt{-3})$.

Implementation. The displayed RHS is the packaged construction `Direct abbrev` specialized to this field.

```
instance : IsCyclotomicExtension {3} ℚ K :=
  CyclotomicField.isCyclotomicExtension 3 ℚ
```

Mathematical meaning. The cyclotomic-extension structure. –

Implementation. `CyclotomicField.isCyclotomicExtension 3 ℚ`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev ζ : K := IsCyclotomicExtension.zeta 3 ℚ K
```

Mathematical meaning. Chosen primitive cube root of unity. –

Implementation. mathlib `IsCyclotomicExtension.zeta`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zeta_isPrimitiveRoot : IsPrimitiveRoot ζ 3
```

Mathematical meaning. The displayed Lean declaration has type `IsPrimitiveRoot ζ 3` –.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.zeta_spec`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem zeta_sq_add_zeta_add_one_eq_zero : ζ ^ 2 + ζ + 1 = 0
```

Mathematical meaning. The displayed Lean declaration has type `ζ ^ 2 + ζ + 1 = 0` – the defining quadratic relation ($\Phi_3(\zeta)=0$).

Implementation. `IsPrimitiveRoot.isRoot_cyclotomic + simpa`. The cyclotomic analogue of `cubeRootThree_cube`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem adjoin_zeta_eq_top : Algebra.adjoin ℚ ({ζ} : Set K) = ⊤
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.adjoin ℚ {ζ} = ⊤` –.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.adjoin_primitive_root_eq_top`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem degree_eq_two : Module.finrank ℚ K = 2
```

Mathematical meaning. The displayed Lean declaration has type `Module.finrank ℚ K = 2` – `[K:ℚ] = φ(3) = 2`.

Implementation. `IsCyclotomicExtension.Rat.finrank` + `Nat.totient_prime`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : Algebra.IsQuadraticExtension ℚ K where
  finrank_eq_two' := degree_eq_two
```

Mathematical meaning. Records degree 2 as a typeclass. –

Implementation. This instance is supplied by `degree_eq_two` after the concrete field has been shown to satisfy its hypotheses.

```
abbrev zetaThreePolynomial : ℚ[X] := X ^ 2 + X + 1
```

Mathematical meaning. The displayed Lean declaration has type `ℚ[X] := X ^ 2 + X + 1` – Φ_3.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
instance : Fact (Irreducible zetaThreePolynomial)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. rewrite as `cyclotomic 3 ℚ`, then `cyclotomic.irreducible_rat`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev KAdjoinRoot : Type := AdjoinRoot zetaThreePolynomial
```

Mathematical meaning. The displayed Lean declaration has type `AdjoinRoot zetaThreePolynomial` – the `AdjoinRoot (X^2+X+1)` model.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem zetaThreePolynomial_eq_minpoly : zetaThreePolynomial = minpoly ℚ ζ
```

Mathematical meaning. The displayed Lean declaration has type `zetaThreePolynomial = minpoly ℚ ζ`.

Implementation. rewrite as `cyclotomic 3 ℚ`, `IsPrimitiveRoot.minpoly_eq_cyclotomic_of_irreducible`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def zetaMinpolyAdjoinRootPresentation : IsAdjoinRoot K (minpoly ℚ ζ) :=
  IsAdjoinRoot.mkOfAdjoinEqTop
    ((IsCyclotomicExtension.integral {3} ℚ K).isIntegral ζ)
    (IsCyclotomicExtension.adjoin_primitive_root_eq_top
    zeta_isPrimitiveRoot)
```

Mathematical meaning. The displayed Lean declaration has type `IsAdjoinRoot K (minpoly ℚ ζ)` –.

Implementation. `IsAdjoinRoot.mkOfAdjoinEqTop` from integrality + `adjoin = ⊤`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev adjoinRootEquivCyclotomic : KAdjoinRoot ≃ₐ[ℚ] K :=
  (AdjoinRoot.algEquivOfEq ℚ zetaThreePolynomial (minpoly ℚ ζ)
    zetaThreePolynomial_eq_minpoly).trans
    zetaMinpolyAdjoinRootPresentation.adjoinRootAlgEquiv
```

Mathematical meaning. The displayed Lean declaration has type `KAdjoinRoot ≃ₐ[ℚ] K` – identifies the `AdjoinRoot` Φ_3 model with the cyclotomic one.

Implementation. `AdjoinRoot.algEquivOfEq` (Φ_3 = minpoly) ∘ `IsAdjoinRoot.adjoinRootAlgEquiv`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem adjoinRootEquivCyclotomic_root :
    adjoinRootEquivCyclotomic (AdjoinRoot.root zetaThreePolynomial) = ζ
```

Mathematical meaning. The displayed Lean declaration has type `adjoinRootEquivCyclotomic (AdjoinRoot.root and related terms) = ζ` –.

Implementation. `AdjoinRoot.algEquivOfEq_root` + `simp`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem two_mul_zeta_add_one_sq : (2 * ζ + 1) ^ 2 = (-3 : K)
```

Mathematical meaning. The displayed Lean declaration has type `(2*ζ + 1)^2 = -3` – exhibits $2\zeta+1 = \sqrt{-3}$.

Implementation. `calc` collapsing $4(\zeta^2+\zeta+1) - 3$ via the relation. –

```
abbrev sqrtNegThreePolynomial : ℚ[X] := X ^ 2 + 3
```

Mathematical meaning. The displayed Lean declaration has type `ℚ[X] := X ^ 2 + 3`. –.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
instance : Fact (Irreducible sqrtNegThreePolynomial)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `irreducible_of_degree_le_three_of_not_isRoot`; no rational root since $x^2+3 > 0$ (`nlinarith`).

```
abbrev QSqrtNegThree : Type := AdjoinRoot sqrtNegThreePolynomial

abbrev sqrtNegThreeRoot : QSqrtNegThree := AdjoinRoot.root
    sqrtNegThreePolynomial

abbrev zetaThreeRoot : KAdjoinRoot := AdjoinRoot.root zetaThreePolynomial
```

Mathematical meaning. The displayed Lean declaration has type `AdjoinRoot (X²+3)`; its root; the root of `AdjoinRoot (X²+X+1)`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtNegThreeRoot_sq : sqrtNegThreeRoot ^ 2 = (-3 : QSqrtNegThree)
```

Mathematical meaning. The displayed Lean declaration has type `sqrtNegThreeRoot ^ 2 = -3` –.

Implementation. `AdjoinRoot.eval₂_root` + `calc`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zetaThreeRoot_relation : zetaThreeRoot ^ 2 + zetaThreeRoot + 1 = 0
```

Mathematical meaning. The displayed Lean declaration has type `zetaThreeRoot ^ 2 + zetaThreeRoot + 1 = 0` –.

Implementation. `AdjoinRoot.eval₂_root`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def sqrtNegThreeToZetaThree : QSqrtNegThree →ₐ[ℚ] KAdjoinRoot :=
  AdjoinRoot.liftAlgHom sqrtNegThreePolynomial (Algebra.ofId ℚ
    KAdjoinRoot)
    (2 * zetaThreeRoot + 1) (by
      calc
        sqrtNegThreePolynomial.eval₂ (Algebra.ofId ℚ KAdjoinRoot) (2 *
    zetaThreeRoot +
    1)
            = (2 * zetaThreeRoot + 1) ^ 2 + 3 := by
              simp [sqrtNegThreePolynomial]
        _ = 4 * (zetaThreeRoot ^ 2 + zetaThreeRoot + 1) := by ring
        _ = 0 := by rw [zetaThreeRoot_relation]; ring)

@[simp]
theorem sqrtNegThreeToZetaThree_root :
    sqrtNegThreeToZetaThree sqrtNegThreeRoot = 2 * zetaThreeRoot + 1
```

Mathematical meaning. The displayed Lean declaration has type `QSqrtNegThree →ₐ[ℚ] KAdjoinRoot`, sending $\sqrt{-3} \mapsto 2\zeta+1$.

Implementation. `AdjoinRoot.liftAlgHom`; relation check $(2\zeta+1)^2+3 = 4(\zeta^2+\zeta+1) = 0$. `_root` is `@[simp]`.

```
def zetaThreeToSqrtNegThree : KAdjoinRoot →ₐ[ℚ] QSqrtNegThree :=
  AdjoinRoot.liftAlgHom zetaThreePolynomial (Algebra.ofId ℚ QSqrtNegThree)
    ((sqrtNegThreeRoot - 1) / 2) (by
      calc
        zetaThreePolynomial.eval₂ (Algebra.ofId ℚ QSqrtNegThree)
            ((sqrtNegThreeRoot - 1) / 2)
            = ((sqrtNegThreeRoot - 1) / 2) ^ 2 + (sqrtNegThreeRoot - 1) /
    2 + 1 := by
              simp [zetaThreePolynomial]
        _ = (sqrtNegThreeRoot ^ 2 + 3) / 4 := by ring
        _ = 0 := by rw [sqrtNegThreeRoot_sq]; norm_num)

@[simp]
theorem zetaThreeToSqrtNegThree_root :
    zetaThreeToSqrtNegThree zetaThreeRoot = (sqrtNegThreeRoot - 1) / 2
```

Mathematical meaning. The displayed Lean declaration has type `KAdjoinRoot →ₐ[ℚ] QSqrtNegThree`, sending $\zeta \mapsto (\sqrt{-3} - 1)/2$.

Implementation. `AdjoinRoot.liftAlgHom`; relation check via $(\sqrt{-3})^2=-3$. `_root` is `@[simp]`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def sqrtNegThreeEquivZetaThree : QSqrtNegThree ≃ₐ[ℚ] KAdjoinRoot where
  __ := sqrtNegThreeToZetaThree
  invFun := zetaThreeToSqrtNegThree
  left_inv x := by
    have hcomp :
        zetaThreeToSqrtNegThree.comp sqrtNegThreeToZetaThree =
          AlgHom.id ℚ QSqrtNegThree := by
      apply AdjoinRoot.algHom_ext
      change zetaThreeToSqrtNegThree (sqrtNegThreeToZetaThree
    sqrtNegThreeRoot) =
        sqrtNegThreeRoot
      rw [sqrtNegThreeToZetaThree_root]
      simp only [map_add, map_mul, map_one, map_ofNat,
    zetaThreeToSqrtNegThree_root]
      ring
    change (zetaThreeToSqrtNegThree.comp sqrtNegThreeToZetaThree) x =
      (AlgHom.id ℚ QSqrtNegThree) x
    rw [hcomp]
  right_inv x := by
    have hcomp :
        sqrtNegThreeToZetaThree.comp zetaThreeToSqrtNegThree =
          AlgHom.id ℚ KAdjoinRoot := by
      apply AdjoinRoot.algHom_ext
      change sqrtNegThreeToZetaThree (zetaThreeToSqrtNegThree
    zetaThreeRoot) =
        zetaThreeRoot
      rw [zetaThreeToSqrtNegThree_root]
      simp only [map_sub, map_div₀, map_one, map_ofNat,
    sqrtNegThreeToZetaThree_root]
      ring_nf
    change (sqrtNegThreeToZetaThree.comp zetaThreeToSqrtNegThree) x =
      (AlgHom.id ℚ KAdjoinRoot) x
    rw [hcomp]

@[simp]
theorem sqrtNegThreeEquivZetaThree_root :
    sqrtNegThreeEquivZetaThree sqrtNegThreeRoot = 2 * zetaThreeRoot + 1
```

Mathematical meaning. The displayed Lean declaration has type `QSqrtNegThree ≃ₐ[ℚ] KAdjoinRoot` – the two `AdjoinRoot` models are the same field.

Implementation. bundle the two maps; `left_inv`/`right_inv` via `AdjoinRoot.algHom_ext` + `ring`. `_root @[simp]`.

```
def sqrtNegThreeEquivCyclotomic : QSqrtNegThree ≃ₐ[ℚ] K :=
  sqrtNegThreeEquivZetaThree.trans adjoinRootEquivCyclotomic

theorem sqrtNegThreeEquivCyclotomic_root :
    sqrtNegThreeEquivCyclotomic sqrtNegThreeRoot = 2 * ζ + 1
```

Mathematical meaning. The displayed Lean declaration has type `QSqrtNegThree ≃ₐ[ℚ] K`, root$\mapsto$ `2ζ+1` – $\mathbb{Q}(\sqrt{-3})$-AdjoinRoot model is the cyclotomic field.

Implementation. The proof applies the packaged result `sqrtNegThreeEquivZetaThree.trans adjoinRootEquivCyclotomic`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
instance : Fact (∀ r : ℚ, r ^ 2 ≠ (-3 : ℚ) + (0 : ℚ) * r)
```

Mathematical meaning. nondegeneracy hypothesis for `QuadraticAlgebra ℚ (-3) 0`. –

Implementation. `nlinarith` ($r^2 \geq 0$). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev QSqrtNegThreeQuadratic : Type := QuadraticAlgebra ℚ (-3 : ℚ) 0

abbrev sqrtNegThreeQuadraticRoot : QSqrtNegThreeQuadratic :=
    QuadraticAlgebra.omega
```

Mathematical meaning. The displayed Lean declaration has type `QuadraticAlgebra ℚ (-3) 0`; its `QuadraticAlgebra.omega`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem sqrtNegThreeQuadraticRoot_sq :
    sqrtNegThreeQuadraticRoot ^ 2 = (-3 : QSqrtNegThreeQuadratic)
```

Mathematical meaning. The displayed Lean declaration has type `sqrtNegThreeQuadraticRoot ^ 2 = -3` –.

Implementation. `QuadraticAlgebra.omega_mul_omega_eq_add` + `simp`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private noncomputable def sqrtNegThreeQuadraticEquivAdjoinRoot :
    QSqrtNegThreeQuadratic ≃ₐ[ℚ] QSqrtNegThree := by
  let f : QSqrtNegThreeQuadratic →ₐ[ℚ] QSqrtNegThree :=
    QuadraticAlgebra.lift ⟨sqrtNegThreeRoot, by
      rw [show sqrtNegThreeRoot * sqrtNegThreeRoot = sqrtNegThreeRoot ^ 2
```

```
    by ring,
        sqrtNegThreeRoot_sq]
      simp⟩
  let g : QSqrtNegThree →ₐ[ℚ] QSqrtNegThreeQuadratic :=
    AdjoinRoot.liftAlgHom sqrtNegThreePolynomial (Algebra.ofId ℚ
    QSqrtNegThreeQuadratic)
      sqrtNegThreeQuadraticRoot (by
        calc
          sqrtNegThreePolynomial.eval₂ (Algebra.ofId ℚ
    QSqrtNegThreeQuadratic)
              sqrtNegThreeQuadraticRoot = sqrtNegThreeQuadraticRoot ^ 2 +
    3 := by
            simp [sqrtNegThreePolynomial]
          _ = 0 := by rw [sqrtNegThreeQuadraticRoot_sq]; ring)
  have hfroot : f sqrtNegThreeQuadraticRoot = sqrtNegThreeRoot := by
    change (QuadraticAlgebra.omega : QSqrtNegThreeQuadratic).re • (1 :
    QSqrtNegThree) +
      (QuadraticAlgebra.omega : QSqrtNegThreeQuadratic).im •
    sqrtNegThreeRoot =
        sqrtNegThreeRoot
    simp
  have hgroot : g sqrtNegThreeRoot = sqrtNegThreeQuadraticRoot := by
    change
      (AdjoinRoot.liftAlgHom sqrtNegThreePolynomial
        (Algebra.ofId ℚ QSqrtNegThreeQuadratic) sqrtNegThreeQuadraticRoot
    _)
        (AdjoinRoot.root sqrtNegThreePolynomial) =
    sqrtNegThreeQuadraticRoot
    exact AdjoinRoot.liftAlgHom_root sqrtNegThreePolynomial
      (Algebra.ofId ℚ QSqrtNegThreeQuadratic) sqrtNegThreeQuadraticRoot _
  refine AlgEquiv.ofAlgHom f g ?_ ?_
  · apply AdjoinRoot.algHom_ext
    change f (g sqrtNegThreeRoot) = sqrtNegThreeRoot
    rw [hgroot]
    exact hfroot
  · apply QuadraticAlgebra.algHom_ext
    change g (f sqrtNegThreeQuadraticRoot) = sqrtNegThreeQuadraticRoot
    rw [hfroot]
    exact hgroot

private theorem sqrtNegThreeQuadraticEquivAdjoinRoot_root :
    sqrtNegThreeQuadraticEquivAdjoinRoot sqrtNegThreeQuadraticRoot =
```

```
    sqrtNegThreeRoot
```

Mathematical meaning. The displayed Lean declaration has type `QSqrtNegThreeQuadratic` $\simeq_a$`[ℚ] QSqrtNegThree` –.

Implementation. `QuadraticAlgebra.lift` $\leftrightarrow$ `AdjoinRoot.liftAlgHom`, glued by `AlgEquiv.ofAlgHom`; `algHom_ext` on both sides. Both private.

```
noncomputable def sqrtNegThreeQuadraticEquivCyclotomic :
    QSqrtNegThreeQuadratic ≃ₐ[ℚ] K :=
  sqrtNegThreeQuadraticEquivAdjoinRoot.trans sqrtNegThreeEquivCyclotomic

theorem sqrtNegThreeQuadraticEquivCyclotomic_root :
    sqrtNegThreeQuadraticEquivCyclotomic sqrtNegThreeQuadraticRoot = 2 *
    ζ + 1
```

Mathematical meaning. The displayed Lean declaration has type `QSqrtNegThreeQuadratic` $\simeq_a$`[ℚ] K`, root $\mapsto$ $2\zeta+1$ – radical-quadratic-algebra model is the cyclotomic field.

Implementation. trans of the previous two equivs.

6.2 `Galois.lean`

```
theorem isGalois : IsGalois ℚ K
```

Mathematical meaning. The displayed Lean declaration has type `IsGalois ℚ K` –.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.isGalois`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
abbrev galEquivZMod : Gal(K / ℚ) ≃* (ZMod 3)ˣ :=
  IsCyclotomicExtension.Rat.galEquivZMod 3 K
```

Mathematical meaning. The displayed Lean declaration has type `Gal(K/ℚ) ≃* (ZMod 3)`$^\times$ – cyclotomic Galois theory identification.

Implementation. The displayed RHS is the packaged construction `IsCyclotomicExtension.Rat.galEquivZMod 3 K` specialized to this field.

```
theorem galEquivZMod_apply_zeta (σ : Gal(K/ℚ)) :
    σ ζ = ζ ^ (galEquivZMod σ).val.val
```

Mathematical meaning. The displayed Lean declaration has type `σ ζ = ζ ^ (galEquivZMod σ).val.val` – cyclotomic character.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.Rat.galEquivZMod_apply_of_pow_eq`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem galoisGroupStructure : Nonempty (Gal(K/ℚ) ≃* Multiplicative (ZMod
    2))
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty (Gal(K/ℚ) ≃* Multiplicative (ZMod 2))` – Gal cyclic of order 2 ($(\mathbb{Z}/3)^\times \cong \mathbb{Z}/2$).

Implementation. card via `IsGalois.card_aut_eq_finrank + finrank + totient_prime`; `mulEquivOfPrimeCardEq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

6.3 `Embeddings.lean`

```
theorem nrRealPlaces_eq_zero : InfinitePlace.nrRealPlaces K = 0
```

Mathematical meaning. The displayed Lean declaration has type `InfinitePlace.nrRealPlaces K = 0` –.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.Rat.nrRealPlaces_eq_zero`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem isTotallyComplex : IsTotallyComplex K
```

Mathematical meaning. The displayed Lean declaration has type `IsTotallyComplex K` –.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.Rat.isTotallyComplex`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem nrComplexPlaces_eq_one : InfinitePlace.nrComplexPlaces K = 1
```

Mathematical meaning. The displayed Lean declaration has type `InfinitePlace.nrComplexPlaces K = 1` –.

Implementation. `and related termsnrComplexPlaces_eq_totient_div_two + totient_prime`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_card_eq_one :
    Fintype.card (NumberField.InfinitePlace K) = 1
```

Mathematical meaning. The displayed Lean declaration has type `Fintype.card (InfinitePlace K) = 1` –.

Implementation. `card_eq_nrRealPlaces_add_nrComplexPlaces` + above. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_isComplex (v : NumberField.InfinitePlace K) :
    NumberField.InfinitePlace.IsComplex v
```

Mathematical meaning. The displayed Lean declaration has type ∀ `v, IsComplex v` –.

Implementation. The proof applies the packaged result `isTtotallyComplex.isComplex`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem exists_complexEmbedding_zeta :
    ∃ σ : K →+* ℂ, σ ζ = Complex.exp (2 * Real.pi * Complex.I / 3)
```

Mathematical meaning. The displayed Lean declaration has type ∃ σ `: K →+* ℂ,` σ ζ `= exp(2`π`i/3)` – exhibits an explicit complex embedding.

Implementation. `Complex.isPrimitiveRoot_exp`; lift via `AdjoinRoot.liftAlgHom` composed with `adjoinRootEquivCyclotomic.symm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

6.4 RingOfIntegers.lean

```
noncomputable def zetaInt : 𝒪 K := zeta_isPrimitiveRoot.toInteger
```

Mathematical meaning. The displayed Lean declaration has type $\mathcal{O}$ `K := zeta_isPrimitiveRoot.toInteger` – ζ as an algebraic integer. Pervasive downstream.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem zeta_pow_three : ζ ^ 3 = 1
```

Mathematical meaning. The displayed Lean declaration has type ζ `^ 3 = 1` –.

Implementation. The proof applies the packaged result `IsPrimitiveRoot.pow_eq_one`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
abbrev integralPowerBasis : PowerBasis ℤ (𝒪 K) :=
  zeta_isPrimitiveRoot.integralPowerBasis
```

Mathematical meaning. explicit integral power basis from a primitive root (cyclotomic-specific).

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem integralPowerBasis_gen : integralPowerBasis.gen =
    zeta_isPrimitiveRoot.toInteger

theorem integralPowerBasis_dim : integralPowerBasis.dim = 2
```

Mathematical meaning. The displayed Lean declaration has type `gen = zeta_isPrimitiveRoot.toInteger; dim = 2`.

Implementation. `simp [integralPowerBasis]; + totient_prime`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev zetaExplicitIntegralBasis : Basis (Fin 2) ℤ (𝓞 K) :=
  integralPowerBasis.basis.reindex (finCongr integralPowerBasis_dim)
```

Mathematical meaning. The displayed Lean declaration has type `Basis (Fin 2) ℤ (𝓞 K)` – the explicit integral basis `[1, ζ]`.

Implementation. reindex `integralPowerBasis.basis` along `finCongr integralPowerBasis_dim`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zetaExplicitIntegralBasis_zero :
    zetaExplicitIntegralBasis 0 = 1

theorem zetaExplicitIntegralBasis_one :
    zetaExplicitIntegralBasis 1 = zeta_isPrimitiveRoot.toInteger
```

Mathematical meaning. The displayed Lean declaration has type `basis 0 = 1; basis 1 = zeta_isPrimitiveRoot.toInteger`.

Implementation. `Basis.reindex_apply` + `basis_eq_pow` + the `Fin`-coe computations. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cyclotomicRing_isIntegralClosure : IsIntegralClosure
    (CyclotomicRing 3 ℤ ℚ) ℤ K
```

Mathematical meaning. The displayed Lean declaration has type `IsIntegralClosure (CyclotomicRing 3 ℤ ℚ) ℤ K` –.

Implementation. The proof applies the packaged result `and related termsRat.cyclotomicRing_isIntegralClosure`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem isIntegralClosure_adjoin_zeta :
    IsIntegralClosure (Algebra.adjoin ℤ ({ζ} : Set K)) ℤ K
```

Mathematical meaning. The displayed Lean declaration has type `IsIntegralClosure (Algebra.adjoin ℤ {ζ}) ℤ K` – i.e. $\mathbb{Z}[\zeta] = \mathcal{O}$ K.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.Rat.isIntegralClosure_adjoin_singleton`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem integralPowerBasis_adjoin_gen_eq_top :
    Algebra.adjoin ℤ ({integralPowerBasis.gen} : Set (𝒪 K)) = ⊤
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.adjoin ℤ {integralPowerBasis.gen} = ⊤` –.

Implementation. The proof applies the packaged result `PowerBasis.adjoin_gen_eq_top`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

6.5 `Discriminant.lean`

```
theorem numberField_discr_eq : NumberField.discr K = -3
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.discr K = -3` –.

Implementation. `IsCyclotomicExtension.Rat.discr_prime 3 K` + `norm_num`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zetaExplicitIntegralBasis_discr :
    Algebra.discr ℤ zetaExplicitIntegralBasis = (-3 : ℤ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℤ zetaExplicitIntegralBasis = -3` –.

Implementation. `NumberField.discr_eq_discr` > `discr_prime`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem integralBasis_discr :
    Algebra.discr ℚ (NumberField.integralBasis K) = (-3 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℚ (integralBasis K) = -3` –.

Implementation. `NumberField.coe_discr` bridge from the $\mathbb{Z}$-disc. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
example
    (P : Matrix (Free.ChooseBasisIndex ℤ (𝒪 K)) (Free.ChooseBasisIndex ℤ
    (𝒪 K)) ℚ) :
    Algebra.discr ℚ (Matrix.mulVec (P.map (algebraMap ℚ K))
    (NumberField.integralBasis
    K)) =
      P.det ^ 2 * Algebra.discr ℚ (NumberField.integralBasis K)

example
```

```
    (P : Matrix (Free.ChooseBasisIndex ℤ (𝒪 K)) (Free.ChooseBasisIndex ℤ
    (𝒪 K)) ℚ)
    (hdet : P.det = 1 ∨ P.det = -1) :
    Algebra.discr ℚ (Matrix.mulVec (P.map (algebraMap ℚ K))
    (NumberField.integralBasis
    K)) =
        (-3 : ℚ)
```

Mathematical meaning. change-of-basis discriminant law: `discr(P • b) = det(P)`^{2} `· discr(b)`; unimodular `det=`$\pm$`1` preserves `-3`.

Implementation. `Algebra.discr_of_matrix_mulVec` + `integralBasis_discr`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

6.6 PrimeSplitting.lean

```
private theorem gen_eq_zetaInt :
    zeta_isPrimitiveRoot.integralPowerBasis.gen = zetaInt

private theorem zetaInt_pow_three : zetaInt ^ 3 = 1

private theorem zetaInt_relation : zetaInt ^ 2 + zetaInt + 1 = 0
```

Mathematical meaning. The displayed Lean declaration has type `integralPowerBasis.gen = zetaInt`; `zetaInt^3 = 1`; `zetaInt`^{2}`+zetaInt+1 = 0`.

Implementation. `integralPowerBasis_gen/rfl`; `toInteger_cube_eq_one`; `isRoot_cyclotomic` + `ext`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private noncomputable def sig2 : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    2 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))

private theorem sig2_zeta : sig2 ζ = ζ ^ 2
```

Mathematical meaning. The displayed Lean declaration has type `K ≃ₐ[ℚ] K`, the nontrivial automorphism $\zeta \mapsto \zeta^2$ (conjugation).

Implementation. `IsCyclotomicExtension.fromZetaAut` from `pow_of_coprime 2`; spec via `fromZetaAut_spec`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem autEq' (σ : K ≃ₐ[ℚ] K) : σ = 1 ∨ σ = sig2
```

Mathematical meaning. The displayed Lean declaration has type `∀ σ, σ = 1 ∨ σ = sig2` – the only two automorphisms.

Implementation. `decide` on `(ZMod 3)ˣ` values via `autToPow_spec`; extensionality through the power basis.

```
private theorem prod_aut_apply (x : K) :
    ∏ σ : K ≃ₐ[ℚ] K, σ x = x * sig2 x
```

Mathematical meaning. The displayed Lean declaration has type `∏ σ, σ x = x * sig2 x` – product over Galois group of size 2.

Implementation. `univ = {1, sig2}`, `Finset.prod_pair`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem minpoly_gen_eq :
    minpoly ℤ zeta_isPrimitiveRoot.integralPowerBasis.gen =
      Polynomial.X ^ 2 + Polynomial.X + Polynomial.C 1
```

Mathematical meaning. The displayed Lean declaration has type `minpoly ℤ integralPowerBasis.gen = X²+X+C 1` – integral minimal polynomial.

Implementation. `minpoly.isIntegrallyClosed_dvd` (divides `X²+X+1` via `zetaInt_relation`) + monic-degree-2 argument forcing the cofactor to be 1.

```
private theorem isMaximal_span_pair (p : ℕ) [Fact p.Prime] (r : ℤ)
    (hr : ((r : ZMod p)) ^ 2 + (r : ZMod p) + 1 = 0) :
    (Ideal.span {(p : 𝒪 K), zetaInt - (r : 𝒪 K)}).IsMaximal

private theorem absNorm_span_pair (p : ℕ) (r : ℤ)
    (hr : ((r : ZMod p)) ^ 2 + (r : ZMod p) + 1 = 0) :
    Ideal.absNorm (Ideal.span {(p : 𝒪 K), zetaInt - (r : 𝒪 K)}) = p
```

Mathematical meaning. The displayed Lean declaration has type for `r²+r+1≡0 (mod p)`: `(span {p, ζ−r}).IsMaximal`; `absNorm = p`.

Implementation. thin wrappers over Common's `isMaximal_span_gen_sub` / `absNorm_span_gen_sub` (Kummer-Dedekind ResidueHom), feeding `minpoly_gen_eq`.

```
theorem norm_eq_normForm (x y : ℚ) :
    Algebra.norm ℚ (x • (1 : K) + y • ζ) = x ^ 2 - x * y + y ^ 2
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.norm ℚ (x•1 + y•ζ) = x² − xy + y²` – the principal binary quadratic form of disc −3.

Implementation. `norm_eq_prod_automorphisms` + `prod_aut_apply` + `linear_combination` against the cyclotomic relation.

```
theorem intNorm_eq_normForm (x y : ℤ) :
    Algebra.norm ℤ (x • (1 : 𝒪 K) + y • zetaInt) = x ^ 2 - x * y + y ^ 2
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.norm ℤ (x•1 + y•zetaInt) = x² − xy + y² (x,y ∈ ℤ)`.

Implementation. `Algebra.coe_norm_int` cast bridge to `norm_eq_normForm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isUnit_iff_normForm (x y : ℤ) :
    IsUnit (x • (1 : 𝒪 K) + y • zetaInt) ↔ x ^ 2 - x * y + y ^ 2 = 1
```

Mathematical meaning. The displayed Lean declaration has type `IsUnit (x•1 + y•zetaInt) ↔ x²−xy+y² = 1` – the unit equation (6 solutions = $\pm 1, \pm\zeta, \pm\zeta^2$).

Implementation. `NumberField.isUnit_iff_norm` + `intNorm_eq_normForm`; rules out −1 via positivity (`nlinarith`/`omega`).

```
theorem norm_one_sub_zeta : Algebra.norm ℚ (1 - ζ) = 3
```

```
theorem norm_one_add_zeta : Algebra.norm ℚ (1 + ζ) = 1
```

Mathematical meaning. The displayed Lean declaration has type `norm ℚ (1−ζ) = 3`; `norm ℚ (1+ζ) = 1`. –.

Implementation. specialise `norm_eq_normForm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zetaInt_sub_one_prime : Prime (zetaInt - 1 : 𝒪 K)
```

Mathematical meaning. The displayed Lean declaration has type `Prime (zetaInt − 1)` –.

Implementation. The proof applies the packaged result `IsPrimitiveRoot.zeta_sub_one_prime'`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem zetaInt_sub_one_dvd_three : (zetaInt - 1 : 𝒪 K) ∣ (3 : 𝒪 K)
```

Mathematical meaning. The displayed Lean declaration has type `(zetaInt − 1) ∣ 3` –.

Implementation. The proof applies the packaged result `toInteger_sub_one_dvd_prime'`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem span_three_eq :
    Ideal.span {(3 : 𝓞 K)} = Ideal.span {zetaInt - 1} ^ 2
```

Mathematical meaning. The displayed Lean declaration has type `span {3} = span {zetaInt - 1} ^ 2` – 3 is totally ramified `(3)=(ζ-1)²`.

Implementation. `span_singleton_pow` + `span_singleton_eq_span_singleton`; explicit unit via `zetaInt_relation`/`pow_three`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem span_zetaInt_sub_one_eq_pair :
    Ideal.span {zetaInt - 1} = Ideal.span {((3 : ℕ) : 𝓞 K), zetaInt - ((1
    : ℤ) : 𝓞 K)}
```

Mathematical meaning. The displayed Lean declaration has type `span {zetaInt-1} = span {(3:ℕ), zetaInt-(1:ℤ)}` – bridges to the Common pair form.

Implementation. `le_antisymm` via `mem_span_pair` and `span_three_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_span_zetaInt_sub_one :
    Ideal.absNorm (Ideal.span {zetaInt - 1}) = 3
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (span {zetaInt-1}) = 3` (residue degree `f=1`).

Implementation. `decide` the mod-3 root, `span_zetaInt_sub_one_eq_pair`, `absNorm_span_pair`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isPrime_span_zetaInt_sub_one : (Ideal.span {zetaInt - 1}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `(span {zetaInt-1}).IsPrime` – the unique prime over 3.

Implementation. `isMaximal_span_pair 3 1`.isPrime. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_span_zetaInt_sub_one :
    (Ideal.span {zetaInt - 1}).LiesOver (Ideal.span {(3 : ℤ)})

theorem inertiaDeg_span_zetaInt_sub_one :
    (Ideal.span {(3 : ℤ)}).inertiaDeg (Ideal.span {zetaInt - 1}) = 1

theorem ramificationIdx_span_zetaInt_sub_one :
    (Ideal.span {(3 : ℤ)}).ramificationIdx (Ideal.span {zetaInt - 1}) = 2
```

Mathematical meaning. The displayed Lean declaration has type `(ζ−1)` lies over `(3)`; `f=1`; `e=2`.

Implementation. `liesOver_span_iff + span_three_eq`; `absNorm_eq_pow_inertiaDeg'`; `ramificationIdx_spec` with a norm-9-vs-27 divisibility contradiction.

```
theorem absNorm_span_two : Ideal.absNorm (Ideal.span {(2 : 𝒪 K)}) = 4
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (span {2}) = 4` – 2 inert, residue degree 2.

Implementation. `absNorm_span_singleton` + `norm_algebraMap` + `finrank = 2`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem quotient_zeta_ne_one_of_two_mem {M : Ideal (𝒪 K)}
    [M.IsMaximal]
    (h2mem : (2 : 𝒪 K) ∈ M) : Ideal.Quotient.mk M zetaInt ≠ 1

private theorem absNorm_maximal_over_span_two_eq_four {M : Ideal (𝒪 K)}
    (hMmax :
    M.IsMaximal)
    (hMle : Ideal.span {(2 : 𝒪 K)} ≤ M) : Ideal.absNorm M = 4
```

Mathematical meaning. The displayed Lean declaration has type in a maximal `M ∋ 2`, `mk M zetaInt ≠ 1`; and `absNorm M = 4`.

Implementation. finite-field order argument: ω `= mk zetaInt` has order 3, so `3 ∣ 2^k − 1`, forcing `k=2`.

```
theorem isPrime_span_two : (Ideal.span {(2 : 𝒪 K)}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `(span {2}).IsPrime` – 2 is inert (Φ_3 irreducible mod 2).

Implementation. take maximal `M ⊇ (2)`, show `absNorm M = 4 = absNorm (2)`, then `eq_of_le_of_absNorm_eq` (Common).

```
theorem liesOver_span_two : (Ideal.span {(2 : 𝒪 K)}).LiesOver (Ideal.span
    {(2 : ℤ)})

theorem inertiaDeg_span_two :
    (Ideal.span {(2 : ℤ)}).inertiaDeg (Ideal.span {(2 : 𝒪 K)}) = 2
```

Mathematical meaning. The displayed Lean declaration has type `(2)` lies over `(2)`; `f=2`. –.

Implementation. `liesOver_span_iff`; `absNorm_eq_pow_inertiaDeg' + pow_right_injective`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem seven_eq_mul : (7 : 𝒪 K) = (3 + zetaInt) * (3 + zetaInt ^
    2)
```

Mathematical meaning. The displayed Lean declaration has type `(7:𝒪 K) = (3+zetaInt)(3+zetaInt²)` –.

Implementation. `linear_combination` with `pow_three`/relation. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_seven_eq :
    Ideal.span {(7 : 𝒪 K)} = Ideal.span {3 + zetaInt} * Ideal.span {3 +
    zetaInt ^ 2}
```

Mathematical meaning. The displayed Lean declaration has type `span {7} = span {3+ζ} * span {3+ζ²}` – `7 ≡ 1 (mod 3)` splits.

Implementation. `span_singleton_mul_span_singleton` + `seven_eq_mul`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_span_seven_factor : Ideal.absNorm (Ideal.span {3 +
    zetaInt}) = 7

theorem isPrime_span_seven_factor : (Ideal.span {3 + zetaInt}).IsPrime

theorem liesOver_span_seven_factor :
    (Ideal.span {3 + zetaInt}).LiesOver (Ideal.span {(7 : ℤ)})

theorem inertiaDeg_span_seven_factor :
    (Ideal.span {(7 : ℤ)}).inertiaDeg (Ideal.span {3 + zetaInt}) = 1
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (3+ζ) = 7`; prime; lies over `(7)`; `f=1`.

Implementation. `intNorm_eq_normForm 3 1`; `isPrime_of_irreducible_absNorm`; `liesOver_span_iff`; `absNorm_eq_pow_inertiaDeg'`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramified_iff_eq_three (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ^ 2) ↔ p =
    3
```

Mathematical meaning. The displayed Lean declaration has type `(∃ P prime, span{p} ≤ P²) ↔ p = 3` – ramifies iff `p | disc = −3`.

Implementation. forward via different ideal: `dvd_differentIdeal_iff` from unramifiedness failure (Common's `not_map_span_natCast_le_pow_succ and related terms / not_`

`isUnramifiedAt_of_` and related terms), then `absNorm_differentIdeal = |disc| = 3`; backward `span_three_eq`.

```
theorem ramificationIdx_span_two :
    (Ideal.span {(2 : ℤ)}).ramificationIdx (Ideal.span {(2 : 𝒪 K)}) = 1

theorem ramificationIdx_span_seven_factor :
    (Ideal.span {(7 : ℤ)}).ramificationIdx (Ideal.span {3 + zetaInt}) = 1
```

Mathematical meaning. The displayed Lean declaration has type `e=1` at `(2)`; `e=1` at `(3+ζ)` over 7 – both unramified.

Implementation. `ramificationIdx_spec` + `ramified_iff_eq_three` contradiction. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem absNorm_span_natCast (p : ℕ) :
    Ideal.absNorm (Ideal.span {((p : ℕ) : 𝒪 K)}) = p ^ 2
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (span {(p:𝒪 K)}) = p^2` –.

Implementation. `absNorm_span_natCast` + `RingOfIntegers.rank` + `degree_eq_two`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem one_mod_three_of_prime_norm (p : ℕ) (hp : p.Prime) (hp3 :
    p ≠ 3) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
    Ideal.absNorm P = p) →
      p % 3 = 1

private theorem exists_prime_norm_of_one_mod_three (p : ℕ) (hp : p.Prime)
    :
    p % 3 = 1 →
      ∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
    Ideal.absNorm P = p

private theorem exists_prime_norm_iff_one_mod_three (p : ℕ) (hp :
    p.Prime) (hp3 : p ≠
    3) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
    Ideal.absNorm P = p) ↔
      p % 3 = 1
```

Mathematical meaning. The displayed Lean declaration has type existence of a norm-`p` prime over `p` $\iff$ `p` $\equiv$ `1 (mod 3)`.

Implementation. forward – `ZMod.ringEquivOfPrime`, `mk zetaInt` gives an order-3 element of `(ZMod p)`$^\times$ so `3 ∣ p−1`; backward – generator `g` of cyclic `(ZMod p)`$^\times$, `g^m` of order 3 is a mod-`p` root of `X`^{2}`+X+1`, feed `isMaximal_span_pair`/`absNorm_span_pair`.

```
private theorem one_mod_three_of_splits (p : ℕ) (hp : p.Prime) (hp3 : p ≠
    3) :
    (∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) → p % 3 = 1

private theorem splits_of_one_mod_three (p : ℕ) (hp : p.Prime) (hp3 : p ≠
    3) :
    p % 3 = 1 →
      ∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
        Ideal.span {(p : 𝒪 K)} = P * Q
```

Mathematical meaning. The displayed Lean declaration has type the two directions of the splitting law in product form.

Implementation. `map_mul absNorm` + `absNorm_span_natCast`; Common's `left_eq_prime_of_mul_eq_prime_sq and related terms`; distinctness via `ramified_iff_eq_three`.

```
theorem splits_iff_one_mod_three (p : ℕ) (hp : p.Prime) (hp3 : p ≠ 3) :
    (∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) ↔ p % 3 = 1
```

Mathematical meaning. The displayed Lean declaration has type for `p ≠ 3` prime: `(∃ P≠Q primes, span{p}=P*Q) ↔ p ≡ 1 (mod 3)`.

Implementation. combine the two private directions.

```
private theorem span_zetaInt_sub_one_dvd_differentIdeal :
    Ideal.span {zetaInt - 1} ∣ differentIdeal ℤ (𝒪 K)
```

Mathematical meaning. The displayed Lean declaration has type `span {zetaInt−1} ∣ differentIdeal ℤ (𝒪 K)` –.

Implementation. `dvd_differentIdeal_iff` from ramification at 3, Common's `not_map_span_natCast and related terms`/`not_isUnramifiedAt and related terms`.

```
theorem absNorm_differentIdeal_eq_three :
    Ideal.absNorm (differentIdeal ℤ (𝒪 K)) = 3
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (differentIdeal ℤ (𝒪 K)) = 3` –.

Implementation. `NumberField.absNorm_differentIdeal` + `numberField_discr_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem prime_dvd_differentIdeal_iff_eq_span_zeta_sub_one
    (P : Ideal (𝒪 K)) (_hP : P.IsPrime) (_hP0 : P ≠ ⊥) :
    P ∣ differentIdeal ℤ (𝒪 K) ↔ P = Ideal.span {zetaInt - 1}
```

Mathematical meaning. The displayed Lean declaration has type for prime `P ≠ ⊥`: `P ∣ different ↔ P = span {zetaInt-1}`.

Implementation. norm divides $3 \Rightarrow$ norm $= 3 \Rightarrow$ `3 ∈ P`, lies over `(3)`, then `IsCyclotomicExtension.Rat.eq_span_zeta_sub_one_of_liesOver'`; converse from the dvd lemma.

6.7 `ClassGroup.lean`

```
abbrev zetaFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  FractionalIdeal.spanSingleton (𝒪 K)⁰ ζ

abbrev ramifiedPrimeFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  (Ideal.span {zetaInt - 1} : FractionalIdeal (𝒪 K)⁰ K)
```

Mathematical meaning. The displayed Lean declaration has type `spanSingleton ζ`; `span {zetaInt-1}` as fractional ideals.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem zeta_mem_fractionalIdeal :
    ζ ∈ zetaFractionalIdeal

theorem zetaFractionalIdeal_absNorm :
    FractionalIdeal.absNorm zetaFractionalIdeal = |Algebra.norm ℚ ζ|
```

Mathematical meaning. The displayed Lean declaration has type `ζ ∈ zetaFractionalIdeal`; `absNorm = |Algebra.norm ℚ ζ|`.

Implementation. `mem_spanSingleton_self`; `absNorm_span_singleton`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramifiedPrimeFractionalIdeal_ne_zero :
    ramifiedPrimeFractionalIdeal ≠ 0
```

Mathematical meaning. The displayed Lean declaration has type `ramifiedPrimeFractionalIdeal ≠ 0` –.

Implementation. `coeIdeal_ne_zero` + and related `termsRat.span_zeta_sub_one_ne_bot` (with `3^(0+1)` reshaping).

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝒪 K)
```

Mathematical meaning. The displayed Lean declaration has type `IsPrincipalIdealRing (𝒪 K)` –.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.Rat.three_pid K`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝒪 K)
```

Mathematical meaning. The displayed Lean declaration has type `UniqueFactorizationMonoid (𝒪 K)` –.

Implementation. `infer_instance` from PID. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.classNumber K = 1` –.

Implementation. `classNumber_eq_one_iff.mpr ( and related termsRat.three_pid K)`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classGroupStructure : Nonempty (ClassGroup (𝒪 K) ≃*
    Multiplicative (ZMod 1))
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty (ClassGroup (𝒪 K) ≃* Multiplicative (ZMod 1))` – class group trivial.

Implementation. `Nat.card = 1` ⇒ `Subsingleton`/`Unique` ⇒ `MulEquiv.ofUnique`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev unitIdealRepresentative : (Ideal (𝒪 K))⁰ := ⟨(⊤ : Ideal (𝒪 K)), by
    simp⟩

theorem classGroupRepresentative (C : ClassGroup (𝒪 K)) :
    C = ClassGroup.mk0 unitIdealRepresentative
```

Mathematical meaning. The displayed Lean declaration has type `⟨⊤, _⟩ : (Ideal (𝒪 K))⁰`; every class `= ClassGroup.mk0 unitIdealRepresentative`.

Implementation. `Subsingleton.elim`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

6.8 `Units.lean`

```
abbrev etaUnit : (𝒪 K)ˣ :=
  (zeta_isPrimitiveRoot.toInteger_isPrimitiveRoot.isUnit (by decide)).unit
```

Mathematical meaning. The displayed Lean declaration has type `(𝒪 K)ˣ` – ζ as a unit. –.

Implementation. The displayed RHS is the packaged construction `toInteger_isPrimitiveRoot.isUnit (decide).unit` specialized to this field.

```
theorem unit_mem_unitsList (u : (𝒪 K)ˣ) :
    u ∈ ([1, -1, etaUnit, -etaUnit, etaUnit ^ 2, -(etaUnit ^ 2)] : List
    (𝒪 K)ˣ)

theorem unit_eq_one_or_neg_one_or_eta (u : (𝒪 K)ˣ) :
    u = 1 ∨ u = -1 ∨ u = etaUnit ∨ u = -etaUnit ∨ u = etaUnit ^ 2 ∨
      u = -(etaUnit ^ 2)
```

Mathematical meaning. The displayed Lean declaration has type every unit is among `[1,-1,`η`,-`η`,`η^2`,-`η^2`]`; the disjunction form.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.Rat.Three.Units.mem`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem torsionOrder_eq_six : Units.torsionOrder K = 6
```

Mathematical meaning. The displayed Lean declaration has type `Units.torsionOrder K = 6` –.

Implementation. `IsCyclotomicExtension.Rat.torsionOrder_eq` + `norm_num`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem unitRank_eq_zero : Units.rank K = 0
```

Mathematical meaning. The displayed Lean declaration has type `Units.rank K = 0` – signature `(0,1)` $\Rightarrow$ r_1`+`r_2`-1 = 0`.

Implementation. unfold `rank`, `card_eq_nrRealPlaces_add_nrComplexPlaces` + the cyclotomic place counts. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem unitGroupStructure : Nonempty ((𝒪 K)ˣ ≃* Multiplicative (ZMod 6))
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty ((𝒪 K)ˣ ≃* Multiplicative (ZMod 6))` – units $\cong$ μ_6.

Implementation. every unit is torsion (`fin_cases` over the 6-element list, each `IsOfFinOrder`); `(𝒪 K)ˣ` cyclic; `Nat.card = 6` via `torsionOrder_eq_six`; `mulEquivOfCyclicCardEq`.

```
theorem regulator_eq_one : Units.regulator K = 1
```

Mathematical meaning. The displayed Lean declaration has type `Units.regulator K = 1` – rank-0 $\Rightarrow$ trivial (empty) log-lattice determinant.

Implementation. `Units.regulator_eq_det'`; the index type `{w ≠ w₀}` has card 0 (from `unitRank_eq_zero` via `equivFinRank`), so `Matrix.det_eq_one_of_card_eq_zero` gives det = 1.

6.9 CMField.lean

```
theorem isCMField : NumberField.IsCMField K
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.IsCMField K` –.

Implementation. `IsCyclotomicExtension.Rat.isCMField K (S := {3})` with witness ⟨`3, and related terms`⟩.

```
theorem cmField_isTotallyComplex : NumberField.IsTotallyComplex K
```

Mathematical meaning. The displayed Lean declaration has type `IsTotallyComplex K` –.

Implementation. `isTotallyComplex` (Embeddings). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cmField_quadratic_over_maximalRealSubfield :
    Algebra.IsQuadraticExtension (NumberField.maximalRealSubfield K) K
```

Mathematical meaning. The displayed Lean declaration has type `IsQuadraticExtension (maximalRealSubfield K) K` – here `maximalRealSubfield` = $\mathbb{Q}$.

Implementation. `IsCMField` $\Rightarrow$ `infer_instance`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cmField_unitsRank_eq_maximalRealSubfield :
    NumberField.Units.rank (NumberField.maximalRealSubfield K) =
      NumberField.Units.rank K
```

Mathematical meaning. The displayed Lean declaration has type `Units.rank (maximalRealSubfield K) = Units.rank K` – CM rank-equality (both 0).

Implementation. `IsCMField.units_rank_eq_units_rank`. – After unfolding the field-specific definitions, these named lemmas supply the essential argument.

6.10 `Euclidean.lean`

```
private theorem abs_norm_int_lt_of_remainder_eq {b r : 𝒪 K} {bK : K} {xa
    xb : ℚ}
    (hbK : bK = algebraMap (𝒪 K) K b)
    (hrK :
      algebraMap (𝒪 K) K r =
        bK * ((xa - ((round xa : ℤ) : ℚ)) • (1 : K) +
          (xb - ((round xb : ℤ) : ℚ)) • ζ))
    (hb_pos : 0 < |Algebra.norm ℚ bK|)
    (hfrac :
      |(xa - ((round xa : ℤ) : ℚ)) ^ 2 -
        (xa - ((round xa : ℤ) : ℚ)) * (xb - ((round xb : ℤ) : ℚ)) +
        (xb - ((round xb : ℤ) : ℚ)) ^ 2| ≤ 3 / 4) :
    |((Algebra.norm ℤ r : ℤ) : ℚ)| < |((Algebra.norm ℤ b : ℤ) : ℚ)|
```

Mathematical meaning. The displayed Lean declaration has type given the remainder decomposition and the fractional-form bound $\leq$ `3/4`, `|N(r)| < |N(b)|` (over $\mathbb{Q}$). –.

Implementation. Common's `abs_intNorm_lt_of_abs_norm_le` with `3/4 < 1`, after `norm_eq_normForm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem exists_coords (x : K) :
    ∃ a b : ℚ, x = a • (1 : K) + b • ζ
```

Mathematical meaning. The displayed Lean declaration has type ∀ `x : K,` ∃ `a b : ℚ, x = a•1 + b•`ζ – coordinates in the `1,`ζ basis.

Implementation. `IsPrimitiveRoot.powerBasis.exists_eq_aeval`; degree $\leq$ `1` so `eq_X_add_C_of_natDegree_le_one`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem normEuclidean (a b : 𝒪 K) (hb : b ≠ 0) :
    ∃ q r : 𝒪 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

Mathematical meaning. The displayed Lean declaration has type ∀ `a b, b ≠ 0` → ∃ `q r, a = b*q + r ∧ |N r| < |N b|` (natAbs) – $\mathbb{Z}[\zeta_3]$ is norm-Euclidean.

Implementation. divide in `K`, take nearest-integer coords (`round`), bound the quadratic-form remainder `|s`^{2} `- st + t`^{2}`| ≤ 3/4` (`nlinarith` with `sq_nonneg (s±t)`); then `abs_norm_int_lt_` and related terms + `natAbs_lt_of_abs_ratCast_lt` (Common). Strengthens `isPrincipalIdealRing`.

6.11 `LocalGlobal.lean`

```
def heightOneThree : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {zetaInt - 1}
  isPrime := isPrime_span_zetaInt_sub_one
  ne_bot := by
    intro h
    have hnorm := absNorm_span_zetaInt_sub_one
    rw [h] at hnorm
    norm_num at hnorm

abbrev vThree : NumberField.FinitePlace K :=
  NumberField.FinitePlace.mk heightOneThree
```

Mathematical meaning. The displayed Lean declaration has type `HeightOneSpectrum (𝒪 K)` at `span {zetaInt-1}`; `FinitePlace.mk` of it.

Implementation. `isPrime_span_zetaInt_sub_one`; `ne_bot` from `absNorm = 3 ≠ 0`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
example :
    (Ideal.span {(3 : ℤ)}).ramificationIdx heightOneThree.asIdeal = 2

example :
    (Ideal.span {(3 : ℤ)}).inertiaDeg heightOneThree.asIdeal = 1
```

Mathematical meaning. `e=2`, `f=1` for the place over 3.

Implementation. `ramificationIdx_span_zetaInt_sub_one / inertiaDeg_ and related terms`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev CompletionAtThree : Type :=
  heightOneThree.adicCompletion K

abbrev InfiniteCompletion (v : NumberField.InfinitePlace K) : Type :=
  v.Completion
```

Mathematical meaning. The displayed Lean declaration has type `heightOneThree.adicCompletion K`; `v.Completion`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem complexEmbedding_norm_zeta_eq_one (φ : K →+* ℂ) : ‖φ ζ‖ =
    1
```

Mathematical meaning. The displayed Lean declaration has type $\|\varphi\ \zeta\| = 1$ for any φ : K $\to+*$ $\mathbb{C}$ –.

Implementation. `Complex.norm_eq_one_of_pow_eq_one` (ζ^3=1). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
example (v : NumberField.InfinitePlace K) :
    Nonempty (InfiniteCompletion v ≃+* ℂ)

theorem infiniteCompletion_zeta_norm (v : NumberField.InfinitePlace K) :
    ‖(algebraMap K (InfiniteCompletion v) ζ)‖ = 1
```

Mathematical meaning. The displayed Lean declaration has type `InfiniteCompletion v` $\simeq+*$ $\mathbb{C}$; $\|$`algebraMap K _ ζ`$\|$ = 1.

Implementation. `Completion.ringEquivComplexOfIsComplex`; `Completion.norm_coe` + `norm_embedding_eq` + above. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem zeta_ne_zero : ζ ≠ 0

example :
    ((∏ w : NumberField.InfinitePlace K, w ζ ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K, v ζ) = 1

example :
    ((∏ w : NumberField.InfinitePlace K,
          w (algebraMap (𝒪 K) K (etaUnit : 𝒪 K)) ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K,
          v (algebraMap (𝒪 K) K (etaUnit : 𝒪 K))) = 1
```

Mathematical meaning. $\zeta \neq$ 0; product formula $\prod$_w $\cdot$ $\prod$_v = 1 for ζ and for `etaUnit`.

Implementation. The proof applies the packaged result `NumberField.prod_abs_eq_one`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
abbrev CyclotomicThreeInfiniteAdeleRing : Type :=
  NumberField.InfiniteAdeleRing K

abbrev infinitePrincipalAdele_zeta : CyclotomicThreeInfiniteAdeleRing :=
  algebraMap K CyclotomicThreeInfiniteAdeleRing ζ

abbrev infinitePrincipalAdele_etaUnit : CyclotomicThreeInfiniteAdeleRing
    :=
  algebraMap K CyclotomicThreeInfiniteAdeleRing (algebraMap (𝒪 K) K
```

```
    (etaUnit : 𝒪 K))

example :
    ‖infinitePrincipalAdele_zeta‖ = |Algebra.norm ℚ ζ|

example :
    DenseRange (algebraMap K CyclotomicThreeInfiniteAdeleRing)
```

Mathematical meaning. infinite adele ring; principal adeles of ζ/`etaUnit`; norm formula + dense range.

Implementation. `InfiniteAdeleRing.coe_norm_eq_abs_norm`; `denseRange_algebraMap`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem canonicalEmbedding_zetaInt_mem_integerLattice :
    NumberField.canonicalEmbedding K (algebraMap (𝒪 K) K zetaInt) ∈
      NumberField.canonicalEmbedding.integerLattice K
```

Mathematical meaning. The displayed Lean declaration has type `canonicalEmbedding K (algebraMap zetaInt) ∈ integerLattice K –`.

Implementation. exhibit preimage `zetaInt`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev mixedEmbedding_zeta : NumberField.mixedEmbedding.mixedSpace K :=
  NumberField.mixedEmbedding K ζ

example
    (f : NumberField.InfinitePlace K → NNReal)
    (hf : ∀ w : NumberField.InfinitePlace K, w ζ < f w) :
    mixedEmbedding_zeta ∈ NumberField.mixedEmbedding.convexBodyLT K f

example :
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone K)

example :
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone.normLeOne K)

theorem polarSpace_nonempty :
    Nonempty (NumberField.mixedEmbedding.polarSpace K)
```

Mathematical meaning. mixed embedding of ζ; convex-body membership; measurability of `fundamentalCone`/`normLeOne`;`Nonempty (polarSpace K)`.

Implementation. `convexBodyLT_mem`; `measurableSet_fundamentalCone`/ and related `termsnormLeOne`; `infer_instance`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem house_zeta :
    NumberField.house ζ = 1

theorem house_etaUnit :
    NumberField.house (algebraMap (𝒪 K) K (etaUnit : 𝒪 K)) = 1
```

Mathematical meaning. The displayed Lean declaration has type `house ζ = 1`; `house etaUnit = 1`.

Implementation. `le_antisymm` via `canonicalEmbedding.norm_le_iff` and $\|\varphi\ \zeta\| = 1$ at each embedding; `etaUnit` by `simpa`.

```
abbrev CyclotomicThreeAdeleRing : Type :=
  NumberField.AdeleRing (𝒪 K) K

abbrev principalAdele_zeta : CyclotomicThreeAdeleRing :=
  algebraMap K CyclotomicThreeAdeleRing ζ

abbrev principalAdele_etaUnit : CyclotomicThreeAdeleRing :=
  algebraMap K CyclotomicThreeAdeleRing (algebraMap (𝒪 K) K (etaUnit : 𝒪
    K))
```

Mathematical meaning. The displayed Lean declaration has type full adele ring and principal adeles.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem principalAdele_zeta_ne_zero : principalAdele_zeta ≠ 0
```

Mathematical meaning. The displayed Lean declaration has type `principalAdele_zeta ≠ 0` –.

Implementation. `AdeleRing.algebraMap_injective` + `zeta_ne_zero`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem principalAdele_etaUnit_is_unit_everywhere :
    ∀ v : NumberField.FinitePlace K,
      v (algebraMap (𝒪 K) K (etaUnit : 𝒪 K)) = 1
```

Mathematical meaning. The displayed Lean declaration has type `∀ v finite, v (etaUnit) = 1` – a unit has trivial valuation at every finite place.

Implementation. `norm_embedding_eq` + `norm_eq_one_iff_notMem`; `etaUnit·etaUnit`$^{-1}$`=1` $\notin$ `maximal ideal`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev cyclotomicThreeDedekindZeta (s : ℂ) : ℂ :=
  NumberField.dedekindZeta K s

abbrev cyclotomicThreeDedekindZetaResidue : ℝ :=
  NumberField.dedekindZeta_residue K

example :
    0 < cyclotomicThreeDedekindZetaResidue

example :
    cyclotomicThreeDedekindZetaResidue =
      (2 ^ NumberField.InfinitePlace.nrRealPlaces K *
          (2 * Real.pi) ^ NumberField.InfinitePlace.nrComplexPlaces K *
          NumberField.Units.regulator K * NumberField.classNumber K) /
        (NumberField.Units.torsionOrder K * Real.sqrt |NumberField.discr
    K|)
```

Mathematical meaning. Dedekind zeta function and its residue; positivity; the class-number-formula residue identity.

Implementation. `dedekindZeta_residue_pos`; `dedekindZeta_residue_def` (involves regulator, classNumber, torsionOrder, discr – all computed in earlier files).

6.12 Summary for $\mathbb{Q}(\zeta_3)$

Folder: `XYin/NumberField/FieldExamples/CyclotomicThree/` Field: `K = CyclotomicField 3 ℚ`, the third cyclotomic field = $\mathbb{Q}(\sqrt{-3})$, built via `IsCyclotomicExtension {3} ℚ K`; generator ζ a primitive cube root with Φ_3 = `X^2+X+1` ($\zeta^2+\zeta+1 = 0$). Degree 2, signature `(0,1)`, ring of integers $\mathbb{Z}[\zeta]$ (Eisenstein integers), disc `-3`, class number 1, unit group μ_6 (rank 0, regulator 1), norm-Euclidean, CM field. Twin of `CubeRootThree` ($\mathbb{Q}(\sqrt[3]{3})$): there the cubic relation `r^3=3` drives everything; here the quadratic cyclotomic relation `r^2+r+1=0` plays the analogous structural role. All declarations in `namespace NumberField.CyclotomicThree` unless noted. Shared helpers from `XYin/NumberField/FieldExamples/Common.lean` (`namespace NumberField.FieldExamples`). Sorry-free: confirmed (grep clean). Rank-0 units, so no Artin-bound blockers (unlike ζ_5/ζ_8). Reduction-mod-p / Kummer-Dedekind machinery uses Common's ResidueHom (`isMaximal_span_gen_sub`, `absNorm_span_gen_sub`) rather than bespoke residue maps; only thin local wrappers remain.

7 $\mathbb{Q}(\zeta_5)$

Folder: `XYin/NumberField/FieldExamples/CyclotomicFive/`. Field `K = ℚ(ζ₅) = CyclotomicField 5 ℚ`, modelled via `IsCyclotomicExtension {5} ℚ K` with generator ζ = `IsCyclotomicExtension.zeta 5 ℚ K`. Degree φ`(5) = 4`, signature `(0,2)` (totally complex), ring

of integers $\mathbb{Z}[\zeta]$ (monogenic), discriminant $125 = 5^3$, class number 1, unit group $\mu_{10} \times \mathbb{Z}$ (Dirichlet rank 1, fundamental unit $1 + \zeta$). CM field: complex conjugation is $\sigma_4 : \zeta \mapsto \zeta^4$, real subfield $\mathbb{Q}(\sqrt{5})$ realised via the quadratic Gauss sum $1 + 2\zeta + 2\zeta^4$ (whose square is 5). Direct port/twin of `CyclotomicEight` (ζ_8) – same route map (`fromZetaAut`, `autEquivPow`, `torsionPowerHom`/`rankOnePowerHom`, Common `genResidueHom` Kummer-Dedekind). Two extra special-topic files `CMField.lean`, `QuadraticSubfield.lean`. 2 sorries (down from 3): `fundamentalUnitSystem_isFundamental` (Units.lean – $1+\zeta$ is fundamental, the cyclotomic-unit index $h^+=1$ obstruction) and `normEuclidean` (Euclidean.lean, covering argument). The former `regulator_eq` sorry is now closed: its proof body is complete (via mathlib's `regOfFamily_eq_det`, exactly as in SqrtTwo/SqrtFive), so the *only* remaining unit-side blocker is `isFundamental`, on which `regulator_eq` still depends transitively. The "regulator value" was never the API gap; the genuine obstruction is the cyclotomic-unit theory needed to certify $1+\zeta$ as fundamental.

Namespace `NumberField.CyclotomicFive` throughout. Shared helpers come from `NumberField.FieldExamples` (`Common.lean`): `eq_of_le_of_absNorm_eq`, `torsionPowerHom`, `rankOnePowerHom`, `not_isUnramifiedAt_of_map_span_natCast_le_sq_of_not_le_pow_succ`, `not_map_span_natCast_le_pow_succ_of_absNorm_span_eq_pow`, and the generic Kummer-Dedekind `ResidueHom` section (`genResidueHom`, `isMaximal_span_gen_sub`, `absNorm_span_gen_sub`).

Each block below begins with the displayed Lean declaration or object definition. For theorem, lemma, instance-proof, and example blocks the proof body is omitted; for definitions and abbreviations the displayed RHS is retained, because it is part of the mathematical object being documented.

7.1 `Basic.lean`

```
abbrev K : Type := CyclotomicField 5 ℚ
```

Mathematical meaning. `CyclotomicField 5 ℚ`, the field $\mathbb{Q}(\zeta_5)$.

Implementation. This is a direct definition: the displayed RHS fixes the concrete model or object, and later proofs use it by unfolding or by its generated API.

```
instance : IsCyclotomicExtension {5} ℚ K :=
  CyclotomicField.isCyclotomicExtension 5 ℚ
```

Mathematical meaning. cyclotomic-extension structure on `K`.

Implementation. `CyclotomicField.isCyclotomicExtension 5 ℚ`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev ζ : K := IsCyclotomicExtension.zeta 5 ℚ K
```

Mathematical meaning. chosen primitive fifth root of unity.

Implementation. `IsCyclotomicExtension.zeta 5 ℚ K`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zeta_isPrimitiveRoot : IsPrimitiveRoot ζ 5
```

Mathematical meaning. ζ is a primitive 5th root.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.zeta_spec`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem zeta_pow_five : ζ ^ 5 = 1
```

Mathematical meaning. order-5 relation.

Implementation. The proof applies the packaged result `zeta_isPrimitiveRoot.pow_eq_one`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem zeta_sum_eq_zero : 1 + ζ + ζ ^ 2 + ζ ^ 3 + ζ ^ 4 = 0
```

Mathematical meaning. cyclotomic relation $\Phi_5(\zeta) = 0$.

Implementation. `geom_sum_eq_zero` + `Finset.sum_range_succ` unfold + `linear_combination`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem degree_eq_four : Module.finrank ℚ K = 4
```

Mathematical meaning. $[K:\mathbb{Q}] = \varphi(5) = 4$.

Implementation. `IsCyclotomicExtension.finrank` with `cyclotomic.irreducible_rat`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev zetaFivePolynomial : ℚ[X] :=
  Polynomial.X ^ 4 + Polynomial.X ^ 3 + Polynomial.X ^ 2 + Polynomial.X +
    Polynomial.C 1
```

Mathematical meaning. $X^4 + X^3 + X^2 + X + 1 = \Phi_5$.

Implementation. This is a direct definition: the displayed RHS fixes the concrete model or object, and later proofs use it by unfolding or by its generated API.

```
instance : Fact (Irreducible zetaFivePolynomial) where
  out
```

Mathematical meaning. Φ_5 irreducible over $\mathbb{Q}$.

Implementation. rewrite to `cyclotomic 5 ℚ` via `cyclotomic_prime`, then `cyclotomic.irreducible_rat`.

```
abbrev KAdjoinRoot : Type := AdjoinRoot zetaFivePolynomial
```

Mathematical meaning. `AdjoinRoot` Φ_5 presentation of the field.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev zetaFiveRoot : KAdjoinRoot := AdjoinRoot.root zetaFivePolynomial
```

Mathematical meaning. formal root `AdjoinRoot.root` Φ_5.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem zetaFiveRoot_relation :
    1 + zetaFiveRoot + zetaFiveRoot ^ 2 + zetaFiveRoot ^ 3 + zetaFiveRoot
    ^ 4 = 0
```

Mathematical meaning. cyclotomic relation in `AdjoinRoot` model.

Implementation. `AdjoinRoot.eval`$_2$`_root` + `ring`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zetaFivePolynomial_eq_minpoly : zetaFivePolynomial = minpoly ℚ ζ
```

Mathematical meaning. Φ_5 is the minimal polynomial of ζ.

Implementation. The proof applies the packaged result `minpoly_eq_cyclotomic_of_irreducible`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
def zetaMinpolyAdjoinRootPresentation : IsAdjoinRoot K (minpoly ℚ ζ) := by
  exact IsAdjoinRoot.mkOfAdjoinEqTop
    ((IsCyclotomicExtension.integral {5} ℚ K).isIntegral ζ)
    (IsCyclotomicExtension.adjoin_primitive_root_eq_top
    zeta_isPrimitiveRoot)
```

Mathematical meaning. `K` is `AdjoinRoot (minpoly ℚ ζ)`.

Implementation. `IsAdjoinRoot.mkOfAdjoinEqTop` + `adjoin_primitive_root_eq_top`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def adjoinRootEquivCyclotomic : KAdjoinRoot ≃ₐ[ℚ] K := by
  refine (AdjoinRoot.algEquivOfEq ℚ zetaFivePolynomial (minpoly ℚ ζ)
    ?_).trans ?_
  · exact zetaFivePolynomial_eq_minpoly
  · exact zetaMinpolyAdjoinRootPresentation.adjoinRootAlgEquiv
```

Mathematical meaning. `AdjoinRoot` $\Phi_5 \cong \mathbb{Q}(\zeta_5)$ as $\mathbb{Q}$-algebras.

Implementation. `AdjoinRoot.algEquivOfEq` ($\Phi_5 = \text{minpoly}$) $\circ$ `adjoinRootAlgEquiv`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem adjoinRootEquivCyclotomic_root :
    adjoinRootEquivCyclotomic zetaFiveRoot = ζ
```

Mathematical meaning. transport of the formal root.

Implementation. `AdjoinRoot.algEquivOfEq_root` + simp. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

7.2 `Galois.lean`

```
noncomputable def idAut : K ≃ₐ[ℚ] K := AlgEquiv.refl
```

Mathematical meaning. The displayed Lean declaration has type `K ≃ₐ[ℚ] K` – identity (`AlgEquiv.refl`).

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
noncomputable def sigmaTwo : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    2 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))

noncomputable def sigmaThree : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    3 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))

noncomputable def sigmaFour : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    4 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))
```

Mathematical meaning. the automorphisms $\zeta \mapsto \zeta^2, \zeta \mapsto \zeta^3, \zeta \mapsto \zeta^4$; σ_4 is complex conjugation.

Implementation. `IsCyclotomicExtension.fromZetaAut` on `zeta_isPrimitiveRoot.pow_of_coprime k`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sigmaTwo_apply_zeta : sigmaTwo ζ = ζ ^ 2
```

```
theorem sigmaThree_apply_zeta : sigmaThree ζ = ζ ^ 3
```

```
theorem sigmaFour_apply_zeta : sigmaFour ζ = ζ ^ 4
```

Mathematical meaning. action spec. $\sigma_4\ \zeta = \zeta^4 = \zeta^{-1}$ (conj; fixed field = $\mathbb{Q}(\zeta+\zeta^{-1}) = \mathbb{Q}(\sqrt{5})$).

Implementation. The proof applies the packaged result `IsCyclotomicExtension.fromZetaAut_spec`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
private theorem autEq' (σ : K ≃ₐ[ℚ] K) :
    σ = idAut ∨ σ = sigmaTwo ∨ σ = sigmaThree ∨ σ = sigmaFour
```

Mathematical meaning. exhaustive list of the 4 automorphisms.

Implementation. `autToPow_spec`, `(t:ZMod 5).val ∈ {1,2,3,4}` by `decide`, power-basis `algHom_ext`.

```
theorem isGalois : IsGalois ℚ K
```

Mathematical meaning. The displayed Lean declaration has type `IsGalois ℚ K`.

Implementation. `IsCyclotomicExtension.isGalois`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem galoisGroupStructure : Nonempty ((K ≃ₐ[ℚ] K) ≃* (ZMod 5)ˣ)
```

Mathematical meaning. Galois group is the cyclotomic character group $(\mathbb{Z}/5)^\times$.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.autEquivPow`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem galoisGroup_isCyclic : IsCyclic (K ≃ₐ[ℚ] K)
```

Mathematical meaning. Galois group cyclic of order 4 (2 is a primitive root mod 5).

Implementation. surjective image of `autEquivPow.symm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem orderOf_sigmaTwo : orderOf sigmaTwo = 4
```

Mathematical meaning. σ_2 generates the Galois group.

Implementation. transport via `autEquivPow` to $\langle 2,3\rangle \in$ `(ZMod 5)ˣ`, `decide` powers, `Nat.dvd_prime_pow`.

```
theorem aut_eq (σ : K ≃ₐ[ℚ] K) :
    σ = idAut ∨ σ = sigmaTwo ∨ σ = sigmaThree ∨ σ = sigmaFour
```

Mathematical meaning. The displayed Lean declaration has type public wrapper of `autEq'`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

7.3 **Embeddings.lean**

```
theorem isTotallyComplex : NumberField.IsTotallyComplex K
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.IsTotallyComplex K`.

Implementation. `IsCyclotomicExtension.Rat.isTotallyComplex`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem nrRealPlaces_eq_zero : NumberField.InfinitePlace.nrRealPlaces K =
    0
```

Mathematical meaning. The displayed Lean declaration has type `nrRealPlaces K = 0`.

Implementation. `IsCyclotomicExtension.Rat.nrRealPlaces_eq_zero`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem nrComplexPlaces_eq_two :
    NumberField.InfinitePlace.nrComplexPlaces K = 2
```

Mathematical meaning. signature `(0,2)`.

Implementation. `card_add_two_mul_card_eq_rank` + `degree_eq_four` + omega. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_card_eq_two :
    Fintype.card (NumberField.InfinitePlace K) = 2
```

Mathematical meaning. The displayed Lean declaration has type `Fintype.card (InfinitePlace K) = 2`.

Implementation. `card_eq_nrRealPlaces_add_nrComplexPlaces`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_isComplex (v : NumberField.InfinitePlace K) :
    NumberField.InfinitePlace.IsComplex v
```

Mathematical meaning. The displayed Lean declaration has type `∀ v, v.IsComplex`.

Implementation. `isTtotallyComplex.isComplex`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem exists_complexEmbedding_zeta :
    ∃ σ : K →+* ℂ, σ ζ = Complex.exp (2 * Real.pi * Complex.I / 5)
```

Mathematical meaning. explicit complex embedding hitting the standard 5th root.

Implementation. `Complex.isPrimitiveRoot_exp`, `AdjoinRoot.liftAlgHom` of Φ_5, transported through `adjoinRootEquivCyclotomic.symm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

7.4 `RingOfIntegers.lean`

```
theorem zeta_integral : ζ ∈ integralClosure ℤ K
```

Mathematical meaning. The displayed Lean declaration has type `ζ ∈ integralClosure ℤ K`.

Implementation. `IsPrimitiveRoot.isIntegral`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def zetaInt : 𝒪 K := ⟨ζ, zeta_integral⟩
```

Mathematical meaning. The displayed Lean declaration has type $\mathcal{O}$ `K` $=\langle$`ζ, zeta_integral`$\rangle$. The integral form of ζ. Public.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem zetaInt_pow_five : zetaInt ^ 5 = 1
```

Mathematical meaning. The displayed Lean declaration has type `zetaInt ^ 5 = 1`.

Implementation. `Subtype.ext` + `zeta_pow_five`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zetaInt_sum_eq_zero : 1 + zetaInt + zetaInt ^ 2 + zetaInt ^ 3 +
    zetaInt ^ 4 = 0
```

Mathematical meaning. The displayed Lean declaration has type `1 + zetaInt + and related terms + zetaInt`^{4} `= 0`.

Implementation. `Subtype.ext` + `zeta_sum_eq_zero`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev integralPowerBasis : PowerBasis ℤ (𝒪 K) :=
  zeta_isPrimitiveRoot.integralPowerBasis
```

Mathematical meaning. The displayed Lean declaration has type `PowerBasis ℤ (𝒪 K) = zeta_isPrimitiveRoot.integralPowerBasis` $(1, \zeta, \zeta^2, \zeta^3)$. Public.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem integralPowerBasis_gen : integralPowerBasis.gen =
    zeta_isPrimitiveRoot.toInteger
```

Mathematical meaning. The displayed Lean declaration has type `gen = zeta_isPrimitiveRoot.toInteger`.

Implementation. The proof applies the packaged result `simp. Public`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem integralPowerBasis_dim : integralPowerBasis.dim = 4
```

Mathematical meaning. The displayed Lean declaration has type `dim = 4`.

Implementation. `integralPowerBasis_dim`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def integralBasisVectors : Fin 4 → K := ![1, ζ, ζ ^ 2, ζ ^ 3]
```

Mathematical meaning. The displayed Lean declaration has type `Fin 4 → K = ![1, ζ, ζ^2, ζ^3]`. Public.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem span_integralBasisVectors_eq :
    Submodule.span ℤ (Set.range integralBasisVectors) =
      Subalgebra.toSubmodule (integralClosure ℤ K)
```

Mathematical meaning. $1, \zeta, \zeta^2, \zeta^3$ spans $\mathcal{O}$ K as a $\mathbb{Z}$-lattice.

Implementation. monogenicity (`isIntegralClosure_adjoin_singleton`), `modByMonic` reduction against monic Φ_5 over $\mathbb{Z}$, `aeval_eq_sum_range'` to land in the span.

```
noncomputable def concreteIntegralBasis : Basis (Fin 4) ℤ (𝒪 K) :=
  integralPowerBasis.basis.reindex (finCongr integralPowerBasis_dim)
```

Mathematical meaning. The displayed Lean declaration has type `Basis (Fin 4) ℤ (𝒪 K)` – reindexed `integralPowerBasis.basis`. Public.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem concreteIntegralBasis_apply (i : Fin 4) :
    (algebraMap (𝒪 K) K) (concreteIntegralBasis i) = integralBasisVectors
    i
```

Mathematical meaning. The displayed Lean declaration has type `algebraMap (concreteIntegralBasis i) = integralBasisVectors i`.

Implementation. `fin_cases` + simp. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_integralBasisVectors_eq_integralBasis :
    Submodule.span ℤ (Set.range integralBasisVectors) =
      Submodule.span ℤ (Set.range (NumberField.integralBasis K))
```

Mathematical meaning. displayed basis and mathlib's integral basis span the same lattice.

Implementation. reduce both to `integralClosure` via `mem_span_integralBasis`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev concreteRingOfIntegers : Subalgebra ℤ K := Algebra.adjoin ℤ ({ζ} :
    Set K)
```

Mathematical meaning. The displayed Lean declaration has type `Subalgebra ℤ K = Algebra.adjoin ℤ {ζ} = ℤ[ζ]`. Public.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem isIntegralClosure_adjoin_zeta :
    IsIntegralClosure (Algebra.adjoin ℤ ({ζ} : Set K)) ℤ K
```

Mathematical meaning. The displayed Lean declaration has type `IsIntegralClosure (adjoin ℤ {ζ}) ℤ K`.

Implementation. `IsCyclotomicExtension.Rat.isIntegralClosure_adjoin_singleton`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem integralPowerBasis_adjoin_gen_eq_top :
    Algebra.adjoin ℤ ({integralPowerBasis.gen} : Set (𝒪 K)) = ⊤
```

Mathematical meaning. The displayed Lean declaration has type `adjoin ℤ {gen} = ⊤`.

Implementation. `PowerBasis.adjoin_gen_eq_top`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ringOfIntegers_eq : integralClosure ℤ K = concreteRingOfIntegers
```

Mathematical meaning. $\mathcal{O}$ `K` = $\mathbb{Z}[\zeta]$ (monogenic).

Implementation. antisymm via `isIntegralClosure_adjoin_zeta`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

7.5 Discriminant.lean

```
theorem numberField_discr_eq : NumberField.discr K = (125 : ℤ)
```

Mathematical meaning. discriminant = 5^3 (odd-prime cyclotomic formula `(-1)^((p-1)/2) p^(p-2)`).

Implementation. `IsCyclotomicExtension.Rat.discr_prime 5` + `norm_num`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem concreteIntegralBasis_discr :
    Algebra.discr ℤ concreteIntegralBasis = (125 : ℤ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℤ concreteIntegralBasis = 125`.

Implementation. `discr_eq_discr` + `numberField_discr_eq`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem integralBasis_discr :
    Algebra.discr ℚ (NumberField.integralBasis K) = (125 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℚ (integralBasis K) = 125`.

Implementation. `coe_discr` + `numberField_discr_eq`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

7.6 PrimeSplitting.lean

```
private theorem gen_eq_zetaInt :
    zeta_isPrimitiveRoot.integralPowerBasis.gen = zetaInt

private noncomputable def sig2 : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    2 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))

private noncomputable def sig3 : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    3 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))

private noncomputable def sig4 : K ≃ₐ[ℚ] K :=
```

```
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    4 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))

private theorem sig2_zeta : sig2 ζ = ζ ^ 2

private theorem sig3_zeta : sig3 ζ = ζ ^ 3

private theorem sig4_zeta : sig4 ζ = ζ ^ 4

private theorem autEq' (σ : K ≃ₐ[ℚ] K) : σ = 1 ∨ σ = sig2 ∨ σ = sig3 ∨ σ
    = sig4

private theorem prod_aut_apply (x : K) :
    ∏ σ : K ≃ₐ[ℚ] K, σ x = x * sig2 x * sig3 x * sig4 x

private theorem intNorm_eq (x : 𝒪 K) (n : ℤ)
    (h : algebraMap (𝒪 K) K x * sig2 (algebraMap (𝒪 K) K x) *
      sig3 (algebraMap (𝒪 K) K x) * sig4 (algebraMap (𝒪 K) K x) = (n :
    K)) :
    Algebra.norm ℤ x = n

private theorem minpoly_gen_eq :
    minpoly ℤ zeta_isPrimitiveRoot.integralPowerBasis.gen =
      Polynomial.X ^ 4 + Polynomial.X ^ 3 + Polynomial.X ^ 2 +
    Polynomial.X +
        Polynomial.C 1

private theorem isMaximal_span_pair (p : ℕ) [Fact p.Prime] (r : ℤ)
    (hr : ((r : ZMod p)) ^ 4 + ((r : ZMod p)) ^ 3 + ((r : ZMod p)) ^ 2 +
    (r : ZMod p) +
    1 = 0) :
    (Ideal.span {(p : 𝒪 K), zetaInt - (r : 𝒪 K)}).IsMaximal

private theorem absNorm_span_pair (p : ℕ) (r : ℤ)
    (hr : ((r : ZMod p)) ^ 4 + ((r : ZMod p)) ^ 3 + ((r : ZMod p)) ^ 2 +
    (r : ZMod p) +
    1 = 0) :
    Ideal.absNorm (Ideal.span {(p : 𝒪 K), zetaInt - (r : 𝒪 K)}) = p
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem norm_one_sub_zeta : Algebra.norm ℚ (1 - ζ) = 5
```

Mathematical meaning. generator of the prime above 5 has norm $\Phi_5(1) = 5$.

Implementation. `sub_one_norm_isPrimePow` + degree-4 norm of -1. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem norm_one_add_zeta : Algebra.norm ℚ (1 + ζ) = 1
```

Mathematical meaning. $1 + \zeta$ is a *cyclotomic unit* ($\Phi_5(-1) = 1$).

Implementation. $1 - \zeta^2 = (1-\zeta)(1+\zeta)$, norm of $1-\zeta^2$ is 5, divide by `norm_one_sub_zeta`.

```
def normForm {R : Type*} [CommRing R] (a b c d : R) : R :=
  a ^ 4 - a ^ 3 * b - a ^ 3 * c - a ^ 3 * d +
    a ^ 2 * b ^ 2 + 2 * a ^ 2 * b * c + 2 * a ^ 2 * b * d +
    a ^ 2 * c ^ 2 - 3 * a ^ 2 * c * d + a ^ 2 * d ^ 2 -
    a * b ^ 3 - 3 * a * b ^ 2 * c + 2 * a * b ^ 2 * d +
    2 * a * b * c ^ 2 - a * b * c * d - 3 * a * b * d ^ 2 -
    a * c ^ 3 + 2 * a * c ^ 2 * d + 2 * a * c * d ^ 2 - a * d ^ 3 +
    b ^ 4 - b ^ 3 * c - b ^ 3 * d + b ^ 2 * c ^ 2 +
    2 * b ^ 2 * c * d + b ^ 2 * d ^ 2 - b * c ^ 3 -
    3 * b * c ^ 2 * d + 2 * b * c * d ^ 2 - b * d ^ 3 +
    c ^ 4 - c ^ 3 * d + c ^ 2 * d ^ 2 - c * d ^ 3 + d ^ 4
```

Mathematical meaning. The displayed Lean declaration has type R – explicit quartic form $N(a + b\zeta + c\zeta^2 + d\zeta^3)$ in the integral basis. Public.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private def zetaCoords (a b c d : ℚ) : K :=
  (a : K) + (b : K) * ζ + (c : K) * ζ ^ 2 + (d : K) * ζ ^ 3

private def normPairA (a b c d : ℚ) : K :=
  a ^ 2 - a * b + b ^ 2 - b * c + c ^ 2 - c * d + d ^ 2

private def normPairB (a b c d : ℚ) : K :=
  -c * d + b * d - b * c + a * d + a * c - a * b

private def normPairC (a b c d : ℚ) : K :=
```

```
  a ^ 2 - a * c - a * d + b ^ 2 - b * d + c ^ 2 + d ^ 2

private def zetaPair : K := ζ ^ 2 + ζ ^ 3

private theorem sig2_zetaCoords (a b c d : ℚ) :
    sig2 (zetaCoords a b c d) =
      (a : K) - c + (d - c) * ζ + (b - c) * ζ ^ 2 - c * ζ ^ 3

private theorem sig3_zetaCoords (a b c d : ℚ) :
    sig3 (zetaCoords a b c d) =
      (a : K) - d + (c - d) * ζ - d * ζ ^ 2 + (b - d) * ζ ^ 3

private theorem sig4_zetaCoords (a b c d : ℚ) :
    sig4 (zetaCoords a b c d) =
      (a : K) - b - b * ζ + (d - b) * ζ ^ 2 + (c - b) * ζ ^ 3

private theorem zetaCoords_mul_sig4 (a b c d : ℚ) :
    zetaCoords a b c d *
        ((a : K) - b - b * ζ + (d - b) * ζ ^ 2 + (c - b) * ζ ^ 3) =
      normPairA a b c d + normPairB a b c d * zetaPair

private theorem sig2_zetaCoords_mul_sig3 (a b c d : ℚ) :
    ((a : K) - c + (d - c) * ζ + (b - c) * ζ ^ 2 - c * ζ ^ 3) *
        (a - d + (c - d) * ζ - d * ζ ^ 2 + (b - d) * ζ ^ 3) =
      normPairC a b c d - normPairB a b c d * zetaPair

private theorem normPair_mul_eq_normForm (a b c d : ℚ) :
    (normPairA a b c d + normPairB a b c d * zetaPair) *
        (normPairC a b c d - normPairB a b c d * zetaPair) =
      algebraMap ℚ K (normForm a b c d)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem norm_eq_normForm (a b c d : ℚ) :
    Algebra.norm ℚ
        (a • (1 : K) + b • ζ + c • ζ ^ 2 + d • ζ ^ 3) =
      normForm a b c d
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.norm ℚ (a•1 + b•ζ + c•ζ^2 + d•ζ^3) = normForm a b c d`.

Implementation. norm_eq_prod_automorphisms + prod_aut_apply, then the normPair factorisation. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem intNorm_eq_normForm (a b c d : ℤ) :
    Algebra.norm ℤ
        (a • (1 : 𝒪 K) + b • zetaInt + c • zetaInt ^ 2 + d • zetaInt ^ 3)
    =
      normForm a b c d
```

Mathematical meaning. The displayed Lean declaration has type integral version Algebra.norm ℤ (and related terms) = normForm a b c d.

Implementation. coe_norm_int cast from norm_eq_normForm. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isUnit_iff_normForm (a b c d : ℤ) :
    IsUnit (a • (1 : 𝒪 K) + b • zetaInt + c • zetaInt ^ 2 + d • zetaInt ^
    3) ↔
      (normForm a b c d).natAbs = 1
```

Mathematical meaning. unit equation in the integral basis.

Implementation. NumberField.isUnit_iff_norm + intNorm_eq_normForm. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_five_eq : Ideal.span {(5 : 𝒪 K)} = Ideal.span {1 - zetaInt}
    ^ 4
```

Mathematical meaning. 5 is totally ramified, (5) = $(1-\zeta)^4$, e = 4 = [K:ℚ].

Implementation. span_singleton_eq_span_singleton with explicit unit, linear_combination. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem span_one_sub_zetaInt_eq_pair :
    Ideal.span {1 - zetaInt} = Ideal.span {((5 : ℕ) : 𝒪 K), zetaInt - ((1
    : ℤ) : 𝒪 K)}
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem isPrime_span_one_sub_zeta : (Ideal.span {1 - zetaInt}).IsPrime
```

Mathematical meaning. `(1−ζ)` is the unique prime over 5.

Implementation. `isMaximal_span_pair 5 1` (root of Φ_5 `mod 5` is 1). Uses Common ResidueHom (via wrapper).

```
theorem absNorm_span_one_sub_zeta : Ideal.absNorm (Ideal.span {1 -
    zetaInt}) = 5
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (span {1−zetaInt}) = 5` (residue degree `f = 1`).

Implementation. `absNorm_span_pair 5 1`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_span_one_sub_zeta :
    (Ideal.span {1 - zetaInt}).LiesOver (Ideal.span {(5 : ℤ)})
```

Mathematical meaning. The displayed Lean declaration has type `(span {1−zetaInt}).LiesOver (span {(5:ℤ)})`.

Implementation. `liesOver_span_iff` + `span_five_eq`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem inertiaDeg_span_one_sub_zeta :
    (Ideal.span {(5 : ℤ)}).inertiaDeg (Ideal.span {1 - zetaInt}) = 1
```

Mathematical meaning. The displayed Lean declaration has type `inertiaDeg = 1`.

Implementation. `absNorm_eq_pow_inertiaDeg'` + `absNorm_span_one_sub_zeta`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramificationIdx_span_one_sub_zeta :
    (Ideal.span {(5 : ℤ)}).ramificationIdx (Ideal.span {1 - zetaInt}) = 4
```

Mathematical meaning. The displayed Lean declaration has type `ramificationIdx = 4`.

Implementation. `ramificationIdx_spec`; lower bound from `span_five_eq`, upper from absNorm of $\mathfrak{P}^5 \nmid$ `(5)`. Public.

```
theorem absNorm_span_two : Ideal.absNorm (Ideal.span {(2 : 𝓞 K)}) = 16
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (span {2}) = 16` (`f = 4`).

Implementation. `absNorm_span_singleton` + `norm_algebraMap` + `finrank = 4`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem quotient_zeta_ne_one_of_two_mem {M : Ideal (𝒪 K)}
    [M.IsMaximal]
    (h2mem : (2 : 𝒪 K) ∈ M) : Ideal.Quotient.mk M zetaInt ≠ 1

private theorem absNorm_maximal_over_span_two_eq_sixteen {M : Ideal (𝒪 K)}
    (hMmax : M.IsMaximal) (hMle : Ideal.span {(2 : 𝒪 K)} ≤ M) :
    Ideal.absNorm M = 16
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem isPrime_span_two : (Ideal.span {(2 : 𝒪 K)}).IsPrime
```

Mathematical meaning. 2 is inert (`ord_{(ℤ/5)ˣ}(2) = 4`, Φ_5 irreducible mod 2).

Implementation. maximal `M ⊇ (2)` has norm 16 = norm(2), so `M = (2)`; Common `eq_of_le_of_absNorm_eq`.

```
theorem liesOver_span_two :
    (Ideal.span {(2 : 𝒪 K)}).LiesOver (Ideal.span {(2 : ℤ)})

theorem inertiaDeg_span_two :
    (Ideal.span {(2 : ℤ)}).inertiaDeg (Ideal.span {(2 : 𝒪 K)}) = 4
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private theorem nineteen_x_eq : (19 : 𝒪 K) = (zetaInt ^ 2 + 5 * zetaInt +
    1) *
    (5 * zetaInt ^ 4 + zetaInt ^ 2 + zetaInt)

private theorem nineteen_y_eq : (19 : 𝒪 K) = (zetaInt ^ 2 - 4 * zetaInt +
    1) *
    (-(4 * zetaInt ^ 4) + zetaInt ^ 2 + zetaInt)

private theorem span_pair_nineteen_left :
    Ideal.span {(19 : 𝒪 K), zetaInt ^ 2 + 5 * zetaInt + 1} =
      Ideal.span {zetaInt ^ 2 + 5 * zetaInt + 1}
```

```
private theorem span_pair_nineteen_right :
    Ideal.span {(19 : 𝒪 K), zetaInt ^ 2 + 15 * zetaInt + 1} =
      Ideal.span {zetaInt ^ 2 - 4 * zetaInt + 1}

private theorem absNorm_span_x :
    Ideal.absNorm (Ideal.span {zetaInt ^ 2 + 5 * zetaInt + 1}) = 361

private theorem absNorm_span_y :
    Ideal.absNorm (Ideal.span {zetaInt ^ 2 - 4 * zetaInt + 1}) = 361

private theorem absNorm_ne_nineteen (P : Ideal (𝒪 K)) : Ideal.absNorm P ≠
    19

private theorem isPrime_of_absNorm_361 {g : 𝒪 K}
    (h361 : Ideal.absNorm (Ideal.span {g}) = 361) : (Ideal.span
    {g}).IsPrime
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem span_nineteen_eq :
    Ideal.span {(19 : 𝒪 K)} =
      Ideal.span {(19 : 𝒪 K), zetaInt ^ 2 + 5 * zetaInt + 1} *
        Ideal.span {(19 : 𝒪 K), zetaInt ^ 2 + 15 * zetaInt + 1}
```

Mathematical meaning. 19 splits into two degree-2 primes ($19 \equiv -1 \bmod 5$, order 2).

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem isPrime_primesAboveNineteen :
    (Ideal.span {(19 : 𝒪 K), zetaInt ^ 2 + 5 * zetaInt + 1}).IsPrime ∧
      (Ideal.span {(19 : 𝒪 K), zetaInt ^ 2 + 15 * zetaInt + 1}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type both factors of norm 361 are prime.

Implementation. `isPrime_of_absNorm_361`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem elevenA : (ζ ^ 2 + ζ - 1) * (ζ ^ 3 + ζ - 1) = ζ ^ 4 - 2 *
    ζ + 2
```

```
private theorem elevenB : (ζ ^ 4 + ζ ^ 2 - 1) * (ζ ^ 4 + ζ ^ 3 - 1) = -(2
    * ζ ^ 4) + ζ
    + 2

private theorem elevenC : (ζ ^ 4 - 2 * ζ + 2) * (-(2 * ζ ^ 4) + ζ + 2) =
    11

private theorem norm_eleven_orbit {w : 𝒪 K}
    (hw : algebraMap (𝒪 K) K w * sig2 (algebraMap (𝒪 K) K w) *
      sig3 (algebraMap (𝒪 K) K w) * sig4 (algebraMap (𝒪 K) K w) =
        (ζ ^ 2 + ζ - 1) * (ζ ^ 3 + ζ - 1) * ((ζ ^ 4 + ζ ^ 2 - 1) * (ζ ^ 4
    + ζ ^ 3 -
    1))) :
    Ideal.absNorm (Ideal.span {w}) = 11

private theorem absNorm_span_u3 :
    Ideal.absNorm (Ideal.span {zetaInt ^ 2 + zetaInt - 1}) = 11

private theorem absNorm_span_u4 :
    Ideal.absNorm (Ideal.span {zetaInt ^ 4 + zetaInt ^ 3 - 1}) = 11

private theorem absNorm_span_u5 :
    Ideal.absNorm (Ideal.span {zetaInt ^ 4 + zetaInt ^ 2 - 1}) = 11

private theorem absNorm_span_u9 :
    Ideal.absNorm (Ideal.span {zetaInt ^ 3 + zetaInt - 1}) = 11

private theorem span_pair_eleven_three :
    Ideal.span {(11 : 𝒪 K), zetaInt - 3} = Ideal.span {zetaInt ^ 2 +
    zetaInt - 1}

private theorem span_pair_eleven_four :
    Ideal.span {(11 : 𝒪 K), zetaInt - 4} = Ideal.span {zetaInt ^ 4 +
    zetaInt ^ 3 - 1}

private theorem span_pair_eleven_five :
    Ideal.span {(11 : 𝒪 K), zetaInt - 5} = Ideal.span {zetaInt ^ 4 +
    zetaInt ^ 2 - 1}

private theorem span_pair_eleven_nine :
    Ideal.span {(11 : 𝒪 K), zetaInt - 9} = Ideal.span {zetaInt ^ 3 +
    zetaInt - 1}
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem span_eleven_eq :
    Ideal.span {(11 : 𝒪 K)} =
      Ideal.span {(11 : 𝒪 K), zetaInt - 3} * Ideal.span {(11 : 𝒪 K),
    zetaInt - 4} *
        Ideal.span {(11 : 𝒪 K), zetaInt - 5} * Ideal.span {(11 : 𝒪 K),
    zetaInt - 9}
```

Mathematical meaning. 11 splits completely (`11 ≡ 1 mod 5`).

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem isPrime_primeAboveEleven (r : ℤ) (hr : r = 3 ∨ r = 4 ∨ r = 5 ∨ r
    = 9) :
    (Ideal.span {(11 : 𝒪 K), zetaInt - (r : 𝒪 K)}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `(11, ζ - r).IsPrime` for `r ∈ {3,4,5,9}`.

Implementation. `isMaximal_span_pair 11 r + decide`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem one_mod_five_of_splits_completely (p : ℕ) (hp : p.Prime)
    (hp5 : p ≠ 5) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
      Ideal.absNorm P = p) → p % 5 = 1

private theorem splits_completely_of_one_mod_five (p : ℕ) (hp : p.Prime) :
    p % 5 = 1 →
      ∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
        Ideal.absNorm P = p
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem splits_completely_iff (p : ℕ) (hp : p.Prime) (hp5 : p ≠ 5) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
    Ideal.absNorm P = p) ↔
      p % 5 = 1
```

Mathematical meaning. the splitting law for $\mathbb{Q}(\zeta_5)$ (decomposition determined by `p mod 5`).

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem ramified_iff_eq_five (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ^ 2) ↔ p =
    5
```

Mathematical meaning. 5 is the only ramified prime (the prime divisors of `disc = 125`).

Implementation. forward via `differentIdeal`, `dvd_differentIdeal_iff`, `absNorm_differentIdeal = 125`; uses Common `not_isUnramifiedAt_of_` and related terms + `not_map_span_natCast_le_pow_succ_` and related terms; reverse via `span_five_eq`.

```
theorem absNorm_differentIdeal_eq_oneHundredTwentyFive :
    Ideal.absNorm (differentIdeal ℤ (𝒪 K)) = 125
```

Mathematical meaning. The displayed Lean declaration has type `absNorm (differentIdeal ℤ (𝒪 K)) = 125`.

Implementation. `NumberField.absNorm_differentIdeal` + `numberField_discr_eq`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem prime_dvd_differentIdeal_iff_eq_span_one_sub_zeta
    (P : Ideal (𝒪 K)) (_hP : P.IsPrime) (_hP0 : P ≠ ⊥) :
    P ∣ differentIdeal ℤ (𝒪 K) ↔ P = Ideal.span {1 - zetaInt}
```

Mathematical meaning. only the ramified prime $(1-\zeta)$ divides the different.

Implementation. forward by norm-divides-125 $\Rightarrow$ contains 5 $\Rightarrow$ contains $(1-\zeta)$; reverse via `pow_sub_one_dvd_differentIdeal (e = 4)`.

7.7 `ClassGroup.lean`

```
abbrev zetaFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  FractionalIdeal.spanSingleton (𝒪 K)⁰ ζ
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev ramifiedPrimeFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  (Ideal.span {1 - zetaInt} : FractionalIdeal (𝒪 K)⁰ K)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem zeta_mem_fractionalIdeal :
    ζ ∈ zetaFractionalIdeal
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `mem_spanSingleton_self`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zetaFractionalIdeal_absNorm :
    FractionalIdeal.absNorm zetaFractionalIdeal = |Algebra.norm ℚ ζ|
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `absNorm_span_singleton`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramifiedPrimeFractionalIdeal_ne_zero :
    ramifiedPrimeFractionalIdeal ≠ 0
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

Mathematical meaning. class number 1 (Minkowski bound $\approx$ `1.7 < 2`).

Implementation. `classNumber_eq_one_iff` $\rightarrow$ `isPrincipalIdealRing_of_isPrincipal_of_pow_le_of_mem_primesOver_of_mem_Icc`; bound the Minkowski floor `< 2` via `nrComplexPlaces=2`, `disc=125`, $\sqrt{125}$ `< 12`, π `> 3.14`, `nlinarith`.

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝒪 K)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `classNumber_eq_one_iff`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝒪 K)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. PID ⇒ UFD instance. Public.

```
theorem classGroupStructure : Nonempty (ClassGroup (𝒪 K) ≃*
    Multiplicative (ZMod 1))
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. trivial group via `Nat.card = 1`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev unitIdealRepresentative : (Ideal (𝒪 K))⁰ := ⟨(⊤ : Ideal (𝒪 K)), by
    simp⟩

theorem classGroupRepresentative (C : ClassGroup (𝒪 K)) :
    C = ClassGroup.mk0 unitIdealRepresentative
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

7.8 **Units.lean**

```
noncomputable def torsionGenerator : (𝒪 K)ˣ where
  val := -zetaInt
  inv := -zetaInt ^ 4
  val_inv := by linear_combination zetaInt_pow_five
  inv_val := by linear_combination zetaInt_pow_five
```

Mathematical meaning. The displayed Lean declaration has type `(𝒪 K)ˣ` – $-\zeta$ with inverse $-\zeta^4$ (`(-ζ)(-ζ⁴) = ζ⁵ = 1`); primitive 10th root. Public.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
def torsionUnits : Set ((𝓞 K)ˣ) := {u | ∃ k : Fin 10, u =
    torsionGenerator ^ k.val}
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem torsionOrder_eq' : NumberField.Units.torsionOrder K = 10
```

```
private theorem neg_zeta_isPrimitiveRoot : IsPrimitiveRoot (-ζ : K) 10
```

```
theorem torsionUnits_eq : {u : (𝓞 K)ˣ | IsOfFinOrder u} = torsionUnits
```

```
theorem torsionOrder_eq_ten : NumberField.Units.torsionOrder K = 10
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `IsCyclotomicExtension.Rat.torsionOrder_eq` (5 odd $\Rightarrow$ 2·5). Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem neg_zeta_isPrimitiveRoot : IsPrimitiveRoot (-ζ : K) 10
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `orderOf_mul` of coprime orders 2 and 5. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem torsionUnits_eq : {u : (𝓞 K)ˣ | IsOfFinOrder u} = torsionUnits
```

Mathematical meaning. the torsion subgroup is exactly μ_{10} `= {±ζ^j}`.

Implementation. `rootsOfUnity_eq_torsion` + `torsionOrder = 10`; `eq_pow_of_pow_eq_one` of $-\zeta$. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem unitRank_eq_one : NumberField.Units.rank K = 1
```

Mathematical meaning. $r_1+r_2-1 = 1$.

Implementation. `nrRealPlaces=0`, `nrComplexPlaces=2`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def fundamentalUnit : (𝒪 K)ˣ where
  val := 1 + zetaInt
  inv := -(zetaInt + zetaInt ^ 3)
  val_inv := by linear_combination -zetaInt_sum_eq_zero
  inv_val := by linear_combination -zetaInt_sum_eq_zero
```

Mathematical meaning. The displayed Lean declaration has type `(𝒪 K)ˣ` – the cyclotomic unit `1 + ζ`, inverse `-(ζ + ζ³) ((1+ζ)(ζ+ζ³) = ζ+ζ²+ζ³+ζ⁴ = -1)`. Public.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
noncomputable def fundamentalUnitSystem : Fin (NumberField.Units.rank K)
    → (𝒪 K)ˣ :=
  fun _ => fundamentalUnit
```

Mathematical meaning. The displayed Lean declaration has type `Fin (rank K) → (𝒪 K)ˣ` – the single-element system `fun _ => 1 + ζ`. Public.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem exists_eq_torsion_mul_fundSystem_zpow (i₀ : Fin
    (NumberField.Units.rank
    K))
    (u : (𝒪 K)ˣ) :
    ∃ ζ' ∈ NumberField.Units.torsion K, ∃ n : ℤ,
      u = ζ' * NumberField.Units.fundSystem K i₀ ^ n
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private theorem fundSystem_zpow_mem_torsion (i₀ : Fin
    (NumberField.Units.rank K)) (n :
    ℤ)
    (hn : NumberField.Units.fundSystem K i₀ ^ n ∈
    NumberField.Units.torsion K) : n = 0
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private theorem eq_torsionGenerator_pow_of_mem_torsion {ζ' : (𝒪 K)ˣ}
    (hζ' : ζ' ∈ NumberField.Units.torsion K) : ∃ k : Fin 10, ζ' =
    torsionGenerator ^
    k.val
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem fundamentalUnitSystem_isFundamental :
    Subgroup.closure (Set.range fundamentalUnitSystem) ⊔
    NumberField.Units.torsion K = ⊤
```

Mathematical meaning. `1 + ζ` together with μ_{10} generates *all* units of $\mathbb{Z}[\zeta]$.

Implementation. SORRIED. This is the cyclotomic-unit index statement `[𝒪ˣ : ⟨μ, cyclotomic units⟩] = h⁺ = 1` for $\mathbb{Q}(\zeta_5)$. Blocked: identifying the *explicit* unit `1 + ζ` with mathlib's *abstract* `fundSystem K 0` needs the index of the cyclotomic-unit group, which mathlib does not provide. (`unit_eq_torsion_mul_ and related terms` and `unitGroupStructure` proceed *assuming* this lemma, but the abstract structure is proved independently from `fundSystem`.)

```
theorem unit_eq_torsion_mul_fundamentalUnit_zpow (u : (𝒪 K)ˣ) :
    ∃ ζ' : (𝒪 K)ˣ, ζ' ∈ NumberField.Units.torsion K ∧ ∃ n : ℤ, u = ζ' *
    fundamentalUnit
    ^ n
```

Mathematical meaning. The displayed Lean declaration has type `∃ ζ' ∈ torsion, ∃ n, u = ζ' · (1+ζ)^n` (every unit is $\pm\zeta$`^j (1+ζ)^n`).

Implementation. `mem_sup` of `fundamentalUnitSystem_isFundamental`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem unitGroupStructure_bijective
    (i₀ : Fin (NumberField.Units.rank K)) (η : (𝒪 K)ˣ)
    (hη : η = NumberField.Units.fundSystem K i₀) (htg10 :
    torsionGenerator ^ 10 = 1)
    (htg_tor : torsionGenerator ∈ NumberField.Units.torsion K) :
    Function.Bijective
      ((torsionPowerHom 10 torsionGenerator htg10).coprod
    (rankOnePowerHom η))
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem unitGroupStructure :
    Nonempty ((𝒪 K)ˣ ≃* (Multiplicative (ZMod 10) × Multiplicative (Fin 1
    → ℤ)))
```

Mathematical meaning. the unit group $\cong \mu_{10} \times \mathbb{Z}$ abstractly.

Implementation. `MulEquiv.ofBijective` of the coprod hom (built on `fundSystem`, *not* on `fundamentalUnit` – independent of the sorry).

```
theorem regulator_eq :
    NumberField.Units.regulator K = 2 * Real.log ((1 + Real.sqrt 5) / 2)
```

Mathematical meaning. regulator = 2 log $\varphi \approx$ 0.962 (φ = (1+$\sqrt{5}$)/2 = |1+ζ| at one complex place).

Implementation. the proof body is now complete – `fundamentalUnitSystem_isFundamental` (index 1) ⇒ `regOfFamily_div_regulator` ⇒ `regulator = regOfFamily`, then `regOfFamily_eq_det` + `Matrix.det_eq_elem_of_subsingleton` evaluate the rank-1 determinant with the archimedean value `|1+ζ|_v` = φ. It is not its own sorry any more, but it *calls* the still-sorried `fundamentalUnitSystem_isFundamental`, so it is true only modulo that. The regulator-value step itself uses only mature mathlib API.

7.9 `Euclidean.lean`

```
theorem normEuclidean (a b : 𝒪 K) (hb : b ≠ 0) :
    ∃ q r : 𝒪 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

Mathematical meaning. $\mathbb{Z}[\zeta_5]$ is norm-Euclidean.

Implementation. SORRIED. Requires the standard covering argument (every element of K is within norm-< 1 of a lattice point of $\mathbb{Z}[\zeta]$ under the canonical embedding) – no mathlib API for norm-Euclideanity of cyclotomic rings; needs a geometry-of-numbers covering estimate.

7.10 `LocalGlobal.lean`

```
def heightOneFive : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {1 - zetaInt}
  isPrime := by
    exact isPrime_span_one_sub_zeta
  ne_bot := by
    intro h
    have hnorm := absNorm_span_one_sub_zeta
```

```
    rw [h] at hnorm
    norm_num at hnorm
```

Mathematical meaning. The displayed Lean declaration has type `HeightOneSpectrum (𝒪 K)` – the ramified prime `(1 − ζ)` as a height-one point.

Implementation. `asIdeal = span {1−zetaInt}`, prime/`ne_bot` from PrimeSplitting. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev vFive : NumberField.FinitePlace K :=
  NumberField.FinitePlace.mk heightOneFive
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example :
    (Ideal.span {(5 : ℤ)}).ramificationIdx heightOneFive.asIdeal = 4

example :
    (Ideal.span {(5 : ℤ)}).inertiaDeg heightOneFive.asIdeal = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. This anonymous example is a smoke test: it instantiates the relevant API in this concrete field and checks the reductions.

```
abbrev CompletionAtFive : Type :=
  heightOneFive.adicCompletion K

abbrev InfiniteCompletion (v : NumberField.InfinitePlace K) : Type :=
  v.Completion
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem complexEmbedding_norm_zeta_eq_one (φ : K →+* ℂ) : ‖φ ζ‖ =
    1
```

```
example (v : NumberField.InfinitePlace K) :
    Nonempty (InfiniteCompletion v ≃+* ℂ)

theorem infiniteCompletion_zeta_norm (v : NumberField.InfinitePlace K) :
    ‖(algebraMap K (InfiniteCompletion v) ζ)‖ = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
example (v : NumberField.InfinitePlace K) :
    Nonempty (InfiniteCompletion v ≃+* ℂ)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. This anonymous example is a smoke test: it instantiates the relevant API in this concrete field and checks the reductions.

```
private theorem zeta_ne_zero : ζ ≠ 0

example :
    ((∏ w : NumberField.InfinitePlace K, w ζ ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K, v ζ) = 1

example :
    ((∏ w : NumberField.InfinitePlace K,
          w (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪 K)) ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K,
          v (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪 K))) = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
abbrev CyclotomicFiveInfiniteAdeleRing : Type :=
  NumberField.InfiniteAdeleRing K

abbrev infinitePrincipalAdele_zeta : CyclotomicFiveInfiniteAdeleRing :=
  algebraMap K CyclotomicFiveInfiniteAdeleRing ζ
```

```
abbrev infinitePrincipalAdele_fundamentalUnit :
    CyclotomicFiveInfiniteAdeleRing :=
  algebraMap K CyclotomicFiveInfiniteAdeleRing (algebraMap (𝒪 K) K
    (fundamentalUnit : 𝒪
    K))

example :
    ‖infinitePrincipalAdele_zeta‖ = |Algebra.norm ℚ ζ|

example :
    DenseRange (algebraMap K CyclotomicFiveInfiniteAdeleRing)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem canonicalEmbedding_zetaInt_mem_integerLattice :
    NumberField.canonicalEmbedding K (algebraMap (𝒪 K) K zetaInt) ∈
      NumberField.canonicalEmbedding.integerLattice K
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof applies the packaged result `range membership. Public`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
abbrev mixedEmbedding_zeta : NumberField.mixedEmbedding.mixedSpace K :=
  NumberField.mixedEmbedding K ζ

example
    (f : NumberField.InfinitePlace K → NNReal)
    (hf : ∀ w : NumberField.InfinitePlace K, w ζ < f w) :
    mixedEmbedding_zeta ∈ NumberField.mixedEmbedding.convexBodyLT K f

example :
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone K)

example :
    MeasurableSet (NumberField.mixedEmbedding.fundamentalCone.normLeOne K)

theorem polarSpace_nonempty :
    Nonempty (NumberField.mixedEmbedding.polarSpace K)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem house_zeta :
    NumberField.house ζ = 1
```

Mathematical meaning. house of a root of unity is 1.

Implementation. `norm_le_iff` (all conjugates norm 1) + lower bound. Public.

```
example :
    0 < NumberField.house (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪 K))
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. This anonymous example is a smoke test: it instantiates the relevant API in this concrete field and checks the reductions.

```
abbrev CyclotomicFiveAdeleRing : Type :=
  NumberField.AdeleRing (𝒪 K) K

abbrev principalAdele_zeta : CyclotomicFiveAdeleRing :=
  algebraMap K CyclotomicFiveAdeleRing ζ

abbrev principalAdele_fundamentalUnit : CyclotomicFiveAdeleRing :=
  algebraMap K CyclotomicFiveAdeleRing (algebraMap (𝒪 K) K
    (fundamentalUnit : 𝒪 K))

theorem principalAdele_zeta_ne_zero : principalAdele_zeta ≠ 0

theorem principalAdele_fundamentalUnit_is_unit_everywhere :
    ∀ v : NumberField.FinitePlace K,
      v (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪 K)) = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
abbrev cyclotomicFiveDedekindZeta (s : ℂ) : ℂ :=
  NumberField.dedekindZeta K s

abbrev cyclotomicFiveDedekindZetaResidue : ℝ :=
  NumberField.dedekindZeta_residue K

example :
    0 < cyclotomicFiveDedekindZetaResidue

example :
    cyclotomicFiveDedekindZetaResidue =
      (2 ^ NumberField.InfinitePlace.nrRealPlaces K *
          (2 * Real.pi) ^ NumberField.InfinitePlace.nrComplexPlaces K *
          NumberField.Units.regulator K * NumberField.classNumber K) /
        (NumberField.Units.torsionOrder K * Real.sqrt |NumberField.discr
    K|)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

7.11 `CMField.lean`

```
theorem isCMField : NumberField.IsCMField K
```

Mathematical meaning. $\mathbb{Q}(\zeta_5)$ is a CM field.

Implementation. `IsCyclotomicExtension.Rat.isCMField K` with witness `5` (odd, `> 2`). Uses mathlib's `NumberField.IsCMField` class directly.

```
theorem cmField_isTotallyComplex : NumberField.IsTotallyComplex K
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `isTotallyComplex`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cmField_quadratic_over_maximalRealSubfield :
    Algebra.IsQuadraticExtension (NumberField.maximalRealSubfield K) K
```

Mathematical meaning. `K` is quadratic over its maximal real subfield $K^+ = \mathbb{Q}(\sqrt{5})$.

Implementation. `infer_instance` given `IsCMField K` (mathlib instance). Uses `maximalRealSubfield`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cmField_unitsRank_eq_maximalRealSubfield :
    NumberField.Units.rank (NumberField.maximalRealSubfield K) =
      NumberField.Units.rank K
```

Mathematical meaning. the unit-rank equality for CM fields (`rank K⁺ = rank K`, here both 1).

Implementation. The proof applies the packaged result `NumberField.IsCMField.units_rank_eq_units_rank`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

7.12 `QuadraticSubfield.lean`

```
theorem gaussSum_sq : (1 + 2 * ζ + 2 * ζ ^ 4) ^ 2 = 5
```

Mathematical meaning. the quadratic Gauss sum for 5 squares to 5 $\Rightarrow \sqrt{5} \in \mathbb{Q}(\zeta_5)$; $\mathbb{Q}(\sqrt{5})$ is the unique quadratic subfield (index-2 subgroup of the cyclic Galois group; Kronecker-Weber in miniature).

Implementation. `linear_combination` of `zeta_pow_five` + `zeta_sum_eq_zero`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev sqrtFivePolynomial : ℚ[X] := Polynomial.X ^ 2 - Polynomial.C 5
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
instance : Fact (Irreducible sqrtFivePolynomial) where
  out
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
abbrev QSqrtFive : Type := AdjoinRoot sqrtFivePolynomial

abbrev sqrtFiveRoot : QSqrtFive := AdjoinRoot.root sqrtFivePolynomial

theorem sqrtFiveRoot_sq : sqrtFiveRoot ^ 2 = (5 : QSqrtFive)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
abbrev gaussSum : K := 1 + 2 * ζ + 2 * ζ ^ 4
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev sqrtFiveSubfield : Subalgebra ℚ K := Algebra.adjoin ℚ ({gaussSum}
    : Set K)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
noncomputable def sqrtFiveToCyclotomic : QSqrtFive →ₐ[ℚ] K := by
  exact AdjoinRoot.liftAlgHom sqrtFivePolynomial (Algebra.ofId ℚ K)
    gaussSum (by
    simp [sqrtFivePolynomial, gaussSum, gaussSum_sq])
```

Mathematical meaning. the embedding $\mathbb{Q}(\sqrt{5}) \hookrightarrow \mathbb{Q}(\zeta_5), \sqrt{5} \mapsto$ `gaussSum`.

Implementation. `AdjoinRoot.liftAlgHom` of `Algebra.ofId ℚ K` using `gaussSum_sq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def sqrtFiveEquivSubfield : QSqrtFive ≃ₐ[ℚ]
    sqrtFiveSubfield := by
  let gs : sqrtFiveSubfield := ⟨gaussSum,
    Algebra.self_mem_adjoin_singleton ℚ gaussSum⟩
  let g : QSqrtFive →ₐ[ℚ] sqrtFiveSubfield :=
    AdjoinRoot.liftAlgHom sqrtFivePolynomial (Algebra.ofId ℚ
    sqrtFiveSubfield) gs (by
      ext
      simp [sqrtFivePolynomial, gs, gaussSum, gaussSum_sq])
  refine AlgEquiv.ofBijective g ⟨RingHom.injective g.toRingHom, ?_⟩
  intro y
  let p : (x : K) → x ∈ Algebra.adjoin ℚ ({gaussSum} : Set K) → Prop :=
```

```
    fun x hx => ∃ z : QSqrtFive, g z = ⟨x, hx⟩
  change p y.1 y.2
  refine Algebra.adjoin_induction (s := ({gaussSum} : Set K)) ?_ ?_ ?_ ?_
    y.2
  · intro x hx
    rcases hx with rfl
    exact ⟨sqrtFiveRoot,
      AdjoinRoot.liftAlgHom_root sqrtFivePolynomial (Algebra.ofId ℚ
    sqrtFiveSubfield)
    gs _⟩
  · intro r
    exact ⟨algebraMap ℚ QSqrtFive r, g.commutes r⟩
  · intro x y hx hy hx' hy'
    rcases hx' with ⟨x', hxeq⟩
    rcases hy' with ⟨y', hyeq⟩
    refine ⟨x' + y', ?_⟩
    rw [map_add, hxeq, hyeq]
    ext
    rfl
  · intro x y hx hy hx' hy'
    rcases hx' with ⟨x', hxeq⟩
    rcases hy' with ⟨y', hyeq⟩
    refine ⟨x' * y', ?_⟩
    rw [map_mul, hxeq, hyeq]
    ext
    rfl
```

Mathematical meaning. $\mathbb{Q}(\sqrt{5}) \cong$ the generated subfield.

Implementation. `AlgEquiv.ofBijective` of the corestricted lift; surjectivity by `Algebra.adjoin_induction` over the single generator `gaussSum`.

```
theorem sqrtFiveToCyclotomic_root : sqrtFiveToCyclotomic sqrtFiveRoot =
    gaussSum

theorem sqrtFiveEquivSubfield_root :
    ((sqrtFiveEquivSubfield sqrtFiveRoot : sqrtFiveSubfield) : K) =
    gaussSum
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. `AdjoinRoot.liftAlgHom_root`. Public. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sqrtFiveToCyclotomic_injective : Function.Injective
    sqrtFiveToCyclotomic
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

7.13 Summary for $\mathbb{Q}(\zeta_5)$

Folder: `XYin/NumberField/FieldExamples/CyclotomicFive/`. Field `K` = $\mathbb{Q}(\zeta_5)$ = `CyclotomicField 5 ℚ`, modelled via `IsCyclotomicExtension {5} ℚ K` with generator ζ = `IsCyclotomicExtension.zeta 5 ℚ K`. Degree φ`(5) = 4`, signature `(0,2)` (totally complex), ring of integers $\mathbb{Z}[\zeta]$ (monogenic), discriminant `125 = 5`^{3}, class number `1`, unit group $\mu_{10} \times \mathbb{Z}$ (Dirichlet rank `1`, fundamental unit `1 + ζ`). CM field: complex conjugation is σ_4 `:` $\zeta \mapsto \zeta^4$, real subfield $\mathbb{Q}(\sqrt{5})$ realised via the quadratic Gauss sum `1 + 2ζ + 2ζ`^{4} (whose square is `5`). Direct port/twin of `CyclotomicEight` (ζ_8) – same route map (`fromZetaAut`, `autEquivPow`, `torsionPowerHom`/`rankOnePowerHom`, Common `genResidueHom` Kummer-Dedekind). Two extra special-topic files `CMField.lean`, `QuadraticSubfield.lean`. 2 sorries (down from 3): `fundamentalUnitSystem_isFundamental` (Units.lean – `1+ζ` is fundamental, the cyclotomic-unit index `h⁺=1` obstruction) and `normEuclidean` (Euclidean.lean, covering argument). The former `regulator_eq` sorry is now closed: its proof body is complete (via mathlib's `regOfFamily_eq_det`, exactly as in SqrtTwo/SqrtFive), so the *only* remaining unit-side blocker is `isFundamental`, on which `regulator_eq` still depends transitively. The "regulator value" was never the API gap; the genuine obstruction is the cyclotomic-unit theory needed to certify `1+ζ` as fundamental.

Namespace `NumberField.CyclotomicFive` throughout. Shared helpers come from `NumberField.FieldExamples` (`Common.lean`): `eq_of_le_of_absNorm_eq`, `torsionPowerHom`, `rankOnePowerHom`, `not_isUnramifiedAt_of_map_span_natCast_le_sq_of_not_le_pow_succ`, `not_map_span_natCast_le_pow_succ_of_absNorm_span_eq_pow`, and the generic Kummer-Dedekind `ResidueHom` section (`genResidueHom`, `isMaximal_span_gen_sub`, `absNorm_span_gen_sub`).

8 $\mathbb{Q}(\zeta_8)$

Field: $\mathbb{Q}$`(ζ₈)`, the eighth cyclotomic field, modelled as `K := CyclotomicField 8 ℚ` via `IsCyclotomicExtension {8} ℚ K`. Degree φ`(8) = 4`, signature `(0,2)` (totally complex, CM), ring of integers $\mathbb{Z}[\zeta_8]$ (monogenic), discriminant `256 = 2`^{8}, class number `1`. The Galois group is $(\mathbb{Z}/8)^\times$, the non-cyclic Klein four-group $\mathbb{Z}/2 \times \mathbb{Z}/2$ (smallest non-cyclic Galois group over $\mathbb{Q}$); every automorphism is an involution. CM field with three quadratic subfields $\mathbb{Q}(\sqrt{2})$, $\mathbb{Q}$`(i)`, $\mathbb{Q}(\sqrt{-2})$. Unit group $\mu_8 \times \mathbb{Z}$ (rank 1), fundamental unit `1 +` $\sqrt{2}$. This is the twin of CyclotomicFive ($\mathbb{Q}(\zeta_5)$, cyclic Galois group); the route map (fromZetaAut automorphisms, residueHom Kummer-Dedekind, `torsionPowerHom`/`rankOnePowerHom` unit structure) is shared. 2 sorries (down from 3): `fundamentalUnitSystem_isFundamental`

(Units.lean – $1+\sqrt{2}$ is fundamental, the cyclotomic-unit index $h^+=1$ obstruction) and `normEuclidean` (Euclidean.lean, geometry-of-numbers covering argument). The former `regulator_eq` sorry is now closed: its body is complete via mathlib's `regOfFamily_eq_det` (as in SqrtTwo/SqrtFive/CyclotomicFive), so it depends only transitively on `isFundamental`. The regulator value was never the API gap – the genuine obstruction is certifying $1+\sqrt{2}$ as the fundamental unit (cyclotomic-unit / real-subfield theory).

All declarations live in `namespace NumberField.CyclotomicEight` unless noted. Shared helpers referenced from `XYin/NumberField/FieldExamples/Common.lean` (`namespace NumberField.FieldExamples`).

Each block below begins with the displayed Lean declaration or object definition. For theorem, lemma, instance-proof, and example blocks the proof body is omitted; for definitions and abbreviations the displayed RHS is retained, because it is part of the mathematical object being documented.

8.1 `Basic.lean`

```
abbrev K : Type := CyclotomicField 8 ℚ
```

Mathematical meaning. The eighth cyclotomic field over $\mathbb{Q}$.

Implementation. Direct abbreviation of `CyclotomicField 8 ℚ`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : IsCyclotomicExtension {8} ℚ K :=
  CyclotomicField.isCyclotomicExtension 8 ℚ
```

Mathematical meaning. Endows `K` with the cyclotomic-extension structure.

Implementation. `CyclotomicField.isCyclotomicExtension 8 ℚ`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev ζ : K := IsCyclotomicExtension.zeta 8 ℚ K
```

Mathematical meaning. A chosen primitive eighth root of unity.

Implementation. Mathlib's canonical `IsCyclotomicExtension.zeta`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zeta_isPrimitiveRoot : IsPrimitiveRoot ζ 8
```

Mathematical meaning. ζ is a primitive eighth root of unity.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.zeta_spec`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem zeta_pow_eight : ζ ^ 8 = 1
```

Mathematical meaning. The displayed Lean declaration has type ζ `^ 8 = 1`.

Implementation. The proof applies the packaged result `IsPrimitiveRoot.pow_eq_one`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem zeta_pow_four : ζ ^ 4 = -1
```

Mathematical meaning. Vanishing of Φ_8 at ζ: ζ^4 `= −1`. Heavily used (every `linear_combination`).

Implementation. `(zeta_isPrimitiveRoot.pow (8 = 4*2)).eq_neg_one_of_two_right`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem degree_eq_four : Module.finrank ℚ K = 4
```

Mathematical meaning. `[`$\mathbb{Q}(\zeta_8)$`:`$\mathbb{Q}$`] = `φ`(8) = 4`.

Implementation. `IsCyclotomicExtension.finrank` + `cyclotomic.irreducible_rat`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev zetaEightPolynomial : Polynomial ℚ := Polynomial.X ^ 4 +
    Polynomial.C (1 : ℚ)
```

Mathematical meaning. The defining polynomial `X`^{4} `+ 1`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem cyclotomic_eight_rat : Polynomial.cyclotomic 8 ℚ =
    zetaEightPolynomial
```

Mathematical meaning. Φ_8 `= X`^{4} `+ 1` over $\mathbb{Q}$.

Implementation. Two `cyclotomic_expand_eq_cyclotomic` steps (2→4→8) from `cyclotomic_two`, then `expand_X`/`ring`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
instance : Fact (Irreducible zetaEightPolynomial) where
  out
```

Mathematical meaning. `X`^{4} `+ 1` is irreducible over $\mathbb{Q}$.

Implementation. rewrite to Φ_8 via `cyclotomic_eight_rat`, then `cyclotomic.irreducible_rat`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev KAdjoinRoot : Type := AdjoinRoot zetaEightPolynomial
```

Mathematical meaning. The field presented as $\mathbb{Q}$[X]/(X^4+1).

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev zetaEightRoot : KAdjoinRoot := AdjoinRoot.root zetaEightPolynomial
```

Mathematical meaning. The formal root generator in the AdjoinRoot model.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem zetaEightRoot_pow_four : zetaEightRoot ^ 4 = (-1 : KAdjoinRoot)
```

Mathematical meaning. The displayed Lean declaration has type zetaEightRoot ^ 4 = (−1 : KAdjoinRoot).

Implementation. AdjoinRoot.eval$_2$_root gives root^4+1=0; calc/ring. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem zetaEightPolynomial_eq_minpoly : zetaEightPolynomial = minpoly ℚ ζ
```

Mathematical meaning. The displayed Lean declaration has type zetaEightPolynomial = minpoly $\mathbb{Q}$ ζ.

Implementation. cyclotomic_eight_rat + IsPrimitiveRoot.minpoly_eq_cyclotomic_of_irreducible. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def zetaMinpolyAdjoinRootPresentation : IsAdjoinRoot K (minpoly ℚ ζ) := by
  exact IsAdjoinRoot.mkOfAdjoinEqTop
    ((IsCyclotomicExtension.integral {8} ℚ K).isIntegral ζ)
    (IsCyclotomicExtension.adjoin_primitive_root_eq_top
    zeta_isPrimitiveRoot)
```

Mathematical meaning. The displayed Lean declaration has type IsAdjoinRoot K (minpoly $\mathbb{Q}$ ζ).

Implementation. IsAdjoinRoot.mkOfAdjoinEqTop + adjoin_primitive_root_eq_top. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def adjoinRootEquivCyclotomic : KAdjoinRoot ≃ₐ[ℚ] K := by
  refine (AdjoinRoot.algEquivOfEq ℚ zetaEightPolynomial (minpoly ℚ ζ)
    ?_).trans ?_
  · exact zetaEightPolynomial_eq_minpoly
  · exact zetaMinpolyAdjoinRootPresentation.adjoinRootAlgEquiv
```

Mathematical meaning. Equivalence between the `AdjoinRoot` model and `CyclotomicField`.

Implementation. `AdjoinRoot.algEquivOfEq` (poly equality) composed with `adjoinRootAlgEquiv`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem adjoinRootEquivCyclotomic_root :
    adjoinRootEquivCyclotomic zetaEightRoot = ζ
```

Mathematical meaning. The displayed Lean declaration has type `adjoinRootEquivCyclotomic zetaEightRoot = ζ`.

Implementation. `AdjoinRoot.algEquivOfEq_root` + `simp`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

8.2 `Galois.lean`

```
noncomputable def idAut : K ≃ₐ[ℚ] K := AlgEquiv.refl
```

Mathematical meaning. Identity automorphism.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
noncomputable def sigmaThree : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    3 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))

noncomputable def sigmaFive : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    5 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))

noncomputable def sigmaSeven : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    7 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))
```

Mathematical meaning. The three nontrivial automorphisms $\zeta \mapsto \zeta^3$ / ζ^5 / ζ^7 (fixing $\mathbb{Q}(\sqrt{-2})$ / $\mathbb{Q}(i)$ / $\mathbb{Q}(\sqrt{2})$ resp.; `sigmaSeven` is complex conjugation).

Implementation. `IsCyclotomicExtension.fromZetaAut` of `zeta_isPrimitiveRoot.pow_of_coprime k`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem sigmaThree_apply_zeta : sigmaThree ζ = ζ ^ 3
```

```
theorem sigmaFive_apply_zeta : sigmaFive ζ = ζ ^ 5
```

```
theorem sigmaSeven_apply_zeta : sigmaSeven ζ = ζ ^ 7
```

Mathematical meaning. The displayed Lean declaration has type `sigmaK ζ = ζ ^ k` (k = 3,5,7).

Implementation. The proof applies the packaged result `IsCyclotomicExtension.fromZetaAut_spec`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem isGalois : IsGalois ℚ K
```

Mathematical meaning. The displayed Lean declaration has type `IsGalois ℚ K`.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.isGalois`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem galoisGroupStructure : Nonempty ((K ≃ₐ[ℚ] K) ≃* (ZMod 8)ˣ)
```

Mathematical meaning. Galois group $\cong (\mathbb{Z}/8)^\times$ (cyclotomic character).

Implementation. The proof applies the packaged result `IsCyclotomicExtension.autEquivPow`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem galoisGroup_klein :
    Nonempty ((K ≃ₐ[ℚ] K) ≃*
      (Multiplicative (ZMod 2) × Multiplicative (ZMod 2)))
```

Mathematical meaning. The Galois group is the Klein four-group.

Implementation. `autEquivPow` transports the involution property; establishes `Nat.card = 4` (`card_aut_eq_finrank`) and `Monoid.exponent = 2` to land `IsKleinFour` on both sides, then `IsKleinFour.nonempty_mulEquiv`; `decide` on `(ZMod 8)ˣ`.

```
theorem aut_sq_eq_one (σ : K ≃ₐ[ℚ] K) : σ * σ = 1
```

Mathematical meaning. Every automorphism is an involution (key non-cyclic witness).

Implementation. `autEquivPow` injectivity + `decide` that `t*t=1` in `(ZMod 8)ˣ`.

```
theorem galoisGroup_not_isCyclic : ¬ IsCyclic (K ≃ₐ[ℚ] K)
```

Mathematical meaning. The Galois group is not cyclic (smallest non-cyclic Galois group over $\mathbb{Q}$).

Implementation. A generator would have order 4 (`card_aut_eq_finrank`), contradicting `aut_sq_eq_one`.

```
theorem aut_eq (σ : K ≃ₐ[ℚ] K) :
    σ = idAut ∨ σ = sigmaThree ∨ σ = sigmaFive ∨ σ = sigmaSeven
```

Mathematical meaning. The four displayed automorphisms exhaust the Galois group.

Implementation. `autToPow_spec` + `decide` on `(ZMod 8)ˣ.val ∈ {1,3,5,7}`; `powerBasis.algHom_ext` to pin maps by their value on ζ.

8.3 `Embeddings.lean`

```
theorem isTotallyComplex : NumberField.IsTotallyComplex K
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.IsTotallyComplex K`.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.Rat.isTotallyComplex`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem nrRealPlaces_eq_zero : NumberField.InfinitePlace.nrRealPlaces K =
    0
```

Mathematical meaning. The displayed Lean declaration has type `InfinitePlace.nrRealPlaces K = 0`.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.Rat.nrRealPlaces_eq_zero`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem nrComplexPlaces_eq_two :
    NumberField.InfinitePlace.nrComplexPlaces K = 2
```

Mathematical meaning. The displayed Lean declaration has type `InfinitePlace.nrComplexPlaces K = 2`.

Implementation. `card_add_two_mul_card_eq_rank` + `nrRealPlaces_eq_zero`, `degree_eq_four`, `omega`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_card_eq_two :
    Fintype.card (NumberField.InfinitePlace K) = 2
```

Mathematical meaning. The displayed Lean declaration has type `Fintype.card (InfinitePlace K) = 2`.

Implementation. `card_eq_nrRealPlaces_add_nrComplexPlaces` + the two place counts. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem infinitePlace_isComplex (v : NumberField.InfinitePlace K) :
    NumberField.InfinitePlace.IsComplex v
```

Mathematical meaning. The displayed Lean declaration has type `(v : InfinitePlace K) → IsComplex v`.

Implementation. The proof applies the packaged result `isTotallyComplex.isComplex`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem exists_complexEmbedding_zeta :
    ∃ σ : K →+* ℂ, σ ζ = Complex.exp (2 * Real.pi * Complex.I / 8)
```

Mathematical meaning. There is a complex embedding sending ζ to the standard primitive root.

Implementation. `Complex.isPrimitiveRoot_exp` gives ξ a root of X^4+1; lift via `AdjoinRoot.liftAlgHom ∘ adjoinRootEquivCyclotomic.symm`; `adjoinRootEquivCyclotomic_root`.

8.4 `RingOfIntegers.lean`

```
theorem zeta_integral : ζ ∈ integralClosure ℤ K
```

Mathematical meaning. The displayed Lean declaration has type `ζ ∈ integralClosure ℤ K`.

Implementation. The proof applies the packaged result `IsPrimitiveRoot.isIntegral`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
noncomputable def zetaInt : 𝒪 K := ⟨ζ, zeta_integral⟩
```

Mathematical meaning. ζ viewed in the ring of integers. Heavily used across the entire folder.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem zetaInt_pow_four : zetaInt ^ 4 = -1
```

Mathematical meaning. The displayed Lean declaration has type `zetaInt ^ 4 = -1`.

Implementation. `Subtype.ext` + `push_cast` + `zeta_pow_four`. The workhorse of all `linear_combination` proofs in PrimeSplitting/Units.

```
abbrev integralPowerBasis : PowerBasis ℤ (𝒪 K) :=
  zeta_isPrimitiveRoot.integralPowerBasis
```

Mathematical meaning. Mathlib's integral power basis `1, ζ, ζ^2, ζ^3`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem integralPowerBasis_gen : integralPowerBasis.gen =
    zeta_isPrimitiveRoot.toInteger
```

Mathematical meaning. The displayed Lean declaration has type `integralPowerBasis.gen = zeta_isPrimitiveRoot.toInteger`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem integralPowerBasis_dim : integralPowerBasis.dim = 4
```

Mathematical meaning. The displayed Lean declaration has type `integralPowerBasis.dim = 4`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
def integralBasisVectors : Fin 4 → K := ![1, ζ, ζ ^ 2, ζ ^ 3]
```

Mathematical meaning. The displayed Lean declaration has type `Fin 4 → K := ![1, ζ, ζ^2, ζ^3]`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem span_integralBasisVectors_eq :
    Submodule.span ℤ (Set.range integralBasisVectors) =
      Subalgebra.toSubmodule (integralClosure ℤ K)
```

Mathematical meaning. `1, ζ, ζ^2, ζ^3` spans the ring of integers as a $\mathbb{Z}$-lattice.

Implementation. `le_antisymm`; reverse inclusion via `isIntegralClosure_adjoin_singleton`, `adjoin_singleton_eq_range_aeval`, `modByMonic` reduction (degree <4), `aeval_eq_sum_range'`.

```
noncomputable def concreteIntegralBasis : Basis (Fin 4) ℤ (𝒪 K) :=
  integralPowerBasis.basis.reindex (finCongr integralPowerBasis_dim)
```

Mathematical meaning. `1, ζ, ζ^2, ζ^3` packaged as a genuine basis. Used in Discriminant + PrimeSplitting (finrank computations).

Implementation. The displayed RHS is the packaged construction `integralPowerBasis.basis.reindex (finCongr integralPowerBasis_dim)` specialized to this field.

```
theorem concreteIntegralBasis_apply (i : Fin 4) :
    (algebraMap (𝒪 K) K) (concreteIntegralBasis i) = integralBasisVectors
    i
```

Mathematical meaning. The displayed Lean declaration has type `algebraMap (𝒪 K) K (concreteIntegralBasis i) = integralBasisVectors i`.

Implementation. `fin_cases` + `simp`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_integralBasisVectors_eq_integralBasis :
    Submodule.span ℤ (Set.range integralBasisVectors) =
      Submodule.span ℤ (Set.range (NumberField.integralBasis K))
```

Mathematical meaning. The displayed basis spans the same lattice as mathlib's integral basis (equality of lattices, not a reindexing).

Implementation. `span_integralBasisVectors_eq` + `mem_span_integralBasis` both directions. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev concreteRingOfIntegers : Subalgebra ℤ K := Algebra.adjoin ℤ ({ζ} :
    Set K)
```

Mathematical meaning. The displayed Lean declaration has type `Subalgebra ℤ K := Algebra.adjoin ℤ {ζ}`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem isIntegralClosure_adjoin_zeta :
    IsIntegralClosure (Algebra.adjoin ℤ ({ζ} : Set K)) ℤ K
```

Mathematical meaning. The displayed Lean declaration has type `IsIntegralClosure (Algebra.adjoin ℤ {ζ}) ℤ K`.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.Rat.isIntegralClosure_adjoin_singleton`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem integralPowerBasis_adjoin_gen_eq_top :
    Algebra.adjoin ℤ ({integralPowerBasis.gen} : Set (𝒪 K)) = ⊤
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.adjoin ℤ {integralPowerBasis.gen} = ⊤`.

Implementation. The proof applies the packaged result `PowerBasis.adjoin_gen_eq_top`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem ringOfIntegers_eq : integralClosure ℤ K = concreteRingOfIntegers
```

Mathematical meaning. $\mathcal{O}$ `K` = $\mathbb{Z}[\zeta]$ (monogenicity).

Implementation. `le_antisymm` via `isIntegralClosure_adjoin_zeta` + `adjoin_le`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

8.5 Discriminant.lean

```
theorem numberField_discr_eq : NumberField.discr K = (256 : ℤ)
```

Mathematical meaning. `disc(ℚ(ζ₈)) = 256 = 2^8`. Used in ClassGroup (Minkowski bound), PrimeSplitting (different/ramification), LocalGlobal.

Implementation. rewrite {8} to {2^3}, `IsCyclotomicExtension.Rat.discr_prime_pow 2 3` + `totient/norm_num`.

```
theorem concreteIntegralBasis_discr :
    Algebra.discr ℤ concreteIntegralBasis = (256 : ℤ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℤ concreteIntegralBasis = 256`.

Implementation. `NumberField.discr_eq_discr` + `numberField_discr_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem integralBasis_discr :
    Algebra.discr ℚ (NumberField.integralBasis K) = (256 : ℚ)
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.discr ℚ (NumberField.integralBasis K) = (256 : ℚ)`.

Implementation. `NumberField.coe_discr` + `numberField_discr_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

8.6 Subfields.lean

```
theorem sqrt_two_sq : (ζ + ζ ^ 7) ^ 2 = 2
```

Mathematical meaning. $\sqrt{2}$ `= ζ + ζ⁻¹` lives in $\mathbb{Q}(\zeta_8)$ (generator of $\mathbb{Q}(\sqrt{2})$, the maximal real subfield).

Implementation. `linear_combination ( and related terms) * zeta_pow_four`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem i_sq : (ζ ^ 2) ^ 2 = -1
```

Mathematical meaning. `i = ζ²` (generator of $\mathbb{Q}$(i)).

Implementation. The proof applies the packaged result `linear_combination zeta_pow_four`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem sqrt_neg_two_sq : (ζ - ζ ^ 7) ^ 2 = -2
```

Mathematical meaning. $\sqrt{-2}$ `= ζ − ζ⁻¹` (generator of $\mathbb{Q}(\sqrt{-2})$); together with the above exhibits all three quadratic subfields $\mathbb{Q}(\sqrt{2})$, $\mathbb{Q}$(i), $\mathbb{Q}(\sqrt{-2})$.

Implementation. `linear_combination ( and related terms) * zeta_pow_four`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

8.7 PrimeSplitting.lean

```
private theorem gen_eq_zetaInt :
    zeta_isPrimitiveRoot.integralPowerBasis.gen = zetaInt
```

Mathematical meaning. The displayed Lean declaration has type `zeta_isPrimitiveRoot.integralPowerBasis.gen = zetaInt`.

Implementation. `integralPowerBasis_gen` + `rfl`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private noncomputable def sig3 : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    3 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))

private noncomputable def sig5 : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    5 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))

private noncomputable def sig7 : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
    7 (by
    norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))
```

Mathematical meaning. Private copies of `sigmaThree/Five/Seven`, needed locally for the norm-via-conjugates machinery (`prod_aut_apply`).

Implementation. `fromZetaAut` of `pow_of_coprime`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem sig3_zeta : sig3 ζ = ζ ^ 3

private theorem sig5_zeta : sig5 ζ = ζ ^ 5

private theorem sig7_zeta : sig7 ζ = ζ ^ 7
```

Mathematical meaning. The displayed Lean declaration has type `sigK ζ = ζ ^ k`.

Implementation. The proof applies the packaged result `fromZetaAut_spec`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
private theorem autEq' (σ : K ≃ₐ[ℚ] K) : σ = 1 ∨ σ = sig3 ∨ σ = sig5 ∨ σ
    = sig7
```

Mathematical meaning. Private duplicate of `Galois.aut_eq` (using `1` for identity).

Implementation. `autToPow_spec` + `decide` + `powerBasis.algHom_ext`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem prod_aut_apply (x : K) :
    ∏ σ : K ≃ₐ[ℚ] K, σ x = x * sig3 x * sig5 x * sig7 x
```

Mathematical meaning. The product over the four Galois conjugates.

Implementation. distinctness via `pow_inj`, `Finset.univ = {1,sig3,sig5,sig7}`, `prod_insert`/`prod_pair`/`ring`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem intNorm_eq (x : 𝒪 K) (n : ℤ)
    (h : algebraMap (𝒪 K) K x * sig3 (algebraMap (𝒪 K) K x) *
      sig5 (algebraMap (𝒪 K) K x) * sig7 (algebraMap (𝒪 K) K x) = (n :
    K)) :
    Algebra.norm ℤ x = n
```

Mathematical meaning. The displayed Lean declaration has type `(x : 𝒪 K) → (conjugate product = (n:K)) → Algebra.norm ℤ x = n`.

Implementation. `norm_eq_prod_automorphisms`, `prod_aut_apply`, `Algebra.coe_norm_int`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def normForm {R : Type*} [CommRing R] (a b c d : R) : R :=
  (a ^ 2 - c ^ 2 + 2 * b * d) ^ 2 + (2 * a * c - b ^ 2 + d ^ 2) ^ 2
```

Mathematical meaning. Explicit norm form of $a+b\zeta+c\zeta^2+d\zeta^3$. Used by Units.lean (unit equation).

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private def zetaCoords (a b c d : ℚ) : K :=
  (a : K) + (b : K) * ζ + (c : K) * ζ ^ 2 + (d : K) * ζ ^ 3
```

Mathematical meaning. The displayed Lean declaration has type `(a b c d : ℚ) → K := a + bζ + cζ^2 + dζ^3`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem sig3_zetaCoords (a b c d : ℚ) :
    sig3 (zetaCoords a b c d) =
      (a : K) + (d : K) * ζ - (c : K) * ζ ^ 2 + (b : K) * ζ ^ 3

private theorem sig5_zetaCoords (a b c d : ℚ) :
    sig5 (zetaCoords a b c d) = (a : K) - (b : K) * ζ + (c : K) * ζ ^ 2 -
      (d : K) * ζ ^ 3

private theorem sig7_zetaCoords (a b c d : ℚ) :
    sig7 (zetaCoords a b c d) =
      (a : K) - (d : K) * ζ - (c : K) * ζ ^ 2 - (b : K) * ζ ^ 3
```

Mathematical meaning. The displayed Lean declaration has type explicit images of `zetaCoords a b c d` under each automorphism.

Implementation. `map_*`, `sig*_zeta`, reduce powers ζ^5 and related termsζ^{21} via `linear_combination` and related variants `zeta_pow_four/eight`, `ring_nf`.

```
private theorem zetaCoords_mul_sig5 (a b c d : ℚ) :
    zetaCoords a b c d * ((a : K) - b * ζ + c * ζ ^ 2 - d * ζ ^ 3) =
      ((a : K) ^ 2 - c ^ 2 + 2 * b * d) + (2 * a * c - b ^ 2 + d ^ 2) * ζ
    ^ 2

private theorem sig3_zetaCoords_mul_sig7 (a b c d : ℚ) :
    ((a : K) + d * ζ - c * ζ ^ 2 + b * ζ ^ 3) *
      ((a : K) - d * ζ - c * ζ ^ 2 - b * ζ ^ 3) =
      ((a : K) ^ 2 - c ^ 2 + 2 * b * d) - (2 * a * c - b ^ 2 + d ^ 2) * ζ
    ^ 2
```

Mathematical meaning. The displayed Lean declaration has type product identities collapsing to $(a^2-c^2+2bd) \pm (2ac-b^2+d^2)\zeta^2$.

Implementation. `ring_nf` + `zeta_pow_four`, $\zeta^6 = -\zeta^2$. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem norm_eq_normForm (a b c d : ℚ) :
    Algebra.norm ℚ
      (a • (1 : K) + b • ζ + c • ζ ^ 2 + d • ζ ^ 3) = normForm a b c d
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.norm` $\mathbb{Q}$ $(a\bullet 1+b\bullet\zeta+c\bullet\zeta^2+d\bullet\zeta^3)$ `= normForm a b c d`.

Implementation. rewrite to `zetaCoords`, `norm_eq_prod_automorphisms`, the four conjugate formulas, then the two product identities.

```
theorem intNorm_eq_normForm (a b c d : ℤ) :
    Algebra.norm ℤ
      (a • (1 : 𝒪 K) + b • zetaInt + c • zetaInt ^ 2 + d • zetaInt ^ 3) =
        normForm a b c d
```

Mathematical meaning. The displayed Lean declaration has type `Algebra.norm ℤ (a•1+b•zetaInt+c•zetaInt²+d•zetaInt³) = normForm a b c d`.

Implementation. `Algebra.coe_norm_int` + `norm_eq_normForm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem isUnit_iff_normForm (a b c d : ℤ) :
    IsUnit (a • (1 : 𝒪 K) + b • zetaInt + c • zetaInt ^ 2 + d • zetaInt ^
    3) ↔
      (normForm a b c d).natAbs = 1
```

Mathematical meaning. Unit equation for $\mathbb{Z}[\zeta_8]$.

Implementation. `NumberField.isUnit_iff_norm`, `RingOfIntegers.coe_norm`, `intNorm_eq_normForm`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem cyclotomic_eight_int : Polynomial.cyclotomic 8 ℤ =
    Polynomial.X ^ 4 + 1
```

Mathematical meaning. The displayed Lean declaration has type `Polynomial.cyclotomic 8 ℤ = X ^ 4 + 1`.

Implementation. same 2→4→8 expand argument as `cyclotomic_eight_rat`, over $\mathbb{Z}$. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem minpoly_gen_eq :
    minpoly ℤ zeta_isPrimitiveRoot.integralPowerBasis.gen = Polynomial.X
    ^ 4 + 1
```

Mathematical meaning. The displayed Lean declaration has type `minpoly ℤ (integralPowerBasis.gen) = X ^ 4 + 1`.

Implementation. `isIntegrallyClosed_dvd` + degree/monic arguments, `eq_one_of_monic_natDegree_zero`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem isMaximal_span_pair (p : ℕ) [Fact p.Prime] (r : ℤ)
    (hr : ((r : ZMod p)) ^ 4 = -1) :
    (Ideal.span {(p : 𝒪 K), zetaInt - (r : 𝒪 K)}).IsMaximal
```

Mathematical meaning. The prime `(p, ζ−r)` is maximal whenever `r` is a fourth root of `−1 mod p`.

Implementation. ← `gen_eq_zetaInt` then `Common.isMaximal_span_gen_sub` (the generic Kummer-Dedekind ResidueHom from Common), with `minpoly_gen_eq`.

```
private theorem absNorm_span_pair (p : ℕ) (r : ℤ) (hr : ((r : ZMod p)) ^
    4 = -1) :
    Ideal.absNorm (Ideal.span {(p : 𝒪 K), zetaInt - (r : 𝒪 K)}) = p
```

Mathematical meaning. The displayed Lean declaration has type `Ideal.absNorm (Ideal.span {p, zetaInt − r}) = p`.

Implementation. ← `gen_eq_zetaInt` then `Common.absNorm_span_gen_sub`, with `minpoly_gen_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_two_eq : Ideal.span {(2 : 𝒪 K)} = Ideal.span {1 - zetaInt} ^
    4
```

Mathematical meaning. `(2) = (1−ζ)^4`, total ramification at 2 (e=4). Used pervasively (LiesOver, ramification, classNumber, different).

Implementation. `span_singleton_pow` + `span_singleton_eq_span_singleton` with explicit unit + `linear_combination` and related variants `zetaInt_pow_four`.

```
private theorem span_one_sub_zetaInt_eq_pair :
    Ideal.span {1 - zetaInt} = Ideal.span {((2 : ℕ) : 𝒪 K), zetaInt - ((1
    : ℤ) : 𝒪 K)}
```

Mathematical meaning. The displayed Lean declaration has type `Ideal.span {1 − zetaInt} = Ideal.span {(2:ℕ), zetaInt − 1}`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem isPrime_span_one_sub_zeta : (Ideal.span {1 - zetaInt}).IsPrime
```

Mathematical meaning. `(1−ζ)` is prime (unique prime over 2). Used in ClassGroup, LocalGlobal, different.

Implementation. `span_one_sub_zetaInt_eq_pair` + `(isMaximal_span_pair 2 1).isPrime`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_span_one_sub_zeta : Ideal.absNorm (Ideal.span {1 -
    zetaInt}) = 2
```

Mathematical meaning. The displayed Lean declaration has type `Ideal.absNorm (Ideal.span {1 − zetaInt}) = 2`.

Implementation. `span_one_sub_zetaInt_eq_pair` + `absNorm_span_pair 2 1`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem liesOver_span_one_sub_zeta :
    (Ideal.span {1 - zetaInt}).LiesOver (Ideal.span {(2 : ℤ)})
```

Mathematical meaning. The displayed Lean declaration has type `(Ideal.span {1 - zetaInt}).LiesOver (Ideal.span {(2:ℤ)})`.

Implementation. `liesOver_span_iff` + `span_two_eq` + `pow_le_self`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem inertiaDeg_span_one_sub_zeta :
    (Ideal.span {(2 : ℤ)}).inertiaDeg (Ideal.span {1 - zetaInt}) = 1
```

Mathematical meaning. The displayed Lean declaration has type `(Ideal.span {(2:ℤ)}).inertiaDeg (Ideal.span {1 - zetaInt}) = 1`.

Implementation. `absNorm_eq_pow_inertiaDeg'` + `absNorm_span_one_sub_zeta` + `Nat.Prime.pow_eq_iff`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem ramificationIdx_span_one_sub_zeta :
    (Ideal.span {(2 : ℤ)}).ramificationIdx (Ideal.span {1 - zetaInt}) = 4
```

Mathematical meaning. The displayed Lean declaration has type `(Ideal.span {(2:ℤ)}).ramificationIdx (Ideal.span {1 - zetaInt}) = 4`.

Implementation. `ramificationIdx_spec`: $\leq$ via `span_two_eq`; $>$ ruled out by `absNorm` ($32 \nmid 16$, using finrank=4 via `concreteIntegralBasis`).

```
private theorem span_pair_three_left :
    Ideal.span {(3 : 𝓞 K), zetaInt ^ 2 + zetaInt + 2} =
      Ideal.span {zetaInt ^ 2 + zetaInt - 1}

private theorem span_pair_three_right :
    Ideal.span {(3 : 𝓞 K), zetaInt ^ 2 + 2 * zetaInt + 2} =
      Ideal.span {zetaInt ^ 2 - zetaInt - 1}
```

Mathematical meaning. The displayed Lean declaration has type `Ideal.span {3, zetaInt`^{2}`+zetaInt+2} = Ideal.span {zetaInt`^{2}`+zetaInt−1}` (and right twin).

Implementation. `mem_span_singleton`, explicit witnesses, `linear_combination` and related variants `zetaInt_pow_four`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem span_three_eq :
    Ideal.span {(3 : 𝓞 K)} =
      Ideal.span {(3 : 𝓞 K), zetaInt ^ 2 + zetaInt + 2} *
        Ideal.span {(3 : 𝓞 K), zetaInt ^ 2 + 2 * zetaInt + 2}
```

Mathematical meaning. 3 splits into two degree-2 primes ($\Phi_8 \equiv$ `(X^2+X+2)(X^2+2X+2) mod 3`).

Implementation. rewrite to principal singletons + `span_singleton_mul` + explicit unit.

```
private theorem absNorm_ne_three (P : Ideal (𝒪 K)) : Ideal.absNorm P ≠ 3
```

Mathematical meaning. No prime with residue field $\mathbb{F}_3$ (X^4+1 has no root mod 3).

Implementation. from card 3 build `ZMod.ringEquivOfPrime`, transport `zetaInt^4 = −1`, `decide`.

```
private theorem isPrime_of_absNorm_nine {g : 𝒪 K}
    (h9 : Ideal.absNorm (Ideal.span {g}) = 9) : (Ideal.span {g}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `Ideal.absNorm (span {g}) = 9 → (span {g}).IsPrime`.

Implementation. take maximal `M ≥`; `absNorm M | 9`, exclude 1 and 3 (`absNorm_ne_three`), so 9; cofactor norm 1.

```
private theorem absNorm_span_g1 :
    Ideal.absNorm (Ideal.span {zetaInt ^ 2 + zetaInt - 1}) = 9

private theorem absNorm_span_g2 :
    Ideal.absNorm (Ideal.span {zetaInt ^ 2 - zetaInt - 1}) = 9
```

Mathematical meaning. The displayed Lean declaration has type `Ideal.absNorm (span {zetaInt^2±zetaInt−1}) = 9`.

Implementation. `intNorm_eq` with conjugate formulas (`sig3/5/7_zeta`), `linear_combination` and related variants `zeta_pow_four`.

```
theorem isPrime_primesAboveThree :
    (Ideal.span {(3 : 𝒪 K), zetaInt ^ 2 + zetaInt + 2}).IsPrime ∧
      (Ideal.span {(3 : 𝒪 K), zetaInt ^ 2 + 2 * zetaInt + 2}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type both primes above 3 are prime.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem absNorm_primeAboveThreeLeft :
    Ideal.absNorm (Ideal.span {(3 : 𝒪 K), zetaInt ^ 2 + zetaInt + 2}) = 9

theorem absNorm_primeAboveThreeRight :
    Ideal.absNorm (Ideal.span {(3 : 𝒪 K), zetaInt ^ 2 + 2 * zetaInt + 2})
    = 9
```

Mathematical meaning. The displayed Lean declaration has type each prime above 3 has norm 9.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private theorem seventeen_mem_span_two :
    (17 : 𝒪 K) ∈ Ideal.span {zetaInt - 2}

private theorem seventeen_mem_span_eight :
    (17 : 𝒪 K) ∈ Ideal.span {zetaInt ^ 3 - 2}

private theorem seventeen_mem_span_nine :
    (17 : 𝒪 K) ∈ Ideal.span {zetaInt ^ 7 - 2}

private theorem seventeen_mem_span_fifteen :
    (17 : 𝒪 K) ∈ Ideal.span {zetaInt ^ 5 - 2}
```

Mathematical meaning. The displayed Lean declaration has type `(17:𝒪 K) ∈ Ideal.span {zetaInt^k − 2}` for k=1,3,7,5.

Implementation. `mem_span_singleton` + explicit cofactor + big `linear_combination` and related variants `zetaInt_pow_four`.

```
private theorem span_pair_seventeen_two :
    Ideal.span {(17 : 𝒪 K), zetaInt - 2} = Ideal.span {zetaInt - 2}

private theorem span_pair_seventeen_eight :
    Ideal.span {(17 : 𝒪 K), zetaInt - 8} = Ideal.span {zetaInt ^ 3 - 2}

private theorem span_pair_seventeen_nine :
    Ideal.span {(17 : 𝒪 K), zetaInt - 9} = Ideal.span {zetaInt ^ 7 - 2}

private theorem span_pair_seventeen_fifteen :
    Ideal.span {(17 : 𝒪 K), zetaInt - 15} = Ideal.span {zetaInt ^ 5 - 2}
```

Mathematical meaning. The displayed Lean declaration has type `Ideal.span {17, zetaInt − r} = Ideal.span {zetaInt^k − 2}`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem span_seventeen_eq :
    Ideal.span {(17 : 𝒪 K)} =
      Ideal.span {(17 : 𝒪 K), zetaInt - 2} * Ideal.span {(17 : 𝒪 K),
    zetaInt - 8} *
```

```
        Ideal.span {(17 : 𝒪 K), zetaInt - 9} * Ideal.span {(17 : 𝒪 K),
    zetaInt - 15}
```

Mathematical meaning. 17 splits completely ($17 \equiv 1 \bmod 8$; roots of X^4+1 mod 17 are the order-8 elements).

Implementation. rewrite each pair to a principal singleton, multiply, explicit product = 17.

```
theorem isPrime_primeAboveSeventeen (r : ℤ) (hr : r = 2 ∨ r = 8 ∨ r = 9 ∨
    r = 15) :
    (Ideal.span {(17 : 𝒪 K), zetaInt - (r : 𝒪 K)}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `(r ∈ {2,8,9,15}) → (Ideal.span {17, zetaInt − r}).IsPrime`.

Implementation. `decide` on `(r:ZMod 17)^4 = −1 + (isMaximal_span_pair 17).isPrime`.

```
private theorem fintype_card_le_prime_sq_of_pow_prime_sq_fixed {F :
    Type*} [CommRing F]
    [IsDomain F] [Fintype F] {p : ℕ} (hp : p.Prime)
    (hfixed : ∀ x : F, x ^ p ^ 2 = x) :
    Fintype.card F ≤ p ^ 2
```

Mathematical meaning. Generic: a finite domain where `x^(p²)=x` has $\leq p^2$ elements.

Implementation. roots of `X^(p²) − X` ⊇ univ; `card_roots'`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem not_isPrime_span_two_natCast :
    ¬ (Ideal.span {((2 : ℕ) : 𝒪 K)}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `¬ (Ideal.span {((2:ℕ):𝒪 K)}).IsPrime`.

Implementation. `span_two_eq` + `mem_of_pow_mem` + `absNorm` contradiction. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem quotient_pow_prime_sq_eq_self_of_isPrime_span_natCast_ne_
    two
    (p : ℕ) (hp : p.Prime) (hp2 : p ≠ 2)
    (hpr : (Ideal.span {((p : ℕ) : 𝒪 K)}).IsPrime) :
    ∀ x : (𝒪 K) / Ideal.span {((p : ℕ) : 𝒪 K)}, x ^ p ^ 2 = x
```

Mathematical meaning. The displayed Lean declaration has type `(p ≠ 2 prime, span {p} prime) → ∀ x : 𝒪 K / span{p}, x^(p²) = x`.

Implementation. residue field has char p, `p² ≡ 1 (mod 8)`, frobenius² fixes ζ; `aeval_algHom_apply`.

```
private theorem not_isPrime_span_natCast_of_ne_two (p : ℕ) (hp : p.Prime)
    (hp2 : p ≠ 2)
    :
    ¬ (Ideal.span {((p : ℕ) : 𝒪 K)}).IsPrime
```

Mathematical meaning. The displayed Lean declaration has type `(p prime, p ≠ 2) → ¬ (Ideal.span {p}).IsPrime`.

Implementation. residue field would have p^4 elements but `x^(p^2)=x` bounds it by $p^2 < p^4$.

```
theorem no_inert_prime (p : ℕ) (hp : p.Prime) :
    ¬ (Ideal.span {(p : 𝒪 K)}).IsPrime
```

Mathematical meaning. No rational prime is inert (hallmark of non-cyclic abelian extensions: every residue degree $\leq$ exponent $2 < 4$).

Implementation. case split p=2/≠2 to the two private lemmas.

```
private theorem orderOf_mk0_eq_eight_of_pow_four_eq_neg_one {p : ℕ} [Fact
    p.Prime]
    (hp2 : p ≠ 2) {s : ZMod p} (hs0 : s ≠ 0) (hs4 : s ^ 4 = (-1 : ZMod
    p)) :
    orderOf (Units.mk0 s hs0) = 8
```

Mathematical meaning. The displayed Lean declaration has type `[Fact p.Prime] (p≠2) (s≠0) (s^4 = −1) → orderOf (Units.mk0 s) = 8`.

Implementation. order | 8, not | 4 (since $s^4 = -1 \neq 1$), so = 8.

```
private theorem unit_val_pow_four_eq_neg_one_of_orderOf_eq_eight {p : ℕ}
    [Fact p.Prime]
    {u : (ZMod p)ˣ} (hu : orderOf u = 8) : (u : ZMod p) ^ 4 = -1
```

Mathematical meaning. The displayed Lean declaration has type `[Fact p.Prime] (orderOf u = 8) → (u:ZMod p)^4 = −1`.

Implementation. u^4 squares to 1 but $\neq 1$, so $= -1$.

```
private theorem one_mod_eight_of_splits_completely (p : ℕ) (hp : p.Prime)
    (hp2 : p ≠ 2)
    :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
      Ideal.absNorm P = p) → p % 8 = 1
```

Mathematical meaning. The displayed Lean declaration has type `(p ≠ 2 prime) → (∃ P prime, ⊇span{p}, norm p) → p % 8 = 1`.

Implementation. residue field $\mathbb{F}$_p, `Units.mk0 s` has order 8, so `8 ∣ p−1`.

```
private theorem splits_completely_of_one_mod_eight (p : ℕ) (hp : p.Prime)
    :
    p % 8 = 1 →
      ∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
        Ideal.absNorm P = p
```

Mathematical meaning. The displayed Lean declaration has type `(p prime) → p % 8 = 1 → ∃ P prime ⊇ span{p}, norm p`.

Implementation. cyclic $(\mathbb{Z}/p)^\times$ generator `g`, `g^m` has order 8, build prime `(p, ζ−r)` via `isMaximal_span_pair`.

```
theorem splits_completely_iff (p : ℕ) (hp : p.Prime) (hp2 : p ≠ 2) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
    Ideal.absNorm P = p) ↔
      p % 8 = 1
```

Mathematical meaning. Splitting law: p splits completely iff `p ≡ 1 (mod 8)`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem ramified_iff_eq_two (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ^ 2) ↔ p =
    2
```

Mathematical meaning. Ramification law: only 2 ramifies (p | disc 256).

Implementation. different ideal route: `dvd_differentIdeal_iff`, `absNorm_differentIdeal`, `numberField_discr_eq`, plus `Common.not_isUnramifiedAt_of_map_span_natCast_le_sq_of_not_le_pow_succ` and `Common.not_map_span_natCast_le_pow_succ_of_absNorm_span_eq_pow`.

```
abbrev zetaEightPolynomialMod (p : ℕ) : (ZMod p)[X] :=
  X ^ 4 + C (1 : ZMod p)
```

Mathematical meaning. The displayed Lean declaration has type `(p : ℕ) → (ZMod p)[X] := X^4 + C 1`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem zetaEightPolynomialMod_eq_cyclotomic (p : ℕ) :
    zetaEightPolynomialMod p = Polynomial.cyclotomic 8 (ZMod p)
```

Mathematical meaning. The displayed Lean declaration has type `zetaEightPolynomialMod p = cyclotomic 8 (ZMod p)`.

Implementation. map `cyclotomic_eight_int` via `Int.castRingHom`, `map_cyclotomic_int`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem absNorm_differentIdeal_eq_twoHundredFiftySix :
    Ideal.absNorm (differentIdeal ℤ (𝒪 K)) = 256
```

Mathematical meaning. The displayed Lean declaration has type `Ideal.absNorm (differentIdeal ℤ (𝒪 K)) = 256`.

Implementation. `absNorm_differentIdeal` + `numberField_discr_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem prime_dvd_differentIdeal_iff_eq_span_one_sub_zeta
    (P : Ideal (𝒪 K)) (_hP : P.IsPrime) (_hP0 : P ≠ ⊥) :
    P ∣ differentIdeal ℤ (𝒪 K) ↔ P = Ideal.span {1 - zetaInt}
```

Mathematical meaning. The different is supported only at $(1-\zeta)$.

Implementation. forward via `absNorm` dvd 2^8 + `mem_of_pow_mem` + maximality; reverse via `pow_sub_one_dvd_differentIdeal`.

8.8 `ClassGroup.lean`

```
abbrev zetaFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  FractionalIdeal.spanSingleton (𝒪 K)⁰ ζ
```

Mathematical meaning. The displayed Lean declaration has type `FractionalIdeal (𝒪 K)⁰ K := spanSingleton _ ζ`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev ramifiedPrimeFractionalIdeal : FractionalIdeal (𝒪 K)⁰ K :=
  (Ideal.span {1 - zetaInt} : FractionalIdeal (𝒪 K)⁰ K)
```

Mathematical meaning. The displayed Lean declaration has type `FractionalIdeal (𝒪 K)⁰ K := (Ideal.span {1 - zetaInt} : and related variants)`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem zeta_mem_fractionalIdeal :
    ζ ∈ zetaFractionalIdeal
```

Mathematical meaning. The displayed Lean declaration has type `ζ ∈ zetaFractionalIdeal`.

Implementation. The proof applies the packaged result `mem_spanSingleton_self`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem zetaFractionalIdeal_absNorm :
    FractionalIdeal.absNorm zetaFractionalIdeal = |Algebra.norm ℚ ζ|
```

Mathematical meaning. The displayed Lean declaration has type `FractionalIdeal.absNorm zetaFractionalIdeal = |Algebra.norm ℚ ζ|`.

Implementation. The proof applies the packaged result `absNorm_span_singleton`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem ramifiedPrimeFractionalIdeal_ne_zero :
    ramifiedPrimeFractionalIdeal ≠ 0
```

Mathematical meaning. The displayed Lean declaration has type `ramifiedPrimeFractionalIdeal ≠ 0`.

Implementation. `coeIdeal_ne_zero` + `span_singleton_eq_bot` $\Rightarrow \zeta = 1 \Rightarrow$ order 1 vs 8 contradiction.

```
private theorem gen_eq_zetaInt :
    zeta_isPrimitiveRoot.integralPowerBasis.gen = zetaInt
```

Mathematical meaning. The displayed Lean declaration has type `zeta_isPrimitiveRoot.integralPowerBasis.gen = zetaInt`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private theorem minpoly_gen_eq :
    minpoly ℤ zeta_isPrimitiveRoot.integralPowerBasis.gen = Polynomial.X
    ^ 4 + 1
```

Mathematical meaning. The displayed Lean declaration has type `minpoly ℤ (integralPowerBasis.gen) = X ^ 4 + 1`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private noncomputable def residueHom (p : ℕ) (r : ℤ) (hr : ((r : ZMod p))
    ^ 4 = -1) :
    𝒪 K →ₐ[ℤ] ZMod p :=
  zeta_isPrimitiveRoot.integralPowerBasis.lift ((r : ℤ) : ZMod p)
    (by rw [minpoly_gen_eq]; simp [hr])
```

Mathematical meaning. Reduction $\zeta \mapsto$ `r` mod `p`.

Implementation. `integralPowerBasis.lift` with `minpoly_gen_eq`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem residueHom_zetaInt (p : ℕ) (r : ℤ) (hr : ((r : ZMod p)) ^
    4 = -1) :
    residueHom p r hr zetaInt = (r : ZMod p)
```

Mathematical meaning. The displayed Lean declaration has type `residueHom p r hr zetaInt = (r : ZMod p)`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private theorem ker_residueHom (p : ℕ) (r : ℤ) (hr : ((r : ZMod p)) ^ 4 =
    -1) :
    RingHom.ker (residueHom p r hr).toRingHom =
      Ideal.span {(p : 𝒪 K), zetaInt - (r : 𝒪 K)}
```

Mathematical meaning. The displayed Lean declaration has type `RingHom.ker (residueHom p r hr).toRingHom = Ideal.span {p, zetaInt − r}`.

Implementation. division `q(ζ)−q(r) ∈ (ζ−r)` via `sub_dvd_eval_sub`; `intCast_zmod_eq_zero_iff_dvd`; both inclusions.

```
private theorem residueHom_surjective (p : ℕ) (r : ℤ) (hr : ((r : ZMod
    p)) ^ 4 = -1) :
    Function.Surjective (residueHom p r hr)
```

Mathematical meaning. The displayed Lean declaration has type `Function.Surjective (residueHom p r hr)`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private theorem isMaximal_span_pair (p : ℕ) [Fact p.Prime] (r : ℤ)
    (hr : ((r : ZMod p)) ^ 4 = -1) :
    (Ideal.span {(p : 𝒪 K), zetaInt - (r : 𝒪 K)}).IsMaximal
```

Mathematical meaning. The displayed Lean declaration has type `[Fact p.Prime] → (Ideal.span {p, zetaInt − r}).IsMaximal`.

Implementation. ← `ker_residueHom` + `ker_isMaximal_of_surjective`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem span_one_sub_zetaInt_eq_pair :
    Ideal.span {1 - zetaInt} = Ideal.span {((2 : ℕ) : 𝒪 K), zetaInt - ((1
    : ℤ) : 𝒪 K)}
```

Mathematical meaning. The displayed Lean declaration has type `Ideal.span {1 − zetaInt} = Ideal.span {(2:ℕ), zetaInt − 1}`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
private theorem eq_span_one_sub_of_two_mem (P : Ideal (𝒪 K)) (hP :
    P.IsPrime)
    (h2 : (2 : 𝒪 K) ∈ P) : P = Ideal.span {1 - zetaInt}
```

Mathematical meaning. The displayed Lean declaration has type `(P prime, 2 ∈ P) → P = Ideal.span {1 − zetaInt}`.

Implementation. $(1-\zeta)^4 \in$ `P`, `mem_of_pow_mem`, maximality (local `isMaximal_span_pair`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

Mathematical meaning. Class number 1.

Implementation. `classNumber_eq_one_iff` + `isPrincipalIdealRing_of_isPrincipal_of_pow_le_of_mem_primesOver_of_mem_Icc`; Minkowski bound $(4/\pi)^2 \cdot 4!/4^4 \cdot \sqrt{256} \approx 2.4 < 3$ via `Nat.floor_lt`, `pi_gt_d2`, `nlinarith`; only p=2 remains, principal $(1-\zeta)$.

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝒪 K)
```

Mathematical meaning. The displayed Lean declaration has type `IsPrincipalIdealRing (𝒪 K)`.

Implementation. ← `classNumber_eq_one_iff` + `classNumber_eq_one`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝒪 K)
```

Mathematical meaning. The displayed Lean declaration has type `UniqueFactorizationMonoid (𝒪 K)`.

Implementation. PIR ⇒ UFM (`infer_instance`). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem classGroupStructure : Nonempty (ClassGroup (𝒪 K) ≃*
    Multiplicative (ZMod 1))
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty (ClassGroup (𝒪 K) ≃* Multiplicative (ZMod 1))`.

Implementation. card 1 ⇒ `Unique` ⇒ `MulEquiv.ofUnique`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev unitIdealRepresentative : (Ideal (𝒪 K))⁰ := ⟨(⊤ : Ideal (𝒪 K)), by
    simp⟩
```

Mathematical meaning. The displayed Lean declaration has type `(Ideal (𝒪 K))⁰ := ⟨⊤, _⟩`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem classGroupRepresentative (C : ClassGroup (𝒪 K)) :
    C = ClassGroup.mk0 unitIdealRepresentative
```

Mathematical meaning. The displayed Lean declaration has type `(C : ClassGroup (𝒪 K)) → C = ClassGroup.mk0 unitIdealRepresentative`.

Implementation. The proof applies the packaged result `Subsingleton.elim`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

8.9 **Units.lean**

```
noncomputable def torsionGenerator : (𝒪 K)ˣ where
  val := zetaInt
  inv := zetaInt ^ 7
  val_inv := by linear_combination (zetaInt ^ 4 - 1) * zetaInt_pow_four
  inv_val := by linear_combination (zetaInt ^ 4 - 1) * zetaInt_pow_four
```

Mathematical meaning. A torsion generator (ζ). Used in unit-group structure.

Implementation. `linear_combination (zetaInt`$^4-1$`) * zetaInt_pow_four` for val_inv/inv_val. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
def torsionUnits : Set ((𝒪 K)ˣ) := {u | ∃ k : Fin 8, u = torsionGenerator
    ^ k.val}
```

Mathematical meaning. The displayed Lean declaration has type `Set (𝒪 K)ˣ := {u | ∃ k : Fin 8, u = torsionGenerator ^ k.val}`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem torsionOrder_eq' : NumberField.Units.torsionOrder K = 8
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.Units.torsionOrder K = 8`.

Implementation. The proof applies the packaged result `IsCyclotomicExtension.Rat.torsionOrder_eq`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem torsionUnits_eq : {u : (𝓞 K)ˣ | IsOfFinOrder u} = torsionUnits
```

Mathematical meaning. Torsion subgroup = $\mu_8 =$ `{ζ^k}` (note $-1 = \zeta^4$ already a power of ζ, unlike $\mathbb{Q}(\zeta_5)$).

Implementation. `rootsOfUnity_eq_torsion`, `torsionOrder_eq'`, `eq_pow_of_pow_eq_one`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem torsionOrder_eq_eight : NumberField.Units.torsionOrder K = 8
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.Units.torsionOrder K = 8`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem unitRank_eq_one : NumberField.Units.rank K = 1
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.Units.rank K = 1`.

Implementation. signature `(0,2)` $\Rightarrow$ rank $= r_1+r_2-1 = 1$ (`card_eq_nrRealPlaces_add_nrComplexPlaces`).

```
noncomputable def fundamentalUnit : (𝓞 K)ˣ where
  val := 1 + zetaInt + zetaInt ^ 7
  inv := zetaInt + zetaInt ^ 7 - 1
  val_inv := by
    linear_combination (zetaInt ^ 10 - zetaInt ^ 6 + 2 * zetaInt ^ 4 +
    zetaInt ^ 2 - 2)
    *
      zetaInt_pow_four
  inv_val := by
    linear_combination (zetaInt ^ 10 - zetaInt ^ 6 + 2 * zetaInt ^ 4 +
    zetaInt ^ 2 - 2)
    *
      zetaInt_pow_four
```

Mathematical meaning. `1 + √2`, the fundamental unit (the fundamental unit of the real subfield $\mathbb{Q}(\sqrt{2})$). Used in LocalGlobal.

Implementation. `linear_combination ( and related terms) * zetaInt_pow_four`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
noncomputable def fundamentalUnitSystem : Fin (NumberField.Units.rank K)
    → (𝒪 K)ˣ :=
  fun _ => fundamentalUnit
```

Mathematical meaning. The displayed Lean declaration has type `Fin (rank K) → (𝒪 K)ˣ := fun _ => fundamentalUnit`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem exists_eq_torsion_mul_fundSystem_zpow (i₀ : Fin
    (NumberField.Units.rank
    K))
    (u : (𝒪 K)ˣ) :
    ∃ ζ' ∈ NumberField.Units.torsion K, ∃ n : ℤ,
      u = ζ' * NumberField.Units.fundSystem K i₀ ^ n
```

Mathematical meaning. The displayed Lean declaration has type `(i₀)(u) → ∃ ζ' ∈ torsion, ∃ n, u = ζ' * fundSystem K i₀ ^ n`.

Implementation. rank 1 ⇒ subsingleton index; `closure_fundSystem_sup_torsion_eq_top`, `mem_sup`, `mem_closure_singleton`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem fundSystem_zpow_mem_torsion (i₀ : Fin
    (NumberField.Units.rank K)) (n :
    ℤ)
    (hn : NumberField.Units.fundSystem K i₀ ^ n ∈
    NumberField.Units.torsion K) : n = 0
```

Mathematical meaning. The displayed Lean declaration has type `fundSystem K i₀ ^ n ∈ torsion → n = 0`.

Implementation. image in `basisModTorsion` repr is a basis vector. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem eq_torsionGenerator_pow_of_mem_torsion {ζ' : (𝒪 K)ˣ}
    (hζ' : ζ' ∈ NumberField.Units.torsion K) : ∃ k : Fin 8, ζ' =
    torsionGenerator ^
    k.val
```

Mathematical meaning. The displayed Lean declaration has type `ζ' ∈ torsion → ∃ k : Fin 8, ζ' = torsionGenerator ^ k.val`.

Implementation. The proof applies the packaged result `torsionUnits_eq`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

```
theorem fundamentalUnitSystem_isFundamental :
    Subgroup.closure (Set.range fundamentalUnitSystem) ⊔
    NumberField.Units.torsion K = ⊤
```

Mathematical meaning. $\{1+\sqrt{2}\}$ together with μ_8 generates all units.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem unit_eq_torsion_mul_fundamentalUnit_zpow (u : (𝒪 K)ˣ) :
    ∃ ζ' : (𝒪 K)ˣ, ζ' ∈ NumberField.Units.torsion K ∧ ∃ n : ℤ, u = ζ' *
    fundamentalUnit
    ^ n
```

Mathematical meaning. every unit is ζ`^j (1+√2)`^{n}.

Implementation. `fundamentalUnitSystem_isFundamental` + `mem_sup`/`mem_closure_singleton`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem unitGroupStructure_bijective
    (i₀ : Fin (NumberField.Units.rank K)) (η : (𝒪 K)ˣ)
    (hη : η = NumberField.Units.fundSystem K i₀) (htg8 : torsionGenerator
    ^ 8 = 1)
    (htg_tor : torsionGenerator ∈ NumberField.Units.torsion K) :
    Function.Bijective
      ((torsionPowerHom 8 torsionGenerator htg8).coprod (rankOnePowerHom
    η))
```

Mathematical meaning. The displayed Lean declaration has type `(i₀)(η = fundSystem K i₀)(htg8)(htg_tor) → Bijective ((torsionPowerHom 8 torsionGenerator htg8).coprod (rankOnePowerHom η))`.

Implementation. injectivity via `fundSystem_zpow_mem_torsion` + `pow_eq_one_iff_dvd`; surjectivity via `exists_eq_torsion_mul_fundSystem_zpow` + `eq_torsionGenerator_pow_of_mem_torsion`.

```
theorem unitGroupStructure :
    Nonempty ((𝒪 K)ˣ ≃* (Multiplicative (ZMod 8) × Multiplicative (Fin 1
    → ℤ)))
```

Mathematical meaning. Unit group $\cong \mu_8 \times \mathbb{Z}$. (Crucially proved independently of the sorry via `fundSystem`/`closure_fundSystem_sup_torsion_eq_top`, NOT via `fundamentalUnit`.)

Implementation. `MulEquiv.ofBijective` of the coprod hom via `unitGroupStructure_bijective`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem regulator_eq :
    NumberField.Units.regulator K = 2 * Real.log (1 + Real.sqrt 2)
```

Mathematical meaning. Regulator $= 2\cdot\log(1+\sqrt{2}) \approx 1.7627$ (complex place weight 2, $|1+\zeta+\zeta^7| = 1+\sqrt{2}$).

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

8.10 `Euclidean.lean`

```
theorem normEuclidean (a b : 𝒪 K) (hb : b ≠ 0) :
    ∃ q r : 𝒪 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

Mathematical meaning. $\mathbb{Q}(\zeta_8)$ is norm-Euclidean.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

8.11 `CMField.lean`

```
theorem isCMField : NumberField.IsCMField K
```

Mathematical meaning. $\mathbb{Q}(\zeta_8)$ is a CM field.

Implementation. `IsCyclotomicExtension.Rat.isCMField` with witness ⟨`8, and related terms`⟩. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cmField_isTotallyComplex : NumberField.IsTotallyComplex K
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.IsTotallyComplex K`.

Implementation. re-export of `isTotallyComplex`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cmField_quadratic_over_maximalRealSubfield :
    Algebra.IsQuadraticExtension (NumberField.maximalRealSubfield K) K
```

Mathematical meaning. `K` is quadratic over its maximal real subfield $\mathbb{Q}(\sqrt{2})$ (defining property of a CM field).

Implementation. from `isCMField` instance, `infer_instance`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem cmField_unitsRank_eq_maximalRealSubfield :
    NumberField.Units.rank (NumberField.maximalRealSubfield K) =
      NumberField.Units.rank K
```

Mathematical meaning. unit rank equal to that of the real subfield (the Dirichlet/CM unit-index statement; both are 1).

Implementation. The proof applies the packaged result `IsCMField.units_rank_eq_units_rank`; the preceding declarations provide the hypotheses needed to use it in this concrete field.

8.12 `LocalGlobal.lean`

```
def heightOneTwo : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {1 - zetaInt}
  isPrime := by
    exact isPrime_span_one_sub_zeta
  ne_bot := by
    intro h
    have hnorm := absNorm_span_one_sub_zeta
    rw [h] at hnorm
    norm_num at hnorm
```

Mathematical meaning. The finite place over 2 (totally ramified prime $(1-\zeta)$).

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
def heightOneThreeLeft : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {(3 : 𝒪 K), zetaInt ^ 2 + zetaInt + 2}
  isPrime := by
    exact isPrime_primesAboveThree.1
  ne_bot := by
    intro h
    have h3mem : (3 : 𝒪 K) ∈ Ideal.span {(3 : 𝒪 K), zetaInt ^ 2 + zetaInt
    + 2} :=
      Ideal.subset_span (by simp)
    rw [h] at h3mem
    norm_num at h3mem

def heightOneThreeRight : HeightOneSpectrum (𝒪 K) where
  asIdeal := Ideal.span {(3 : 𝒪 K), zetaInt ^ 2 + 2 * zetaInt + 2}
  isPrime := by
    exact isPrime_primesAboveThree.2
  ne_bot := by
```

```
    intro h
    have h3mem : (3 : 𝒪 K) ∈ Ideal.span {(3 : 𝒪 K), zetaInt ^ 2 + 2 *
    zetaInt + 2} :=
      Ideal.subset_span (by simp)
    rw [h] at h3mem
    norm_num at h3mem
```

Mathematical meaning. The displayed Lean declaration has type `HeightOneSpectrum (𝒪 K)` for the two primes over 3.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev vTwo : NumberField.FinitePlace K :=
  NumberField.FinitePlace.mk heightOneTwo
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.FinitePlace K := FinitePlace.mk heightOneTwo`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
example :
    (Ideal.span {(2 : ℤ)}).ramificationIdx heightOneTwo.asIdeal = 4

example :
    (Ideal.span {(2 : ℤ)}).inertiaDeg heightOneTwo.asIdeal = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. This anonymous example is a smoke test: it instantiates the relevant API in this concrete field and checks the reductions.

```
theorem vThreeLeft_inertiaDeg :
    (Ideal.span {(3 : ℤ)}).inertiaDeg heightOneThreeLeft.asIdeal = 2

theorem vThreeRight_inertiaDeg :
    (Ideal.span {(3 : ℤ)}).inertiaDeg heightOneThreeRight.asIdeal = 2
```

Mathematical meaning. The displayed Lean declaration has type `(span {(3:ℤ)}).inertiaDeg heightOneThree*.asIdeal = 2`.

Implementation. `liesOver_span_iff` + `absNorm_eq_pow_inertiaDeg'` + `absNorm_primeAboveThree*` + `Nat.pow_right_injective`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev CompletionAtTwo : Type :=
  heightOneTwo.adicCompletion K
```

Mathematical meaning. The displayed Lean declaration has type `heightOneTwo.adicCompletion K`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
abbrev InfiniteCompletion (v : NumberField.InfinitePlace K) : Type :=
  v.Completion
```

Mathematical meaning. The displayed Lean declaration has type `(v : InfinitePlace K) → Type := v.Completion`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
private theorem complexEmbedding_norm_zeta_eq_one (φ : K →+* ℂ) : ‖φ ζ‖ =
    1
```

Mathematical meaning. The displayed Lean declaration has type `(φ : K →+* ℂ) → ‖φ ζ‖ = 1`.

Implementation. `Complex.norm_eq_one_of_pow_eq_one` ($\varphi\zeta^8 = 1$). After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
example (v : NumberField.InfinitePlace K) :
    Nonempty (InfiniteCompletion v ≃+* ℂ)
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. This anonymous example is a smoke test: it instantiates the relevant API in this concrete field and checks the reductions.

```
theorem infiniteCompletion_zeta_norm (v : NumberField.InfinitePlace K) :
    ‖(algebraMap K (InfiniteCompletion v) ζ)‖ = 1
```

Mathematical meaning. The displayed Lean declaration has type `‖algebraMap K (InfiniteCompletion v) ζ‖ = 1`.

Implementation. `Completion.norm_coe`, `norm_embedding_eq`, `complexEmbedding_norm_zeta_eq_one`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
private theorem zeta_ne_zero : ζ ≠ 0
```

Mathematical meaning. The displayed Lean declaration has type $\zeta \neq 0$.

Implementation. $\zeta^8 = 1$ contradicts 0.

```
example :
    ((∏ w : NumberField.InfinitePlace K, w ζ ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K, v ζ) = 1

example :
    ((∏ w : NumberField.InfinitePlace K,
          w (algebraMap (𝓞 K) K (fundamentalUnit : 𝓞 K)) ^ w.mult) *
        ∏ᶠ v : NumberField.FinitePlace K,
          v (algebraMap (𝓞 K) K (fundamentalUnit : 𝓞 K))) = 1
```

Mathematical meaning. This records the mathematical assertion or concrete object displayed in the Lean declaration above.

Implementation. This anonymous example is a smoke test: it instantiates the relevant API in this concrete field and checks the reductions.

```
abbrev CyclotomicEightInfiniteAdeleRing : Type :=
  NumberField.InfiniteAdeleRing K

abbrev infinitePrincipalAdele_zeta : CyclotomicEightInfiniteAdeleRing :=
  algebraMap K CyclotomicEightInfiniteAdeleRing ζ

abbrev infinitePrincipalAdele_fundamentalUnit :
    CyclotomicEightInfiniteAdeleRing :=
  algebraMap K CyclotomicEightInfiniteAdeleRing (algebraMap (𝓞 K) K
    (fundamentalUnit :
    𝓞 K))
```

Mathematical meaning. The displayed Lean declaration has type infinite adele ring + two principal adeles.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem canonicalEmbedding_zetaInt_mem_integerLattice :
    NumberField.canonicalEmbedding K (algebraMap (𝓞 K) K zetaInt) ∈
      NumberField.canonicalEmbedding.integerLattice K
```

Mathematical meaning. The displayed Lean declaration has type `canonicalEmbedding K (algebraMap (𝓞 K) K zetaInt) ∈ integerLattice K`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
abbrev mixedEmbedding_zeta : NumberField.mixedEmbedding.mixedSpace K :=
  NumberField.mixedEmbedding K ζ
```

Mathematical meaning. The displayed Lean declaration has type `mixedEmbedding.mixedSpace K := mixedEmbedding K ζ`.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem polarSpace_nonempty :
    Nonempty (NumberField.mixedEmbedding.polarSpace K)
```

Mathematical meaning. The displayed Lean declaration has type `Nonempty (mixedEmbedding.polarSpace K)`.

Implementation. The proof reduces the displayed assertion to previously established field-specific lemmas and the relevant Mathlib API.

```
theorem house_zeta :
    NumberField.house ζ = 1
```

Mathematical meaning. The displayed Lean declaration has type `NumberField.house ζ = 1`.

Implementation. `le_antisymm` via `canonicalEmbedding.norm_le_iff` + `complexEmbedding_norm_zeta_eq_one`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
abbrev CyclotomicEightAdeleRing : Type :=
  NumberField.AdeleRing (𝒪 K) K

abbrev principalAdele_zeta : CyclotomicEightAdeleRing :=
  algebraMap K CyclotomicEightAdeleRing ζ

abbrev principalAdele_fundamentalUnit : CyclotomicEightAdeleRing :=
  algebraMap K CyclotomicEightAdeleRing (algebraMap (𝒪 K) K
    (fundamentalUnit : 𝒪 K))
```

Mathematical meaning. The displayed Lean declaration has type adele ring + two principal adeles.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

```
theorem principalAdele_zeta_ne_zero : principalAdele_zeta ≠ 0
```

Mathematical meaning. The displayed Lean declaration has type `principalAdele_zeta ≠ 0`.

Implementation. `AdeleRing.algebraMap_injective` + `zeta_ne_zero`. After unfolding the field-specific definitions, these named lemmas supply the essential argument.

```
theorem principalAdele_fundamentalUnit_is_unit_everywhere :
    ∀ v : NumberField.FinitePlace K,
      v (algebraMap (𝒪 K) K (fundamentalUnit : 𝒪 K)) = 1
```

Mathematical meaning. The displayed Lean declaration has type ∀ v : FinitePlace K, v (algebraMap (𝒪 K) K fundamentalUnit) = 1.

Implementation. norm_eq_one_iff_notMem; a unit can't lie in a maximal ideal.

```
abbrev cyclotomicEightDedekindZeta (s : ℂ) : ℂ :=
  NumberField.dedekindZeta K s

abbrev cyclotomicEightDedekindZetaResidue : ℝ :=
  NumberField.dedekindZeta_residue K
```

Mathematical meaning. The displayed Lean declaration has type Dedekind zeta function + residue.

Implementation. Defined directly by the displayed RHS; the point is to name this concrete object so later theorems can unfold or transport it.

8.13 Summary for $\mathbb{Q}(\zeta_8)$

Field: $\mathbb{Q}(\zeta_8)$, the eighth cyclotomic field, modelled as K := CyclotomicField 8 ℚ via IsCyclotomicExtension {8} ℚ K. Degree $\varphi(8) = 4$, signature (0,2) (totally complex, CM), ring of integers $\mathbb{Z}[\zeta_8]$ (monogenic), discriminant $256 = 2^8$, class number 1. The Galois group is $(\mathbb{Z}/8)^\times$, the non-cyclic Klein four-group $\mathbb{Z}/2 \times \mathbb{Z}/2$ (smallest non-cyclic Galois group over $\mathbb{Q}$); every automorphism is an involution. CM field with three quadratic subfields $\mathbb{Q}(\sqrt{2})$, $\mathbb{Q}(i)$, $\mathbb{Q}(\sqrt{-2})$. Unit group $\mu_8 \times \mathbb{Z}$ (rank 1), fundamental unit $1 + \sqrt{2}$. This is the twin of CyclotomicFive ($\mathbb{Q}(\zeta_5)$, cyclic Galois group); the route map (fromZetaAut automorphisms, residueHom Kummer-Dedekind, torsionPowerHom/rankOnePowerHom unit structure) is shared. 2 sorries (down from 3): fundamentalUnitSystem_isFundamental (Units.lean – $1+\sqrt{2}$ is fundamental, the cyclotomic-unit index $h^+=1$ obstruction) and normEuclidean (Euclidean.lean, geometry-of-numbers covering argument). The former regulator_eq sorry is now closed: its body is complete via mathlib's regOfFamily_eq_det (as in SqrtTwo/SqrtFive/CyclotomicFive), so it depends only transitively on isFundamental. The regulator value was never the API gap – the genuine obstruction is certifying $1+\sqrt{2}$ as the fundamental unit (cyclotomic-unit / real-subfield theory).

All declarations live in namespace NumberField.CyclotomicEight unless noted. Shared helpers referenced from XYin/NumberField/FieldExamples/Common.lean (namespace NumberField.FieldExamples).

9 Final summary: current Mathlib **NumberField** API

The clean conclusion of the updated reports is that Mathlib's arithmetic NumberField API is already strong for the classical invariant package. For cyclotomic fields, the API is particularly natural:

`IsCyclotomicExtension` gives the field model, primitive root, finrank, integral closure, discriminant formulas, Galois automorphisms, total-complexity, and place counts in the form one would mathematically expect. For quadratic and small radical fields, the same invariants are still available, but the formal proof naturally contains more presentation work: irreducibility, adjoin-root equivalences, Eisenstein arguments, explicit basis comparisons, and coordinate-level norm computations.

The API is also mature for prime arithmetic and class-number computations. Ideals, absolute norms, inertia degree, ramification index, the different ideal, `ClassGroup`, `NumberField.classNumber`, `IsPrincipalIdealRing`, and `UniqueFactorizationMonoid` all support mathematically meaningful statements. The $\mathbb{Q}(\sqrt{-6})$ folder is a good stress test: it proves class number two, non-PID, non-UFD, and explicit irreducible/non-prime phenomena using the standard predicates rather than inventing replacement notions.

For units, the important correction is that the regulator bridge is not the missing API. Once a chosen unit system is known to be fundamental, `regOfFamily_div_regulator`, `regOfFamily_eq_det`, and the rank-one or rank-zero determinant lemmas compute the regulator naturally. This is why $\mathbb{Q}(\sqrt{2})$ and $\mathbb{Q}(\sqrt{5})$ are now sorry-free: the hard work was certifying the explicit fundamental unit, and after that the regulator value follows by existing Mathlib machinery. The remaining fundamental-unit sorries for $\mathbb{Q}(\sqrt[3]{3})$, $\mathbb{Q}(\zeta_5)$, and $\mathbb{Q}(\zeta_8)$ should not be read as a regulator API failure.

There is nevertheless a layer of results that is formalizable but stylistically unnatural for a mathematician who does not already think in Lean. Finite group structure is often stated as `Nonempty (G ≃* Multiplicative (ZMod n))` instead of "cyclic of order n" or "Klein four". Congruence splitting laws are rebuilt from order computations and residue maps rather than obtained from a packaged Kummer–Dedekind/congruence-law interface. Cyclotomic quadratic subfields are exhibited through generator-square relations and explicit `AdjoinRoot.liftAlgHom` maps rather than as convenient `IntermediateField` objects connected to the Galois correspondence. These are API-shape issues: the mathematics is present, but the proof script does not read like the usual mathematical proof.

The genuinely open cases split into two families. First, certifying a displayed fundamental unit is hard mathematics. In the real quadratic cases this is now solved, by Pell/Zsqrtd for $\mathbb{Q}(\sqrt{2})$ and by a real-embedding window reduction for $\mathbb{Q}(\sqrt{5})$. For the pure cubic field the same kind of real argument remains to be supplied. For the totally complex cyclotomic cases, however, a reusable cyclotomic-units library would be the natural API: a `cyclotomicUnit` theory and the index theorem $[\mathcal{O}_K^\times : C] = h^+$ would turn the "is this explicit unit fundamental?" proof into standard cyclotomic infrastructure.

Second, the open norm-Euclidean statements are primarily geometry-of-numbers problems. The small quadratic and $\mathbb{Q}(\zeta_3)$ examples are discharged by elementary remainder bounds and `nlinarith`; the cubic and quartic cyclotomic cases require real covering arguments. A Mathlib predicate such as `IsNormEuclidean` plus a Euclidean-to-PID hook would make these statements more natural and reusable, but it would not by itself prove the covering bounds.

Thus the priority routes are: package cyclic and small finite-group structure; expose a Kummer–Dedekind/residue-kernel interface for concrete splitting; model cyclotomic subfields via `IntermediateField`; add finite-place and integer-norm helpers; introduce `IsNormEuclidean`; and develop cyclotomic-units/index-theorem API. The remaining hard cases mostly reflect hard mathematics, not a weak `NumberField` API.